达梦数据库

新备份与还原

达梦技术丛书

前言

概述

本文档主要介绍备份还原的基本概念、技术原理，如何使用 DM 的命令行工具 DIsql 和 DMRMAN（DM RECOVER MANAGER）、客户端工具 MANAGER（管理工具）和 CONSOLE（控制台工具）进行备份还原，它们所提供的功能、以及详细的参数介绍。

读者对象

本文档主要适用于 DM 数据库的：

- 开发工程师
- 测试工程师
- 技术支持工程师
- 数据库管理员

通用约定

在本文档中可能出现下列标志，它们所代表的含义如下：

表 0.1 标志含义

标志	说明
 警告：	表示可能导致系统损坏、数据丢失或不可预知的结果。
 注意：	表示可能导致性能降低、服务不可用。
 小窍门：	可以帮助您解决某个问题或节省您的时间。
 说明：	表示正文的附加信息，是对正文的强调和补充。

在本文档中可能出现下列格式，它们所代表的含义如下：

表 0.2 格式含义

格式	说明
宋体	表示正文。
Courier new	表示代码或者屏幕显示内容。
粗体	表示命令行中的关键字（命令中保持不变、必须照输的部分）或者正文中强调的内容。标题、警告、注意、小窍门、说明等内容均采用粗体。
<>	语法符号中，表示一个语法对象。
::=	语法符号中，表示定义符，用来定义一个语法对象。定义符左边为语法对象，右边为相应的语法描述。
|	语法符号中，表示或者符，限定的语法选项在实际语句中只能出现一个。
{ }	语法符号中，大括号内的语法选项在实际的语句中可以出现 0…N 次(N 为大于 0 的自然数)，但是大括号本身不能出现在语句中。
[ ]	语法符号中，中括号内的语法选项在实际的语句中可以出现 0…1 次，但是中括号本身不能出现在语句中。
关键字	关键字在 DM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字既可以大写也可以小写。

访问相关文档

如果您安装了 DM 数据库，可在安装目录的“\doc”子目录中找到 DM 数据库的各种手册与技术丛书。

您也可以通过访问我们的网站 www.dameng.com 阅读或下载 DM 的各种相关文档。

联系我们

如果您有任何疑问或是想了解达梦数据库的最新动态消息，请联系我们：

网址：www.dameng.com

技术服务电话：400-991-6599

技术服务邮箱：dmtech@dameng.com

目录

1 DM7 备份与还原简介 1

1.1 概述 1

1.2 基本概念 2

2 DM7 备份与还原技术原理 18

2.1 DM7 备份与还原技术 18

2.2 应用场景分析 34

2.2.3 恢复方式选择 39

3 DM7 备份与还原实战 42

3.1 准备工作 42

3.1.1 支持与限制 42

3.1.2 归档配置 43

3.2 使用联机执行 SQL 语句进行备份还原 46

3.2.1 概述 46

3.2.2 数据备份 47

3.2.3 数据备份高级主题 53

3.2.4 管理备份 55

3.2.5 数据还原 73

3.2.6 数据还原高级主题 84

3.3 使用脱机工具 DMRMAN 进行备份还原 89

3.3.1 DMRMAN 概述 89

3.3.2 启动和配置 DMRMAN 90

3.3.3 数据备份 97

3.3.4 管理备份 108

3.3.5 数据库还原和恢复 127

3.3.6 归档还原 148

3.3.7 归档修复 151

3.3.8 表空间还原 152

3.3.9 查看操作日志 155

3.4 使用图形化客户端工具进行备份还原 165

3.4.1 使用 MANAGER 工具进行联机备份还原 165

3.4.2 使用 CONSOLE 工具进行脱机备份还原 174

1 DM7 备份与还原简介

数据库备份是 DBA 日常最重要的工作内容。备份不是目的，还原与恢复才是备份的目的。DBA 不仅要保证备份成功，还要保证一旦数据库发生故障时备份可还原可恢复。

本章主要介绍 DM7 备份与还原的概述和基本概念，为理解归档配置、备份还原的原理与工具使用奠定基础，内容主要包括：

- 概述
- 基本概念

1.1 概述

DM7 数据库中的数据存储在数据库的物理数据文件中，数据文件按照页、簇和段的方式进行管理，数据页是最小的数据存储单元。任何一个对 DM7 数据库的操作，归根结底都是对某个数据文件页的读写操作。

因此，DM7 备份的本质就是从数据库文件中拷贝有效的数据页保存到备份集中，这里的有效数据页包括数据文件的描述页和被分配使用的数据页。而在备份的过程中，如果数据库系统还在继续运行，这期间的数据库操作并不是都会立即体现到数据文件中，而是首先以日志的形式写到归档日志中，因此，为了保证用户可以通过备份集将数据恢复到备份结束时间点的状态，还需要将备份过程中产生的归档日志也保存到备份集中。

还原与恢复是备份的反过程。还原是将备份集中的有效数据页重新写入目标数据文件的过程。恢复则是指通过重做归档日志，将数据库状态恢复到备份结束时的状态；也可以恢复到指定时间点和指定 LSN。恢复结束以后，数据库中可能存在处于未提交状态的活动事务，这些活动事务在恢复结束后的第一次数据库系统启动时，会由 DM7 数据库服务器自动进行回滚。

备份、还原与恢复的关系如图 1.1 所示。

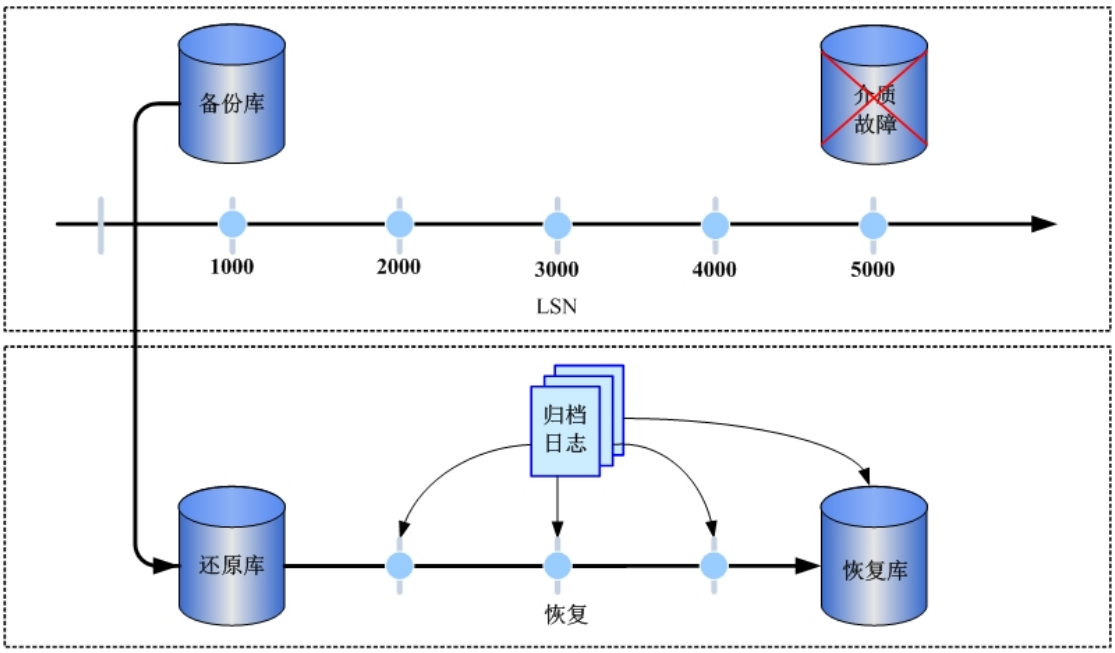


图 1.1 备份、还原与恢复

1.2 基本概念

本节介绍与 DM7 备份与还原相关的一些基本概念，了解这些基本概念是阅读后续章节的基础。

1.2.1 表空间与数据文件

DM7 数据库的表空间是一个逻辑概念，其目的主要是为了方便数据库的管理，数据库的所有对象在逻辑上都存放在某个表空间中，而物理上都存储在所属表空间的数据文件中。一个表空间由一个或多个数据文件组成。

数据文件是数据库中最重要的文件类型，是真实数据存储的地方。DM7 中数据文件的扩展名为.dbf，分为系统默认生成的数据文件和用户自己创建的数据文件两类。

在创建 DM7 数据库时，系统会自动创建 5 个表空间：SYSTEM 表空间、ROLL 表空间、MAIN 表空间、TEMP 表空间和 HMAIN 表空间。DM7 自动为这几个自动创建的表空间分别生成默认的数据文件：SYSTEM.DBF、ROLL.DBF、MAIN.DBF 和 TEMP.DBF，HMAIN 表空间没有默认的数据文件。

用户也可以创建自己的表空间，由用户创建的表空间统称为用户自定义表空间，在创建自定义表空间时需要为表空间指定数据文件。用户可通过为已存在的表空间增加数据文件或

创建一个新的表空间来创建数据文件。

因此，DM7 中的表空间有以下几类：

SYSTEM 表空间 存放了 DM7 数据库全局字典信息和全局系统数据，是 DM7 数据库能够正常运行的必要前提，默认对应数据文件 SYSTEM.DBF。CREATE TABLE 等 DDL 操作会修改 SYSTEM 表空间数据。

ROLL 表空间 存放 DM7 数据库运行过程中产生的所有回滚记录。DM7 中几乎所有的数据库修改操作都会生成回滚记录，并保存在 ROLL 表空间的数据文件中。ROLL 表空间是数据库全局对象，不论修改哪一个表空间，生成的回滚记录都是写入 ROLL 表空间，该表空间由系统自动维护，默认数据文件为 ROLL.DBF。

TEMP 表空间 存放临时表数据以及数据库运行过程中产生的临时数据。在数据库运行过程中，SORT、HASH JOIN 等操作都可能会生成临时结果集，它们作为临时数据存放在 TEMP 表空间中。TEMP 表空间是数据库全局对象，由系统自动维护。若数据库重启，保存在 TEMP 表空间中的所有数据都会丢失。TEMP 表空间的默认数据文件为 TEMP.DBF。

MAIN 表空间 在创建用户时，如果没有指定默认表空间，系统自动指定 MAIN 表空间为用户默认的表空间。

HMAIN 表空间 DM7 的 HUGE 表空间，用来存放 HUGE 表数据文件。HMAIN 表空间和 HUGE 表相关知识可参考《DM_DBA》。



注意：DM7 的物理备份还原不包括 HUGE 表空间，即数据库备份和还原不包括 HMAIN 和用户创建的 HUGE 表空间；也不支持对 HMAIN 和用户创建的 HUGE 表空间进行表空间备份和还原。DM7 提供了 HUGE 表备份还原系统函数支持 HUGE 表的联机备份还原，可参考《DM_SQL》。

用户自定义表空间 即用户通过创建表空间相关操作生成的表空间。

图 1.2 显示了一个完整的 DM7 数据库的表空间和数据文件分布情况，其中 TSDA 为用户自定义表空间。

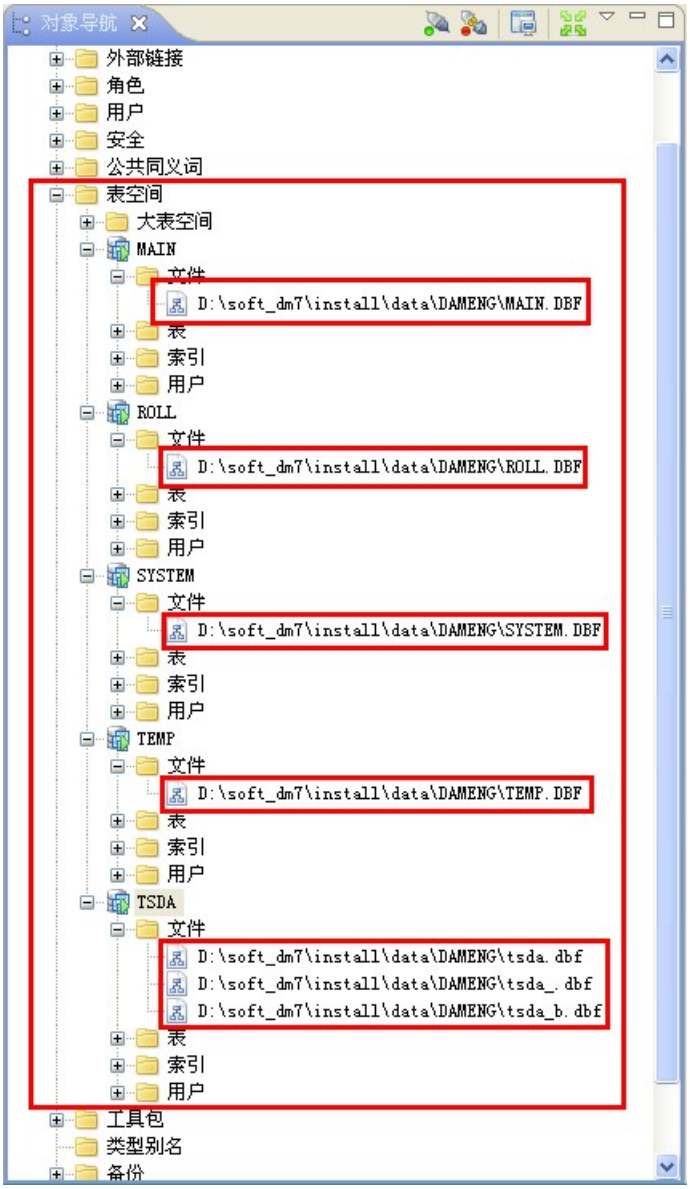


图 1.2 一个完整 DM7 数据库的表空间和数据文件分布

1.2.2 重做日志

重做日志，又叫 REDO 日志，忠实记录了所有物理页的修改，基本信息包括操作类型、表空间号、文件号、页号、页内偏移、实际数据等。数据库中 INSERT、DELETE、UPDATE 等 DML 操作以及 CREATE TABLE 等 DDL 操作最终都会转化为对某些数据文件、某些数据页的修改。因此，在系统故障重启时，通过重做 REDO 日志，可以将数据库恢复到故障时的状态。

DM7 数据库默认包含两个扩展名为 log 的日志文件，用来保存 REDO 日志，称为联机重做日志文件。这两个文件循环使用。任何数据页从内存缓冲区写入磁盘之前，必须保证其

对应的 REDO 日志已经写入到联机日志文件。

1.2.3 归档日志

DM7 数据库可以在归档和非归档两种模式下运行，归档模式的具体配置参考 3.1 小节。DM7 支持多种归档方式，本文中若无特殊说明，均指本地归档。当数据库处于归档模式下且配置了本地归档时，REDO 日志先写入联机日志文件，然后再异步写入归档日志文件。当数据故障退出时，归档日志文件中 REDO 可能会少于联机日志中内容。归档日志文件以配置的归档名称和文件创建时间命名，扩展名也是 log。

采用归档模式会对系统的性能产生影响，然而系统在归档模式下运行会更安全，当出现故障时其丢失数据的可能性更小，这是因为一旦出现介质故障，如磁盘损坏时，利用归档日志，系统可被恢复至故障发生的前一刻，也可以还原到指定的时间点，而如果没有归档日志文件，则只能利用备份集文件恢复数据库到备份数据时的状态。

除了表备份还原，其他的联机备份与还原必须运行在归档模式下。

1.2.4 PWR 记录

DM7 的 REDO 日志中包含有一种特殊的记录，称为 PWR（Page Written Record）日志。PWR 日志包含表空间 ID、文件 ID、数据在文件中的页号和页的当前 LSN 信息。每次数据页刷盘时，系统都会生成一条对应的 PWR 日志。PWR 日志的 LSN 与上一个日志的 LSN 相同（关于 LSN 的介绍见 1.2.5 检查点与日志序列号）。

PWR 记录可用来提升系统故障恢复速度。在引入 PWR 记录之前，数据库故障重启时需要重做所有有效的 REDO 日志记录，但是由于数据缓存淘汰等原因，往往有很多 REDO 日志对应的数据页修改已经写入到磁盘中了。这些无效的重做操作，引发了很多无效 IO；根据 PWR 记录可以快速判断数据页是否已刷盘，有效避免无用 IO。

某些情况下，PWR 记录也可以用来提升增量备份性能。比如，系统包含大量数据页，但自上次备份后，系统只修改少量数据页情况下，可以扫描归档日志文件，通过 PWR 记录快速获取到需要备份的数据页，避免扫描所有数据文件造成大量磁盘 IO。

1.2.5 检查点与日志序列号

DM7 数据库运行过程中，用户的所有操作都在内存中进行。每修改一条记录都必须先

把记录所在的数据页加载到 BUFFER 缓冲区中，然后进行修改。事务运行时，会把生成的 REDO 日志保留在日志缓冲区 RLOG_BUF 中，每条日志记录对应一个 LSN，当事务提交或日志缓冲区满或执行检查点时会进行日志刷盘。

检查点（ckpt）是一个数据库事件，它的功能是按照数据页第一次被修改的顺序，依次将 BUFFER 缓冲区中的脏页写入磁盘，并在这个过程中动态调整 CKPT_LSN 值，释放日志空间。

DM7 的检查点分为两种：完全检查点和部分检查点：

✓ **完全检查点**：会将内存缓冲区中的所有脏页写入磁盘，并调整 CKPT_LSN，在数据库正常关闭时会产生一个完全检查点。

✓ **部分检查点**：根据 dm.ini 配置文件中的参数 CKPT_FLUSH_RATE 和 CKPT_FLUSH_PAGES，确定每次检查点刷脏页的数量。执行部分检查点的过程中，DDL/DML 操作都可以正常执行，DM7 系统中绝大多数情况下触发的都是部分检查点。

下面介绍 DM7 中几个 LSN 值的意义，这些 LSN 值的定义与备份还原范围的确定有很大关系，能让读者更好地理解 DM7 备份还原。

- LSN

LSN（LOG SEQUENCE NUMBER），即日志序列号，表示 REDO 日志产生的顺序，系统为每个 REDO 日志分配一个 LSN 值。LSN 是一个 64 位整数，系统从 1 开始分配 LSN 值。除了 PWR 日志的 LSN 值为 CUR_LSN（与前一个 REDO 日志的 LSN 值相同）外，其他类型的 REDO 日志 LSN 值都是递增的、唯一的。

- CUR_LSN

系统当前 LSN，代表已经分配的最大 LSN 值。物理事务提交时，系统会为其分配一个唯一的 LSN 值（CUR_LSN + 1），同时修改 CUR_LSN 值为 CUR_LSN + 1。

- FILE_LSN

文件 LSN，已经写入联机重做日志文件的最大 LSN 值。每次将 RLOG_BUF 写入联机重做日志文件后，都会修改 FILE_LSN 值。

- FIRST_MODIFIED_LSN

数据页内存控制结构的一个字段，记录数据页第一次被修改时所产生 REDO 日志的 LSN 值，BUFFER 缓冲区中的所有脏页按照 FIRST_MODIFIED_LSN 值排序。

- FLUSH_LSN

FLUSH_LSN 是已经发起日志刷盘请求，但还没有真正写入联机，Redo 日志文件的最

大 LSN 值。

- CKPT_LSN

检查点 LSN。DM7 处理检查点时，将 BUFFER 缓冲区中以 FIRST_MODIFIED_LSN 值从小到大排序的脏页依次写入磁盘，检查点完成时 CKPT_LSN 被置为最后一个写入磁盘的脏页的 FIRST_MODIFIED_LSN 值。

由此可见，CKPT_LSN 表示此 LSN 对应的 REDO 日志已经写入磁盘，并且此 LSN 之前 REDO 日志对应修改的数据页已经全部写入磁盘。因此，数据库故障重启时，CKPT_LSN 之前的 REDO 日志不需要重做，只需要从 CKPT_LSN+1 开始重做 REDO 日志，就可以将系统恢复到故障前状态。并且，在联机重做日志文件中，LSN 值<=CKPT_LSN 的 REDO 日志都可以被覆盖。

根据以上介绍的几个 LSN 值，我们为确定备份范围特别定义了两个 LSN：

- START_LSN

为了保证备份的完整性和有效性，必须包含的归档日志起始 LSN 值。START_LSN = 备份开始时 CKPT_LSN + 1。

- END_LSN

为了保证备份的完整性和有效性，必须包含的归档日志结束 LSN 值。END_LSN = 备份结束时 FILE_LSN。

如果 START_LSN 等于 END_LSN + 1，则表明备份过程中，没有任何数据被修改。

在做增量备份时，要求其 START_LSN 必须大于等于基准备份的 END_LSN 值，这个要求可以简化还原过程，避免还原基准备份后必须先进行恢复再还原增量备份。

下图说明了联机日志文件、日志缓冲区 RLOG_BUF 以及相关各 LSN 之间的关系。

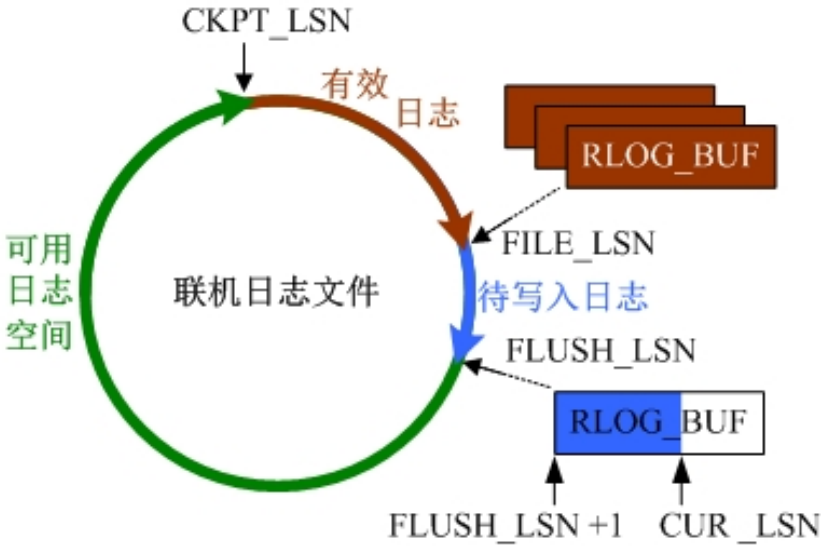


图 1.3 联机日志文件与日志缓冲区

在联机日志文件中，可以覆盖写入 REDO 日志的文件长度为可用日志空间；不能被覆盖的 REDO 日志，系统故障重启需要重做的 REDO 日志为有效日志，有效日志的 LSN 取值范围是(CKPT_LSN,FILE_LSN]。

数据库运行过程中产生的待写入日志首先写入日志缓冲区 RLOG_BUF，当日志刷盘时一起写入联机日志文件中。

1.2.6 备份集

备份集用来存放备份过程中产生的备份数据及备份信息。一个备份集对应了一次完整的备份。

一个备份集为一个目录，它由一个或多个备份片文件和一个元数据文件组成。备份片文件后缀为.bak，用来存放备份数据。元数据文件后缀为.meta，用来存放备份信息。

1.2.6.1 备份片

备份片是用来存储备份数据的文件。备份时，目标数据文件内容或归档日志内容经过处理后，都会存放到各自的备份片文件中。

备份片是用来存储备份数据的文件。备份片的个数，会按照指定的 MAXPIECESIZE 大小限制，陆续增加。

备份片中备份数据根据备份类型不同而不同:

✓ 库级备份时，备份数据包括数据文件和备份过程中产生的归档日志文件，先生成数据文件备份的备份片，后生成归档日志文件备份的备份片，数据文件与归档日志文件不会共用备份片。

- 当联机备份库时，由于备份过程中数据库服务器仍在工作且库备份开始时，只是取了当前的 CKPT_LSN，并未强制调整，有部分 REDO 日志对应数据修改还未来得及写入到数据文件中，这部分 REDO 日志会存在于本地归档文件中。，为确保备份数据的完整性，则会在备份结束时，也会将这部分 REDO 日志进行备份，并写入到独立的备份片中，若要备份的 REDO 日志量很大，则可能会占用多个备份片。
- 当脱机备份时，由于 REDO 日志会先刷入联机日志文件，再刷入本地归档日志文件中，如果库是异常退出，可能会存在部分 REDO 日志已经刷入联机日志文件，但并未刷入本地归档日志文件，利用本地归档恢复无法恢复到故障前的最新状态。

为避免这种情况的出现，备份时也会将这部分 REDO 日志备份到独立的备份片中，同样，如果日志量很大，也可能会占用多个备份片。

- 若联机库备份时，指定 WITHOUT_LOG，则不会备份日志，恢复时，可以利用本地归档日志文件，将库恢复到最新状态；脱机备份，不能指定 WITHOUT_LOG，若存在需要备份的 REDO 日志，则采用强制备份。

✓ 表空间级备份，仅备份指定表空间下的数据文件。

✓ 表级备份，备份目标表中的数据页。

✓ 归档备份，仅备份指定范围内的归档日志文件。

1.2.6.2 元数据

元数据文件用来存储备份信息。通过元数据文件，可以了解整个备份集信息。

元数据文件中包含的备份信息包括如下内容：

✓ 备份集本身相关的信息，如备份是否联机备份，备份的范围，备份的加密信息，以及备份的压缩信息等；

✓ 备份源库的建库参数信息，如 RAC 的节点数，是否大小写敏感，PAGE_CHECK 属性等；

✓ 备份集中备份数据文件信息，如备份哪些数据文件，以及每个数据文件从属表空间，在备份片中的起始位置等；

✓ 备份集中备份片的信息，如包含哪些备份片文件、备份片文件的编号，长度等。

✓ 备份集生成时备份库的 dm.ini 和秘钥文件（dm_service.private 或者 dm_external.config，若指定 usbkey 加密，则不备份）。

1.2.6.3 备份集搜索目录

备份集搜索目录，用于搜集目标备份集的文件路径。

当访问备份集存储介质为 TAPE 时，仅使用用户通过 WITH BACKUPDIR 指定的路径作为备份集搜索路径，具体如何搜集由第三方 SBT 接口实现决定；

当访问备份集存储介质为 DISK 时，备份集目录由三部分组成。一是通过 WITH BACKUPDIR 手动指定外；二是若访问备份集时指定 DATABASE 或者联机操作，则 DATABASE 对应库的默认备份路径或者联机操作中当前系统默认备份路径也作为备份集搜索目录；三是在指定还原或者增量备份时，指定的 BACKUPSET 和 BASE ON BACKUPSET 的上级目录也

作为备份集搜集目录。

备份集在 DISK 介质上为一个目录。从备份集搜索目录中查找备份集，就是判断指定目录是否为备份集目录，以及指定目录中是否存在备份集目录的过程。具体过程为：

第 1 步，先判断当前目录是否为备份集目录，即目录中是否存在 meta 文件，若是备份集目录，则认为当前目录即为备份集目录；否则，继续第 2 步；

第 2 步，遍历当前目录的所有一级子目录，判断一级子目录是否为备份集目录，若是，则收集。所有一级子目录判断结束后，结束该当前目录的查找，不再进入下层目录。

1.2.7 备份

1.2.7.1 概念

备份的目的就是，当数据库遇到损坏的情况下，可以执行还原恢复操作，把数据库复原到损坏前的某个时间点。用于还原恢复数据库的载体是备份集，生成备份集的过程便是备份了。

备份概念可以理解为，将待备份的数据经过处理如加密、压缩等后，写到备份片文件中，并将相关备份信息写到元数据文件中的过程。而一次备份的结果就是一个备份集。

备份库指需要进行备份的库，又称源库或源备份库。

1.2.7.2 分类

按不同的划分标准，备份可以有不同的分类方式，DM7 中对备份的划分标准有如下几种：

- 备份组织形式
- 备份时，数据库是否运行
- 备份的粒度大小
- 备份的内容分类
- 备份数据的一致性
- 备份数据的完整性

本节将依次根据这些划分标准的划分结果详细介绍各种备份分类的概念。

■ **逻辑备份和物理备份**

依据备份的组织形式，可以把备份划分为逻辑备份和物理备份。

物理备份指从物理存储层面，将目标备份对象下所属数据文件（库级、表空间级）、日志文件（本地归档日志）或者数据页（表级），经过压缩或者加密处理后，拷贝到备份片文件中的备份。这种备份是在文件层次进行的。

逻辑备份，指利用 DM7 提供的逻辑导出工具 DEXP，将指定对象（库级、模式级、表级）的数据导出到文件的备份。还原时再使用导入工具 DIMP 将之前导出数据重新导入到数据库中。这种备份是针对数据库对象的数据进行的。

两种方式中，物理备份是更强健的数据保护方式，也是备份策略中的首选。逻辑备份是物理备份的补充方式，相对物理备份而言，具有更大的灵活性。本书重点介绍物理备份，如果读者想了解 DM7 的逻辑备份，可参考《dexp 和 dimp 使用手册》。

- **联机备份和脱机备份**

按照备份时数据库是否处于运行状态，可以把备份划分为联机备份和脱机备份。

联机备份指数据库处于运行状态时，连接到服务器实例上，通过执行 SQL 语句进行的备份。当前许多系统都要求 7*24 小时提供服务，联机备份是最常用的备份形式之一。联机备份时，大量的事务处于活动状态，为确保备份数据的一致性，需要同时备份一段日志（备份期间产生的 REDO 日志）。按照这个需求，要求数据库必须配置本地归档，且归档必须处于开启状态。

脱机备份与联机备份相对应，指数据库处于关闭状态时，使用 DMRMAN 工具执行的备份。



注意：只有已经关闭的数据库才允许执行脱机备份。正在运行的数据库，无法成功执行脱机备份，系统会报错。

DM7 备份与还原中，可以对数据库本身和本地归档日志执行联机备份和脱机备份还原，表空间允许联机备份还原和脱机还原，表只能执行联机备份还原。

- **库备份、表空间备份和表备份**

按照备份的粒度大小，可以将备份划分为库备份、表空间备份和表备份。

库备份，顾名思义就是对整个数据库执行的备份，又称为库级备份。库备份支持联机备份和脱机备份。库备份的对象是数据库中所有数据文件和备份过程中的归档日志，可选择是否备份日志。

表空间备份即对指定表空间执行的备份，又称为表空间级备份。表空间备份仅仅支持联机备份。表空间备份的过程就是拷贝表空间内所有数据文件的有效数据的过程。TEMP 表空间存放的系统运行过程中生成的临时数据，不允许执行备份。RLOG 表空间对应数据库的联机日志，不允许备份。

表备份拷贝指定表所使用的所有数据页到备份集中，并记录各个数据页之间的逻辑关系用来恢复表数据结构。表备份均为联机完全备份，不需要备份归档日志，不存在增量备份之说。DM7 仅支持单个用户表备份或者分区表的单个子分区表的备份。相对库备份和表空间备份，表备份仅针对某个特定的用户表，灵活性有很大提高。

- **库数据备份和库日志备份**

按照备份内容不同，可以分为库中数据备份和库操作过程中产生的归档日志备份。库中数据备份，前面已有说明，包括库级、表空间级和表级备份。

库日志备份，简称归档备份，指对目标库操作过程中产生的 REDO 日志归档文件的备份。归档备份支持联机备份和脱机备份，联机归档备份，是收集第一路有效的归档目录下的归档文件进行备份；脱机备份是收集配置的第一路的归档目录下的归档文件进行备份。归档备份会把收集到的归档文件，备份到备份集中的备份片中。必要的时候可以使用归档备份集，还原归档。

- **一致性备份和非一致性备份**

按照备份集中的数据是否满足一致性，可以将备份划分为一致性备份和非一致性备份。

一致性备份指备份集中包含了全部的备份数据。可以仅利用备份集中的备份数据就把数据库恢复到备份时的状态，如联机库备份（带日志），脱机库备份等。

非一致性备份指单独使用备份集中的数据还不足以把数据库还原到备份时某个数据一致性的点，需要借助归档来恢复，如表空间备份、联机备份（不带日志）等。

- **完全备份和增量备份**

按照备份数据完整性，可将备份划分为完全备份和增量备份。仅库级和表空间级备份存在完全备份和增量备份。

完全备份指备份中包含了指定的库（或者表空间）的全部的有效数据页，这样的一个备份通常会很大，且备份持续时间也比较长。对于一个需要经常备份的系统，执行完全备份是比较消耗时间和空间的。

增量备份指基于指定的库（或者表空间）的某个备份（完全备份或者增量备份），备份自该备份以来所有发生修改了的数据页。这个基于的备份称为该增量备份的基备份。相对而言，增量备份通常很小，备份也较快且占用空间也会比较少。

由于增量备份是基于某个已经存在备份集进行的备份，这样的依赖的关系，就构成了一个备份集链表。一个完整的备份集链表，必须包含一个完全备份，且这个完全备份一定是链表中的第一个备份。若备份集链表中存在多个备份集，则其他位置的备份集均为增量备份集，且越往后备份集越新，最后一个备份集为最新生成的备份集。若备份集链表中只有一个备份集，那么这个备份集一定为完全备份的备份集。

1.2.8 还原

1.2.8.1 概念

还原是备份的逆过程，就是把备份集中的备份数据经过处理后，写回到还原目标库中相应的数据文件中的过程。

还原的数据是不可以直接提供服务的，因为还原回来的数据通常是处于非一致性状态，需要执行恢复操作，使得目标数据库数据一致，才能对外提供服务。

还原库指用来做还原的库，又称目标库，或还原目标库。

1.2.8.2 分类

与备份的分类类似，还原也可按照以下标准进行划分：

- 还原组织形式
- 还原时，数据库是否运行
- 还原的粒度大小
- 还原内容分类
- 还原数据的完整性

下面分别对还原的不同分类进行说明。

■ **逻辑还原和物理还原**

按照还原组织形式，分为逻辑还原和物理还原，分别对应逻辑备份和物理备份。

逻辑还原是逻辑备份的反过程，即使用 DM7 提供 DIMP 工具，把使用 DEXP 导出的备

份数据重新导入的过程。

物理还原是物理备份的逆过程，可以通过联机执行 SQL 语句，或者通过 DMRMAN 等脱机工具，把备份集中备份的数据内容（数据文件、数据页、归档文件）还原到目标位置的过程。

同样，本书不介绍逻辑还原，下文提到的还原都是物理还原。

- **联机还原和脱机还原**

按照还原时目标数据库是否处于运行状态，将还原划分为联机还原和脱机还原。

联机还原指数据库处于运行状态时，通过执行 SQL 语句完成的还原过程。目前联机还原仅支持表空间还原和表还原。表空间的联机还原要求目标表空间不能处于 ONLINE 状态。

脱机还原指数据库处于脱机状态时执行的还原，通过 DMRMAN 工具进行。目前脱机还原支持库级还原、表空间级还原和归档还原。脱机还原指定的目标库必须处于关闭状态。

- **库还原、表空间还原、归档还原和表还原**

按照备份集中备份粒度大小：库级、表空间级和表级，将还原分为数据库还原、表空间还原和表还原。数据库还原，仅能使用库级备份执行，将备份中所有的数据文件还原到指定位置的过程。

表空间还原，可以使用库级备份和表空间级备份，将备份中目标表空间中所有的数据文件还原到目标位置并重做 REDO 日志执行恢复的过程。可以通过指定 UNTIL LSN 和 UNTIL TIME 恢复到某个点，也可以不指定恢复到最新状态。此外，DM7 还支持从目标备份集中还原出指定表空间中指定文件的操作。表空间联机还原时，要求目标表空间不能处于 ONLINE 状态，而 SYSTEM 和 ROLL 表空间内的信息除了涉及到表空间自身，还包含许多涉及整个数据库的数据信息，一旦 OFFLINE 整个数据库将无法工作，所以 SYSTEM 和 ROLL 表空间不允许执行联机还原，允许脱机还原。TEMP 表空间和 RLOG 表空间本身就没备份，所以也不支持还原。其他用户数据表空间允许联机还原和脱机还原。

表级还原，仅可使用表级备份执行，将备份中目标表包含的所有数据页还原到目标库中，并在目标库中重建备份时表上的索引和约束的过程。

- **库数据还原和库日志还原**

从还原内容类型，分为库数据还原和库日志还原。

库数据还原指前面说明的库级、表空间级和表级还原。

库日志还原，是指归档日志的还原。归档还原只能使用归档备份执行，归档还原的实质，是把备份集中备份的归档，重新还原回来。用户也可以根据情况需要，指定还原备份集中满足条件的归档文件。

■ **完全备份还原和增量备份还原**

根据还原备份集中备份数据是否完整，将还原分为完全备份还原和增量备份还原。仅库级和表空间级备份的还原存在。

完全备份还原指目标还原备份集为完全备份。完全备份还原可以不依赖其他备份集直接完成还原操作。

增量备份还原，指目标还原备份集为增量备份。增量备份还原需要完整的备份集链表才能完成还原操作。因此，增量备份还原时需要用户确保完整备份集链表中各备份集都存在，否则将无法执行。

•1.2.9 恢复

1.2.9.1 概念

恢复是重做本地归档日志或者备份集中备份的归档日志的过程。利用恢复操作，可以使数据库恢复到备份时，或者某个最新状态。没有经过恢复的还原数据库是不允许启动的。

表空间和表还原均为联机执行，都不需要再执行恢复操作。因为表空间的还原、恢复操作，是一次性完成的，而表还原是联机完全备份还原，不需要借助本地归档日志，所以也不需要恢复。归档备份还原，仅仅是归档备份集中归档还原出来，与库中数据无关，无所谓的恢复操作，但还原出来的归档日志，可用于恢复操作。因此本节所说的恢复是指用脱机工具 DMRMAN 或控制台工具 CONSOLE 完成的数据库恢复操作。

恢复库指已经做过恢复的库。

1.2.9.2 分类

根据恢复过程中是否重做归档，以及重做归档日志的来源，可将恢复分为更新 DB_MAGIC、从备份集恢复和从归档恢复三类。

■ **更新 DB_MAGIC**

首先介绍一下 DM7 的 PERMANENT_MAGIC 和 DB_MAGIC，两者都是 DM7 在初始化数

据库时自动生成的用来标识数据库唯一性的值。PERMANENT_MAGIC 一经生成，永久不变，称为数据库永久魔数。每一个库有且仅有一个数据库永久魔数。DB_MAGIC 则记录着数据库的变化，当数据库经过备份还原并恢复后，DB_MAGIC 就会改变。

可以通过下列语句查看系统的 PERMANENT_MAGIC 和 DB_MAGIC 值。

```
SELECT PERMANENT_MAGIC;
SELECT DB_MAGIC FROM V$RLOG;
```

当执行一个完整备份集链表或一个完全备份的备份集还原后，且最后的备份集在备份过程中无日志生成（如正常退出库的脱机备份），那么此时还原数据库中数据与备份时数据状态一致。在不需要重做归档日志恢复数据的情况下，可以直接更新 DB_MAGIC 来完成最后的恢复工作，否则还原目标库将无法启动。

■ **从备份集恢复**

如果执行还原的备份集在备份过程中生成了日志，且这些日志在备份集中有完整备份（如默认联机数据库备份），那么在执行备份集还原后，可以重做备份集中备份的日志，将数据库恢复到备份时的状态。这个利用备份集中备份日志的恢复过程，可以看做是从备份集恢复。

■ **从归档恢复**

利用重做本地归档日志来恢复数据的过程，称为从归档恢复。这些归档日志可以本地已经存在的，也可以是从归档备份中还原到本地的。从归档恢复允许恢复到指定的时间点及指定的 LSN 值。若同时指定了时间点和 LSN，则以较早的为结束点。从归档恢复仅能在源库上执行（经过备份集还原的库或之前的某个时间通过拷贝的库也可以，即 PERMANENT_MAGIC 一定要相等）。使用归档恢复的前提是能够访问本地归档日志。

需要执行从归档恢复的场景有以下两种：

✓ 对于某个已经处于数据一致性状态的数据库，需要某个更新的时间点或者 LSN 的数据库数据；

✓ 对于某个已经执行过备份集还原的数据库，需要重做归档使数据库数据处于一致性状态。

1. 恢复到指定时间点

DM7 中事务提交时，系统会生成一个特殊的 REDO 日志，记录事务提交的时间。重做归

档日志时，一旦碰到比指定时间点更大的事务提交时间 REDO 日志，马上终止重做归档日志过程。用户可以通过指定一个时间点，使数据库恢复到这个指定的时间点。比如：用户在下午 5 点做了一个误操作，删除了某些重要数据；我们可以指定恢复时间点到下午 4:59 分，恢复被误删除的数据。

2． 恢复到指定 LSN

DM7 中每条 REDO 日志记录都有一个 LSN 值，用户可以指定一个 LSN 值，将数据库恢复到产生指定 LSN 值的状态。

3． 备份集恢复

从备份集恢复，要求备份集中备份了归档日志，如故障库的脱机库备份，以及未指定 WITHOUT LOG 的联机库备份。备份集恢复就是要重做这些归档日志，且不允许指定恢复到的时间点或者 LSN。

4． 恢复到最新

若不指定恢复到的时间点和恢复到的 LSN，则会重做所有本地归档，将数据库恢复到尽可能新的状态。

2 DM7 备份与还原技术原理

本章介绍 DM7 备份与还原的原理与技术实现，读者通过对应用场景分析的学习，能够正确选择备份、还原与恢复的方式。

本章内容主要包括：

- DM7 备份与还原技术
- 应用场景分析

2.1 DM7 备份与还原技术

2.1.1 备份

DM7 的备份是以物理存储结构和归档日志作为备份的理论基础的。任何一个对 DM7 数据库的操作，归根结底都是对某个数据文件页的读写操作。物理备份的任务之一就是把这些数据文件中有效的数据备份起来。另外，由于数据库系统运行过程中，不是所有的操作都会立即体现到数据文件中，而是首先以日志的形式写到归档日志中，所以对于这些归档日志的备份是 DM7 物理备份的另外一个任务。

2.1.1.1 备份过程

库级备份和表空间级备份，以及归档备份均属于物理备份。表级备份可简单地看做是逻辑和物理备份的混合过程，它介于逻辑备份和物理备份之间，比纯逻辑备份效率要高。

所有备份操作均包括当前库配置文件 dm.ini 和服务器秘钥文件（dm_service.private 或者 dm_external.config）的备份。若目标库指定 usbkey 加密，则不备份服务器秘钥文件。

- **库级和表空间级备份**

DM7 的联机库级和表空间级备份要求必须配置本地归档并启动归档模式，即在 dm.ini 中将 ARCH_INI 置为 1，dmarch.ini 中配置本地归档。

库级和表空间级备份都属于物理备份，备份对象都是物理文件：数据文件和归档日志，都可以进行完全备份和增量备份。

下面结合图 2.1 分别对完全备份和增量备份进行说明。

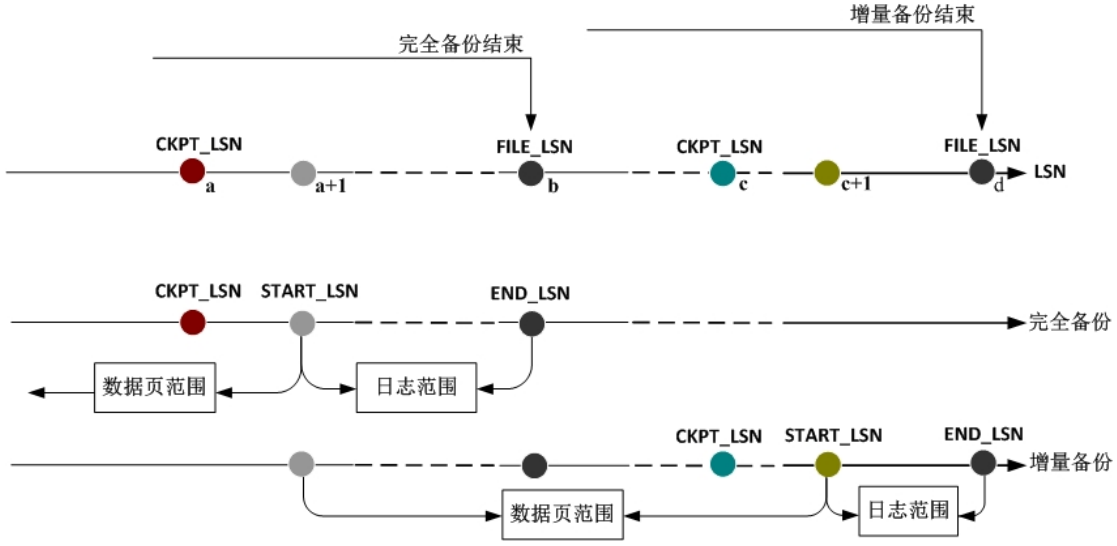


图 2.1 备份对象范围界定

（一） 完全备份

备份数据文件

数据文件备份，就是数据文件中数据页经过简单校验后，从数据文件中拷贝到备份片文件中的过程。

表空间级备份相对库级备份来说，备份粒度较小。表空间级备份针对的是表空间内的数据文件，而库级备份针对的是整个数据库的所有数据文件。参照图 1.2，若执行库级备份，则备份数据文件包括除 TEMP 表空间以外的其他表空间内的所有数据文件；若执行表空间级备份，以用户表空间 TSDA 为例，那么备份数据文件仅为 TSDA 内的数据文件，包括 tsda.dbf、tsda_.dbf 和 tsda_b.dbf。

由于备份表空间的字典等控制信息存储于备份库的 SYSTEM 表空间中，所以表空间级备份恢复时只能在源备份库上联机执行，且要求源库一定能够正常运行。

数据文件备份结束后，START_LSN 之前修改的所有数据页都被备份下来。

备份归档日志

归档日志备份，就是将备份过程中产生的归档日志拷贝到备份片文件中，归档日志占用独立的备份片文件。如果在备份数据文件的过程中没有产生新的归档日志，则归档日志不用备份。若指定备份归档日志，那么备份归档日志的范围从 START_LSN 开始，到 END_LSN 结束，如图 2.1 所示。而 START_LSN 和 END_LSN 值以元信息的形式记录到 META 文件中。

备份归档日志的起始点和结束点分别为 START_LSN 和 END_LSN。START_LSN 和

END_LSN 的界定如下：

START_LSN

系统在做检查点 CKPT 时内存中修改的数据被刷入的数据文件中，因此数据文件的备份是备份至系统上一个 CKPT 时刻，即 CKPT_LSN，如图 2.1 中第一个横轴中的 a 点。此后，如果数据库有新的修改，这些新的修改也需要备份下来。而这些修改都记录在归档日志中，因此需要备份 CKPT_LSN 之后的归档日志。所以将 CKPT_LSN+1 作为备份归档日志的起始点，即 START_LSN，如图 2.1 中第二个横轴所示。

END_LSN

数据文件备份结束时，归档日志备份也要结束。此时归档日志中的最后一个 LSN 就是 FILE_LSN，因此，规定 FILE_LSN 作为备份归档日志的结束点，称为 END_LSN。

若备份时，指定了加密或者压缩，则备份内容（数据页和归档日志）先经过加密或者压缩处理后，再写入到备份片文件中。元信息不加密，直接写入 meta 文件。

（二） 增量备份

库级和表空间级备份支持增量备份。所谓增量备份，就是备份自上次备份以来（END_LSN 值之后）的所有更新内容，包括数据文件和归档日志。上一个备份称为增量备份的基备份。第一个基备份一定是一个完全备份。

增量备份操作对象也包括数据文件与归档日志文件。

数据文件备份

增量备份只备份上次备份结束(END_LSN)之后更新的内容，即增量备份的 START_LSN 一定要大于其基备份的 END_LSN。DM7 规定增量备份中备份数据页的范围是从基备份 START_LSN 到本次增量备份的 START_LSN，如图 2.1 中第三根横轴所示。

为避免增量备份还原过程中重做其基备份的归档日志，要求增量备份时其基备份中的所有未写入数据文件的更新操作都刷入数据文件。

DM7 增量备份根据数据页上的 LSN 值和基备份 END_LSN 信息，过滤掉已经在基备份中备份过的数据页（PAGE_LSN <= 基备份 END_LSN）。因为增量备份数据文件中的数据已经包含了其基备份归档日志中记录的内容，因此增量备份还原的时候，就不再重做其基备份的归档日志了。

与完全备份一样，若增量备份时指定了加密或者压缩，备份内容（数据页和归档记录）也是经过加密或者压缩处理后，再写入备份片文件。

归档日志备份

归档日志文件的备份过程与完全备份中描述的一样，备份 START_LSN 和 END_LSN 之间的归档日志，如图 2.1 中第三根横轴所示。

增量备份时，必须确保其基备份可访问，以便获取其基备份信息。

- **归档备份**

此处说的归档备份，和上文的归档日志备份略有不同。

上文提到的归档日志备份，特指备份数据文件期间，新生成的 REDO 日志（已经归档了的 REDO 日志）的备份。归档日志区间是指一个 START_LSN 到 END_LSN 之间。归档日志的数据是顺序存储的，都是有效的数据，不用做过滤无效页的操作。

此处的归档备份，是把归档目录下，符合条件的归档日志文件都备份下来，与上文提到的备份数据文件类似。

两者虽然备份的都是 REDO 日志，但前文提到的归档日志备份是为了确保备份数据文件中数据的完整性的辅助备份；而此处的归档备份，仅仅是针对本地归档日志文件的备份，与数据文件无关。

归档备份仅备份属于目标库的归档日志文件，即归档日志文件中 DB_MAGIC 与当前库 DB_MAGIC 一致的归档日志文件。不同库，即使本地归档目录配置到同一个目录，其他库的归档日志也不会备份；对于同一个库，当经过数据库还原恢复之后，DB_MAGIC 发生了变量，那么再执行归档备份时，数据库还原恢复之前的归档日志也不会备份。所以，可能会出现同一个本地归档目录中已经存在的归档日志文件个数与实际备份的归档文件个数不同的情况，也是正常的。

- **表级备份**

表级备份实现过程分为两部分：数据部分和元信息部分。

在 DM7 中，每个普通表都有一个聚集索引，数据通过聚集索引键排序。表级数据备份就针对表的聚集索引进行，表上的其他约束和索引都是在聚集索引的基础上建立的二级索引，不在表级数据备份之列。

表级备份的元信息包括建表语句、重建约束语句、重建索引语句，以及其他相关属性信息。

表级备份均为完全备份和一致性备份，不需要使用归档，所以不要求配置本地归档；实现过程采用表的内存逻辑存储结构，所访问的页均为有效页，故也无需进行智能抽取。

2.1.1.2 智能抽取

智能抽取仅限于在备份数据文件过程中使用，它依赖于 DM 的物理存储结构的实现。备份数据文件的过程中，在遍历各文件中数据页时，根据描述页的标记判断数据页是否被分配、使用，将未使用的数据页剔除，仅保留有效数据页进行备份。

由于剔除了未被使用的数据页，因此相对于直接拷贝数据文件方式，物理备份产生的备份集往往会更小，更节省存储空间。并且由于只挑选有效数据页进行备份，还可以有效减少 IO 数量，提升备份还原的效率。

从单个数据文件和数据页来看，以下几种情况是特殊处理的：

1. 处于 RES_OFFLINE 状态的表空间仅以 0 号文件记录表空间状态和表空间 ID，不再备份具体内容。

2. 其他非 RES_OFFLINE 表空间中的其他数据文件。在库级备份过程中，若指定数据文件不存在，且不是 SYSTEM 表空间或者 ROLL 表空间文件，则当前数据文件不备份并置所属表空间状态为 RES_OFFLINE 状态，继续执行下一个文件备份；表空间备份过程中，若指定数据文件不存在，则报错，终止备份。

3. 数据文件头校验过程，若文件首页为空，则认为是文件创建成功，但是尚未刷盘引起的，因此不作校验失败处理，此时仅备份数据文件的元信息，比如文件路径、所属表空间名、是否加密等，不备份文件内容；否则文件首页不为空，但校验失败，比如解析文件头获取的文件大小和偏移非法等，将报错处理。例如，以非 DDL_AUTO_COMMIT 方式，创建一个表空间或者添加一个数据文件，然后，强杀服务器，就有可能出现这种情况。

4. 一个数据页校验过程中若发生页断裂（一般 OS 刷盘单位 512 字节，而数据库数据页至少 8K，数据库正常工作过程中执行备份，有可能出现数据页断裂的情况），先在间隔几十毫秒的时候重试 5 次，若重试过程中成功，则认为成功，执行页备份。若 5 次后仍然失败，若目标库没有指定表空间或者全库加密，则认为是由于刷数据页未完成，待恢复过程中补齐，正常进行备份；否则，认为校验失败，停止备份，因为指定了加密的数据页刷盘时会再次加密，出现页断裂的情况时一个完整数据页的加密情况肯定不完整，恢复时无法区分。

所有数据文件数据页的校验失败原因均会记录到备份还原日志文件 dm_BAKRES_xxx.log 中，若执行备份失败，可通过查看日志分析原因。

2.1.1.3 加密和压缩

DM7 支持对备份数据进行压缩和加密处理。压缩和加密均由用户显式指定，也就是说如果用户没有指定压缩或加密，则备份过程中不会对备份数据进行相关处理，具体使用见 3 DM7 备份与还原实战 中的相关介绍。

压缩级别支持 1~9 级，级别越高压缩比越高，但同时压缩速度越慢。默认采用压缩级别 1。

加密从三方面进行：加密密码、加密类型和加密算法。一是加密密码，通过使用 IDENTIFIED BY <加密密码>来指定。二是加密类型，分为不加密、简单加密和完全加密，简单加密仅仅对部分数据进行加密，加密速度快，但是安全系数低；完全加密会对所有的数据执行加密，安全系数高。三是加密算法，关于加密算法可参见《DM_DBA》加密章节。加密类型和算法，用户均可手动指定。若用户指定了加密密码，但又未特别指定加密类型和算法，则使用默认加密算法进行简单加密。如果用户指定了加密密码，又特别指定加密类型为不加密，则采用不加密。

若同时指定加密和压缩，则先压缩后加密。加密压缩处理的对象，包括经过智能抽取的有效数据文件数据页，和待备份的归档日志。

对于增量备份，若指定加密，且基备份也存在加密，则使用的加密算法和加密密码必须与其基备份中一致；若其基备份未进行加密处理，则对增量备份使用的加密算法和密码没有特殊要求。

2.1.1.4 PWR 优化

PWR（Page Written Record）就是记录下每个数据页的刷盘动作，把这些记录作为特殊的日志写到归档日志文件中。PWR 日志会记录每次刷盘时数据在数据页中的编号及所属数据页的 LSN 信息等。

增量备份的常规实现需要扫描所有的数据文件页，然后依次取出 LSN 大于或等于其基备份 START_LSN 的数据页。而采用 PWR 优化，是通过扫描归档日志中 PWR 日志，直接比较日志 LSN 值与基备份的 START_LSN，就可以确定需要备份的数据页了。

由于增量备份是差别备份，需要备份的数据量不会很大。在数据库比较小的情况下，两者对性能的影响差别还可以忽略；但如果数据库很大，而需要备份的数据页又很少，常规扫描动作则会占用大量的时间，而采用 PWR 仅扫描归档中的 PWR 日志，就会快很多。因为一

般情况下，归档日志文件量会远小于数据文件量。

增量备份实现过程中，如果发现没有指定归档目录，或者从指定归档目录搜集的归档日志不连续，缺失等导致搜集 PWR 出错，或者搜集到日志量大于总的数据文件量，则会使用常规备份方式完成备份。用户可在备份语句中指定 USE PWR 来启动增量备份的 PWR 优化，默认不使用。

2.1.1.5 并行备份

DM7 支持对库级、归档备份和表空间级的并行备份，用户可通过关键字 PARALLEL 指定是否执行并行备份，以及执行并行备份的并行数。并行数没有上限，实际执行时具体使用的并行数由用户指定的并行数和实际可使用的最大并行数决定。并行备份以数据文件为单位，也就是说一个数据文件仅可能出现在一个并行分支中，那么实际使用中并行数首先为以目标备份文件数作为上限，然后再取创建进程个数作为最后使用的并行数。默认并行数为 4。若未指定关键字 PARALLEL，则认为非并行备份。

增量备份是否并行及并行数与其基备份无关，取值方法与完全备份一致。

并行备份的结果包括一个主备份集目录和几个子备份集目录，每个目录都是一个完整的备份集。子备份集不能作为常规备份集使用，不能单独执行还原或者作为其他备份集的基备份。子备份集目录个数与并行备份时使用并行数相等。若存在归档日志备份，那么归档日志备份到主备份集目录中。

以下为一个并行数为 4 的脱机并行备份集（左）和非并行脱机备份集（右）比较截图：



图 2.2 并行备份集目录结构示意图

并行备份仅适用于文件备份，包括数据文件和归档日志文件备份过程。PWR 优化和 TAPE 介质存储不支持并行。并行备份集中子备份集不能单独还原或者恢复，不能作为增量备份的基备份集。

2.1.2 还原与恢复

还原是备份的逆过程，它是将已经备份的数据信息按照原样还原回来的过程。恢复是为了确保数据一致性，重做 REDO 日志的过程。所有的备份都存在还原操作，但仅库级还原和表空间级还原后，都需要恢复操作。

所有的还原和使用备份集的恢复操作，都需要先校验备份集的合法性，比如备份集是否完好、用户指定的解密密码是否与备份集中一致、备份集的备份级别对象是否与还原对象匹配等。以下说明说中跳过备份集本身合法性校验操作，直接说明具体还原原理。

2.1.2.1 还原过程

- **库还原**

库还原包括库中数据还原和库配置信息还原。库中数据还原，就是将备份集中记录的数据文件重建，并将备份的数据文件页按原样写回到目标数据文件中，即还原后的数据页内容与位置均与备份时一致；库配置信息还原，是指库配置文件 dm.ini、控制文件 dm.ctl、库中服务器秘钥文件（dm_service.private 或者 dm_external.config）还原，以及联机日志文件重建。库还原只能使用库备份集。

库还原允许用户通过指定 dm.ini 的完整路径来指定还原到某个具体库，或者通过指定文件目录来指定还原到某个指定目录中。若指定 dm.ini 还原，则指定的 dm.ini 必须存在且各项配置信息有效，其中 CTL_PATH 配置路径，要求必须配置且路径必须有效；若指定目录还原，则指定目录作为数据库系统目录处理。

库还原过程简单概括为：

1. 还原信息构造，即指根据备份集中备份信息和还原时指定的目标，构造库配置文件和库数据文件路径。指定目录还原时，若构造的库配置文件已经存在，则报错，可通过指定 OVERWRITE 关键执行覆盖操作；指定 dm.ini 还原时，直接覆盖；

- 库配置文件路径构造策略：指定 dm.ini 还原，控制文件 dm.ctl 文件路径使用 dm.ini 文件中 CTL_PATH 的配置值，系统目录使用 SYSTEM_PATH 的配置值，服务器秘

钥文件(dm_service.private 或者 dm_external.config)、联机日志文件均认为在 SYSTEM_PATH 下；指定目录还原，系统目录使用指定还原目录，所有库配置文件均认为在指定还原目录下。联机日志文件命名规则，单机环境为 db_name+文件编号.log，其中 db_name 取自备份集备份库的名称，文件编号从 1 开始，如 DAMENG01.log、DAMENG02.log；RAC 环境命名规则为 RAC+站点号_文件编号.log，其中站点号和文件编号均为实际编号+1，如两站点 RAC 环境，0 号站点联机日志文件名为 RAC01_01.log，RAC01_02.log；1 号站点联机日志文件名为 RAC02_01.log、RAC02_02.log。联机日志文件至少 2 个。

- 库数据文件路径构造策略：1)若指定 mapped file 还原，则使用 mapped file 中指定的文件路径,包括镜像文件路径,均与 mappedfile 中保持一致;未在 mapped file 中指定的数据文件路径遵循下一条；2)若还原库系统目录与备份集中备份库系统目录一致，则保持备份集中各数据文件路径不变；否则，保持文件名不变，调整到还原库系统目录下。调整路径过程中，若发现调整后的路径，与已经调整的其他路径存在重复，则重命名后放入当前系统目录中，命名规则为:db_name+序号.dbf，其中，db_name 取自备份集备份库名。3)实际创建文件过程中，若目标文件已经存在，则重命名后也放入还原库系统目录下，直至无重复为止，重命名规则与 2)描述一致。

2. 库配置文件还原，为避免非法配置文件存在，库配置文件还原其实就是根据第 1 步中构造的配置文件路径，删除重建的过程。对于库配置文件 dm.ini，仅会在指定目录还原时，执行重建操作，并将备份集中备份的 dm.ini 文件内容还原到目标 dm.ini 中；对于指定 dm.ini 的还原，则会默认保持不变，用户也可通过 REUSE DMINI 关键字，指定重用备份集中备份的 dm.ini 中优化配置信息。服务器秘钥文件(dm_service.private 或者 dm_external.config)，仅在备份集中备份库非 usbkey 加密的情况下重建，并使用备份集中备份的秘钥内容进行还原。

3. 库数据文件还原，即先删除还原库中所有数据文件，然后，创建第 1 步中构造的目标数据文件，若已经存在，则重命名；最后，将备份片中备份的数据页经过必要的解密或者解压缩后，按原样写到新的数据文件中。数据文件还原后，将当前文件系统刷入控制文件 dm.ctl 中。

4. 更新库信息，具体包括：1)日志信息，设置当前 CKPT_LSN 为备份集中 START_LSN-1，并设置日志文件状态为 INACTIVE；2)更新 DB_MAGIC，还原后，库中 DB_MAGIC、PERMNANT_MAGIC 均与备份集中相同；3)设置还原标志，标识当前库为指定

过库还原的库，不允许使用。

还原过程中若指定 mapped file，则最终还原的数据文件路径和镜像文件路径，若不是已经存在的文件路径，则将与 mapped file 中保持完全一致。Mapped file 中数据文件路径一经指定，那么还原后文件路径理论上一定与 mapped file 中指定一致。镜像文件路径是可选，若 mapped file 中某个文件节点未设置镜像文件路径，那么即使备份时相应的镜像文件路径存在，也会被置空，即还原后，不会生成镜像文件；若 mapped file 中某个文件节点设置了镜像文件路径，那么即使备份时相应的镜像文件即使未设置，也会根据 mapped file 中指定设置。

由于还原是要确保数据库数据的完整性，所以，对于增量备份的还原，需要搜集完整的备份集链表，然后从前到后，逐个还原备份集中数据。鉴于增量备份 START_LSN 确定规则，增量备份的还原过程中，不需要重做任何归档日志。

■ 表空间还原

表空间还原，与库还原中数据文件还原类似，但范围限制在指定目标表空间中。具体参照库还原过程中的数据文件还原部分的相关说明。

与库还原不同的是：

1. 表空间还原后直接执行了目标表空间上 REDO 日志的重做操作，允许指定 UNTIL LSN 和 UNTIL TIME，若未指定，则将表空间恢复到最新状态。表空间还原时会自动将目标库配置的第一路本地归档目录也作为归档收集目录。重做的 REDO 日志优先取指定和自动加入的本地归档目录中的日志，若未在指定归档目录或者当前库配置的本地归档目录中找到目标归档，则尝试使用备份集中备份的 REDO 日志。

RAC 环境不支持使用 UNTIL，且要保证归档使用的完整性。

由于表空间还原恢复不会执行回滚，所以 redo 日志中未 commit 的事务修改也会有效。比如，插入数据并提交时 LSN 值是 50000，如果因指定 UNTIL 或者数据库故障恢复到 LSN=49999（此时事务处于未提交状态）。事实上，与恢复到 LSN=50000 时数据一致。



警告：表空间还原不建议使用 UNTIL LSN 或者 UNTIL TIME，若要使用，则确保备份集生成之后未执行过 DDL 操作，否则将可能出现不可预知的后果。

2. 还原后数据文件的路径，若未指定 mapped file，则认为当前系统目录与备份集中一致，不会事先调整。实际创建过程中，若发现已经存在或者创建失败后，处理方式与数据库还原中处理一致。

3. 表空间联机还原要求目标表空间必须处于 OFFLINE 或者 RES_OFFLINE 状态。表空间还原执行之前，先将表空间状态置为 RES_OFFLINE，若还原失败，则表空间将处于 RES_OFFLINE 状态，允许再次执行还原操作；还原成功，则将表空间状态调整初始状态，若初始状态为 RES_OFFLINE，则调整为 OFFLINE 状态。

表空间还原，可以使用表空间级备份集还原，也可以使用库级备份集还原。若目标表空间未被破坏（即未处于 RES_OFFLINE 状态），也支持目标表空间中指定文件的还原。



说明：库还原时：ONLINE（联机状态）、OFFLINE（脱机状态）、RES_OFFLINE（还原故障状态）。可通过动态视图 V$TABLESPACE 的 STATUS$列值进行查看：0 代表 ONLINE，1 代表 OFFLINE，2 代表 RES_OFFLINE。

■ 表还原

与表级备份相对应，表还原就是将备份集中的表数据在目标表中还原，并将表约束、索引等元数据信息在目标表上重建的过程。表还原分三部分进行：表结构还原、表数据还原，以及索引和约束的重建。表还原仅支持使用表级备份集。

首先，表结构还原。分为两种情况：一是目标表已存在，则清除表中已有的数据、二级索引和约束；二是若目标表不存在，则还原时会利用备份集中记录的建表语句重建。除此之外，如果备份时备份表中存在附加列（快速加列），那么还原时目标表必须存在，且目标表所有列的实际物理存储格式应与备份表完全一致。

其次，表数据还原，即将备份集中备份表的数据写入目标表的过程。

最后，在数据还原成功的目标表上恢复备份表上的约束和二级索引。

表还原均为联机且完全备份还原，不需要归档日志，所以也不需要恢复操作。

由于表还原允许跨库还原，所以要求还原目标库上的建库参数与备份表所在库上的部分建库参数保持一致。用户可通过 DMRMAN 的 SHOW 功能查看备份信息，需要匹配的建库参数见表 2.1 。

表 2.1 还原库与备份库需匹配建库参数

名称	描述
PAGE_SIZE	数据页大小
BLANK_PAD_MODE	空格填充模式，可选值：0/1
CASE_SENSITIVE	字符大小写是否敏感

CHARSET/UNICODE_FLAG	字符集(0)，可选值：0[GB18030]，1[UTF-8]，2[EUC-KR]
USE_NEW_HASH	是否使用新的 HASH 算法
LENGTH_IN_CHAR	VARCHAR 类型长度是否以字符为单位(N)

■ 归档还原

归档还原就是将备份集中的备份的归档日志还原到指定目录中。还原可以指定归档还原到目标库中（即需要指定一个目标库的 dm.ini），或者还原指定的归档目录中。归档还原仅能使用归档备份集中执行。

还原过程概述：

1. 从用户指定的备份集中，收集归档文件，根据用户指定的过滤条件，过滤需要还原哪些归档；

2. 根据用户指定的 OVERWRITE 参数，确定如果目标归档文件已经存在处理策略。1）如果归档文件存在，添加一条日志继续还原其他归档文件；2）如果归档文件存在，直接终止还原，报错范围；3）如果归档文件存在强制覆盖处理。

3. 执行归档还原，从备份片中把数据去除，必要时经过解压缩和压缩处理后回写到新的归档文件中，还原结束。

2.1.2.2 恢复过程

恢复过程，就是重做 REDO 日志的过程，仅适用于库级和表空间级操作。由于表空间级恢复已经在表空间还原后直接执行，此处将着重说明数据库级恢复过程。执行恢复操作的数据库，可以指定执行过还原操作的数据库，也可以是未执行过还原操作的原始库。对于执行过还原操作的库，备份集的还原操作已经将备份集中的所有数据页重新写入到了目标库对应数据文件中，重做 REDO 日志，可以使数据库处于一致性状态。

执行过还原操作的库，必须再执行恢复操作，否则将无法使用。执行过还原操作库的恢复有以下几种情况：

✓ 如果还原备份集中备份了归档日志，那么只要指定从备份集恢复，将备份集中备份的归档日志读出并生成到一个临时归档文件中，然后重做该临时归档文件即可；

✓ 如果还原备份集中没有备份归档日志，那么只需要重做本地归档，至少要重做备份集 START_LSN 到 END_LSN 之间的一段归档；

✓ 如果还原备份集 START_LSN 和 END_LSN 之间无日志，即 START_LSN =

END_LSN+1，那么只要更新 DB_MAGIC 即可。

✓ RAC 环境恢复不支持使用 UNTIL，且要保证归档使用的完整性。

恢复也支持将数据库恢复到最新状态。不管采用哪种情况，START_LSN 和 END_LSN 之间的归档即备份过程中产生的归档都是一定要重做的，否则恢复失败。

- **指定备份集恢复**

指定备份集恢复，一般要求使用哪个备份集还原就是用哪个备份集恢复。对于增量备份集还原后的恢复工作，由于还原已经确保数据的完整和连续性，所以，只需要使用还原时指定备份集恢复即可。与还原过程相同，使用备份集恢复过程也需要校验备份集的有效性。使用备份集恢复要求备份集一定是备份了归档日志的库级备份集，不能直接使用归档备份集执行恢复操作，但可以将归档备份集中归档日志进行还原，然后作为本地归档日志使用。

使用备份集恢复过程简单概括如下：

1. 先根据元数据文件中记录的日志备份的信息，读取日志备份的备份片，并生成临时归档文件；

2. 重做该临时归档文件，重做成功后，删除临时归档文件；

3. 更新数据库相关信息，具体包括：1）日志信息，设置当前 CKPT_LSN 为最后重做 REDO 日志的 LSN 值，并设置日志文件状态为 INACTIVE；2）更新 DB_MAGIC，恢复操作后，仅更新数据库相关信息。库中 DB_MAGIC 重新生成，标识一个新库，但 PERMNANT_MAGIC 与备份集中相同；3）取消还原标志，允许正常使用。

- **使用归档恢复**

使用本地归档日志恢复相对简单，只需要要重做的本地归档日志的所在目录即可。使用归档恢复过程中，除了指定的归档目录外，也会将目标库配置的第一路本地归档目录作为归档收集目录。使用本地归档日志后，与使用备份集恢复一样，也需要更新数据库相关信息。

从本地归档日志恢复操作，允许重复执行，也允许指定恢复到的范围。每次恢复操作过程中对于数据页的修改都将写到数据文件中，且仅在恢复操作成功结束的时候，才会调整数据库日志信息。若执行中途失败或者强杀，导致已经恢复的日志并未更新数据库日志文件中，那么下次执行仍从上次执行起点（CKPT_LSN+1）开始，若这次执行指定恢复到 LSN 或者 TIME 小于上次执行的 LSN 或者 TIME，那么即使恢复操作成功了，启动数据库之后也有可能出现问题。例如，上次恢复操作数据起点为 LSN=10000，执行失败或者强杀时，执行到 LSN=50000，下次执行指定 UNTIL LSN 为 45000，那么即使恢复成功了，数据库数据也已经被破坏，且无法修复。最好的办法就是，若前面执行失败了，那么最后一次执行不指定

UNTIL TIME 或者 LSN，让所有归档 REDO 日志都得到重做，前提是最后一次恢复操作之前，本地归档日志没有丢失。

2.1.2.3 解密和解压缩

备份集的解密和解压缩操作对应备份过程中加密和压缩操作。若备份时未指定加密或压缩，还原和恢复过程中也不需要执行解密或者解压缩。

对于解密操作，需用户手动指定与备份时一致的加密密码和加密算法。如果备份时未指定加密算法，那么还原或恢复时也可以不用指定。

DM7 还原恢复时的解密过程如下：首先，通过比对用户输入与从元数据文件得到备份集加密信息来检验加密密码和算法是否正确；然后，从备份片文件中读取备份数据之后，写到目标文件（包括目标数据文件和临时归档文件）之前执行解密操作。

增量备份集还原过程中，只要完整的备份集链表中有一个备份集进行过加密，那么还原过程就要指定解密密码和算法；在增量备份集恢复过程中，因仅涉及完整备份集链表中的最后一个，即指定的备份集，所以，如果最后一个备份集没有进行加密的话，就可以不用指定解密密码和算法。

与解密不同，解压缩是自动进行的，不需要用户手动指定什么。DM7 通过读取元数据文件确定当前备份集是否进行过压缩，若进行过压缩，则同解密过程一样，在备份数据从备份文件中读取之后，写到目标文件之前，执行解压缩操作。

若备份集同时存在加密和压缩，那么与备份过程处理相反，会先解密再解压缩，然后将处理的最终数据写入到目标文件中。

2.1.2.4 并行还原

并行还原对应并行备份集的还原，非并行备份集不能执行并行还原。并行还原并行数以并行备份集的并行数作为最大值，实际执行时根据实际创建成功的并行子进程数作为最后的还原并行数执行并行还原。

除了支持并行还原外，并行备份集还支持非并行还原，用户可通过指定关键字 NOT PARALLEL 来实现并行备份集的非并行还原。对于非并行备份集，不论是否指定 PARALLEL 关键字，均采用非并行还原。

2.1.3 介质管理层

DM7 实现了介质管理层，简称 MML，用于处理备份、还原和恢复过程中备份片文件和元数据文件在不同存储介质上的所有读写操作。为了统一管理和方便扩展，DM7 根据自身实现自定义了一套 API 接口 SBT 来负责备份数据在存储介质上的实际读写操作，其实现是以动态库的形式存在的，部署在 DM7 数据库系统的 bin 目录下（与 dmserver 同目录）。整个存储介质上的读写操作，由 MML 调用 SBT 完成。

目前 DM7 备份还原支持磁盘（DISK）和磁带（TAPE）两种存储介质。不同存储介质的接口 SBT 实现不一样，但是对于 MML 调用来说是一样的。对于这两种存储介质类型，DM7 均提供了相应的 SBT 接口实现，使用过程中均以动态方式加载。磁盘介质实现为 dmsbt，相应库文件在 WINDOWS 上命名为 dmsbt.dll，其他非 WINDOWS 上（如 LINUX）命名为 dmsbt.so;磁带介质实现为 dmsbtex,相应库文件名在 WINDOWS 上命名为 dmsbtex.dll，其他非 WINDOWS 上命名为 dmsbtex.so(仅提供了 LINUX 系统实现)。其他第三方磁带存储实现，仅需要将库文件名命名为 dmsbtex.dll（WINDOWS）或者 dmsbtex.so（非 WINDOWS），并替换掉 DM7 提供的相应 dmsbtex 库文件即可。

若未指定存储介质类型，则默认为 DISK。

2.1.4 归档切换与修复

归档切换与归档修复的前提都是已经成功配置了归档。

2.1.4.1 归档切换

归档切换，就是将尚未归档的联机日志文件进行归档的过程。通过以下三个命令中的任意一个命令都可以完成归档切换。

```
alter database archivelog current;
alter system archive log current;
alter system switch logfile;
```

2.1.4.2 归档修复

归档修复，指利用目标库中的联机日志文件对本地归档日志文件进行修复的过程。归档修复不算是真正意义上的备份还原操作，它只是在数据库服务器故障退出后，利用归档恢复

无法达到最新状态时，一种辅助操作。

DM7 实现中对操作过程中产生的 REDO 日志，先写入联机日志文件，再异步写入本地归档日志文件中。若数据库服务器正常退出时，所有写入联机日志文件的 REDO 日志页都写入本地归档日志文件中，可以认为所有 REDO 日志都写入本地归档日志文件中；若数据库服务器是故障退出，此时就可能存在部分 REDO 日志已经写入联机日志文件，但未写入本地归档日志文件中的情况。归档修复就是提供一种手段将这部分未写入本地归档日志文件的 REDO 日志追加到本地归档日志文件中。

每个日志文件的头部记录本日志文件的属性信息，包括：日志文件状态（ACTIVE 或 INACTIVE）、日志文件大小、日志文件有效偏移等。当日志文件正在使用时置为 ACTIVE，正常关闭时置为 INACTIVE。DM7 中 REDO 日志的写入单位为 512 字节的日志页，每个日志页都对应一个全局的序列号 SEQ。在每个检查点生成或者日志文件切换时均会重刷日志文件头，记录当前已经写入日志文件的 LSN 信息，包括所在日志文件中偏移、日志页序号等，此时的 LSN 在联机日志文件中称为 LAST_CKPT_LSN，在归档日志文件中称为 CLSN；而每次写入 REDO 日志时仅会在内存中记录当前已经写入日志文件的 LSN 信息，此时的 LSN 称为 FILE_LSN。FILE_LSN 仅对联机日志有效。FILE_LSN 一定大于或者等于 LAST_CKPT_LSN。数据库服务器正常退出时，所有日志文件处于 INACTIVE 状态，FILE_LSN 等于 LAST_CKPT_LSN；故障退出时，所有正在使用的日志文件状态仍保持 ACTIVE 状态，FILE_LSN 大于或者等于 LAST_CKPT_LSN。

要进行归档修复，必须先确定归档修复的起始点和结束点。首先对目标库中联机日志状态进行判断若为 INACTIVE，则认为正常退出，不执行修复操作，直接返回；否则，分析获取联机日志取出 LAST_CKPT_LSN 作为归档修复的起始 BEGIN_LSN；FILE_LSN 是有效 REDO 日志信息中最大的 LSN，它对应联机日志中最新的日志，因此将它作为归档修复的结束 END_LSN。 从 BEGIN_LSN+1 到 END_LSN 这段日志称为有效的 REDO 日志，这正是需要恢复到归档日志中的部分。

归档修复过程中，分别对 dmarch.ini 中配置的每个本地归档目录进行扫描，收集与目标库相匹配的归档日志文件，根据具体情况进行修复或者跳过修复。

满足以下条件跳过对应本地归档目录中日志文件的修复，否则，均执行修复：

- 未收集到归档日志文件，则认为这个归档目录无效，跳过修复；
- 收集到归档日志文件，且最后一个处于 INACTIVE 状态且 CLSN 与联机日志中得到的 END_LSN 相等，则认为本地归档日志文件与联机日志文件中 REDO 日志一致，跳过修

复；

- 收集到归档日志文件，若 CLSN 大于联机日志中的 END_LSN，则认为本地归档日志非法，跳过修复。

具体修复流程如下：

1. 将收集到的最后一个本地日志文件截断，删除有效偏移之后的无效日志内容，调整日志文件大小等于有效偏移，刷入文件头；

2. 创建新本地归档日志文件，用于存放联机日志中修复的 REDO 日志；

3. 比较最后一个归档日志的 CLSN 和从联机日志文件中读取的 BEGIN_LSN，确定待拷贝的联机日志。

如果 CLSN 大于 BEGIN_LSN，说明归档连续，那么只需拷贝联机日志中从 CLSN+1 到 END_LSN 之间（包括 END_LSN）的 REDO 日志到新的本地归档日志文件中。

否则，说明不连续，此时的归档日志存在已经不再起作用。于是需要将联机日志中所有有效 REDO 日志（从 BEGIN_LSN + 1 到 END_LSN）拷贝到新的归档日志文件中。

最后，将修复归档信息刷入新的本地归档日志文件头，并关闭文件；

4. 至此，整个修复过程结束。

2.2 应用场景分析

DM7 提供了多种类型和级别的备份、还原和恢复方式，管理员应根据应用系统的实际需求选择合适的方式。



说明：无论管理员选用哪一种备份方式，都要注意不应将备份产生的备份集与源备份库存放在同一磁盘或同一存储介质上，以避免存储介质发生硬件故障时，源备份库与备份集同时被毁坏。

2.2.1 备份方式选择

DM7 支持多种类型的备份，不同的备份适合不同的应用场景。

2.2.1.1 数据库备份

数据库备份是最基础的备份方式，所有情况的数据破坏都可以通过数据库备份还原进行修复。但数据库的备份也存在缺点，数据库备份数据量大，执行时间长，对数据库服务本身

的工作影响相对较大。另外，数据级别的还原操作仅能在数据库处于关闭状态下进行。

数据库备份分为联机库备份和脱机库备份。联机库备份是指在数据库正常工作的情况下，执行的备份；而脱机库备份是指数据库停止工作，处于关闭状态，或者因故障退出的情况下，执行的备份。

联机数据库备份可以通过关键字 WITHOUT LOG，选择是否备份 REDO 日志，默认备份。若备份了 REDO 日志，数据库还原恢复，不需要再另外指定本地归档日志，使用比较方便，一般建议备份 REDO 日志；脱机数据库备份强制将联机日志中未刷盘的 REDO 日志进行备份。若数据库故障退出，此时的数据库状态肯定是最新的，所以此时执行的脱机数据库备份肯定也是最新的。

数据库备份支持存在增量备份和完全备份，且数据库的联机备份和脱机备份可以混合使用，比如脱机增量备份可以以联机备份为基备份，联机增量备份也可以以脱机备份为基备份。增量备份相对完全备份，备份数据量较少。但增量备份还原，需要完整的基备份链表，即要递归找到所有的基备份，直至基备份完全找到为止。若基备份链表特别长，又势必会影响还原效率。由于脱机数据库备份会强制备份未刷盘的 REDO 日志，所以，脱机数据备份的恢复操作不需要额外的本地归档 REDO 日志。

综合考虑，联机数据库备份的应用场景为：

一般常规性的数据库维护工作，即在不影响数据库正常运行的情况下，建议定期执行联机数据库备份，且完全备份和增量备份结合使用。执行两次完全备份的时间间隔可以尽量长一点，在两次完全备份之间执行一定数量的增量备份，比如，可以选择每周执行一次完全备份，一周内每天执行一次增量备份。为了尽量减少对数据库正常工作的影响，建议备份时间，选择在工作量较少的时间，比如深夜。

对于脱机数据库备份，要求数据库运行工作已经停止且未发生介质故障。所以在以下两种场景中，可以根据数据库维护周期内已经生成的增量备份的个数，适当的选择脱机数据库增量备份或者完全备份。脱机数据库备份应用场景可能为：

1. 执行数据库中数据迁移，需要暂停数据库服务器运行；

2. 数据库运行过程发生故障退出，暂时不需要通过重启恢复数据库运行，或者目前环境无法满足通过重启恢复数据库运行条件，比如资源不足等。

2.2.1.2 表空间备份

表空间备份的主要目的是为了维护表空间的正常工作，以免表空间故障后，无法使用。

表空间备份仅允许备份用户表空间。表空间备份仅支持联机备份，也存在增量备份和完全备份。表空间还原也仅支持联机操作，且仅允许在原数据库上还原原来的表空间，不能跨数据库或者跨表空间还原。

相对于数据库备份，表空间备份的备份范围仅限于某个表空间，备份数据量相对较小，对数据库正常工作影响也相对较小。

DM7 支持从数据库备份集中还原指定表空间，甚至表空间中的指定数据文件，所以，若无特殊情况，可不必单独执行表空间备份。仅在通过数据库备份无法满足数据库维护需求，可适当选择表空间备份。比如：

1．无数据库正常工作维护操作，无数据库备份；

2． 存在数据库正常工作维护操作，但鉴于数据库维护周期较长，无法满足表空间的维护需求，如两次数据库备份维护期间，表空间会频繁操作，若表空间故障，使用已经存在的数据库备份执行还原，会重做较多的 REDO 日志；

3． 表空间为新创建表空间，在已经存在的数据库备份中未备份，但在下一次数据库备份执行之前，会被频繁操作，为避免在此期间表空间出现故障。

2.2.1.3 表备份

表备份的备份目标是某个具体的表，相对于数据库和表空间备份，其备份数据量更少，对于数据库正常工作的影响更小。与表空间还原一样，表的备份还原都需要联机进行；但是与表空间还原不同的是，表还原可以跨表、跨表空间、跨数据库还原，可以是系统表空间上的表，相对于表空间还原要灵活很多。

表备份无增量备份或者完全备份之说，每次备份都会备份目标表中最新的数据、二级索引和约束信息。表空间还原总是将表空间中数据状态恢复到最新，若是用户通过客户端导致的表数据、索引或者约束的误操作，无法通过表空间还原修复；通过数据库还原也很难准确的恢复到误操作之前的状态。

表备份适用场景如下：

1．需要对目标表的某个状态进行保存；

2． 需要将目标表上的数据、二级索引或者约束完整的赋值到另外一张表上，这张表可以与原目标表在或者不在同一个表空间、在或者不在同一个数据库中。

2.2.1.4 归档备份

归档备份与数据库本身无关，仅是将已经归档的本地归档 REDO 日志文件进行备份，方便归档日志的维护工作。归档备份支持联机和脱机两种方式，无增量备份和完全备份之说。归档备份允许指定是否删除已经备份的归档日志文件，在生成归档备份的同时可以通过删除归档日志文件，腾出磁盘空间，避免因空间不足导致刷入归档日志失败。

归档备份适用于本地归档日志文件需要维护的情况，比如：

1．避免因存放本地归档日志文件的磁盘出现故障，导致归档日志文件无法使用；

2． 避免因本地归档日志文件过多，可能导致磁盘溢出的情况，通过执行归档备份将本地归档保存在到其他地方，并删除已经备份的归档日志文件，来腾出更多的磁盘空间；

3． 数据库备份未备份 REDO 日志，或者存在表空间备份需要还原，需要备份相应的本地归档日志，来完成数据库还原和表空间还原后的恢复操作。

2.2.1.5 并行备份

DM7 并行备份实现通过创建多路备份还原子进程来实现并发，以待备份的数据文件或者本地归档日志文件为单位，每个子进程执行完一个文件备份后，继续取下一个文件继续备份。直到所有待备份文件备份结束。对主库配置有一定要求。

DM7 的数据库备份、表空间备份、归档备份都只支持并行备份。并行备份的适用场景可概括为：主库配置性能较好，待备份数据文件较多的情况下，建议选择并行备份来提升备份性能。

2.2.2 还原方式选择

与备份相对应，DM7 也支持相应库级、表空间级和表级、本地归档日志的还原，以及并行备份的并行还原。以下将对各种还原方式的应用场景进行分析。

2.2.2.1 数据库还原

数据库还原只能使用数据库备份集，数据库还原操作只能在脱机情况下进行。所有类型的数据库故障，都可以使用数据库还原进行修复。对于某个表空间故障或者某张表意外删除的情况，若存在相应的表空间备份或者表备份，则建议优先使用相应备份进行还原修复。

数据库还原的使用场景如下：

1．数据库故障退出，无法通过故障重启操作恢复数据库正常运行；

2． 将数据库移植到某个移植到某个已知配置的数据库中，也就是说目标还原库的配置文件 dm.ini 存在且有效，可以使用指定 dm.ini 还原；

3． 将数据库移植到某个已知目录中，这个目录可以已经存在的数据库目录，也可以是其他的文件目录或者空目录，可以使用指定目录还原，这个目录将作为数据库系统目录处理。若这个目录是已经存在的数据库目录且相应的 dm.ini 文件存在，则建议使用 2 中提到的指定 dm.ini 还原。

2.2.2.2 表空间还原

表空间还原，可以使用数据库备份，也可以使用表空间备份。目前 DM7 支持联机和脱机两种方式执行表空间还原。联机还原不支持 SYSTEM 表空间和 ROLL 表空间。

表空间还原使用场景如下：

1． 表空间中数据文件的介质故障，如文件丢失，可使用表空间还原修复。如果是已知某个或某几个数据文件破坏且未执行过表空间还原，则建议仅还原丢失的数据文件；

2． 表空间处于故障状态(RES_OFFLINE)，可以通过指定表空间还原修复。

3． 确保备份集生成之后，未执行过 DDL 操作的前提下，通过指定 UNTIL TIME 或者 UNTIL LSN 可以将目标表空间恢复到指定状态。比如，备份前创建了表 T1 插入 100 条数据，备份后又两次插入 100 条数据。此时发现，最后一次插入的数据有问题，则可以通过使用 UNTIL TIME 指定倒数第二次插入操作时间，将 T1 表内容恢复到倒数第二次插入的状态。

2.2.2.3 表还原

目前，DM7 表还原仅支持从表备份集中还原，且仅支持联机方式。表还原仅能将目标表还原到备份时的状态，可选择还原后是否重建二级索引和约束。

表还原使用场景如下：

1．同一个数据库中，某个表因误操作，表中数据或者表本身被删除；

2． 希望将表中数据完整移植到另外一张表中，这张表可以与备份表同库，也可以是不同库，甚至是不同库中一张空表。

2.2.2.4 归档还原

归档还原相对简单，就是将归档备份集中备份的归档日志文件还原到本地。当且仅当本地的归档日志文件被破坏或者缺失的情况下使用。可以选择还原到某个库的本地归档目录中，也可以还原到其他不相关的文件目录中。

2.2.2.5 并行还原

仅并行备份集可以使用并行还原。

同并行备份一样，并行还原也是通过创建多个备份还原子进程分别执行各子备份集的还原，来实现并行的，对主库性能要求比较高。若主库性能比较好，又是还原的并行备份集，建议采用并行还原；否则，建议使用非并行还原，避免因资源不足，导致还原失败。

2.2.3 恢复方式选择

由于表空间还原后的恢复操作，在表空间还原后直接执行，此处仅说明数据库的恢复操作。DM7 要求执行过还原的数据库不能直接使用，必须经过恢复操作才可以；而对于未执行过还原或者已经执行过恢复操作，也可以执行恢复操作，使数据状态恢复到更新的状态。

数据库恢复分为从备份集恢复、从本地归档日志恢复和直接更新 DB_MAGIC 三种形式。从备份集恢复和从本地归档日志恢复可以指定是否采用重做归档优化。以下将从三种恢复方式和重做归档优化四个方面进行说明。

2.2.3.1 从备份集恢复

从备份集恢复，是指重做备份集中备份的 REDO 日志，只能恢复到备份时的状态，要求从备份集恢复时使用的备份集与库还原时使用备份集为同一个备份集。从备份集恢复，要求备份必须为库备份集且备份时未指定 WITHOUT LOG，或者未脱机库备份集（脱机库备份强制备份 REDO 日志）。

从备份集恢复的应用场景如下：

1．单机环境且执行过还原的数据库，仅希望将库恢复到备份时的状态；

2．使用脱机库备份执行的单机库还原，建议使用从备份集恢复；

3．使用 DDL CLONE 库备份集执行过还原的数据库，只能使用从备份集恢复。

2.2.3.2 从本地归档日志恢复

从本地归档日志恢复，是指重做本地归档 REDO 日志文件，这些日志文件可以是通过归档还原的归档日志，也可以本地已经存在的 REDO 日志为文件。除 DDL CLONE 库备份集还原的数据库外，其他库的恢复操作都可以使用从本地归档日志恢复。

从本地归档日志文件恢复应用场景：

1. 恢复数据库到最新状态；

2. 恢复到指定 LSN 或者指定时间点的恢复操作，若执行过还原的数据库恢复，则指定 LSN 或者 TIME 对应的 LSN 值必须大于备份集的 END_LSN；

3. RAC 环境数据库恢复操作。

使用本地归档日志恢复，要求本地归档日志为连续的归档日志。可通过归档检查工具 dmrachk 进行校验。



关于 dmrachk 的使用，请使用帮助命令 dmrachk help，查看使用说明

说明：

2.2.3.3 更新 DB_MAGIC

更新 DB_MAGIC 恢复方式，是最简单的恢复方式，它不重做 REDO 日志，仅仅是更新库的 DB_MAGIC 值和库状态。

更新 DB_MAGIC 仅适用于执行过还原的数据库，且还原时使用备份生成过程中未产生日志的情况，即还原时使用备份集的 START_LSN 等于 END_LSN+1。可通过 DMRMAN 工具的 SHOW 操作来查看备份集的 START_LSN 和 END_LSN 值情况。一般正常退出的数据库的脱机备份集会满足这种情况。

2.2.3.4 重做归档优化和不优化

重做归档日志过程中，可以选择是否采用重做归档优化。从备份集恢复和从本地归档日志恢复均支持通过关键字 TYPE 设置是否采用重做归档日志优化，0 不采用，1 采用，默认不采用。

不优化方式重做归档日志实现机制是按已经收集到的归档日志文件顺序，逐个日志 REDO。优点是每个日志文件仅会读取一次，缺点是，若日志涉及修改数据页范围较广，就会导致内存中数据页重复加载释放。

优化方式重做归档日志实现是为了去除不优化方式中内存中数据页会重复加载释放的问题。实现中，首先按归档日志顺序收集日志中涉及到的修改数据页并保存到内存中；然后按已经收集到的数据页分批重做 REDO 日志。再执行每一批数据页相关 REDO 日志重做时，都会将所有的本地归档日志文件重新扫描一遍，确保本批次数据页的所有修改日志都被重做到。这种优化方式，事实仅避免了内存中数据页的重复加载释放，但是对于本地归档日志文件较多的情况下，就会出现对归档日志文件重复读取的问题，也会导致性能下降。

由以上分析可以看出，日志文件数不多且日志量较大时，宜采用优化方式重做归档，其他采用默认非优化方式重做归档即可。比如从备份集恢复，一般仅生成一个临时本地归档日志文件且日志量不是很大，可选择非优化方式重做归档。

2.2.4 归档修复

归档修复，是指将数据库故障退出时刷入联机日志文件中而未刷入本地归档日志文件中的 REDO 日志重新刷入本地归档日志。解决因本地归档日志不足，导致的使用本地归档日志恢复，无法将数据库恢复到故障时最新状态的问题。

归档恢复应用场景应该是只要数据库因故障退出，就应该先使用 DMRMAN 工具先执行归档修复，避免因故障数据库执行其他操作破坏联机日志的情况，如原库执行还原。

3 DM7 备份与还原实战

本章详细介绍如何利用 DM7 提供的各种工具进行备份还原与恢复的操作，包括 DIsql 工具、DMRMAN 工具、图形化客户端管理工具 MANAGER 和 CONSOLE。DIsql 工具用于执行联机的数据备份与数据还原，包括数据库备份、表空间备份与还原和表备份与还原；DMRMAN 工具用于脱机的数据备份还原与恢复，只能备份还原和恢复数据库； 客户端工具 MANAGER 和 CONSOLE 对应命令行工具 DIsql 和 DMRMAN 的功能，分别用于联机和脱机备份还原数据。这四种工具都可以独立使用，也可以相互配合，如使用 DIsql 或 MANAGER 工具联机备份的数据库备份文件可以用 DMRMAN 或 CONSOLE 工具还原。读者在阅读完本章的内容后，就可以动手配置归档，并进行备份与还原操作了。

本章内容主要包括：

- 准备工作
- 使用联机执行 SQL 语句进行备份还原
- 使用脱机工具 DMRMAN 进行备份还原
- 使用图形化客户端工具进行备份还原



警告： **OPEN 状态支持备份、还原和恢复的操作；**

MOUNT 状态支持归档备份、表空间级还原；

SUPEND 状态均不支持。

3.1 准备工作

本节介绍着手备份与还原之前的准备工作，主要内容包括：

- 支持与限制
- 归档配置

3.1.1 支持与限制

在着手备份与还原之前，先详细了解一下 DM7 对备份与还原进行了哪些支持与限制？如下进行详细说明。

- **联机备份**

对联机备份的支持与限制：

1）MPP 环境仅允许库和归档备份，且各站点会都会执行，生成相应的备份集，支持 PWR 和 DDL CLONE；

2）RAC 环境仅允许库备份，不支持 PWR；

3）MOUNT 状态仅支持归档备份；

4）SUSPEND 状态所有备份均不支持；

5）OPEN 状态支持所有备份，支持 PWR 和 DDL CLONE；

6）PRIMARY 模式支持所有备份，支持 PWR 和 DDL CLONE；

7）STANDBY 模式仅支持库级、表空间级和归档备份，支持 PWR 和 DDL CLONE；

8）DDL CLONE 必须备份归档，不允许指定 WITHOUT LOG。

- **联机还原：**

仅支持表空间和表级还原，对联机还原的支持与限制：

1）MPP/RAC 不支持；

2）PRIMARY 支持，STANDBY 只支持表空间级还原；

3）MOUNT 支持表空间级还原，SUSPEND 均不支持；

4）OPEN/NORMAL 支持。

- **脱机备份**

脱机备份跟目标库所处的模式和状态无关，仅支持库级和归档备份。

1）MPP 视同单机环境，仅当前站点执行备份操作，允许异常退出后备份，支持 DDL_CLONE，不支持 PWR；

2）RAC 仅支持库备份，不支持 PWR。

- **脱机还原：**

脱机还原跟目标库所处的模式、状态以及集群环境（MPP 和 RAC）无关，仅允许库级和归档还原。

3.1.2 归档配置

3.1.1.1 概述

DM7 服务器可以运行在归档模式或非归档模式下。如果是归档模式，联机日志文件中的

内容保存到硬盘中，形成归档日志文件；如果是非归档模式，则不会形成归档日志。

采用归档模式会对系统的性能产生影响，然而系统在归档模式下运行会更安全，当出现故障时其丢失数据的可能性更小，这是因为一旦出现介质故障，如磁盘损坏时，利用归档日志，系统可被恢复至故障发生的前一刻，也可以还原到指定的时间点，而如果没有归档日志文件，则只能利用备份进行恢复。

通过dm.ini和dmarch.ini可以配置本地归档。dmarch.ini生效的前提是dm.ini中的参数ARCH_INI置为1。

DM7的dmarch.ini可以进行本地归档和远程归档的设置，其中远程归档在数据守护和MPP中使用，不是本文的讨论内容，DM备份与还原过程中使用的日志均为本地归档日志。

dmarch.ini中与备份还原相关的配置参数及其介绍见下表。

表3. 1 dmarch.ini相关配置项

配置项	配置含义
[ARCH_NAME]	REDO日志归档名
ARCH_TYPE	REDO日志归档类型，LOCAL表示本地归档
ARCH_DEST	归档文件存放路径
ARCH_FILE_SIZE	单个REDO日志归档文件大小，取值范围（64M~2048M），缺省128M
ARCH_SPACE_LIMIT	REDO日志归档空间限制，当所有本地归档文件达到限制值时，系统自动删除最老的归档文件。0表示无空间限制，取值范围（1024M~4294967294M），缺省为0

3.1.2.2 何时配置归档

联机备份数据库、备份还原表空间时必须要配置归档。联机备份时，大量的事务处于活动状态，为确保备份数据的一致性，需要同时备份一段日志（备份期间产生的REDO日志），因此要求数据库必须配置本地归档且归档必须处于开启状态。

脱机备份还原数据库可配置归档也可以不配置。因为脱机备份时数据库处于关闭状态不需要备份REDO日志，因此这种情况下数据库可以选择不配置归档。但是如果要指定从归档恢复，则这种情况下数据库必须配置本地归档目录。

表备份为联机完全备份，不需要备份归档日志，所以不需要配置归档。

3.1.2.3 配置归档

归档配置有两种方式：一是联机归档配置，数据库服务器启动情况下，使用 SQL 语句完成 dmarch.ini 和 ARCH_INI 配置；二是手动配置归档，数据库服务器未启动的情况下，手动编写 dmarch.ini 文件和设置参数 ARCH_INI。下面将分别说明这两种归档如何配置。

联机配置归档

使用 SQL 语句配置本地归档。

语法如下：

```
ALTER DATABASE <ADD|MODIFY|DELETE> ARCHIVELOG <归档配置语句>;
<归档配置语句>::= 'DEST = <归档目标>,TYPE = <归档类型>'
<归档类型>::=LOCAL [,FILE_SIZE = <文件大小>][,SPACE_LIMIT = <空间大小限制>]
```

使用 SQL 语句开启和关闭归档模式。

语法如下：

```
ALTER DATABASE ARCHIVELOG | NOARCHIVELOG;
```


警告：**在归档模式下，不允许删除本地归档。**

例如，联机归档配置如下：

1)修改数据库为 MOUNT 状态。

```
SQL>ALTER DATABASE MOUNT;
```

2)配置本地归档。

```
SQL>ALTER DATABASE ADD ARCHIVELOG 'DEST = /home/dm_arch/arch, TYPE = local,
FILE_SIZE = 1024, SPACE_LIMIT = 2048';
```

3)开启归档模式。

```
SQL> ALTER DATABASE ARCHIVELOG;
```

4)修改数据库为 OPEN 状态。

```
SQL>ALTER DATABASE OPEN;
```

手动配置归档

1)手动编辑 dmarch.ini 文件，之后保存在 dm.ini 所在的目录。dmarch.ini 文件内容如下：

```
[ARCHIVE_LOCAL1]

ARCH_TYPE = LOCAL

ARCH_DEST = d:\dm_arch\arch

ARCH_FILE_SIZE = 1024

ARCH_SPACE_LIMIT = 2048
```

2)编辑 dm.ini 文件，设置参数 ARCH_INI=1，保存。

3)启动服务器，服务器已运行于归档模式。



说明： **多路归档，指配置多个本地归档。配置的第一个归档，称为第一路归档，后面依次是第二路、第三路.....**

3.2 使用联机执行 SQL 语句进行备份还原

本节介绍使用 DIsql 工具如何实现数据文件的备份、管理及还原。

本节内容主要包括：

- 概述
- 数据备份
- 数据备份高级主题
- 管理备份
- 数据还原
- 数据还原高级主题

3.2.1 概述

DM 支持通过联机执行 SQL 语句方式对数据库执行备份还原操作。联机方式支持数据库、用户表空间、用户表的备份和用户表空间、用户表的还原，在进行联机库级备份、表空间备份和表空间还原时，必须保证系统处于归档模式，否则联机备份还原不能进行。

联机备份之前，请设置 INI 参数 BAK_POLICY，选用合适的备份策略。BAK_POLICY 取值 0、1 或 2，缺省为 0。为 0 表示同时支持 bak1 和 bak2 版本；为 1 时只能使用 bak1 版本；为 2 表示只支持 bak2 版本。bak1 为备份还原的老版本，bak2 为备份还原的新版本。bak1 和 bak2 使用上略有不同。如 bak1 版本不支持备份集 BAKUPSET 的使用。



注意： **在使用的 SQL 语句中，若指定的备份名、加密密码、加密算法名长度超过 128 个字节，会导致语法分析错误。**

3.2.2 数据备份

本节主要介绍使用 DIsql 工具如何备份数据库、用户表空间及用户表。

3.2.2.1 数据库备份

本节描述了如何使用 DIsql 完成最基本的数据库备份及实施一些备份策略，如限制备份集大小、加密备份等。主要内容包括：

- 概述
- 设置备份选项
- 备份数据库

3.2.2.1.1 概述

在 DIsql 工具中使用 BACKUP 语句你可以备份整个数据库。通常情况下，在服务器配置归档后输入以下语句即可备份数据库：

```
SQL> BACKUP DATABASE BACKUPSET 'db_bak_01';
```

语句执行完后会在默认的备份路径下生成名为“db_bak_01”的备份集目录，默认的备份路径为 dm.ini 同级目录的 bak 目录。这是最简单的数据库备份语句，如果要设置其他的备份选项需了解联机备份数据库的语法。

语法如下：

```
BACKUP DATABASE [[[FULL] [DDL_CLONE]] | INCREMENT [WITH BACKUPDIR '<基备份搜索目录>'{,'<基备份搜索目录>'} [BASE ON <BACKUPSET '<基备份目录>'] [USE PWR]][TO <备份名>]BACKUPSET ['<备份集路径>']
[DEVICE TYPE <介质类型> [PARMS '<介质参数>']]
[BACKUPINFO '<备份描述>'] [MAXPIECESIZE <备份片限制大小>]
[IDENTIFIED BY <密码>[WITH ENCRYPTION<TYPE>][ENCRYPT WITH <加密算法>]]
[COMPRESSED [LEVEL <压缩级别>]] [WITHOUT LOG]
```

```
[TRACE FILE '< TRACE 文件名>'] [TRACE LEVEL < TRACE 日志级别>]
[PARALLEL [<并行数>]];
```

FULL：备份类型。FULL 表示完全备份，可不指定，服务器会默认为完全备份。

DDL_CLONE：数据库克隆。该参数只能用于完全备份中，表示仅拷贝所有的元数据不拷贝数据。如对于数据库中的表来说，只备份表的定义不备份表中数据。表空间和表备份不支持该参数。RAC 环境不支持 DDL_CLONE。


说明：

DDL_CLONE 会克隆 HUGE 表的元数据信息，之后库还原时会还原 HUGE 表的表定义。

INCREMENT：备份类型。INCREMENT 表示增量备份，若要执行增量备份必须指定该参数。

WITH BACKUPDIR：用于增量备份中，指定基备份的搜索目录，最大长度为 256 个字节。若不指定，服务器自动在默认备份目录下搜索基备份。如果基备份不在默认的备份目录下，增量备份必须指定该参数。

BASE ON：用于增量备份中，指定基备份集目录。

USE PWR：用于增量备份中，指定备份过程中使用 PWR 优化，默认不使用。当数据库很大而需要备份的数据页又很少时可指定该参数来提高备份效率，否则可任意选择是否使用该参数。关于 PWR 的介绍见 2.1.1.4 PWR 优化 。

TO：指定生成备份名称。若未指定，系统随机生成，默认备份名格式为：DB_备份类型_数据库名_备份时间。其中，备份时间为开始备份时的系统时间。

BACKUPSET：指定当前备份集生成路径。若指定为相对路径，则在默认备份路径中生成备份集。若不指定，则在默认备份路径中按约定规则，生成默认备份集目录。库级备份默认备份集目录名生成规则：DB_库名_时间，如 DB_DAMENG_20160518_143057_123456。表明该备份集为 2016 年 5 月 18 日 14 时 30 分 57 秒 123456 毫秒时生成的库名为 DAMENG 的数据库备份集。若库名超长，使上述完整名称长度大于 128 个字节，则去掉库名字段，调整为 DB_时间。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

PARMS：只对介质类型为 TAPE 时有效。

BACKUPINFO：备份的描述信息。最大不超过 256 个字节。

MAXPIECESIZE：最大备份片文件大小上限，以 M 为单位，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

IDENTIFIED BY：指定备份时的加密密码。密码应用双引号括起来，这样避免一些特殊字符通不过语法检测。

WITH ENCRYPTION：指定加密类型，0 表示不加密，不对备份文件进行加密处理；1 表示简单加密，对备份文件设置口令，但文件内容仍以明文存；2 表示完全数据加密，对备份文件进行完全的加密，备份文件以密文方式存储。

ENCRYPT WITH：加密算法。缺省情况下，算法为 AES256_CFB。


说明：

加密算法包括：

DES_ECB、DES_CBC、DES_CFB、DES_OFB、DESEDE_ECB、DESEDE_CBC、DESEDE_CFB、DESEDE_OFB、AES128_ECB、AES128_CBC、AES128_CFB 、AES128_OFB、AES192_ECB、AES192_CBC、AES192_CFB 、AES192_OFB、AES256_ECB、AES256_CBC、AES256_CFB 、AES256_OFB 、RC4

COMPRESSED：取值范围 0~9。0 表示不压缩，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。若未指定，但指定 COMPRESSED，则默认 1；否则，默认 0。

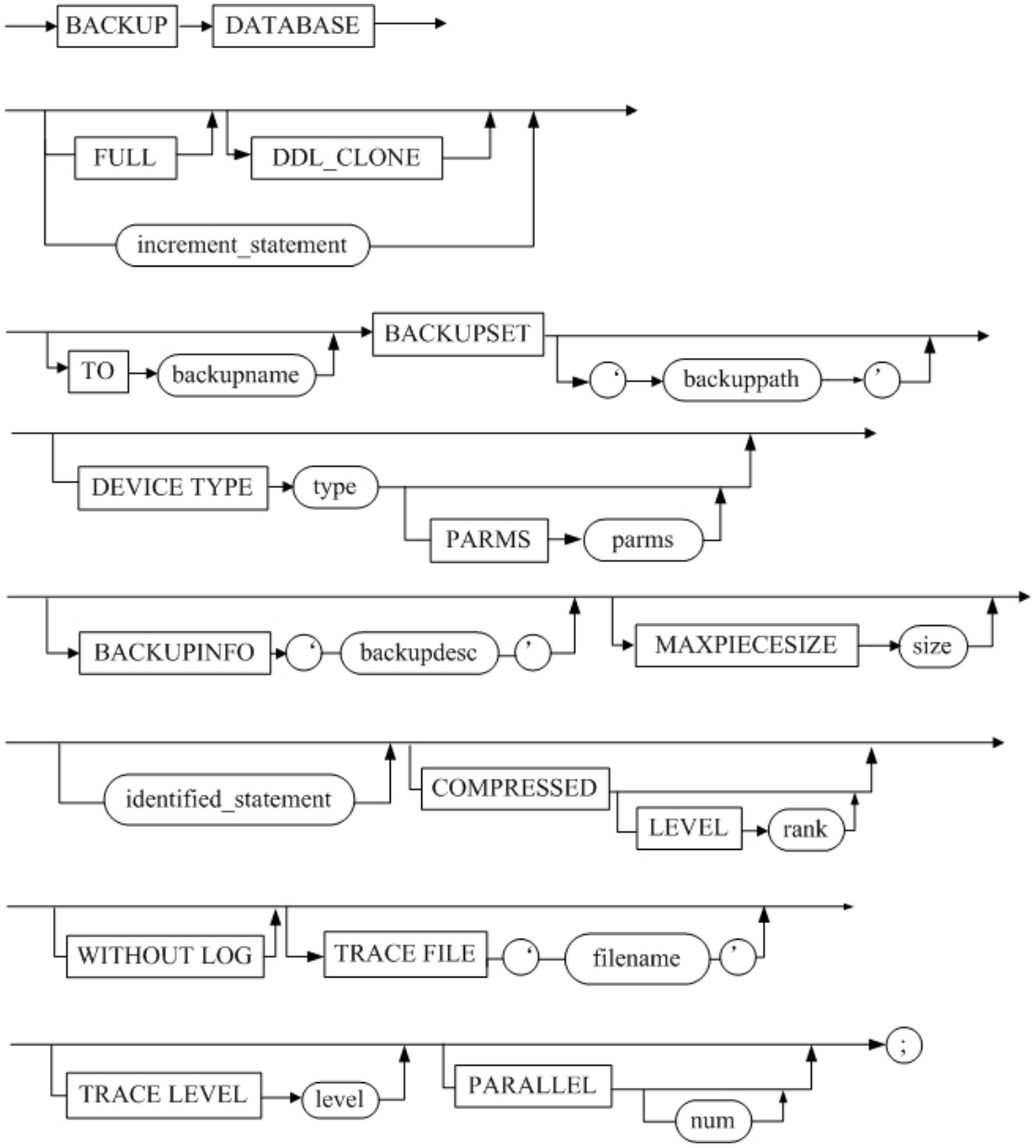
WITHOUT LOG：联机数据库备份是否备份日志。如果使用，则表示不备份，否则表示备份。如果使用了 WITHOUT LOG 参数，则使用 DMRMAN 工具还原时，必须指定 WITH ARCHIVEDIR 参数。

TRACE FILE： 指定生成的 TRACE 文件。启用 TRACE，但不指定 TRACE FILE 时，默认在 DM 数据库系统的 log 目录下生成 DM_SBTTRACE_年月.log 文件；若使用相对路径，则生成在执行码同级目录下。

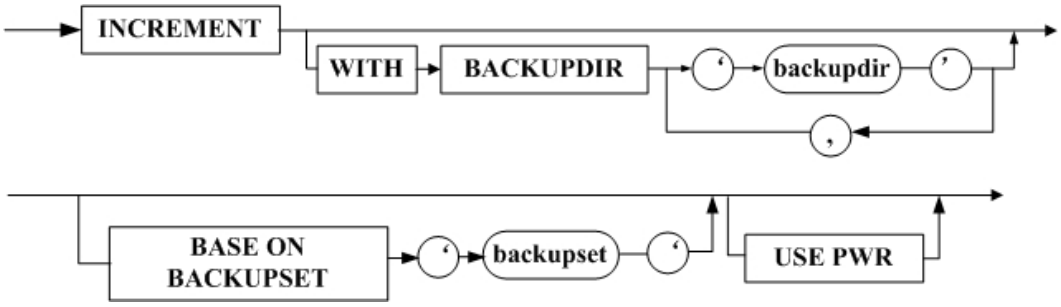
TRACE LEVEL：有效值 1、2，默认为 1 表示不启用 TRACE，此时若指定了 TRACE FILE，会生成 TRACE 文件，但不写入 TRACE 信息；为 2 启用 TRACE 并写入 TRACE 相关内容。

PARALLEL：指定并行备份的并行数，若不指定，则默认为 4，指定 0 或者 1 均认为为非并行备份。增量备份中强制并行数与基备份一致。若未指定关键 PARALLEL，则认为非并行备份。并行备份不支持 PWR 和存在介质为 TAPE 的备份。

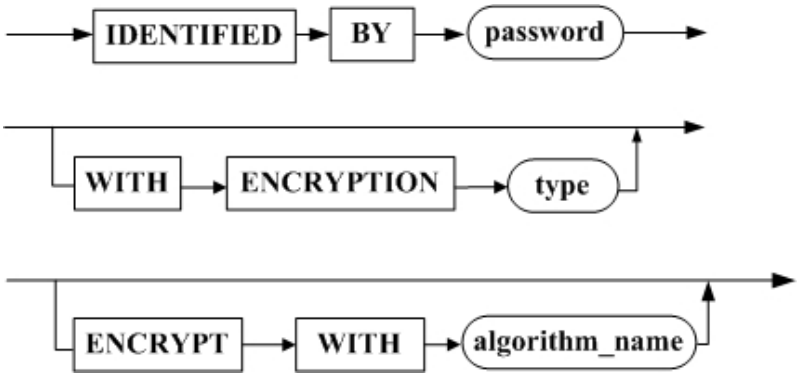
图例：



increment_statement：



identified_statement：



使用说明：

1. 备份成功后会在<备份集路径>或者备份默认目录下生成备份集。备份集中包括一个备份元数据文件，后缀.meta，一个或多个备份片文件，后缀.bak。

2. 当备份数据超过限制大小时，会生成新的备份文件，新的备份文件名是初始文件名后加文件编号。

3. 系统处于归档模式下时，才允许进行数据库联机备份。

4. MOUNT 模式下不允许进行数据库备份。

5. DDL_CLONE 库备份集不能作为增量备份的基备份，仅能用于库级还原。

3.2.2.1.2 设置备份选项

DIsql 中备份语句如果仅指定了必选参数如“BACKUP DATABASE BACKUPSET 'db_bak_01';”，那么服务器会默认地指定备份介质类型、备份路径、备份片大小及备份联机日志等。用户备份时也可以指定这些参数来修改默认值，本节将介绍以下几种常用的备份选项：

- 设置联机数据库备份路径
- 设置备份名
- 增量备份指定基备份目录
- 指定介质类型
- 添加备份描述
- 限制备份片大小
- 加密备份
- 备份压缩

了解更多备份选项请参考 3.2.3 小节，数据库备份高级主题。

设置联机数据库备份路径

备份语句中的 BACKUPSET 参数用于指定备份集的输出路径，例 3.1 备份数据库到指定的路径“/home/dm_bak/db_bak_3_01”。

例 3.1 指定备份集路径为'/home/dm_bak/db_bak_3_01'

```
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_3_01';
```

BACKUPSET 为必选参数。如果该参数仅指定备份集名称即指定为相对路径，备份集会生成到默认的备份路径下。如果仅指定该关键字，不指定目标备份集目录，那么将会在默认备份路径下自动生成。

设置备份名

系统为每个备份指定一个备份名，它可作为识别备份的一种方式。备份时用户可以采用系统生成的备份名也可以指定属于自己的备份名。

备份名可以有效地表示备份的目的或者不同类型备份的用途。比如对于一个用于还原的数据库完全备份可设置备份名为“db_full_for_restore”。

如果不显式地指定备份名，系统会默认为备份创建一个备份名。备份名的格式为：DB_备份类型_数据库名_备份时间，DB 表示备份为数据库备份，备份类型表示该备份为完全备份还是增量备份，数据库名为当前连接数据库的名称，备份时间为开始执行备份的系统时间。

若一次备份产生多个备份片，那么每个备份片的备份名是相同的。不同目录下的备份名可相同，同目录下的备份名不能相同。

备份名最大长度为 128 个字节，如果超长服务器会报语法分析出错。备份名的设置不可以使用特殊的格式，如%NAME。

例 3.2 创建备份集，备份名设置为“WEEKLY_FULL_BAK”。

```
SQL>BACKUP DATABASE TO WEEKLY_FULL_BAK BACKUPSET '/home/dm_bak/db_bak_3_02';
```

增量备份指定基备份目录

请参考 3.2.2.2.2 小节。

指定介质类型

DEVICE TYPE 参数用于指定存储备份集到磁盘还是磁带。若需要备份数据到磁带可使用 BACKUP...DEVICE TYPE TAPE 语句。

例 3.3 指定备份数据库到磁带。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_3_03' DEVICE TYPE TAPE;
```

若备份时不指定介质类型参数服务器会默认存储备份集到磁盘。

添加备份描述

联机备份可选择对执行的备份添加描述信息，相比备份名参数，描述信息可以更详细地对备份类型、用途、场景等进行说明。描述信息最大长度支持 256 个字节。

例 3.4 创建备份为备份集添加描述信息为“完全备份”。

```
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_3_04' BACKUPINFO '完全备份
';
```

用户如果不指定该参数，备份集对应的描述信息属性则为空。

限制备份片大小

MAXPIECESIZE 参数用于控制单个备份片的大小，当介质管理器对单个文件的大小有限制或者需要备份的数据文件很大时就可以使用这个参数。

MAXPIECESIZE 指定了单个备份片文件大小的上限，单位为 M，最小为 128M，32 位系统最大可设置为 2G，64 位系统最大可设置为 128G。如果不设置 32 系统默认为 2G，64 位系统默认为 4G。例如，要限制备份片大小不超过 300M，可指定 MAXPIECESIZE 300，备份时服务器会限定所有的备份片大小不超过 300M。

限制备份片大小主要用于解决文件系统或介质管理器对文件最大值的限制小于备份片默认值的情况，当备份片较大时会导致无法存储。

例 3.5 创建备份限制备份片大小为 300M。

```
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_3_05' MAXPIECESIZE 300;
```

备份压缩

所有的备份语句都可以设置压缩选项，指定 COMPRESSED 参数后备份集会先被压缩然后再写到磁盘或磁带。压缩选项有不同的压缩级别可以选择，取值范围为 0~9。0 表示不压缩，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。若仅指定 COMPRESSED，压缩级别为 1。 实际应用中用户可根据存储空间、数据文件大小、备份效率等确定合适的压缩级别。

例 3.6 执行备份压缩,压缩级别设置为 5。

```
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_3_06'COMPRESSED LEVEL 5;
```

并行备份

DM 支持对库级和表空间级的并行备份，待备份数据文件很大时使用并行备份可以显著提高备份效率。用户可通过关键字 PARALLEL 指定是否执行并行备份，以及执行并行备份的并行数。执行备份时实际使用的并行数由用户指定的并行数和实际可使用的最大并行数决定，而实际可使用的最大并行数由数据文件的个数决定。关于并行备份的更多内容请参考

2.1.1.5 小节。

例 3.7 创建并行备份，指定并行数为 8。

```
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_3_07'PARALLEL 8;
```

3.2.2.1.3 备份数据库

完全备份和增量备份是数据库备份的最常用策略，本节将详细介绍这两种类型备份。

完全备份

执行数据库备份，数据库必须处于 OPEN 状态，MOUNT 和 SUSPEND 状态下不允许执行数据库备份。

数据库完全备份中包含了指定库的全部有效数据页，为了保证数据安全应该对数据库定期执行完全备份。多久执行一次完全备份才合适，一般规则是数据库经过频繁修改后最好执行一次完全备份。

完全备份数据库步骤如下：

1) 配置归档，参考 3.1 节。

2) 保证数据库处于 OPEN 状态。

3) DIsql 中输入备份数据库语句，最简单的不设置其他参数的完全备份语句如下：

```
SQL> BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_full_bak_01';
```

备份语句中的参数 FULL 参数可以省略，不指定备份类型服务器会默认指定备份类型为完全备份。

增量备份

增量备份指基于指定的库（或者表空间）的某个备份（完全备份或者增量备份），备份自该备份以来所有发生修改了的数据页。执行增量备份的主要目的是快速备份数据库中的修改，减少备份时间和避免重复的备份。

如何制定备份策略要根据可接受的最小恢复时间。例如，每周执行一次完全备份，每天执行一次增量备份，那么恢复时要重做的归档就不会超过一天。

增量备份数据库步骤如下：

1) 配置归档，参考 3.1 节。

2) 保证数据库处于 OPEN 状态。

3) DIsql 中输入备份数据库语句，最简单的不设置其他参数的增量备份语句如下：

```
SQL> BACKUP DATABASE INCREMENT WITH BACKUPDIR '/home/dm_bak' BACKUPSET
```

```
'/home/dm_bak/db_increment_bak_02';
```

备份语句中的 INCREMENT 参数不可省略，该参数用来指定执行的备份类型为增量备份。WITH BACKUPDIR 参数用来指定基备份集的搜索目录，如果基备份集不在默认目录该参数不可省略。

3.2.2.2 表空间备份

本节描述了如何使用 DIsql 完成最基本的表空间备份及实施一些备份策略，如限制备份集大小、加密备份等。主要内容包括：

- 概述
- 设置备份选项
- 备份表空间

3.2.2.2.1 概述

在 DIsql 工具中使用 BACKUP 语句也可以备份单个表空间。同备份数据库一样，执行表空间备份服务器也必须运行在归档模式下，启动 DIsql 输入以下语句即可备份表空间：

```
SQL> BACKUP TABLESPACE MAIN BACKUPSET 'ts_bak_01';
```

备份集“ts_bak_01”会生成到默认的备份路径下。如要设置其他备份选项需参考下文的联机备份表空间语法。

语法如下：

```
BACKUP TABLESPACE <表空间名> [FULL | INCREMENT [WITH BACKUPDIR '<基备份搜索目录>'{,'<基备份搜索目录>'}][BASE ON  BACKUPSET '<基备份集目录>']
[USE PWR]][TO <备份名>] BACKUPSET ['<备份集路径>']
[DEVICE TYPE <介质类型> [PARMS '<介质参数>']]
[BACKUPINFO '<备份集描述>'] [MAXPIECESIZE <备份片限制大小>]
[IDENTIFIED BY <加密密码>[WITH ENCRYPTION<TYPE>][ENCRYPT WITH <加密算法>]]
[COMPRESSED [LEVEL <压缩级别>]]
[TRACE FILE '<TRACE 文件名>'] [TRACE LEVEL <TRACE 日志级别>]
[PARALLEL [<并行数>]];
```

表空间名：指定备份的表空间，只能备份用户表空间。

FULL|INCREMENT：备份类型，FULL 表示完全备份，INCREMENT 表示增量备份。若

不指定，默认为完全备份。

WITH BACKUPDIR：用于增量备份中，指定备份目录，最大长度为 256 个字节。若不指定，服务器自动在默认备份目录下搜索基备份。如果基备份不在默认的备份目录下，增量备份必须指定该参数。

BASE ON：用于增量备份中，指定基备份集目录。

USE PWR： 用于增量备份中，指定备份过程中使用 PWR 优化，默认不使用。当表空间很大而需要备份的数据页又很少时可指定该参数来提高备份效率，否则可任意选择是否使用该参数。关于 PWR 的介绍见 2.1.1.4 PWR 优化 。

TO：指定生成备份名称。若未指定，系统随机生成，默认备份名格式为：DB_备份类型_表空间名_备份时间。其中，备份时间为开始备份的系统时间。

BACKUPSET：指定当前备份集生成路径。若指定为相对路径，则在默认备份路径中生成备份集。若不指定，则在默认备份路径下以约定规则生成默认的表空间备份集目录。表空间级备份默认备份集目录名生成规则：TS_表空间名_时间，如 TS_MAIN_20160518_143057_123456。表明该备份集为 2016 年 5 月 18 日 14 时 30 分 57 秒 123456 毫秒时生成的表空间名为 MAIN 的表空间备份集。若表空间名称超长，使上述完整名称长度大于 128 个字节，则去掉表空间名字段，调整为 TS_时间。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

PARMS：只对介质类型为 TAPE 时有效。

BACKUPINFO：备份的描述信息。最大不超过 256 个字节。

MAXPIECESIZE：最大备份片文件大小上限，以 M 为单位，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

IDENTIFIED BY：指定备份时的加密密码。密码应用双引号括起来，这样避免一些特殊字符通不过语法检测。

WITH ENCRYPTION：指定加密类型，0 表示不加密，不对备份文件进行加密处理；1 表示简单加密，对备份文件设置口令，但文件内容仍以明文存；2 表示完全数据加密，对备份文件进行完全的加密，备份文件以密文方式存储。

ENCRYPT WITH：加密算法。缺省情况下，算法为 AES256_CFB。



说明：

加密算法包括：

DES_ECB、DES_CBC、DES_CFB、DES_OFB、DESEDE_ECB、DESEDE_CBC、DESEDE_CFB、DESEDE_OFB、AES128_ECB、AES128_CBC、AES128_CFB 、AES128_OFB、AES192_ECB、AES192_CBC、AES192_CFB 、AES192_OFB、AES256_ECB、AES256_CBC、AES256_CFB 、AES256_OFB 、RC4

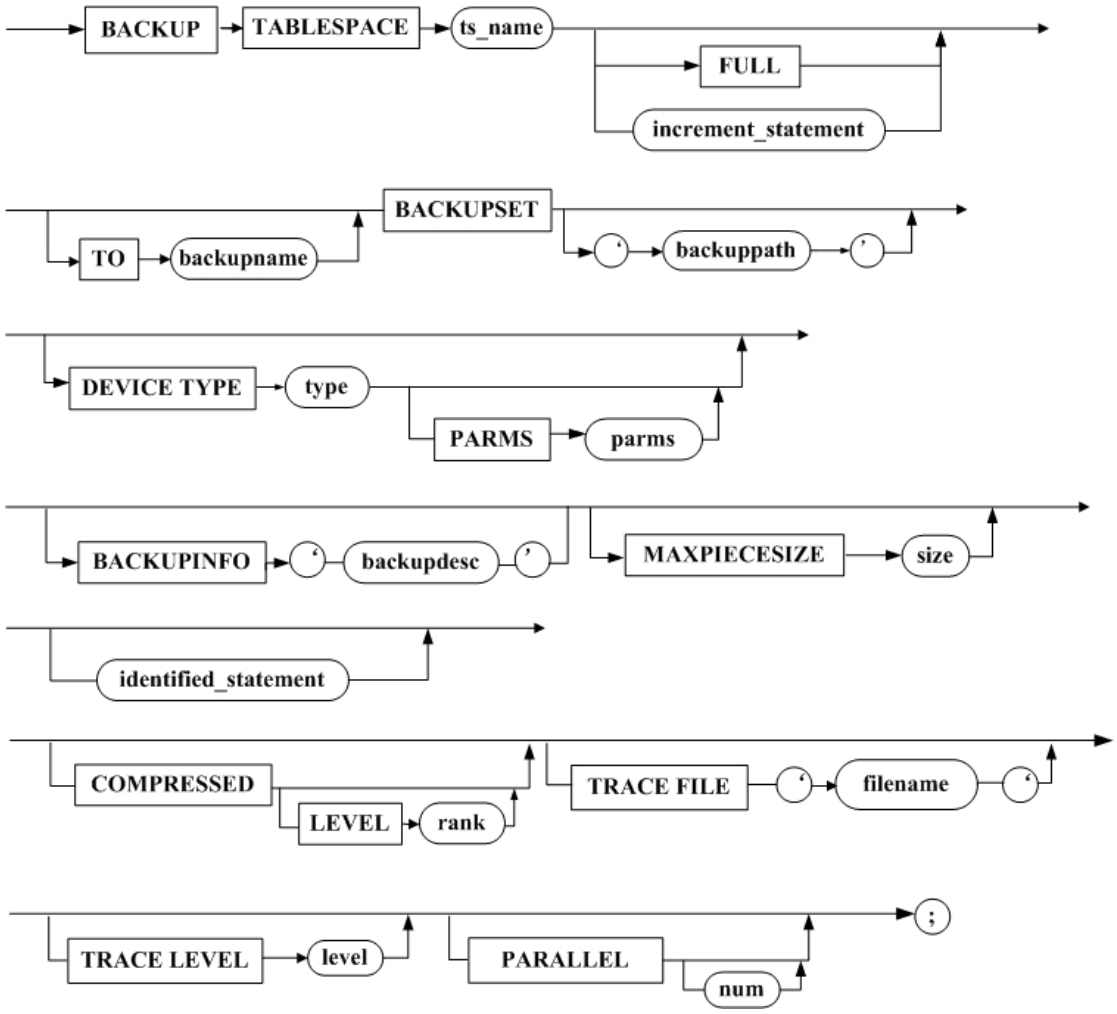
COMPRESSED：取值范围 0~9。0 表示不压缩，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。若未指定，但指定 COMPRESSED，则默认 1；否则，默认 0。

TRACE FILE：指定生成的 TRACE 文件。启用 TRACE，但不指定 TRACE FILE 时，默认在 DM 数据库系统的 log 目录下生成 DM_SBTTRACE_年月.LOG 文件；若使用相对路径，则生成在执行码同级目录下。

TRACE LEVEL：有效值 1、2，默认为 1 表示不启用 TRACE，此时若指定了 TRACE FILE，会生成 TRACE 文件，但不写入 TRACE 信息；为 2 启用 TRACE 并写入 TRACE 相关内容。

PARALLEL：指定并行备份的并行数，若不指定，则默认为 4，指定 0 或者 1 均认为为非并行备份。增量备份中强制并行数与基备份一致。若未指定关键 PARALLEL，则认为非并行备份。并行备份不支持 PWR 和存在介质为 TAPE 的备份。

图例：



increment_statement、identified_statement：请参考 3.2.2.1.1 节数据库备份图例

使用说明：

1. 当备份数据超过限制大小时，会生成新的备份文件，新的备份文件名是初始文件名后加文件编号。

2. 系统处于归档模式下时，才允许进行表空间备份。

3. MOUNT 状态下，不允许进行表空间备份。

4. MPP 和 RAC 环境均不允许进行表空间备份。

3.2.2.2.2 设置备份选项

表空间备份也可指定备份集路径、介质类型、备份名等备份选项，如何设置这些选项可参考 3.2.2.1.2 小节。本节主要介绍 BASE ON 参数的使用。

增量备份指定基备份目录

BASE ON 参数用于增量备份中，用来指定基备份集的目录。如果不指定该参数，会在备份搜索目录中搜索最近一次的完全备份或增量备份作为这次增量备份的基备份。若需要在特定的备份集基础上执行增量备份就需要使用该参数。

下面以增量备份用户 MAIN 表空间为例，指定 BASE ON 参数执行增量备份：

```
SQL>BACKUP TABLESPACE MAIN BACKUPSET 'ts_full_bak_01';
SQL>BACKUP TABLESPACE MAIN INCREMENT BACKUPSET 'ts_increment_bak_01';
SQL>BACKUP TABLESPACE MAIN INCREMENT BASE ON BACKUPSET'ts_full_bak_01' BACKUPSET
'ts_increment_bak_02';
```

上述示例中，增量备份 ts_increment_bak_02 若不指定备份集 ts_full_bak_01 作为基备份，那么默认会使用最近一次的备份集 ts_increment_bak_01 作为基备份。

3.2.2.2.3 备份表空间

同数据库备份一样，表空间备份也可分为完全备份和增量备份。本节主要内容包括：

- 完全备份
- 增量备份

完全备份

执行表空间备份，表空间必须处于 OPEN 状态，MOUNT 和 SUSPEND 状态下不允许执行表空间备份。表空间备份就是拷贝表空间内所有数据文件有效数据的过程。DM7 仅支持表空间联机备份，完全备份一个表空间步骤如下：

1) 配置归档，参考 3.1 节。

2) 保证数据库处于 OPEN 状态。

3) DIsql 中输入备份表空间语句，最简单的不设置其他参数的完全备份表空间语句，如下所示：

```
SQL> BACKUP TABLESPACE MAIN FULL BACKUPSET '/home/dm_bak/ts_full_bak_01';
```

备份语句中的 FULL 参数可以省略，不指定备份类型服务器会默认指定备份类型为完全备份。

增量备份

执行表空间增量备份的主要目的同数据库增量备份一样是为了快速备份数据库中的修改，减少备份时间和避免重复的备份。

增量备份表空间步骤如下：

1) 配置归档，参考 3.1 节。

2) 保证数据库处于 OPEN 状态。

3) DIsql 中输入备份表空间语句，最简单的不设置其他参数的增量备份语句如下：

```
SQL> BACKUP TABLESPACE MAIN INCREMENT WITH BACKUPDIR '/home/dm_bak' BACKUPSET
'/home/dm_bak/ts_increment_bak_02';
```

备份语句中指定的 INCREMENT 参数表示执行的备份类型为增量备份,不可省略。若基备份不在默认备份目录，WITH BACKUPDIR 参数必须指定，用于搜索基备份集。

3.2.2.3 表备份

本节主要介绍使用 DIsql 完成表备份及实施一些备份策略，如限制备份集大小、加密备份等。主要内容包括：

- 概述
- 设置备份选项
- 备份表空间

3.2.2.3.1 概述

与备份数据库与表空间不同，备份表不需要服务器配置归档，DIsql 中输入以下即可备份用户表：

```
SQL> BACKUP TABLE TAB_01 BACKUPSET 'tab_bak_01';
```

备份集“tab_bak_01”会生成到默认的备份路径下。如要设置其他备份选项需参考下文的备份表语法。

语法如下：

```
BACKUP TABLE <表名> [WITH INDEX <ALL | <索引名称>{,<索引名称>}>]]
[TO <备份名>]
BACKUPSET ['<备份集路径>'] [DEVICE TYPE <介质类型> [PARMS '<介质参数>']]
[BACKUPINFO '<备份集描述>']
[MAXPIECESIZE <备份片限制大小>]
[IDENTIFIED BY <加密密码>[WITH ENCRYPTION<TYPE>][ENCRYPT WITH <加密算法>]]
[COMPRESSED [LEVEL <压缩级别>]]
```

```
[TRACE FILE '<trace 文件名>'] [TRACE LEVEL <trace 日志级别>]
```

TABLE：指定备份的表，只能备份用户表。

WITH INDEX：备份索引。若指定，还原操作时会在还原数据之前创建二级索引，否则在之后创建。指定 ALL，备份所有二级索引数据，否则需指定具体的二级索引名称。全文索引、虚索引、位图连接索引、位图连接虚索引、空间或者数组索引不支持使用该参数。使用 WITH INDEX 指定备份的索引，如果还原时指定 WITH INDEX DATA，则还原过程为先创建相应的索引，然后再还原备份的数据。

TO：指定生成备份名称。若未指定，系统随机生成，默认备份名格式为：DB_备份类型_表名_备份时间。其中，备份时间为开始备份的系统时间。

BACKUPSET：指定当前备份集生成路径，若指定为相对路径，则在默认备份路径中生成备份集。若不指定具体备份集路径，则在默认备份路径下以约定规则生成默认的表备份集目录。表备份默认备份集目录名生成规则：TAB_表名_时间，如 TAB_T1_20160518_143057_123456。表明该备份集为 2016 年 5 月 18 日 14 时 30 分 57 秒 123456 毫秒时生成的表名为 T1 的表备份集。若表名超长，使上述完整名称长度大于 128 个字节，则去掉表名字段，调整为 TAB_时间。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

PARMS：只对介质类型为 TAPE 时有效。

BACKUPINFO：备份的描述信息。最大不超过 256 个字节。

MAXPIECESIZE：最大备份片文件大小上限，以 M 为单位，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

IDENTIFIED BY：指定备份时的加密密码。密码应用双引号括起来，这样避免一些特殊字符通不过语法检测。

WITH ENCRYPTION：指定加密类型，0 表示不加密，不对备份文件进行加密处理；1 表示简单加密，对备份文件设置口令，但文件内容仍以明文存；2 表示完全数据加密，对备份文件进行完全的加密，备份文件以密文方式存储。

ENCRYPT WITH：加密算法。缺省情况下，算法为 AES256_CFB。

说明：

加密算法包括：

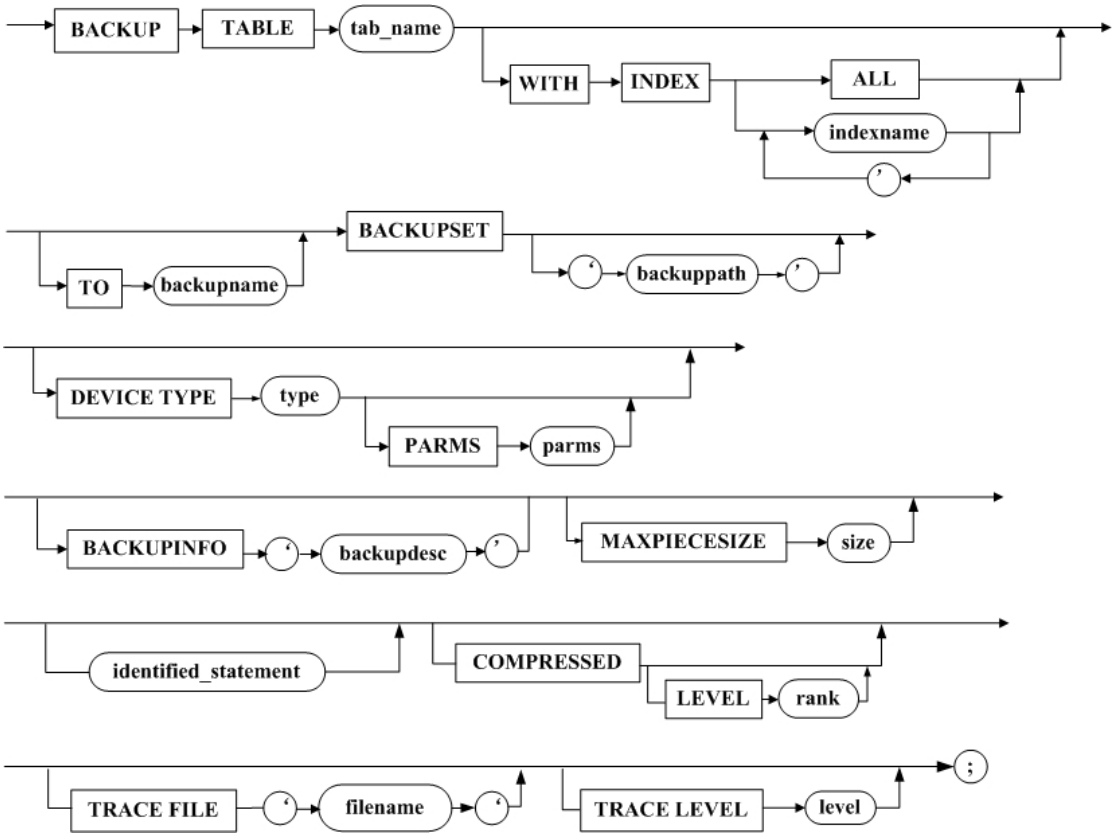
DES_ECB、DES_CBC、DES_CFB、DES_OFB、DESEDE_ECB、DESEDE_CBC、DESEDE_CFB、DESEDE_OFB、AES128_ECB、AES128_CBC、AES128_CFB 、AES128_OFB、AES192_ECB、AES192_CBC、AES192_CFB 、AES192_OFB、AES256_ECB、AES256_CBC、AES256_CFB 、AES256_OFB 、RC4

COMPRESSED：取值范围 0~9。0 表示不压缩，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。若未指定，但指定 COMPRESSED，则默认 1；否则，默认 0。

TRACE FILE：指定生成的 TRACE 文件。启用 TRACE，但不指定 TRACE FILE 时，默认在 DM 数据库系统的 log 目录下生成 DM_SBTTRACE_年月.LOG 文件；若使用相对路径，则生成在执行码同级目录下。

TRACE LEVEL：有效值 1、2，默认为 1 表示不启用 TRACE，此时若指定了 TRACE FILE，会生成 TRACE 文件，但不写入 TRACE 信息；为 2 启用 TRACE 并写入 TRACE 相关内容。

图例：



increment_statement：请参考 3.2.2.1.1 节数据库备份图例

使用说明：

1. 仅支持对普通用户表进行备份，包括堆表。其中，系统表、临时表、物化视图表、物化视图附属表和日志表、特定模式（DBG_PKG/INFORMATION_SCHEMA/INFO_SCHEM/SYSREP/SYSGEO/SYSJOB/SYSCPT/SYS）下表，以及使用 LOB_REC 存储的表不支持备份。表列类型为对象类型的表不支持表备份。对于指定索引备份，堆表不支持索引备份。另外，聚集索引、全文索引、虚索引、位图连接索引和位图连接虚索引、空间索引和数组索引不支持备份。表备份不备份表上的注释，default 表达式中函数定义，所以还原时需用户自行确认。

2. 当备份数据超过限制大小时，会生成新的备份文件，新的备份文件名是初始文件名后加文件编号。

3. 表备份时，其所属表空间必须处于联机状态。

4. 数据守护环境下，仅允许在主库上进行表备份。

5. MOUNT 状态下不允许进行表备份及还原。

6. MPP 和 RAC 环境均不允许进行表备份及还原。

3.2.2.3.2 设置备份选项

表备份常用的备份选项有设置备份名、设置备份集路径、指定介质参数、添加备份描述等，设置方式同数据库备份相同，具体参考 3.2.2.1.2 小节。

3.2.2.3.3 备份表

表备份拷贝指定表所使用的所有数据页到备份集中，并记录各个数据页之间的逻辑关系用来恢复表数据结构。表备份均为联机完全备份，不需要备份归档日志，不存在增量备份之说。当数据库中某张表比较重要而又没必要备份整个数据库或表空间时就可以选择表备份。

完整的备份表步骤如下：

1) 保证数据库处于 OPEN 状态。

2) 创建待备份的表 TAB_01:

```
SQL> CREATE TABLE TAB_01(C1 INT);
```

3) DIsql 中输入备份表语句，简单的备份语句如下：

```
SQL> BACKUP TABLE TAB_01 BACKUPSET '/home/dm_bak/tab_bak_01';
```

3.2.2.4 归档备份

本节主要介绍使用 DIsql 完成归档备份及实施一些备份策略，如限制备份集大小、加密备份等。主要内容包括：

- 概述
- 设置备份选项
- 备份归档

3.2.2.4.1 概述

在 DIsql 工具中使用 BACKUP 语句可以备份归档日志。使用归档备份的前提：一是，归档文件的 db_magic 值和库的 db_magic 值必须一样；二是，服务器必须配置归档；三是，归档日志必须连续，如果出现不连续的情况，前面的会忽略，仅备份最新的连续部分。如果未收集到指定范围内的归档，则不会备份。联机备份的时候经常会切换归档文件，最后一个归档总是空的，所以最后一个归档不会被备份。

DIsql 中输入以下即可备份归档：

```
SQL>BACKUP ARCHIVE LOG ALL BACKUPSET 'arch_bak_01';
```

备份集“arch_bak_01”会生成到默认的备份路径下。如要设置其他备份选项需参考下文的备份归档语法。

语法如下：

```
BACKUP<ARCHIVE LOG |ARCHIVELOG>
[ALL | [FROM LSN <lsn>]| [UNTIL LSN <lsn>]|[LSN BETWEEN <lsn> AND <lsn>] | [FROM
TIME '<time>']|[UNTIL TIME '<time>']|[TIME BETWEEN '<time>'> AND '<time>'
]][<notBackedUpSpec>][DELETE INPUT]
[TO <备份名>]
BACKUPSET ['<备份集路径>'] [DEVICE TYPE <介质类型> [PARMS '<介质参数>']]
[BACKUPINFO '<备份描述>']
[MAXPIECESIZE <备份片限制大小>]
[IDENTIFIED BY <密钥>[WITH ENCRYPTION<TYPE>][ENCRYPT WITH <加密算法>]]
[COMPRESSED [LEVEL <压缩级别>]]
[WITHOUT LOG]
```

```
[TRACE FILE '<trace 文件名>'] [TRACE LEVEL <trace 日志级别>][PARALLEL [<并行数>]];
```

ALL：备份所有的归档；

FROM LSN：指定备份的起始 lsn。

UNTIL LSN：指定备份的截止 lsn。



归档日志的有效 LSN 范围（起始 lsn,截止 lsn）可以通过 V$ARCH_FILE 查看，或者通过 dmclvt 工具分析日志的结果查看。

如果用户无法确定准确的 lsn，也可以指定一个模糊的 lsn 值，取值范围（1~9223372036854775807）。当 FROM LSN 指定的 lsn 小于有效起始 lsn 时，以有效起始 lsn 为准；当 UNTIL LSN 指定的 lsn 超过截止 lsn 时，会报出警告信息，不会生成备份文件。

FROM TIME：指定备份的开始的时间点。

UNTIL TIME：指定备份的截止的时间点。

BETWEEN ... AND ...：指定备份的的区间，仅仅被备份区间内的归档文件。

<notBackedUpSpec>：搜索过滤。搜索过滤仅限于根据备份指定条件能找到的所有归档备份集。1） num TIMES，取值范围为 0~2147483647，指若归档文件已经备份了 num 次，则不再备份；否则备份。如 num=3，则认为已经备份了 3 次的归档文件就不再备份。若 num=0，则认为所有都不需要备份。2）SINCE TIME 'datetime_String'，指定时间开始没有备份的归档文件进行备份。3）若以上两种均未指定，则备份所有未备份过的归档日志文件。

DELETE INPUT：用于指定备份完成之后，是否删除归档操作。

TO：指定生成备份名称。若未指定，系统随机生成，默认备份名格式为：ARCH_备份时间。其中，备份时间为开始备份的系统时间。

BACKUPSET：指定当前备份集生成路径，若指定为相对路径，则在默认备份路径中生成备份集。若不指定具体备份集路径，则在默认备份路径下，以约定归档备份集命名规则生成默认的归档备份集目录。归档备份默认备份集目录名生成规则：ARCH_时间，如 ARCH_20160518_143057_123456。表明该备份集为 2016 年 5 月 18 日 14 时 30 分 57 秒 123456 毫秒时生成的归档备份集。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

PARMS：只对介质类型为 TAPE 时有效。

BACKUPINFO：备份的描述信息。最大不超过 256 个字节。

MAXPIECESIZE：最大备份片文件大小上限，以 M 为单位，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

IDENTIFIED BY：指定备份时的加密密码。密码应该使用双引号括起来，这样避免一些特殊字符通不过语法检测。

WITH ENCRYPTION：指定加密类型，0 表示不加密，不对备份文件进行加密处理；1 表示简单加密，对备份文件设置口令，但文件内容仍以明文存储；2 表示完全数据加密，对备份文件进行完全的加密，备份文件以密文方式存储。

ENCRYPT WITH：加密算法。缺省情况下，算法为 AES256_CFB。


说明：

加密算法包括：

DES_ECB、DES_CBC、DES_CFB、DES_OFB、DESEDE_ECB、DESEDE_CBC、DESEDE_CFB、DESEDE_OFB、AES128_ECB、AES128_CBC、AES128_CFB 、AES128_OFB、AES192_ECB、AES192_CBC、AES192_CFB 、AES192_OFB、AES256_ECB、AES256_CBC、AES256_CFB 、AES256_OFB 、RC4

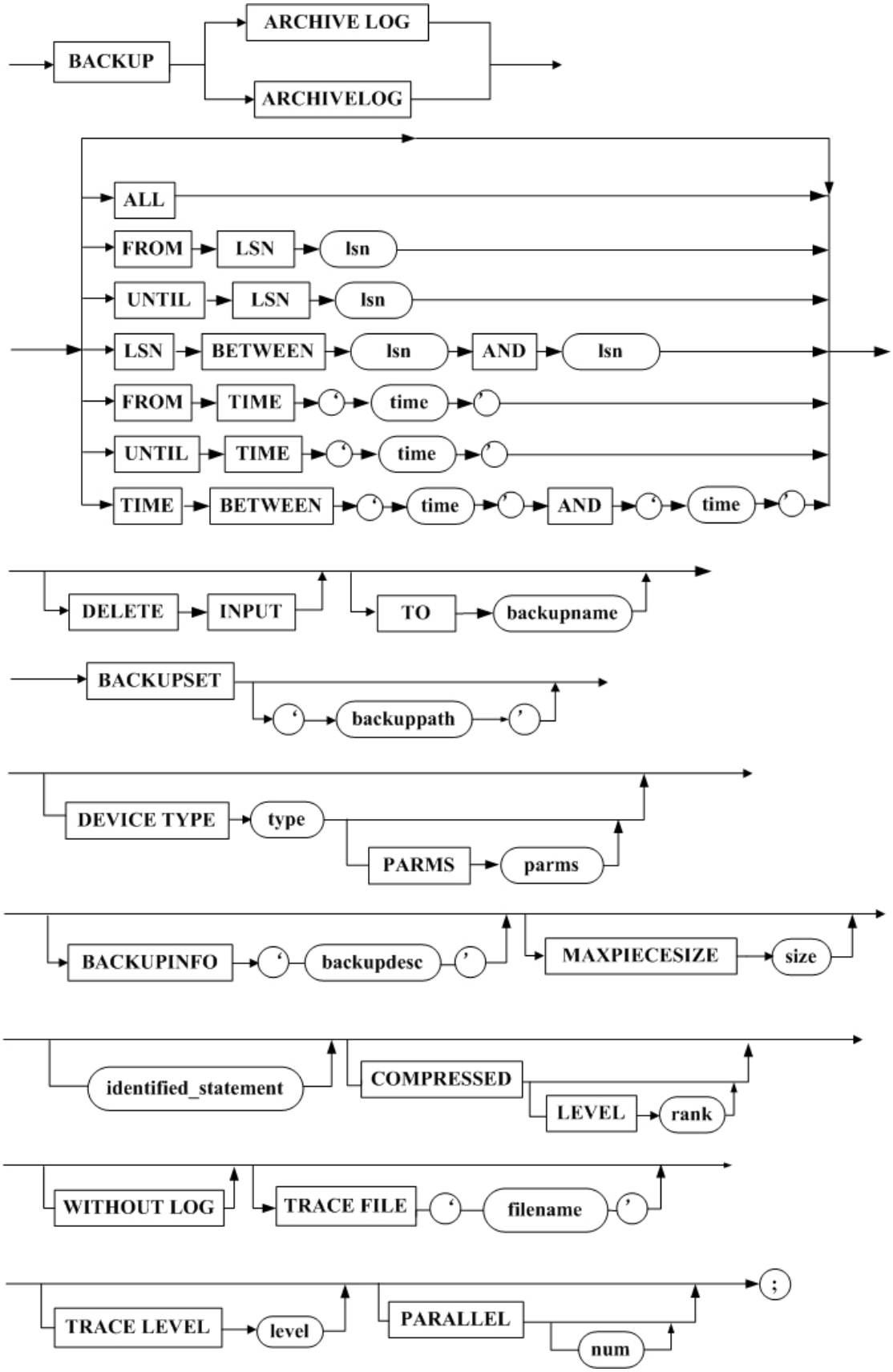
COMPRESSED：取值范围 0~9。0 表示不压缩，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。若未指定，但指定 COMPRESSED，则默认 1；否则，默认 0。

WITHOUT LOG：只是语法支持，不起任何作用。

TRACE FILE：指定生成的 TRACE 文件。启用 TRACE，但不指定 TRACE FILE 时，默认在 DM 数据库系统的 log 目录下生成 DM_SBTTRACE_年月.LOG 文件；若使用相对路径，则生成在执行码同级目录下。

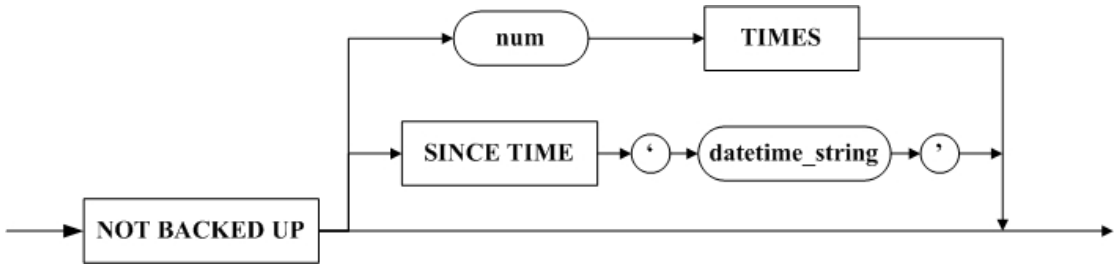
TRACE LEVEL：有效值 1、2，默认为 1 表示不启用 TRACE，此时若指定了 TRACE FILE，会生成 TRACE 文件，但不写入 TRACE 信息；为 2 启用 TRACE 并写入 TRACE 相关内容。

图例：



increment_statement：请参考 3.2.2.1.1 节数据库备份图例

notBackedUpSpec：



3.2.2.4.2 设置备份选项

归档备份常用的备份选项有设置备份名、设置备份集路径、指定介质参数、添加备份描述等，设置方式同数据库备份相同，具体参考 3.2.2.1.2 小节。

3.2.2.4.3 备份归档

归档备份拷贝指定归档目录下的所有的归档文件到备份集中，并记录各个归档文件的属性，文件大小，LSN 区间等。归档备份不存在增量备份之说。当需要保存库的归档时，可以使用归档备份。

完整的备份归档步骤如下：

1) 配置归档，参考 3.1 节。

2) 数据库处于 OPEN 或者 MOUNT 状态。

3) DIsql 中输入备份数据库语句。

例如，备份归档日志，通过 LSN BETWEEN ... AND ...来指定起始和截至 LSN。

首先，确定 LSN 范围。

```
SQL>select ARCH_LSN, CLSN, PATH from V$ARCH_FILE;
```

```
查询结果为：
ARCH_LSN    CLSN    PATH
38758   40301   d:\arch\ARCHIVE_LOCAL1_20160727105417748.log
40302   40303   d:\arch\ARCHIVE_LOCAL1_20160727111411079.log
40304   40305   d:\arch\ARCHIVE_LOCAL1_20160727111513679.log
……
50412   50413   d:\arch\ARCHIVE_LOCAL1_20160727171801098.log
50414   50478   d:\arch\ARCHIVE_LOCAL1_20160727171849712.log
……
```

```
通过查询结果选出备份的起始 LSN 和截至 LSN。 比如 50414  50478
```

其次，备份归档。

```
SQL>BACKUP ARCHIVELOG  LSN BETWEEN 50414  AND 50478 BACKUPSET
'/home/dm_bak/arch_bak_time_14-78';
```

3.2.3 数据备份高级主题

本节介绍高级的联机备份过程，主要内容包括：

- 加密备份
- 设置跟踪日志文件

DM7 的数据库、表空间和表备份均支持创建加密备份和备份时设置跟踪日志文件，本节仅以数据库为例说明这两种类型备份的创建。

加密备份

DM7 提供加密备份的方式保护用户的备份集，没有权限的用户无法访问加密的备份集。

备份语句中通过指定 IDENTIFIED BY...WITH ENCRYPTION...ENCRYPT WITH... 执行加密备份。其中，参数 IDENTIFIED BY 指定加密密码，长度不超过 128 字节，若长度超长服务器会报语法分析出错；参数 WITH ENCRYPTION 指定加密类型，加密类型分为简单加密和复杂加密，简单加密是对备份文件设置口令，但文件内容仍以明文存储，复杂加密则对备份文件进行完全的加密，备份文件以密文方式存储，用户可根据备份数据的重要程度选择加密类型；参数 ENCRYPT WITH 指定加密算法，不同加密算法具体见参数说明，也可通过“SELECT * FROM V$CIPHERS”语句查询 DM7 支持的加密算法，其中算法 MD5 和 SHA1 不能在此处使用。默认使用的加密算法为 AES256_CFB。

加密备份过程中参数 IDENTIFIED BY 必须指定，参数 WITH ENCRYPTION 和参数 ENCRYPT WITH 可不指定，此时 WITH ENCRYPTION 默认值为 1，ENCRYPT WITH 默认值为 AES256_CFB。例如，以下两种加密备份语句都是合法的：

```
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_encrypt' IDENTIFIED BY
"cdb546";
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_encrypt' IDENTIFIED BY
"cdb546" ENCRYPT WITH RC4;
```

若指定了加密密码，但加密类型 WITH ENCRYPTION 参数指定为 0，则为非加密备份，

如下所示：

```
SQL> BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_encrypt' IDENTIFIED BY
"cdb546" WITH ENCRYPTION 0;
```

下面以数据库完全备份为例，创建加密密码为“cdb546”，加密算法为“rc4”的复杂类型数据库加密备份，完整步骤如下：

1) 配置归档，参考 3.1 节。

2) 保证数据库处于 OPEN 状态。

3) 备份数据库，输入以下语句：

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_encrypt_01' IDENTIFIED
BY "cdb546" WITH ENCRYPTION 2 ENCRYPT WITH RC4;
```

对于增量备份加密，如果基备份存在加密，则使用的加密算法和加密密码必须与基备份中一致；如果基备份未进行加密处理，则对增量备份使用的加密密码和算法没有特殊要求。

设置跟踪日志文件

DM7 备份时支持设置跟踪日志文件，日志记录了 SBT 接口的调用过程，用户通过查看日志可跟踪备份还原过程。

与生成跟踪日志文件相关的参数有两个：TRACE FILE 和 TRACE LEVEL。TRACE FILE 用于指定生成的跟踪日志文件路径，TRACE LEVEL 表示是否启用 TRACE。TRACE LEVEL 有效值包括 1 和 2。1 表示不启用 TRACE 功能，2 表示启用，系统默认值为 1。

指定参数 TRACE FILE 但 TRACE LEVEL 值设置为 1 即不启用 TRACE 功能，会生成 TRACE 文件，但不会写入 TRACE 信息。如下所示：

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_trace' TRACE
FILE'/home/dm_log/db_trace.log' TRACE LEVEL 1;
```

TRACE LEVEL 值设置为 2 即启用 TRACE 功能，但若 TRACE FILE 没有指定，系统默认在执行码路径的 log 目录下生成 DM_SBTTRACE_年月.log 文件。如下所示：

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_trace' TRACE LEVEL 2;
```

若 TRACE FILE 使用相对路径，日志文件生成在执行码同级目录下。

以数据库完全备份为例，为备份设置跟踪日志文件的操作步骤如下：

1) 配置归档，参考 3.1 节。

2) 保证数据库处于 OPEN 状态。

3) 备份数据库，输入以下语句：

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_trac_01' TRACE FILE'/home/dm_log/db_bak_trace.log' TRACE LEVEL 2;
```

如果指定的 TRACE 文件已存在，服务器不会覆盖已存在的文件而是在已有文件基础上继续记录日志。

3.2.4 管理备份

本章节主要介绍如何使用 DIsql 工具管理数据库备份、表空间备份、表备份，以及归档备份。本节内容主要包括：

- 概述
- 备份目录管理
- 备份集管理
- 备份信息查看

3.2.4.1 概述

管理备份一个重要的目的是删除不再需要的备份，DM7 没有提供自动删除过期备份的功能，删除备份需要手动执行。

备份管理相关系统过程与函数总结如下：

- SF_BAKSET_BACKUP_DIR_ADD：添加备份目录。
- SF_BAKSET_BACKUP_DIR_REMOVE：指定删除内存中的备份目录。
- SF_BAKSET_BACKUP_DIR_REMOVE_ALL：删除内存中全部的备份目录。
- SF_BAKSET_CHECK：对备份集进行校验。
- SF_BAKSET_REMOVE：删除指定设备类型和指定备份集目录的备份集。
- SF_BAKSET_REMOVE_BATCH：批量删除满足指定条件的所有备份集。
- SP_DB_BAKSET_REMOVE_BATCH：批量删除指定时间之前的数据库备份集。
- SP_TS_BAKSET_REMOVE_BATCH：批量删除指定表空间对象及指定时间之前的表空间备份集。
- SP_TAB_BAKSET_REMOVE_BATCH：批量删除指定表对象及指定时间之前的表备份集。
- SP_ARCH_BAKSET_REMOVE_BATCH：批量删除指定条件的归档备份集。

备份管理相关动态视图总结如下：

- V$BACKUPSET：显示备份集基本信息。
- V$BACKUPSET_DBINFO：显示备份集的数据库相关信息。
- V$BACKUPSET_DBF：显示备份集中数据文件的相关信息。
- V$BACKUPSET_ARCH：显示备份集的归档信息。
- V$BACKUPSET_BKP：显示备份集的备份片信息。
- V$BACKUPSET_SEARCH_DIRS：显示备份集搜索目录。
- V$BACKUPSET_TABLE：显示表备份集中备份表信息。
- V$BACKUPSET_INDEX：显示表备份集中备份索引信息。
- V$BACKUPSET_SUBS：显示并行备份中生成的子备份集信息。



注意：SF_BAKSET_BACKUP_DIR_ADD 添加备份目录仅对当前会话有效。调用删除备份等函数或查看动态视图时要先调用 SF_BAKSET_BACKUP_DIR_ADD 添加备份目录，否则仅搜索默认备份路径下的备份集。

各个函数和动态视图的使用，下文将进行详细介绍。

3.2.4.2 备份目录管理

这里的备份目录是指备份集搜索目录，这些目录被记录在内存中，当执行动态视图（参见 3.2.4.4 备份信息查看）或批量删除备份集时，均会从这些指定目录中先搜索所有备份集信息。

本节主要内容包括：

- SF_BAKSET_BACKUP_DIR_ADD
- SF_BAKSET_BACKUP_DIR_REMOVE
- SF_BAKSET_BACKUP_DIR_REMOVE_ALL

■ **SF_BAKSET_BACKUP_DIR_ADD 函数**

添加备份目录。若添加目录已经存在或者为库默认备份路径，则认为已经存在，不添加，但也不报错。

定义：

```
INT SF_BAKSET_BACKUP_DIR_ADD(
    device_type varchar,
```

```
    backup_dir varchar(256)
)
```

参数说明：

device_type：待添加的备份目录对应存储介质类型，DISK 或者 TAPE。

backup_dir：待添加的备份目录，不能为空。DISK 介质需检查是否为合法文件目录；TAPE 介质仅检查是否超过最大长度 256 字节。

返回值：

1：目录添加成功；其它情况下报错。

举例说明：

```
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
```

执行结果

```
1
```

■ **SF_BAKSET_BACKUP_DIR_REMOVE 函数**

删除备份目录。若删除目录为库默认备份路径，不进行删除，认为删除失败。若指定目录存在于记录的合法目录中，则删除；不存在或者为空则跳过，正常返回。

定义：

```
INT SF_BAKSET_BACKUP_DIR_REMOVE (
    device_type varchar,
    backup_dir varchar(256)
)
```

参数说明：

device_type：待删除的备份目录对应存储介质类型。待删除的备份目录对应存储介质类型，DISK 或者 TAPE。

backup_dir：待删除的备份目录。DISK 介质需检查是否为合法文件目录；TAPE 介质仅检查是否超过最大长度 256 字节。

返回值：

1：目录删除成功；其他情况报错。

举例说明：

```
SQL>SELECT SF_BAKSET_BACKUP_DIR_REMOVE('DISK','/home/dm_bak');
```

执行结果

```
1
```

- **SF_BAKSET_BACKUP_DIR_REMOVE_ALL 函数**

清理全部备份目录，默认备份目录除外。

定义：

```
INT SF_BAKSET_BACKUP_DIR_REMOVE_ALL ()
```

返回值：

1：目录全部清理成功；其它情况下报错。

举例说明：

```
SQL>SELECT SF_BAKSET_BACKUP_DIR_REMOVE_ALL();
```

执行结果

```
1
```

3.2.4.3 备份集校验与删除

本节介绍备份管理中最重要的功能，备份集校验和备份集删除。单个备份集删除时并行备份中地子备份集不允许单独删除;在给定备份集搜集目录中发现存在引用删除备份集作为基备份的需要执行级联删除,默认报错。批量删除备份集时,跳过收集到的单独的子备份集。主要内容如下：

- SF_BAKSET_CHECK
- SF_BAKSET_REMOVE
- SF_BAKSET_REMOVE_BATCH
- SP_DB_BAKSET_REMOVE_BATCH
- SP_TS_BAKSET_REMOVE_BATCH
- SP_TAB_BAKSET_REMOVE_BATCH
- SP_ARCH_BAKSET_REMOVE_BATCH

- **SF_BAKSET_CHECK 函数**

对备份集进行校验。

定义：

```
INT SF_BAKSET_CHECK(
    device_type varchar,
    bakset_path varchar(256)
)
```

参数说明：

device_type：设备类型，disk 或 tape。

bakset_path：待校验的备份集目录。

返回值：

1：备份集目录存在且合法；否则报错。

举例说明：

```
SQL>BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_check';
SQL> SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/ db_bak_for_check');
```

执行结果

```
1
```

- **SF_BAKSET_REMOVE 函数**

删除指定设备类型和指定备份集目录的备份集。一次只检查一个合法.meta 文件，然后删除对应备份集；若存在非法或非正常备份的.meta 文件，则报错或直接返回，不会接着检查下一个.meta 文件；若同一个备份集下还存在其它备份文件或备份集，则只删除备份文件，不会删除整个备份集。

定义：

```
INT SF_BAKSET_REMOVE (
    device_type varchar,
    backsetpath varchar(256),
    option integer
)
```

参数说明：

device_type：设备类型，disk 或 tape。

backsetpath：待删除的备份集目录。

Option:删除备份集选项，0 默认删除，1 级联删除。可选参数。并行备份集中子备份

集不允许单独删除。目标备份集被其他备份集引用为基备份的，默认删除，报错；级联删除情况下，会递归将相关的增量备份也删除。

返回值：

1：备份集目录删除成功，其它情况下报错。

举例说明：

```
SQL>BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove';

SQL>BACKUP DATABASE INCREMENT BACKUPSET '/home/dm_bak/db_bak_for_remove_incr';

SQL> SELECT SF_BAKSET_REMOVE('DISK','/home/dm_bak/db_bak_for_remove');
```

执行结果

```
0
```

```
SQL> SELECT SF_BAKSET_REMOVE('DISK','/home/dm_bak/db_bak_for_remove',1);
```

执行结果

```
1
```

- **SF_BAKSET_REMOVE_BATCH 函数**

批量删除满足指定条件的所有备份集。

定义：

```
INT SF_BAKSET_REMOVE_BATCH (
    device_type varchar,
    end_time    datetime,
    range       int,
    obj_name    varchar(257)
)
```

参数说明：

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

range：指定删除备份的级别。1 代表库级，2 代表表空间级，3 代表表级，4 代表归档备份。若指定 NULL，则忽略备份集备份级别的区分。

obj_name：待删除备份集中备份对象的名称，仅表空间级和表级有效。若为表级备份删除，则需指定完整的表名（模式.表名），否则，将认为删除会话当前模式下的表备份。若指定为 NULL，则忽略备份集中备份对象名称区分。

返回值：

1：备份集目录删除成功，其它情况下报错。

举例说明：

```
SQL>BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove';
SQL>BACKUP TABLESPACE MAIN FULL BACKUPSET '/home/dm_bak/ts_bak_for_remove';
SQL>SELECT SF_BAKSET_REMOVE_BATCH ('DISK', now(), NULL, NULL);
```

执行结果

```
1
```

- **SP_DB_BAKSET_REMOVE_BATCH 过程**

批量删除指定时间之前的数据库备份集。使用该方法前，需要先使用 SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径下的备份集。

定义：

```
SP_DB_BAKSET_REMOVE_BATCH (
    device_type varchar,
    end_time    datetime
)
```

参数说明：

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

举例说明：

```
SQL>BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_batch_del';
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
SQL>CALL SP_DB_BAKSET_REMOVE_BATCH('DISK', NOW());
```

- **SP_TS_BAKSET_REMOVE_BATCH 过程**

批量删除指定表空间对象及指定时间之前的表空间备份集。使用该方法前，需要先使用 SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径下的备份集。

定义：

```
SP_TS_BAKSET_REMOVE_BATCH (
    device_type   varchar,
    end_time      datetime,
    ts_name       varchar(128)
)
```

参数说明：

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

ts_name：表空间名，若未指定，则认为删除所有满足条件的表空间备份集。

举例说明：

```
SQL>BACKUP TABLESPACE MAIN BACKUPSET '/home/dm_bak/ts_bak_for_batch_del';
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
SQL>CALL SP_TS_BAKSET_REMOVE_BATCH('DISK',NOW(),'MAIN');
```

- **SP_TAB_BAKSET_REMOVE_BATCH 过程**

批量删除指定表对象及指定时间之前的表备份集。使用该方法前，需要先使用 SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径下的备份集。

定义：

```
SP_TAB_BAKSET_REMOVE_BATCH (
    device_type varchar,
    end_time    datetime,
    sch_name varchar(128),
    tab_name varchar(128)
)
```

参数说明：

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

sch_name：表所属的模式名。

tab_name：表名，只要模式名和表名有一个指定，就认为需要匹配目标；若均指定为 NULL，则认为删除满足条件的所有表备份。

举例说明：

```
SQL>CREATE TABLE TAB_FOR_BATCH_DEL(C1 INT);
SQL>BACKUP TABLE TAB_FOR_BATCH_DEL BACKUPSET'/home/dm_bak/tab_bak_for_batch_del';
SQL> SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
SQL> CALL SP_TAB_BAKSET_REMOVE_BATCH('DISK',NOW(),'SYSDBA','TAB_FOR_BATCH_DEL');
```

- **SP_ARCH_BAKSET_REMOVE_BATCH 过程**

批量删除指定时间之前的归档备份集。使用该方法前，需要先使用 SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径下的备份集。

定义：

```
SP_ARCH_BAKSET_REMOVE_BATCH (
    device_type varchar,
    end_time    datetime
)
```

参数说明：

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

举例说明：

```
SQL>BACKUP ARCHIVELOG BACKUPSET '/home/dm_bak/arch_bak_for_batch_del';
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
SQL>CALL SP_ARCH_BAKSET_REMOVE_BATCH('DISK', NOW());
```

3.2.4.4 备份信息查看

DM7 提供了一系列动态视图供用户查看备份集相关信息，在查看之前应先使用 SF_BAKSET_BACKUP_DIR_ADD 添加备份集目录，否则只显示默认备份路径下的备份集信息，使用方法如下例所示：

```
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');
```

下面逐个介绍 DM7 提供的备份相关动态视图。

- **V$BACKUPSET**

V$BACKUPSET 显示备份集基本信息。

序号	列	数据类型	说明
1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型
2	BACKUP_ID	INTEGER	备份 ID
3	PARENT_ID	INTEGER	并行备份的子备份集所属备份的 ID
4	BACKUP_NAME	VARCHAR(129)	备份名
5	BACKUP_PATH	VARCHAR(257)	备份路径
6	TYPE	INTEGER	0：基备份，1：增量备份，2：表备份，3：归档备份
7	LEVEL	INTEGER	是否脱机备份。0：联机备份，1：脱机备份
8	RANGE#	INTEGER	1：库备份，2：表空间备份，3：表级备份，4：归档备份
9	OBJECT_NAME	VARCHAR(513)	对象名：数据库名、表空间名或者表名
10	OBJECT_ID	INTEGER	对象 ID，表备份时无效
11	BASE_NAME	VARCHAR(129)	基备份名，表备份时无效
12	BACKUP_TIME	DATETIME(6)	备份时间
13	DESC#	VARCHAR(513)	备份描述信息
14	ENCRYPT_TYPE	INTEGER	加密类型
15	COMPRESS_LEVEL	INTEGER	压缩级别
16	WITHOUT_LOG	INTEGER	联机数据库备份是否备份日志，表备份时无效

17	USE_PWR	INTEGER	增量备份过程中是否使用 PWR 优化
18	PKG_SIZE	INTEGER	数据包大写标志，内部实现
19	BEGIN_LSN	BIGINT	备份的起始 LSN 值，表备份时无效
20	END_LSN	BIGINT	结束备份的 LSN 值，表备份时无效
21	BKP_NUM	INTEGER	备份片个数，即备份集中.bak 文件个数
22	DBF_NUM	INTEGER	备份集中包含的数据库\表空间数据文件个数，表备份时无效
23	PARALLEL_NUM	INTEGER	并行备份的并行数，0 或者 1 为非并行备份集
24	DDL_CLONE	INTEGER	DDL_CLONE 库备份标识，0 不是，1 是
25	MPP_FLAG	INTEGER	MPP 库备份标识，0 不是，1 是

下面以创建数据库备份为例，查看备份集的介质类型、备份路径、备份类型等基本信息：

```
SQL>BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_info';

SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');

SQL>SELECT DEVICE_TYPE, BACKUP_PATH, TYPE, RANGE# FROM V$BACKUPSET;
```

查询结果：

```
LINEID    DEVICE_TYPE BACKUP_PATH                TYPE       RANGE#

---------- ----------- --------------------------- ----------- -----------

1         DISK       /home/dm_bak/db_bak_for_info 0          1
```

- **V$BACKUPSET_DBINFO**

V$BACKUPSET_DBINFO 显示备份集的数据库相关信息。

序号	列	数据类型	说明
1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型
2	BACKUP_ID	INTEGER	备份 ID
3	BACKUP_NAME	VARCHAR(129)	备份名
4	BACKUP_PATH	VARCHAR(257)	备份路径
5	EXTENT_SIZE	INTEGER	数据文件使用的簇大小
6	PAGE_SIZE	INTEGER	页大小
7	LOG_PAGE_SIZE	INTEGER	日志文件页大小

8	CASE_SENSITVE	INTEGER	大小写敏感标志
9	DB_MAGIC	INTEGER	数据库的 magic
10	PM_DB_MAGIC	INTEGER	永久魔数（permenant_magic）
11	UNICODE_FLAG	INTEGER	unicode 标志
12	DB_VERSION	INTEGER	数据库版本
13	GLOBAL_VERSION	VARCHAR(512)	数据库全局版本信息
14	ENABLE_POLICY	INTEGER	安全策略
15	ARCH_FLAG	INTEGER	归档是否打开的标志
16	RAC_NODE	INTEGER	高性能集群的节点数目
17	PAGE_CHECK	INTEGER	数据页校验配置
18	RLOG_ENCRYPT	INTEGER	归档日志是否加密
19	EX_CIPHER_NAME	VARCHAR(512)	外部加密算法名称
20	EX_CIPHER_ID	INTEGER	外部加密算法名称对应的 ID
21	EX_HASH_NAME	VARCHAR(512)	外部 HASH 算法名称
22	EX_HASH_ID	INTEGER	外部 HASH 算法名称对应的 ID
23	LENGTH_IN_CHAR	INTEGER	VARCHAR 类型长度是否以字符为单位
24	USE_NEW_HASH	INTEGER	是否使用改进的字符类型 HASH 算法
25	BLANK_PAD_MODE	INTEGER	数据库空格填充模式

表还原时要求目标库的特定建库参数要与源库一致，如页大小、簇大小，通过查看备份文件的数据库信息可确定目标库需要设置哪些建库参数。下面以创建表备份为例，查看备份集的页大小、簇大小、大小写是否敏感、UNICODE_FLAG 等数据库信息：

```
SQL>BACKUP TABLE TAB_01 BACKUPSET '/home/dm_bak/tab_bak_01';

SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');

SQL>SELECT   BACKUP_PATH,   PAGE_SIZE,   EXTENT_SIZE,   CASE_SENSITIVE   FROM
V$BACKUPSET_DBINFO WHERE BACKUP_PATH='/home/dm_bak/tab_bak_01';
```

查询结果：

```
LINEID    BACKUP_PATH                PAGE_SIZE   EXTENT_SIZE CASE_SENSITIVE
---------- ---------------------------- ----------- ----------- -------
1        /home/dm_bak/db_bak_for_dbinfo    8192        16          1
```

■ **V$BACKUPSET_DBF**

显示备份集中数据文件的相关信息，表备份时无效。

序号	列	数据类型	说明
1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型
2	BACKUP_ID	INTEGER	备份 ID
3	BACKUPNAME	VARCHAR(129)	备份名
4	BACKUPPATH	VARCHAR(257)	备份路径
5	FILE_SEQ	INTEGER	备份的数据文件序号
6	TS_ID	INTEGER	表空间 ID
7	FILE_ID	INTEGER	数据文件 ID
8	TS_STATE	INTEGER	表空间状态
9	TS_NAME	VARCHAR(129)	表空间名
10	FILE_NAME	VARCHAR(257)	包含完整路径的数据文件名
11	MIRROR_PATH	VARCHAR(257)	镜像文件路径
12	FILE_LEN	BIGINT	数据文件占用的字节大小
13	MAX_LIMIT_SIZE	VARCHAR(512)	文件最大大小，以 M 为单位
14	AUTO_EXTEND	INTEGER	是否支持自动扩展：1 支持，0 不支持
15	NEXT_SIZE	INTEGER	文件每次扩展大小，以 M 为单位
16	START_BKP_SEQ	INTEGER	起始备份片编号
17	START_BKP_OFF	BIGINT	起始备份片偏移
18	END_BKP_SEQ	INTEGER	结束备份片编号
19	END_BKP_OFF	BIGINT	结束备份片偏移

数据库和表空间备份集中记录了备份的数据文件具体信息，如果想了解备份集中包含了哪些数据文件且这些数据文件有什么属性，可通过查询 V$BACKUPSET_DBF 实现。下面以表空间备份为例，查看备份集中的数据文件信息。

```
SQL>CREATE TABLESPACE TS_FOR_DBF DATAFILE'TS_FOR_DBF_01.DBF' SIZE 128;

SQL>ALTER TABLESPACE TS_FOR_DBF ADD DATAFILE'TS_FOR_DBF_02.DBF' SIZE 128;

SQL>BACKUP TABLESPACE TS_FOR_DBF BACKUPSET '/home/dm_bak/ts_bak_for_dbf';
```

```
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');

SQL>SELECT FILE_SEQ, TS_ID, FILE_ID, TS_NAME, FILE_NAME FROM V$BACKUPSET_DBF
WHERE BACKUPPATH ='/home/dm_bak/ts_bak_for_dbf';
```

查询结果：

```
LINEID    FILE_SEQ    TS_ID      FILE_ID    TS_NAME    FILE_NAME

---------- ---------- ---------- ---------- ----------

1       1       5       0          TS_FOR_DBF /home/xm/DAMENG/TS_FOR_DBF_01.DBF

2       2       5       1          TS_FOR_DBF /home/xm/DAMENG/TS_FOR_DBF_02.DBF
```

- **V$BACKUPSET_ARCH**

显示备份集中数据文件的相关信息，表备份时无效。

序号	列	数据类型	说明
1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型
2	BACKUP_ID	INTEGER	备份 ID
3	BACKUPNAME	VARCHAR(129)	备份名
4	BACKUPPATH	VARCHAR(257)	备份路径
5	FILE_SEQ	INTEGER	备份的数据文件序号
6	FILE_NAME	VARCHAR(257)	归档文件路径
7	FILE_LEN	BIGINT	归档文件大小
8	BEGIN_LSN	BIGINT	备份归档的起始 LSN 值
9	BEGIN_SEQNO	INTEGER	起始备份归档序号
10	BEGIN_RPAG_OFF	INTEGER	备份归档起始 LSN 对应归档页内偏移
11	END_LSN	BIGINT	结束备份归档的 LSN 值
12	CREATE_TIME	DATETIME	归档文件的创建时间
13	CLOSE_TIME	INTEGER	归档文件的关闭时间
14	START_BKP_SEQ	INTEGER	起始备份片编号
15	START_BKP_OFF	BIGINT	起始备份片偏移
16	END_BKP_SEQ	INTEGER	结束备份片编号
17	END_BKP_OFF	BIGINT	结束备份片偏移

备份集日志信息可以是联机库备份备份开始到备份结束这段时间服务器产生的日志，也可以是归档备份中备份的归档信息。接下来以数据库备份为例查询备份集日志信息。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_arch';SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');
SQL>SELECT BACKUPPATH, FILE_SEQ, BEGIN_LSN, END_LSN FROM V$BACKUPSET_ARCH WHERE BACKUPPATH='/home/dm_bak/db_bak_for_arch';
```

查询结果：

```
LINEID     BACKUPPATH                FILE_SEQ    BEGIN_LSN        END_LSN
---------- ------------------------- ----------- ---------- ----------
1         /home/dm_bak/db_bak_for_arch    6            280037     281471
```

- **V$BACKUPSET_BKP**

显示备份集的备份片信息。

序号	列	数据类型	说明
1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型
2	BACKUP_ID	INTEGER	备份 ID
3	BACKUPNAME	VARCHAR(129)	备份名
4	BACKUPPATH	VARCHAR(257)	备份路径
5	BKP_NTH	INTEGER	备份片文件编号
6	FILE_NAME	VARCHAR(257)	备份文件名
7	BKP_LEN	BIGINT	备份片长度

以数据库备份为例，查看备份集中的备份片信息。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_bkp';
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');
SQL>SELECT BACKUPPATH, BKP_NTH, FILE_NAME, BKP_LEN FROM V$BACKUPSET_BKP WHERE BACKUPPATH='/home/dm_bak/db_bak_for_bkp';
```

查询结果：

```
LINEID     BACKUPPATH                BKP_NTH     FILE_NAME           BKP_LEN
---------- ------------------------- ------ -------------------- --------
1         /home/dm_bak/db_bak_for_bkp   0       db_bak_for_bkp.bak   15378944
```

```
2         /home/dm_bak/db_bak_for_bkp   1       db_bak_for_bkp_1.bak 9216
```

■ **V$BACKUPSET_SEARCH_DIRS**

显示备份集搜索目录。

序号	列	数据类型	说明
1	DIR	VARCHAR(257)	备份集搜索目录

查询 V$BACKUPSET_SEARCH_DIRS 显示当前会话已添加的备份目录，即备份集搜索目录。若用户没有添加备份目录，那么仅显示默认的备份目录。

```
SQL>SELECT SF_BAKSET_BACKUP_DIR_REMOVE_ALL();

SQL>SELECT * FROM V$BACKUPSET_SEARCH_DIRS;
```

查询结果：

```
LINEID    DIR

---------- ------------------

1         /home/xm/DAMENG/bak
```

如果添加备份目录，查询结果包括默认备份目录和用户添加的备份目录。

```
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');

SQL>SELECT * FROM V$BACKUPSET_SEARCH_DIRS;
```

查询结果：

```
LINEID    DIR

---------- ------------------

1         /home/dm_bak

2         /home/xm/DAMENG/bak
```

■ **V$BACKUPSET_TABLE**

显示表备份集中备份表信息，仅表备份有效。

序号	列	数据类型	说明
1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型
2	BACKUP_ID	INTEGER	备份 ID
3	BACKUPNAME	VARCHAR(129)	备份名
4	BACKUPPATH	VARCHAR(257)	备份路径

5	SCHEMANAME	VARCHAR(129)	备份表所属的模式名
6	USERNAME	VARCHAR(129)	执行表备份的用户名
7	TSNAME	VARCHAR(129)	备份表存储的表空间名
8	TABLENAME	VARCHAR(129)	备份表名
9	TABLETYPE	INTEGER	表类型
10	INITSQL	VARCHAR(4097)	完整建表语句，忽略引用约束（语句可能会被截断）
11	DCONS_SQL	VARCHAR(4096)	备份表中被禁用约束的创建语句（语句可能会被截断，DMRMAN 可查看完整语句）
12	DIDX_SQL	VARCHAR(4096)	备份表中无效二级索引的创建语句（语句可能会被截断，DMRMAN 可查看完整语句）
13	BIDX_NUM	INT	备份集中备份的二级索引个数
14	META_VERSION	INT	当前表备份的元信息的版本号

下面创建表备份并查看备份中备份名、备份路径、表名等信息。

```
SQL>CREATE TABLE TAB_FOR_INFO(C1 INT);
SQL>BACKUP TABLE TAB_FOR_INFO TO TAB_BAK_FOR_INFO BACKUPSET
'/home/dm_bak/tab_bak_for_info';
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');
SQL>SELECT BACKUPNAME, TABLENAME FROM V$BACKUPSET_TABLE WHERE
BACKUPPATH='/home/dm_bak/tab_bak_for_info';
```

查询结果：

```
LINEID    BACKUPNAME      TABLENAME
---------- ---------------- ------------
1         TAB_BAK_FOR_INFO TAB_FOR_INFO
```

- **V$BACKUPSET_INDEX**

显示表备份集中备份索引信息，仅表备份有效。

序号	列	数据类型	说明

1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型
2	BACKUP_ID	INTEGER	备份 ID
3	BACKUPNAME	VARCHAR(512)	备份名
4	BACKUPPATH	VARCHAR(1024)	备份路径
5	SCHEMANAME	VARCHAR(512)	备份表所属的模式名
6	TABLENAME	VARCHAR(512)	备份表名
7	INDEXNAME	VARCHAR(512)	备份索引名称
8	VALID	CHAR	索引是否有效，Y/N
9	OF_CONS	CHAR	索引是否为因建约束内建，Y/N
10	CONS_VALID	CHAR	若索引为因建约束内建，则约束是否有效，Y/N
11	CRT_SQL	VARCHAR(4096)	创建该索引的语句（可能会被截断，DMRMAN 中可查看完整版）

如需查看表备份中的索引备份信息，可通过□V$BACKUPSET_INDEX视图查看，下面举例说明如何查看表备份中的索引是否有效。

```
SQL>CREATE TABLE TAB_FOR_INDEX (C1 INT);
SQL>BACKUP TABLE TAB_FOR_INDEX WITH INDEX ALL BACKUPSET
'/home/dm_bak/tab_bak_for_index';
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');
SQL>SELECT TABLENAME,INDEXNAME,VALID FROM V$BACKUPSET_INDEX WHERE
BACKUPPATH='/home/dm_bak/tab_bak_for_index';
```

查询结果:

```
LINEID    TABLENAME    INDEXNAME      VALID
---------- ------------- --------------- -----
1         TAB_FOR_INDEX I_TAB_FOR_INDEX N
```

- **V$BACKUPSET_SUBS**

显示并行备份中生成的子备份集信息。

序号	列	数据类型	说明
1	DEVICE_TYPE	VARCHAR(10)	备份集存储介质类型

2	BACKUPNAME	VARCHAR(512)	备份名
3	BACKUP_ID	INTEGER	备份 ID
4	PARENT_ID	INTEGER	子备份集所属主备份集 ID
5	BACKUPPATH	VARCHAR(1024)	各备份集绝对路径
6	BKP_NUM	INTEGER	各备份集中备份片文件个数，可能为 0
7	DBF_NUM	INTEGER	各备份集中备份数据文件个数，可能为 0

数据库和表空间支持并行备份，备份后会在主备份集中生成多个子备份集，查询视图 V$BACKUPSET_SUBS 可获取子备份集中包含的备份片文件个数及备份数据文件个数等信息。以创建数据库并行备份为例，查看子备份集的相关信息。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_bak_for_subs' PARALLEL 3;
SQL>SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK', '/home/dm_bak');
SQL>SELECT BACKUPPATH, BKP_NUM, DBF_NUM FROM V$BACKUPSET_SUBS;
```

查询结果：

```
LINEID    BACKUPPATH                                    BKP_NUM    DBF_NUM
---------- ---------------------------------------------- -----------
1         /home/dm_bak/db_bak_for_subs/db_bak_for_subs_0 1          1
2         /home/dm_bak/db_bak_for_subs/db_bak_for_subs_1 1          1
3         /home/dm_bak/db_bak_for_subs/db_bak_for_subs_2 1          1
```

3.2.5 数据还原

DM7 仅支持表空间和表的联机还原，数据库和归档日志的还原必须通过脱机工具 DMRMAN 执行。本章节主要介绍如何使用 DIsql 工具还原表空间和表。

本章内容主要包括：

- 表空间还原
- 表还原

3.2.5.1 表空间还原

本节阐述使用 DIsql 工具完成基本的表空间还原操作及实施一些还原策略。主要内容

包括：

- 概述
- 表空间还原

3.2.5.1.1 概述

在 DIsql 中使用 RESTORE 语句可以还原表空间，还原表空间需要服务器运行在归档模式下且还原之前需要将表空间脱机。启动 DIsql 输入以下语句即可还原表空间：

```
SQL> ALTER TABLESPACE MAIN OFFLINE;
SQL> RESTORE TABLESPACE MAIN FROM BACKUPSET 'ts_bak_01';
SQL> ALTER TABLESPACE MAIN ONLINE;
```

指定备份集“ts_bak_01”为相对路径，服务器自动会在默认备份路径下搜索备份集。如果备份集不在默认路径下，备份集路径应指定为绝对路径。

语法如下：

```
RESTORE TABLESPACE <表空间名>
[DATAFILE< <文件编号> {,<文件编号>} | '<文件路径>' {,'<文件路径>'}>]
FROM BACKUPSET '<备份集路径>' [DEVICE TYPE <介质类型> [PARMS '<介质参数>']]
[IDENTIFIED BY <加密密码>] [ENCRYPT WITH <加密算法>]
[WITH BACKUPDIR '<备份目录>' {,'<备份目录>'}]
[WITH ARCHIVEDIR '归档目录'{,'归档目录'}]
[MAPPED FILE '<映射文件>']
[TRACE FILE '<TRACE 文件名>'] [TRACE LEVEL <TRACE 日志级别>][NOT PARALLEL]
[UNTIL TIME '<时间串>'] [UNTIL LSN <LSN>];
```

TABLESPACE：指定还原的表空间，只能还原用户表空间。

DATAFILE：还原指定的数据文件。可以指定数据文件编号或数据文件路径。文件编号，对应动态视图 V$DATAFILE 中 ID 列的值；文件路径，对应动态视图 V$DATAFILE 中 PATH 或者 MIRROR_PATH 列的值，也可以仅指定数据文件名称（相对路径），与表空间中数据文件匹配时，会使用 SYSTEM 目录补齐。

BACKUPSET：指定还原备份集的路径。若指定为相对路径，会在默认备份目录下搜索备份集。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示备份集存储介质为磁盘，TAPE 表示备份集存储介质为磁带。

PARMS：介质参数，只对介质类型为 TAPE 时有效。

IDENTIFIED BY：加密备份表空间时，用户设置的密码。

ENCRYPT WITH：加密算法。缺省情况下，算法为 AES256_CFB。具体包含哪些加密算法请参考表空间备份的参数说明。

WITH BACKUPDIR：指定备份搜索目录，最大长度为 256 个字节。使用完全备份还原中，若指定的备份集路径为相对路径可通过设置此参数搜索备份集；增量备份还原中设置该参数除上述功能外还用于搜索基备份集。

WITH ARCHIVEDIR：归档日志搜索目录。如果归档日志不在配置文件中指定的目录下，或者归档日志分散在多个目录，此时就需要指定该参数用于搜索归档日志。

MAPPED FILE ：指定存放还原目标路径的文件，参考 3.3.5.2.1 小节。

TRACE FILE：指定生成的 TRACE 文件。启用 TRACE，但不指定 TRACE FILE 时，默认在 DM 数据库系统的 log 目录下生成 DM_SBTTRACE_年月.LOG 文件；若使用相对路径，则生成在执行码同级目录下。

TRACE LEVEL：有效值 1、2，默认为 1 表示不启用 TRACE，此时若指定了 TRACE FILE，会生成 TRACE 文件，但不写入 TRACE 信息；为 2 启用 TRACE 并写入 TRACE 相关内容。

NOT PARALLEL：指定并行备份集使用非并行方式还原。对于非并行备份集，不论是否指定该关键字，均采用非并行还原。

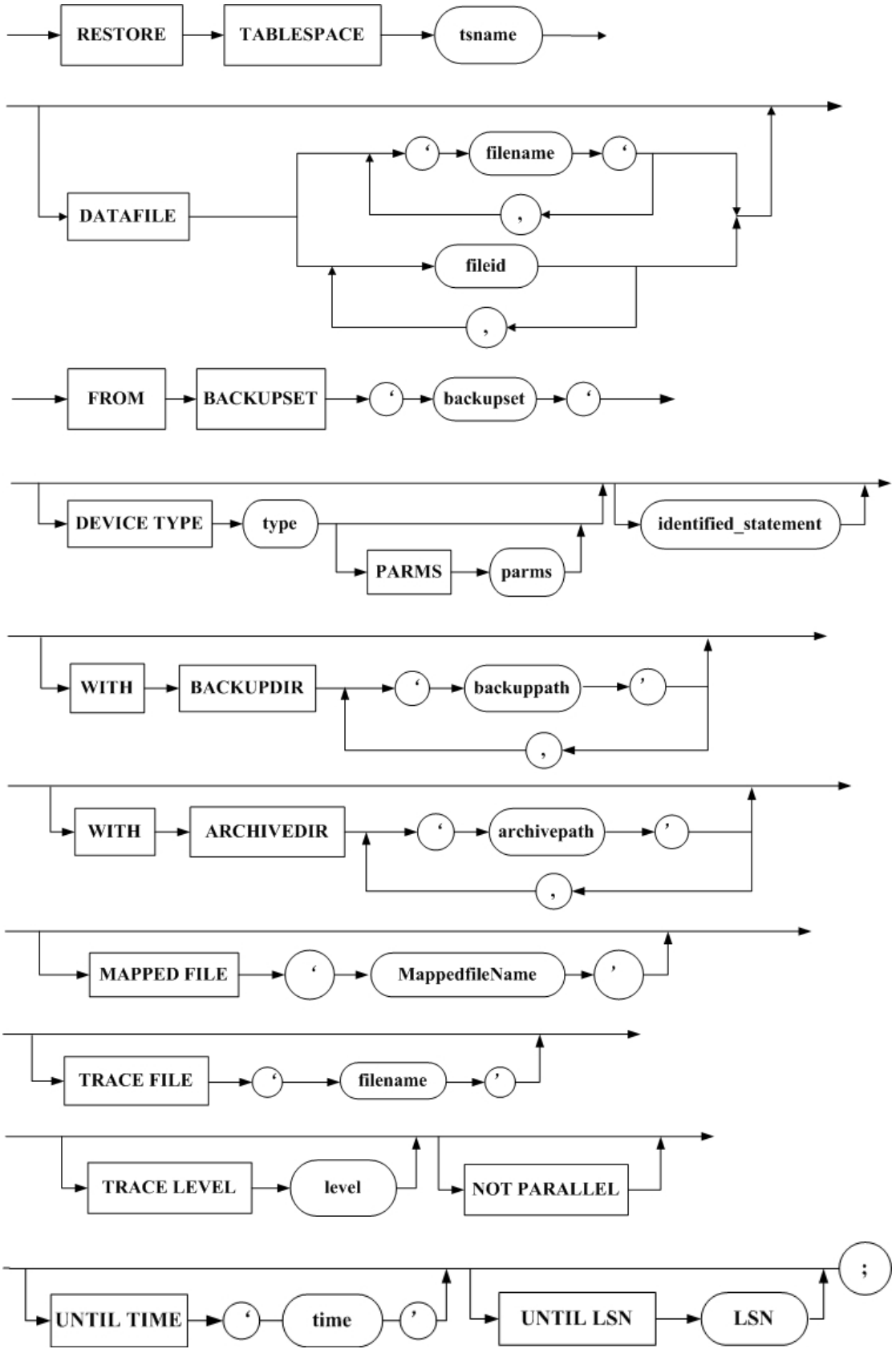
UNTIL TIME：恢复表空间到指定的时间点。主备和 RAC 环境不支持。

UNTIL LSN：恢复表空间到指定的 LSN。主备和 RAC 环境不支持。

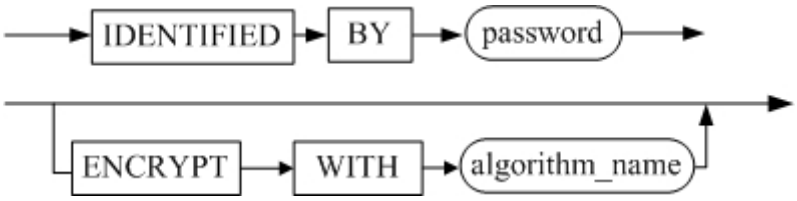


警告：表空间还原不建议使用 UNTIL LSN 或者 UNTIL TIME，若要使用，则确保备份集生成之后未执行过 DDL 操作，否则将可能出现不可预知的后果。

图例：



identified_statement：



使用说明：

1. 支持从非 DDL_CLONE 的库级备份集和表空间级备份集中还原出指定表空间，若还原目标表空间状态不是 RES_OFFLINE，还还原指定表空间中指定数据文件。

2. 备份集路径是指备份集所在目录，其中应包含完整备份数据，包括元数据文件(.meta)和备份片文件(.bak)。

3. 还原前应该先将还原的表空间脱机，只能还原用户表空间。

4. 该功能在配置本地归档后才起作用。

5. MPP 和 RAC 环境均不允许进行表空间还原。

6. 若指定 NOT PARALLEL，则认为并行备份不使用并行还原，默认并行备份执行并行还原。对于非并行备份集还原，则忽略。

7. SYSTEM 表空间和 ROLL 表空间不允许指定 UNTIL LSN 或者 UNTIL TIME 还原；其他用户表空间可以。若未指定 UNTIL TIME 或者 UNTIL LSN，则均还原到最新状态。使用 UNTIL TIME（或者 UNTIL LSN）的同时不能指定 DATAFILE 还原，否则报语法错误。

3.2.5.1.2 表空间还原

本节介绍使用 DIsql 工具如何还原表空间。本节主要内容包括：

- 使用表空间完全备份还原表空间
- 使用表空间增量备份还原表空间
- 使用数据库备份还原表空间

使用表空间完全备份还原表空间

执行表空间还原，数据库必须处于 OPEN 状态，MOUNT 和 SUSPEND 状态下不允许执行表空间还原。

表空间的还原、恢复操作是一次性完成，因此还原后不需要执行恢复操作。使用表空间完全备份还原表空间的完整步骤如下：

1) 配置归档，具体步骤见 3.1.2 归档配置 。

2） 保证数据库处于 OPEN 状态。

3） 备份表空间。

```
SQL>BACKUP TABLESPACE MAIN BACKUPSET '/home/dm_bak/ts_full_bak_01';
```

4） 校验表空间备份。此步骤为可选，如果确定备份文件合法可不进行备份校验。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/ts_full_bak_01');
```

5） 修改表空间为脱机。

```
SQL>ALTER TABLESPACE MAIN OFFLINE;
```

6） 还原表空间。

```
SQL>RESTORE TABLESPACE MAIN FROM BACKUPSET '/home/dm_bak/ts_full_bak_01';
```

7） 修改表空间为联机。

```
SQL>ALTER TABLESPACE MAIN ONLINE;
```

使用表空间增量备份还原表空间

在增量备份与基备份同目录情况下，使用增量备份还原表空间步骤与完全备份一致。当增量备份与基备份不在同一个目录下时需要使用 RESTORE TABLESPACE...WITH BACKUPDIR 语句还原。这里以后面一种情况为例说明使用增量备份还原表空间。步骤如下：

1） 配置归档，具体步骤见 3.1.2 归档配置 。

2） 保证数据库处于 OPEN 状态。

3） 完全备份表空间。

```
SQL>BACKUP TABLESPACE MAIN BACKUPSET '/home/dm_bak/ts_full_bak_02';
```

4） 多次增量备份表空间。

```
SQL>BACKUP TABLESPACE MAIN INCREMENT WITH BACKUPDIR '/home/dm_bak' BACKUPSET
'/home/dm_bak2/ts_incre_bak_02_01';
SQL> BACKUP TABLESPACE MAIN INCREMENT WITH BACKUPDIR '/home/dm_bak',
'/home/dm_bak2' BACKUPSET '/home/dm_bak3/ts_incre_bak_02_02';
```

5） 校验表空间备份。此步骤为可选，如果确定备份文件合法可不进行备份校验。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak3/ts_incre_bak_02_02');
```

6） 修改表空间为脱机。

```
SQL>ALTER TABLESPACE MAIN OFFLINE;
```

7） 使用增量备份 ts_incre_bak_02_02 还原表空间。

```
SQL>RESTORE TABLESPACE MAIN FROM BACKUPSET '/home/dm_bak3/ts_incre_bak_02_02'
```

```
WITH BACKUPDIR '/home/dm_bak2','/home/dm_bak';
```

8) 修改表空间为联机。

```
SQL>ALTER TABLESPACE MAIN ONLINE;
```

使用数据库备份还原表空间

使用数据库备份还原表空间可以快速还原一个或多个表空间，而不影响数据库中其他的表空间和对象。在数据库中仅有特定的表空间损坏且没有表空间备份的情况下，可以选择使用此种方式还原。

具体步骤如下：

1) 配置归档，具体步骤见 3.1.2 归档配置 。

2) 保证数据库处于 OPEN 状态。

3) 创建数据库完全备份，备份集存放目录为“/home/dm_bak”。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_full_bak_for_ts';
```

4) 校验表空间备份。此步骤为可选，如果确定备份文件合法可不进行备份校验。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/db_full_bak_for_ts ');
```

5) 修改表空间为脱机。

```
SQL>ALTER TABLESPACE MAIN OFFLINE;
```

6) 还原表空间。

```
SQL> RESTORE TABLESPACE MAIN FROM BACKUPSET '/home/dm_bak/db_full_bak_for_ts';
```

7) 修改表空间为联机。

```
SQL>ALTER TABLESPACE MAIN ONLINE;
```

3.2.5.2 表还原

3.2.5.2.1 概述

表还原与表空间还原不同，不需要配置归档，DIsql 中输入以下简单的 RESTORE 语句就可还原表：

```
SQL> RESTORE TABLE TAB_01 FROM BACKUPSET 'tab_bak_01';
```

语法如下：

```
RESTORE TABLE [<表名>][STRUCT]
```

```
[WITH INDEX | WITH INDEX DATA | WITHOUT INDEX] [WITH CONSTRAINT|WITHOUT CONSTRAINT]
FROM BACKUPSET'<备份集路径>' [DEVICE TYPE <介质类型> [PARMS '<介质参数>']]
[IDENTIFIED BY <密码>] [ENCRYPT WITH <加密算法>]
[TRACE FILE '<TRACE 文件名>'] [TRACE LEVEL <TRACE 日志级别>];
```

TABLE：需要还原的表名称。

STRUCT：执行表结构还原，若未指定，则认为是表中数据还原；表数据还原要求还原目标表结构与备份集中完全一致，否则报错，所以表结构还原可以在表数据还原之前执行，减少报错。

WITH INDEX/WITHOUT INDEX：指定还原数据后是否重建二级索引，默认重建。

WITH INDEX DATA：还原索引数据。指定 WITH INDEX DATA 执行数据还原时，会将备份集中备份的二级索引数据还原到目标表中相应的二级索引中，执行还原前，待还原的二级索引必须在还原目标表中存在。该二级索引可以是用户创建索引也可以是由于创建约束而系统自动创建的；若还原数据时，指定 WITH INDEX DATA，那么还原表结构时也需要指定。指定 WITH INDEX DATA 的表结构还原时，则会在目标表中将待还原二级索引数据对应的二级索引重建。指定 WITH INDEX DATA 时，默认 WITH INDEX 有效。

WITH CONSTRAINT/WITHOUT CONSTRAINT：指定还原数据后是否重建约束，默认重建。

BACKUPSET:表空间备份时指定的备份集路径。若指定为相对路径，会在默认备份目录下搜索备份集。

DEVICE TYPE:指存储备份集的介质类型，暂支持 DISK 和 TAPE，默认 DISK。DISK 表示备份集存储介质为磁盘，TAPE 表示备份集存储介质为磁带。

PARMS:介质参数，只对介质类型为 TAPE 时有效。

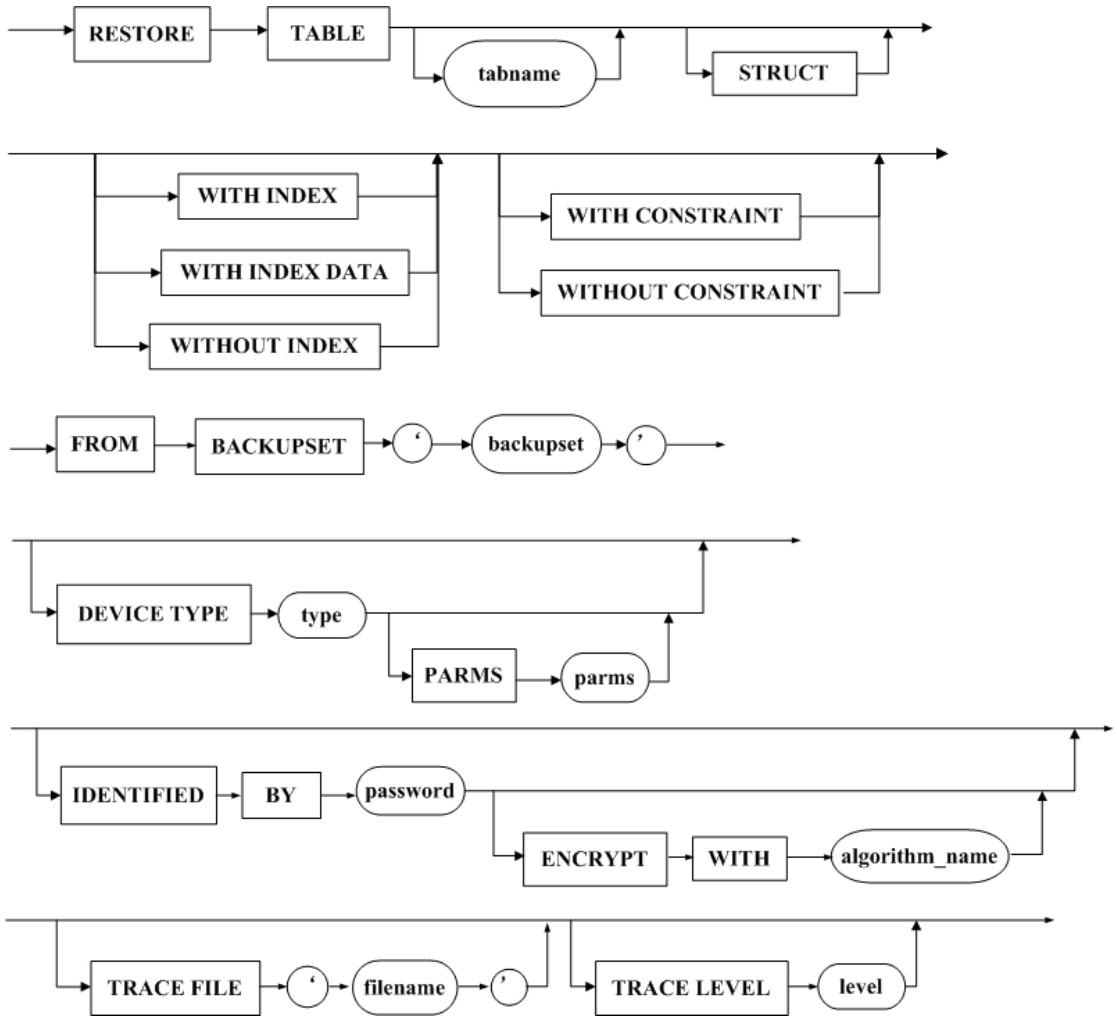
IDENTIFIED BY:加密备份表时，用户设置的密码。

ENCRYPT WITH:加密算法。缺省情况下，算法为 AES256_CFB。

TRACE FILE:指定生成的 TRACE 文件名。

TRACE LEVEL:有效值 1、2，默认为 1 表示不启用 TRACE，为 2 表示启用 TRACE。

图例:



使用说明:

1. 仅支持对普通用户表进行还原，包括堆表。其中，系统表、临时表、物化视图表、物化视图附属表和日志表、特定模式（DBG_PKG/INFORMATION_SCHEMA/INFO_SCHEM/SYSREP/SYSGEO/SYSJOB/SYSCPT/SYS）下表，以及使用 LOB_REC 存储的表不支持还原。表列类型为对象类型的表也不支持表还原。若还原表中存在位图连接索引和位图连接虚索引也不支持还原。

2. 备份集路径指备份集所在目录，其中应包含完整备份数据，包括元数据文件(.meta)和备份片文件(.bak)。仅支持从表备份集中还原表。

3. 表名设置为可选参数。若指定，则表必须存在且表定义必须与备份表严格一致；若不指定，则使用备份集中记录的备份表作为还原目标表。

4. 若指定 WITHOUT INDEX，则还原后目标表中无索引信息，即使目标表本来存在，也会被删除；若未指定，则在数据还原后重新创建备份集中索引，目标表原来的索引信息会被删除。

5. 若指定 WITHOUT CONSTRAINT，则还原后目标表中无约束信息，聚集主键除外，即使目标表本来存在，也会被删除；若未指定，则在数据还原后重新创建备份集中约束，目标表原来的约束信息会被删除。

6. 目标表所在的表空间必须处于联机状态。

7. 数据守护环境下，主库允许表备份还原，备库不允许。

8. MOUNT 和 SUSPEND 状态下不允许进行表还原。

9. MPP 和 RAC 环境均不允许进行表还原。

10. 若在语句中指定 STRUCT 关键字，则执行表结构还原。表结构还原会根据备份集中备份表还原要求，对目标表定义进行校验，并删除目标表中已存在的二级索引和约束；若未指定 WITHOUT CONSTRAINT，则会将备份集中的 CHECK 约束在目标表上重建；若指定 WITH INDEX DATA，会将待还原数据的二级索引在目标表上重建。

11. 不指定 STRUCT 关键字，则执行表数据还原，表数据还原默认仅会将备份表中聚集索引上数据进行还原，若指定 WITH INDEX DATA，则会将备份时备份的二级索引数据也进行还原。表数据还原默认仅会在目标表定义与备份表一致且不存在二级索引和约束的情况下执行；若还原时未指定 WITHOUT CONSTRAINT，也允许目标表中存在与备份表中相同的 CHECK 约束。若指定 WITH INDEX DATA，则要求目标表中必须存在于备份集中待还原数据二级索引定义完全一样的二级索引。

12. 若在未指定 STRUCT 的情况下，执行还原出现存在二级索引或冗余约束的错误；或者不指定目标表时，报目标不存在的情况，可先执行 STRUCT 还原后，再继续执行实际数据的还原。

13. 数据还原（非 STRUCT）时指定 WITH INDEX DATA，会将备份集中备份的二级索引数据还原到目标索引中，不考虑该二级索引是用户创建还是由于创建约束而系统自动创建的；结构还原（STRUCT）时，若指定 WITH INDEX DATA，则会在目标表中将待还原二级索引数据对应的二级索引重建。

14. WITH /WITHOUT INDEX 用于指定还原数据后是否重建二级索引，默认重建；WITH INDEX DATA 是指使用备份集中备份的二级索引还原相应的二级索引，不指定时直接在聚集索引数据基础上重建；WITH/WITHOUT CONSTRAINT 指定还原数据后是否在目标表上重建约束，默认重建。对于备份表中存在的无效索引和约束，仅备份创建语句，还原时并不创建，用户如有需要，可通过相关动态视图或 DMRMAN 工具查看语句后手动创建。但是若指定 WITH INDEX DATA，且备份时备份了无效索引，那么还原时会将备份的无效索引在目

标表中还原，且状态仍处于无效状态。

3.2.5.2.2 表还原

执行表还原，数据库必须处于 OPEN 状态，MOUNT 和 SUSPEND 状态下不允许执行表还原。与表空间还原不同，表还原不需要配置归档。表还原是联机完全备份还原，不需要借助本地归档日志，所以还原后不需要恢复。完整的表备份还原步骤如下：

1) 保证数据库为 OPEN 状态。

2) 创建待备份的表。

```
SQL>CREATE TABLE TAB_FOR_RES_01(C1 INT);
```

3) 备份表数据。

```
SQL>BACKUP TABLE TAB_FOR_RES_01 BACKUPSET '/home/dm_bak/tab_bak_for_res_01';
```

4) 校验备份。此步骤为可选。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/tab_bak_for_res_01');
```

5) 还原表数据。

```
SQL>RESTORE TABLE TAB_FOR_RES FROM BACKUPSET '/home/dm_bak/tab_bak_for_res_01';
```

表还原实质是表内数据的还原，以及索引和约束等的重建。如果备份文件与目标表中都包含索引或约束该如何还原呢？下面以表中包含索引为例说明如何还原表，具体步骤如下：

1) 保证数据库为 OPEN 状态。

2) 创建待备份的表。

```
SQL>CREATE TABLE TAB_FOR_RES_02(C1 INT);
```

3) 创建索引。

```
SQL>CREATE INDEX I_TAB_FOR_RES_02 ON TAB_FOR_RES_02(C1);
```

4) 备份表。

```
SQL>BACKUP TABLE TAB_FOR_RES_02 BACKUPSET '/home/dm_bak/tab_bak_for_res_02';
```

5) 校验备份。此步骤为可选。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/tab_bak_for_res_02');
```

6) 执行表结构还原。表备份和表中都包含索引，如果直接执行表数据还原会报错：还原表中存在二级索引或冗余约束。

```
SQL>RESTORE TABLE TAB_FOR_RES_02 STRUCT FROM BACKUPSET
```

```
'/home/dm_bak/tab_bak_for_res_02';
```

7) 执行表数据还原。

```
SQL>RESTORE TABLE TAB_FOR_RES_02 FROM BACKUPSET
'/home/dm_bak/tab_bak_for_res_02';
```

3.2.6 数据还原高级主题

本节介绍使用 DIsql 工具联机数据还原的高级过程，DIsql 工具仅支持对表空间及表进行还原。

本节主要内容包括：

- 表空间还原
- 表还原

3.2.6.1 表空间还原

前面的 3.2.5.1 章节已经对表空间的最基本最常用的还原场景进行了描述，本节将介绍一些不常用且较复杂的还原场景。

本节主要内容包括：

- 指定归档目录还原
- 还原表空间中指定的数据文件
- 指定映射文件还原

指定归档目录还原

由于磁盘空间的影响，服务器归档可能出现分布在多个目录的情况。出现这种情况时就需要指定归档目录还原。还原时指定多个归档目录的操作步骤如下：

1) 备份用户表空间 MAIN。

```
SQL> BACKUP TABLESPACE MAIN BACKUPSET '/home/dm_bak/ts_bak_for_arch';
```

2) 校验备份。此步骤可选。

```
SQL>SELECT SF_RMAN_BAKSET_CHECK('DISK','/home/dm_bak/ts_bak_for_arch');
```

3) 还原用户表空间 MAIN。假设归档日志分布在目录为/home/dm_arch1 和 /home/dm_arch2 两个目录下。

```
SQL> ALTER TABLESPACE MAIN OFFLINE;
```

```
SQL>RESTORE TABLESPACE MAIN FROM BACKUPSET '/home/dm_bak/ts_bak_for_arch' WITH
ARCHIVEDIR'/home/dm_arch1', '/home/dm_arch2';
SQL>ALTER TABLESPACE MAIN ONLINE;
```

还原表空间中指定的数据文件

DM7 不仅支持从数据库备份和表空间备份中还原表空间，还支持还原表空间中特定的数据文件。若表空间已经被破坏，则不允许执行特定数据文件的还原。使用数据库备份还原表空间或表空间中的数据文件与使用表空间备份操作类似，区别在于 RESTORE 语句中指定的备份集为数据库备份集。因此，本节仅以使用表空间备份为例说明还原数据文件的操作步骤。

1) 创建待备份的表空间 TS_FOR_RES_01，并在库目录下创建 3 个数据文件。

```
SQL> CREATE TABLESPACE TS_FOR_RES_01 DATAFILE'ts_for_res_01_01.dbf' SIZE 128;
SQL> ALTER TABLESPACE TS_FOR_RES_01 ADD DATAFILE'ts_for_res_01_02.dbf' SIZE 128;
SQL> ALTER TABLESPACE TS_FOR_RES_01 ADD DATAFILE'ts_for_res_01_03.dbf' SIZE 128;
```

2) 备份表空间。

```
SQL> BACKUP TABLESPACE TS_FOR_RES_01 BACKUPSET '/home/dm_bak/ts_bak_for_dbf';
```

3) 查询数据文件的文件编号或路径。还原指定的数据文件需知道数据文件对应的文件编号或路径，相关信息可通过查询动态视图 V$DATAFILE 获取。

```
SQL> SELECT ID, PATH FROM V$DATAFILE;
```

查询结果：

```
行号      ID          PATH
--------- ----------- -----------------------------------
1         0           /home/xm/DAMENG/SYSTEM.DBF
2         0           /home/xm/DAMENG/ROLL.DBF
3         0           /home/xm/DAMENG/MAIN.DBF
4         0           /home/xm/DAMENG/TEMP.DBF
5         0           /home/xm/DAMENG/ts_for_res_01_01.dbf
6         1           /home/xm/DAMENG/ts_for_res_01_02.dbf
7         2           /home/xm/DAMENG/ts_for_res_01_03.dbf
```

如果想还原 ts_for_res_01_02.dbf 和 ts_for_res_01_03.dbf 两个数据文件，通过查询结果可知它们的文件编号（1 和 2）以及相应的路径。

4) 校验备份。此步骤为可选。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/ts_bak_for_dbf');
```

5) 通过文件编号还原表空间 TS_FOR_RES_01 中的数据文件 ts_for_res_01_02.dbf 和 ts_for_res_01_03.dbf。

```
SQL> ALTER TABLESPACE TS_FOR_RES_01 OFFLINE;
SQL>RESTORE TABLESPACE TS_FOR_RES_01 DATAFILE 1, 2 FROM BACKUPSET
'/home/dm_bak/ts_bak_for_dbf';
SQL>ALTER TABLESPACE TS_FOR_RES_01 ONLINE;
```

如果不想使用文件编号还原，使用指定数据文件路径还原数据文件的语句如下：

```
SQL>RESTORE TABLESPACE TS_FOR_RES_01 DATAFILE
'/home/xm/DAMENG/ts_for_res_01_02.dbf',
'/home/xm/DAMENG/ts_for_res_01_03.dbf' FROM BACKUPSET
'/home/dm_bak/ts_bak_for_dbf';
```

指定映射文件还原

映射文件用于指定存放还原目标路径，即备份集里面的数据文件路径。指定映射文件还原可以重新指定备份集中数据文件的路径。下面以 MAIN 表空间为例说明如何使用映射文件还原。

1) 备份 MAIN 表空间。

```
SQL> BACKUP TABLESPACE MAIN BACKUPSET '/home/dm_bak/ts_bak_for_map';
```

2) 使用 DMRMAN 的 DUMP 命令生成映射文件 map_file.txt，存至 /home/dm_mappedfile 目录。文件中指定数据文件 MAIN.DBF 还原后的路径为 /home/dm_dbf/MAIN.DBF。关于映射文件的详细内容参见下文的3.3.5.2.1 数据库还原。

```
/**============================================================**/
/*[DAMENG_MAIN_FIL_0]*/
fil_id        = 0
ts_id         = 4
ts_name       = MAIN
data_path     = /home/dm_dbf/MAIN.DBF
/***************************** END ****************************/
```

3) 还原 MAIN 表空间。

```
SQL> ALTER TABLESPACE MAIN OFFLINE;
```

```
SQL> RESTORE TABLESPACE MAIN FROM BACKUPSET '/home/dm_bak/ts_bak_for_map' MAPPED
FILE'/home/dm_mappedfile/map_file.txt';
SQL> ALTER TABLESPACE MAIN ONLINE;
```

3.2.6.2 表还原

前面的 3.2.5.2 表还原 已经介绍了表备份还原的基本常用操作，本节将介绍一些表还原的高级过程。主要内容包括：

- 指定还原时不重建索引
- 指定还原时不重建约束
- 指定还原时还原二级索引数据

指定还原时不重建索引

表备份时会默认备份表中的索引,还原时使用RESTORE TABLE...WITHOUT INDEX...语句可选择不还原索引。完整示例如下：

1) 保证数据库为 OPEN 状态。

2) 准备数据。创建待备份的表及索引。

```
SQL>CREATE TABLE TAB_FOR_IDX_01(C1 INT);
SQL>CREATE INDEX I_TAB_FOR_IDX_01 ON TAB_FOR_IDX_01 (C1 INT);
```

3) 备份表数据。

```
SQL>BACKUP TABLE TAB_FOR_IDX_01 BACKUPSET '/home/dm_bak/tab_bak_for_res_01';
```

4) 校验备份。此步骤为可选。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/tab_bak_for_res_01');
```

5) 还原表数据，但重建索引。

```
SQL>RESTORE TABLE TAB_FOR_RES FROM BACKUPSET '/home/dm_bak/tab_bak_for_res_01';
```

指定还原时不重建约束

表备份时会默认备份表中的索引定义，还原时使用 RESTORE TABLE...WITHOUT CONSTRAINT...语句可选择还原时不重建约束。完整示例如下：

1) 保证数据库为 OPEN 状态。

2) 准备数据。创建待备份的表及索引。

```
SQL>CREATE TABLE TAB_FOR_CONS_01(C1 INT);
```

```
SQL>CREATE INDEX I_TAB_FOR_CONS_01 ON TAB_FOR_CONS_01 (C1 INT);
```

3) 备份表数据。

```
SQL>BACKUP TABLE TAB_FOR_CONS_01 BACKUPSET '/home/dm_bak/tab_bak_for_res_01';
```

4) 校验备份。此步骤为可选。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/tab_bak_for_res_01');
```

5) 还原表数据，但不还原约束。

```
SQL>RESTORE TABLE TAB_FOR_CONS_01 WITHOUT CONSTRAINT FROM BACKUPSET
'/home/dm_bak/tab_bak_for_res_01';
```

指定还原时还原二级索引数据

表备份时会默认备份表中的索引定义，但不会备份索引数据。备份时，指定 WITH INDEX，也会备份二级索引数据。还原时使用 RESTORE TABLE...WITH INDEX DATA...语句可选择备份集中备份的二级索引数据进行还原。完整示例如下：

6) 保证数据库为 OPEN 状态。

7) 准备数据。创建待备份的表及索引。

```
SQL>CREATE TABLE TAB_FOR_IDX_DATA_01(C1 INT);
SQL>CREATE INDEX I_TAB_FOR_IDX_DATA_01 ON TAB_FOR_IDX_DATA_01 (C1 INT);
SQL>INSERT INTO TAB_FOR_IDX_DATA_01 VALUES(100);
```

8) 备份表数据，包括二级索引数据。

```
SQL>BACKUP TABLE TAB_FOR_IDX_DATA_01 WITH INDEX BACKUPSET
'/home/dm_bak/tab_bak_for_res_01';
```

9) 校验备份。此步骤为可选。

```
SQL>SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/tab_bak_for_res_01');
```

10) 还原表结构，包括重建待还原二级索引

```
SQL>RESTORE TABLE TAB_FOR_IDX_DATA_01_STRUCT WITH INDEX DATA FROM BACKUPSET
'/home/dm_bak/tab_bak_for_res_01';
```

11) 还原表数据

```
SQL>RESTORE TABLE TAB_FOR_IDX_DATA_01 WITHOUT WITH INDEX DATA FROM BACKUPSET
'/home/dm_bak/tab_bak_for_res_01';
```

3.3 使用脱机工具 DMRMAN 进行备份还原

DMRMAN（DM RECOVERY MANEGER）是 DM7 的脱机备份还原管理工具，由它来统一负责库级脱机备份、脱机还原、数据库恢复等相关操作，该工具支持命令行指定参数方式和控制台交互方式执行，降低了用户的操作难度。

通过 DMRMAN 工具执行脱机操作过程中，仅会使用通过关键字 DATABASE 指定的目标库的本地归档配置信息，不会对本地归档配置文件中其他类型归档配置信息进行校验。例如：DATABASE 指定目标库为/opt/dmdbms/data/dm.ini，为单机环境且配置参数 ARCH_INI 为 1，那么只会相应的读取 dmarch.ini 中本地归档日志信息。此时，即使 dmarch.ini 中配置了 REALTIME、MARCH、TIMELY 等其他类型的归档日志，也不会去校验 dmmal.ini、dmtimer.ini 等是否有效。

本章节内容主要包括：

- DMRMAN 概述
- 启动和配置 DMRMAN
- 数据备份
- 管理备份
- 数据库还原和恢复
- 查看操作日志

3.3.1 DMRMAN 概述

DMRMAN 是 DM7 提供的命令行工具，无需额外地安装。DMRMAN 的结构比较复杂，为了更好地使用它我们需要了解它的结构。接下来将对 DMRMAN 的重要组成部分进行介绍。

源库

源库是待备份的数据库，即是需要数据库管理员重点维护的数据库。使用 DMRMAN 工具只可对数据库进行脱机备份，联机备份需要通过 DIsql 工具实现。DMRMAN 可以针对整个数据库执行脱机完全备份和增量备份，数据库可以配置归档也可以不配置。

目标库

目标库是待还原的数据库，即是用来做还原的库，也称为目标还原库。目标库必须是新初始化或者已使用过得数据库，但必须处于关闭状态。目前目标库的还原仅支持脱机还原，

可通过 DMRMAN 和 CONSOLE 工具实现。

DMRMAN 客户端

DMRMAN 客户端是 DM7 软件的一部分，用来执行目标数据库的备份和恢复操作。DMRMAN 客户端是一个命令行工具，命令行的好处是管理员可以编写复杂的脚本，将 DMRMAN 和操作系统的任务调度结合起来可以实现备份的自动化。

备份集

利用 DMRMAN 工具将数据库中的一个或多个数据文件、数据库信息等备份到一个称为“备份集”的逻辑结构中，备份集的格式是特定的只能由 DMRMAN 创建和访问。一个备份集中会包含一个或多个二进制文件，这个二进制文件被称为备份片。备份数据库会产生一个备份集，这个备份集可能包含一个或者多个备份片。备份片的个数由数据库中数据文件的大小和文件系统对文件大小的限制共同决定。如果备份的数据文件大小超过了文件系统对文件大小的限制就会产生多个备份片。

在对数据库进行备份时可以指定备份片的大小来适应存储设备的容量。例如，一个数据库的产生的备份集大小约为 40GB，一盘磁带的容量为 10GB，这时可以指定备份片大小为 10G，备份后备份集将包含 4 个备份片，正好使用 4 盘磁带。

3.3.2 启动和配置 DMRMAN

本节介绍如何启动命令行工具 DMRMAN 并进行交互。主要内容包括：

- 启动和退出 DMRMAN
- 输入 DMRMAN 命令
- DMRMAN 环境配置

3.3.2.1 启动和退出 DMRMAN

安装 DM 数据库后，DMRMAN 可执行程序与数据库其他可执行程序一样位于安装路径的执行码目录下。比如，LINUX 上数据库的执行码目录为/opt/dmdbms/bin，转到执行码目录直接在操作系统的命令行中输入以下命令就可启动 DMRMAN：

```
./dmrman
```

启动后控制台中输入 exit 命令即可退出 DMRMAN 环境：

```
RMAN>exit
```

3.3.2.2 输入 DMRMAN 命令

DMRMAN 工具支持控制台输入命令、命令行设置参数两种操作方式。下面分别对这两种操作方式进行介绍。

DMRMAN 控制台输入命令

当 DMRMAN 客户端已经启动且可以输入命令时会显示命令提示符。如下所示：

```
RMAN>
```

输入以下用于执行的 DMRMAN 命令：

```
RMAN>HELP
RMAN>BACKUP DATABASE'/opt/dmdbms/data/DAMENG/dm.ini';
```

大多数的 DMRMAN 命令会设置多个参数，命令的结尾可以加分号结束，也可以不使用分号。

DMRMAN 控制台还可以执行脚本。如创建一个名为 cmd_file.txt 的文件，文件中包含“BACKUP DATABASE'/opt/dmdbms/data/DAMENG/dm.ini'”命令，保存到/home/dm_cmd 目录下，执行方式如下：

```
RMAN>`/home/dm_cmd/cmd_file.txt
```

DMRMAN 是按行解析命令，如果输入一行不完整的命令执行会报错，如一条完整的命令分三行输入会导致报错：

```
BACKUP
DATABASE
'/opt/dmdbms/data/DAMENG/dm.ini';
```

DMRMAN 命令行设置参数执行

DMRMAN 命令行设置参数执行又可分为命令行指定脚本、命令行指定语句两种执行方式。DMRMAN 支持的参数如表 3.3.1 所示。

表 3.3.1 DMRMAN 参数

参数	含义	备注
CTLFILE	指定执行语句所在的文件路径。脚本文件格式支持*.txt	可选

		参数
CTLSTMT	指定待执行语句。如： CTLSTMT="BACKUP DATABASE '/home/dmdbms/data/DAMENG/dm.ini'"	可选参数
DCR_INI	指定 dmdcr.ini 路径；若未指定且当前目录中 dmdcr.ini 存在，则使用当前目录中的 dmdcr.ini。可单独使用，也可与其他参数配合使用。主要用于 rac 环境	可选参数
HELP	打印帮助信息	可选参数

命令行指定脚本执行，要求我们先创建一个包含 DMRMAN 命令的文件，然后设置 CTLFILE 参数，参数后面指定文件的路径。例如，创建一个名为 cmd_rman.txt 的文件，保存到/home/dm_cmd 目录下，文件中包含一行如下所示的 DMRMAN 命令：

```
BACKUP DATABASE'/opt/dmdbms/data/DAMENG/dm.ini';
```

设置 CTLFILE 参数启动 DMRMAN 工具，指定脚本文件中的命令将被执行：

```
./dmrman CTLFILE='/home/dm_cmd/cmd_rman.txt'
```

命令执行完后 DMRMAN 工具会自动退出。

命令行执行语句执行，是 DMRMAN 工具设置 CTLSTMT 参数，参数后面执行需要执行的 DMRMAN 命令，使用方法如下所示：

```
./dmrman  CTLSTMT="BACKUP DATABASE'/opt/dmdbms/data/DAMENG/dm.ini';"
```



注意：CTLFILE 中存在多条语句或 CTLSTMT 中一次输入多条语句时，每条语句都必须以分号结尾；执行单条语句，语句结尾可以加分号也可以不加。

CTLSTMT 中，待执行语句外层双引号一定要加。执行完毕后工具自动退出。

3.3.2.3 DMRMAN 环境配置

使用 CONFIGURE 命令进行 DMRMAN 的默认参数配置，配置默认的存储介质类型、备份集搜集目录、归档日志搜集目录、跟踪日志文件。

- 显示和清除现有参数的默认配置
- 配置存储介质类型：DISK 或 TAPE
- 配置备份集搜集目录
- 配置归档日志搜集目录
- 配置跟踪日志文件

语法如下：

```
CONFIGURE |
CONFIGURE CLEAR |
CONFIGURE DEFAULT <sub_conf_stmt>
<sub_conf_stmt>::=
DEVICE [[TYPE<介质类型> [PARMS <第三方参数>]]|CLEAR] |
TRACE [[FILE <跟踪日志文件路径>][TRACE LEVEL <跟踪日志等级>]|CLEAR] |
BACKUPDIR [[ADD|DELETE] '<基备份搜索目录>'{,'<基备份搜索目录>' }|CLEAR] |
ARCHIVEDIR [[ADD|DELETE] '<归档日志目录>'{,'<归档日志目录>'}
{'<归档日志目录>'{,'<归档日志目录>'} }|CLEAR]
```

CONFIGURE：查看设置的默认值。

CLEAR：清理参数的默认值。

DEVICE TYPE：备份集存储的介质类型，DISK 或者 TAPE，默认 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

PARMS：介质参数，供第三方存储介质管理使用。

BACKUPDIR：默认搜集备份的目录，可以设置为不存在但是在系统中是有效的路径。

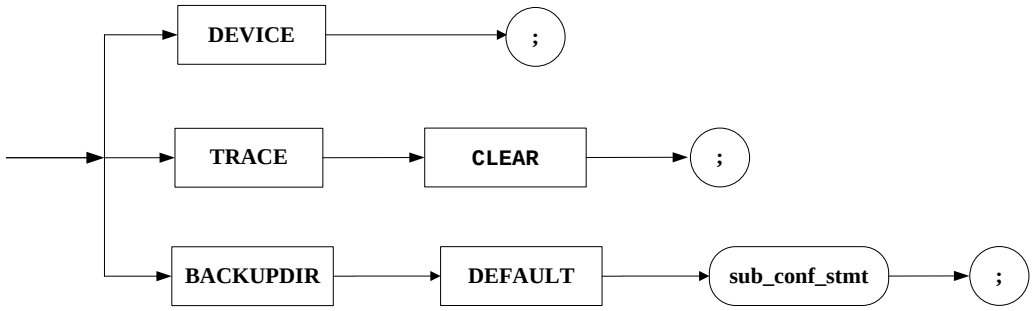
ARCHIVEDIR ：默认搜集归档的目录，可以设置为不存在但是在系统中是有效的路径。以站点为单位进行增加或删除；配置或增加时，若未指定站点号，则默认为当前站点。站点号只用于 RAC 环境，取值为 0-15，目前有效的为 0 和 1。各实例对应各 RAC 站点的站点编号，可在 V$INSTANCE 查看 RAC_SEQNO。

ADD：添加默认备份集搜索目录或归档日志目录，若已经存在，则替换原来的。

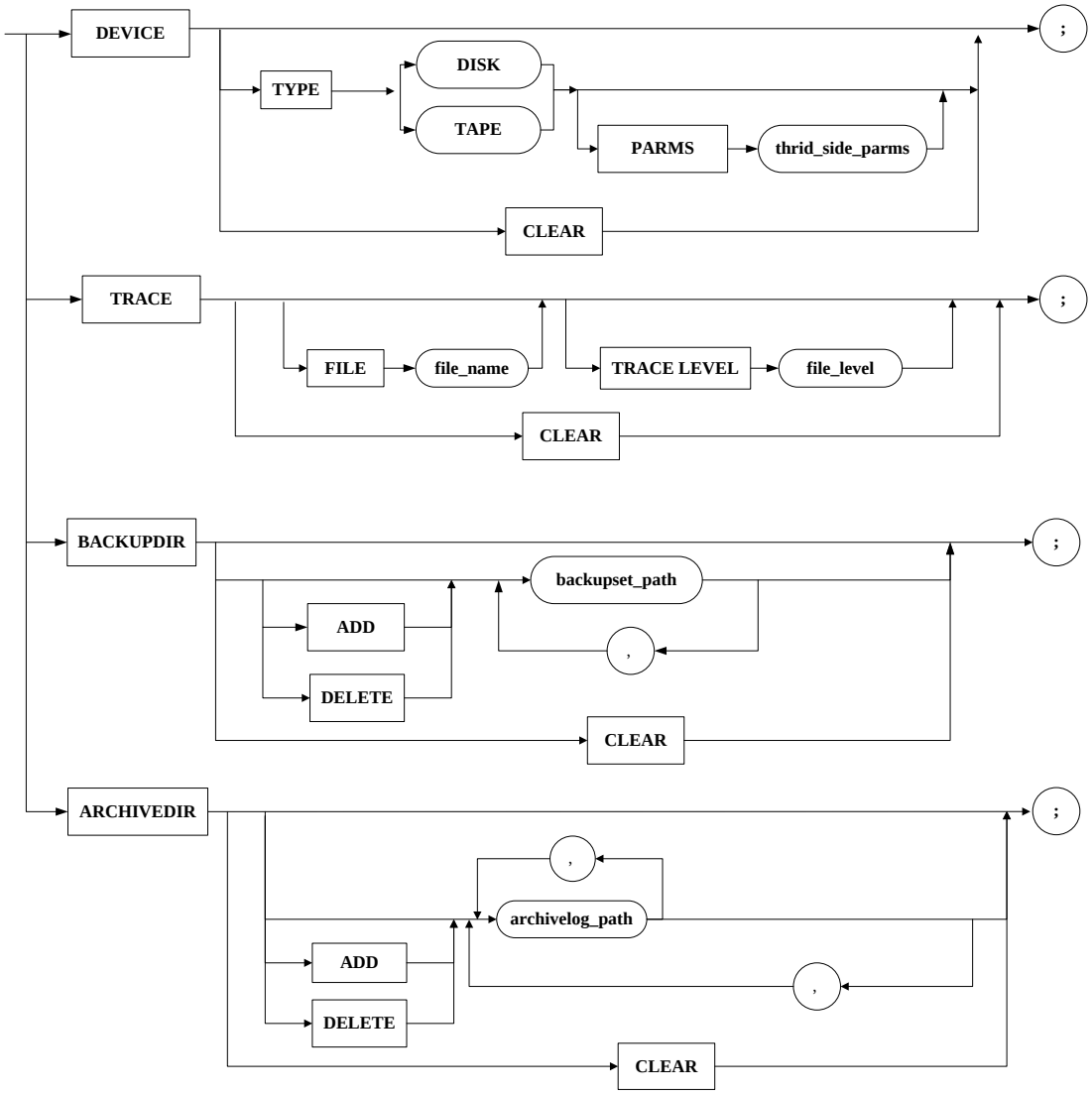
DELETE：删除指定默认备份集搜索目录或者归档日志目录。

TRACE：介质存储过程中使用的跟踪日志配置，包括文件路径（TRACE FILE）和日志级别（TRACE LEVEL），其中日志级别有效值（1/2，不写/写；默认为 1，不写）。

图例：



sub_conf_stmt：



使用说明：

1. 设置的参数默认值仅在此 DMRMAN 实例存活期间有效。

2. DMRMAN 命令中如果指定了相同的参数，会覆盖 CONFIGURE 的默认设置。

显示和清除所有默认配置项

使用 CONFIGURE 命令就可显示 DMRMAN 配置项的当前值。示例如下：

```
RMAN>CONFIGURE;

THE DMRMAN DEFAULT SETTING:


DEFAULT DEVICE:
          MEDIA : DISK
DEFAULT TRACE :
          FILE  :
          LEVEL : 1
DEFAULT BACKUP DIRECTORY:
          TOTAL COUNT  :0
DEFAULT ARCHIVE DIRECTORY:
          TOTAL COUNT  :0


RMAN>CONFIGURE CLEAR;
```

使用 CONFIGURE...CLEAR 命令可恢复任意一个配置项到默认值。示例如下：

```
RMAN>CONFIGURE DEFAULT DEVICE CLEAR;
```

或使用 CONFIGURE...CLEAR 命令清除所有的默认配置。示例如下：

```
RMAN>CONFIGURE CLEAR;
```

配置存储介质类型：DISK 或 TAPE

备份时如果没指定备份介质类型参数，则会使用 CONFIGURE 中配置的默认介质类型。DMRMAN 预先配置的介质类型为 DISK，不需要特别指定。

修改介质类型默认配置的步骤如下：

1) 启动 DMRMAN。

2) 执行 CONFIGURE 命令查看介质类型当前的默认值。

```
RMAN>CONFIGURE;
```

3) 执行 CONFIGURE DEFAULT DEVICE TYPE 修改默认值。

```
RMAN>CONFIGURE DEFAULT DEVICE TYPE TAPE PARMS 'command';
```

配置备份集搜集目录

备份集搜索目录用于增量备份还原中搜索基备份。单个目录最大长度为 256 个字节，可配置的备份集搜索目录没有限制。如果不指定备份集搜集目录只会在库的默认备份目录和当前备份执行备份集目录的上级目录下搜索备份集。

配置备份集搜索目录步骤如下：

```
RMAN>CONFIGURE DEFAULT BACKUPDIR;

RMAN>CONFIGURE DEFAULT BACKUPDIR '/home/dm_bak1' ,'/home/dm_bak2';
```

若要增加或删除部分备份集搜索目录，不需要对所有的目录重新进行配置，只要添加或删除指定的目录即可。

```
RMAN>CONFIGURE DEFAULT BACKUPDIR ADD '/home/dm_bak3';

RMAN>CONFIGURE DEFAULT BACKUPDIR DELETE '/home/dm_bak3';
```

配置归档日志搜集目录

归档日志搜索目录用于增量备份还原中搜索归档日志。单个目录最大长度为 256 个字节，可配置的归档日志搜索目录没有限制。

配置归档日志搜索目录步骤如下：

```
RMAN>CONFIGURE DEFAULT ARCHIVEDIR;

RMAN>CONFIGURE DEFAULT ARCHIVEDIR '/home/dm_arch1' ,'/home/dm_arch2';
```

若要增加或删除部分归档日志搜索目录，不需要对所有的目录重新进行配置，只要添加或删除指定的目录即可。

```
RMAN>CONFIGURE DEFAULT ARCHIVEDIR ADD '/home/dm_arch3';

RMAN>CONFIGURE DEFAULT ARCHIVEDIR DELETE '/home/dm_arch3';
```

配置跟踪日志文件

跟踪日志文件记录了 SBT 接口的调用过程，用户通过查看日志可跟踪备份还原过程（见 3.2.3 数据备份高级主题）。DMRMAN 备份还原命令中不支持设置跟踪日志文件，只能用 CONFIGURE 命令配置，默认配置不记录跟踪日志。

配置跟踪日志文件步骤如下：

1) 显示 TRACE 文件的默认配置。

```
RMAN>CONFIGURE DEFAULT TRACE;
```

2）配置默认 TRACE 文件。

```
RMAN>CONFIGURE DEFAULT TRACE FILE '/home/dm_trace/trace.log';
```

3）配置默认 TRACE 级别。

```
RMAN>CONFIGURE DEFAULT TRACE LEVEL 2;
```

其中，TRACE 文件和 TRACE 级别也可以同时设置：

```
RMAN>CONFIGURE DEFAULT TRACE FILE '/home/dm_trace/trace.log ' TRACE LEVEL 2;
```

3.3.3 数据备份

本节介绍使用 DMRMAN 工具如何执行数据库和归档的备份操作。

本节内容主要包括：

- 备份数据库
- 备份归档

3.3.3.1 备份数据库

3.3.3.1.1 概述

在 DMRMAN 工具中使用 BACKUP 命令你可以备份整个数据库。使用 DMRMAN 备份数据库不需要设置归档，关闭服务器在 DMRMAN 中输入以下命令即可备份数据库：

```
RMAN> BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini';
```

命令执行完后会在默认的备份路径下生成备份集目录，默认的备份路径为 dm.ini 同级目录的 bak 目录。这是最简单的脱机数据库备份语句，如果要设置其他备份选项请参考下文的语法及使用说明。

语法如下：

```
BACKUP DATABASE '<INI 文件路径>' [[[FULL][DDL_CLONE]] |INCREMENT [WITH BACKUPDIR
'<基备份搜索目录>'{,'<基备份搜索目录>'}] [BASE ON BACKUPSET '<基备份集目录>']]
[TO <备份名>] [BACKUPSET '<备份集目录>'] [DEVICE TYPE <介质类型>[PARMS '<介质参数>']
[BACKUPINFO '<备份描述>'] [MAXPIECESIZE <备份片限制大小>]
[IDENTIFIED BY <加密密码>[WITH ENCRYPTION<TYPE>][ENCRYPT WITH <加密算法>]]
```

```
[COMPRESSED [LEVEL <压缩级别>]] [WITHOUT LOG] [PARALLEL [<并行数>]];
```

DATABASE：必选参数。指定还原目标库的 INI 文件路径。

FULL：备份类型。FULL 表示完全备份，可不指定，DMRMAN 会默认为完全备份。

DDL_CLONE:数据库克隆。该参数只能用于完全备份中，表示仅拷贝所有的元数据不拷贝数据。如对于数据库中的表来说，只备份表的定义不备份表中数据。RAC 环境不支持 DDL_CLONE。

INCREMENT: 备份类型。INCREMENT 表示增量备份，若要执行增量备份必须指定该参数。

WITH BACKUPDIR:用于增量备份中，指定基备份的搜索目录，最大长度为 256 个字节。若不指定，服务器自动在默认备份目录下搜索基备份。如果基备份不在默认的备份目录下，增量备份必须指定该参数。

BASE ON BACKUPSET：用于增量备份中，为增量备份指定基备份集目录。

TO: 指定生成备份名称。若未指定，系统随机生成，默认备份名格式为：DB_备份类型_数据库名_备份时间。

BACKUPSET: 指定当前备份集生成目录。若指定为相对路径，则在默认备份路径中生成备份集。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

PARMS：只对介质类型为 TAPE 时有效。

BACKUPINFO：备份的描述信息。最大不超过 256 个字节。

MAXPIECESIZE：最大备份片文件大小上限，以 M 为单位，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

IDENTIFIED BY：指定备份时的加密密码。密码应用双引号括起来，这样避免一些特殊字符通不过语法检测。

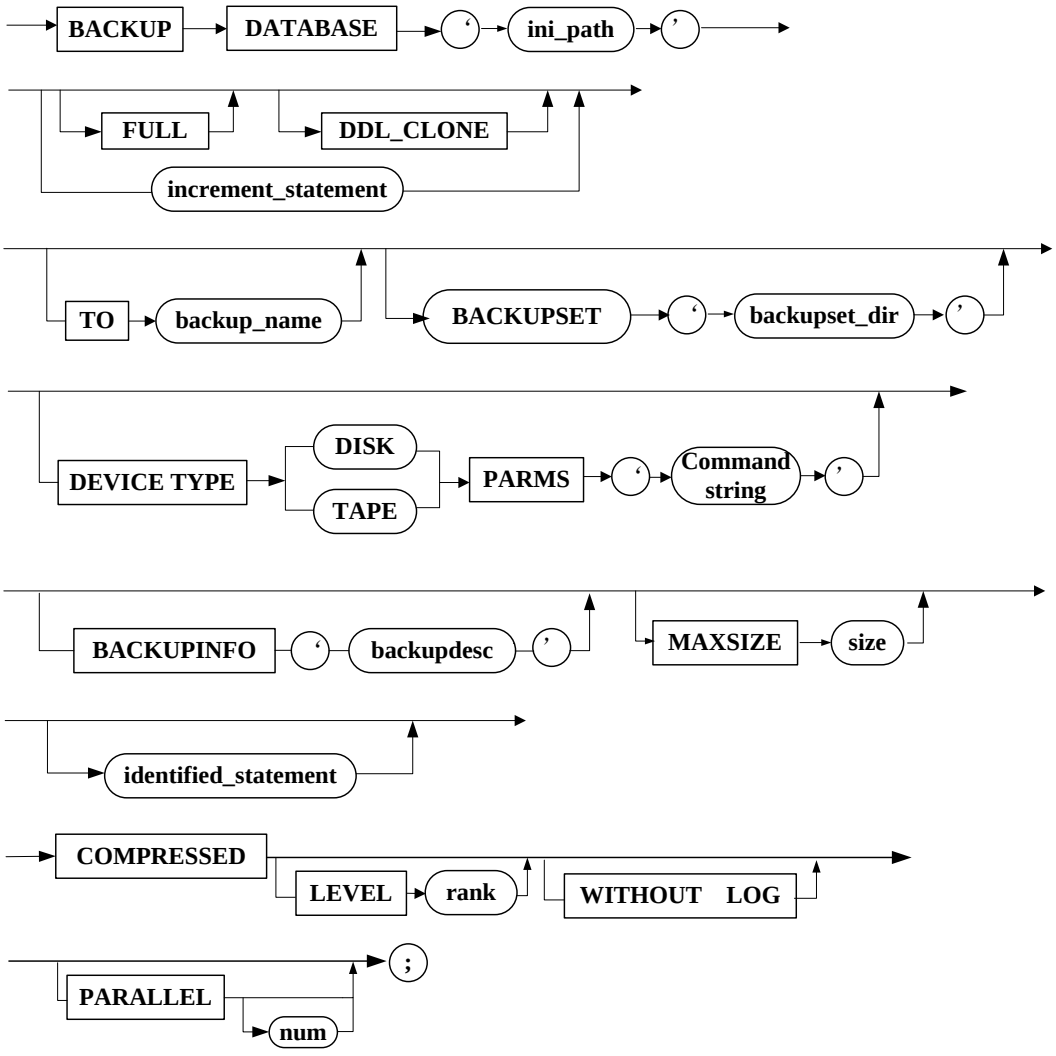
WITH ENCRYPTION：指定加密类型，0 表示不加密，不对备份文件进行加密处理；1 表示简单加密，对备份文件设置口令，但文件内容仍以明文存；2 表示完全数据加密，对备份文件进行完全的加密，备份文件以密文方式存储。

ENCRYPT WITH：加密算法。缺省情况下，算法为 AES256_CFB。具体可以使用的加密算法参考联机数据库备份章节的参数说明。

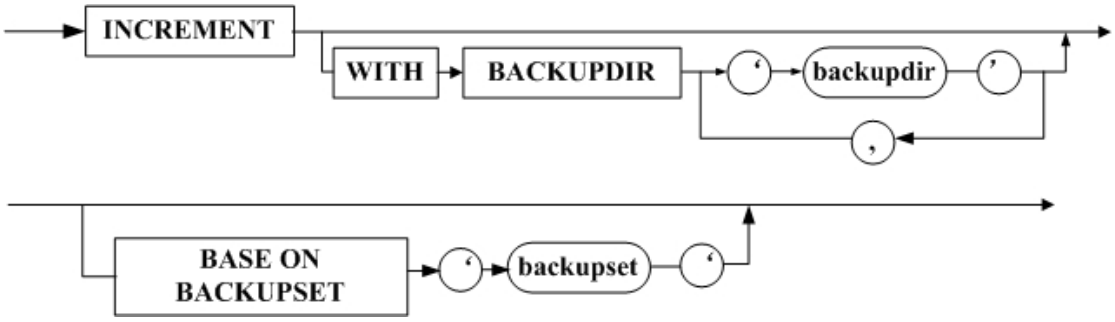
COMPRESSED：取值范围 0~9。0 表示不压缩，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。若未指定，但指定 COMPRESSED，则默认 1；否则，默认 0。

WITHOUT LOG：脱机数据库备份是否备份日志。如果使用，则表示不备份，否则表示备份。如果使用了 WITHOUT LOG 参数，则使用 DMRMAN 工具还原时，必须指定 WITH ARCHIVEDIR 参数。

图例：



increment_statement：



identified_statement：请参考 3.2.2.1.1 节数据库备份图例。

使用说明：

1．备份成功后会在<备份集目录>或者备份默认目录下生成备份集。备份集中包括一个备份元数据文件，后缀.meta，一个或多个备份片文件，后缀.bak。

2．对于并行备份的备份集，备份集中还包括其他子备份集目录，但每个子备份集目录中也都包含一个 meta 文件，0 个或者多个备份片文件。

3．DDL_CLONE 库备份集不能作为增量备份的基备份，仅能用于库级还原。

4．脱机备份的数据库可以正常退出库，也可以是故障退出的数据库。若是故障退出的数据库，则备份前，需先进行归档修复。

3.3.3.1.2 备份数据库

本节主要描述使用 DMRMAN 如何执行基本的脱机数据库备份及实施一些备份策略，包括：

- 设置备份选项
- 创建完全备份
- 创建增量备份

设置备份选项

备份命令如果仅指定了必选参数如“BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'”，那么 DMRMAN 会根据配置的环境及内置的参数默认值自动指定备份介质类型、备份路径、备份片大小等参数。用户备份时也可以指定这些参数来覆盖默认值，常见的备份选项有设置备份集路径、指定备份名、限制备份片大小、添加描述信息、并行备份等。DMRMAN 脱机备份数据库的语法与使用 DIsql 联机备份类似，关于备份选项的详细介绍及使用参见 3.2.2.1.2 设置备份选项。

创建完全备份

执行数据库备份要求数据库处于脱机状态。与联机备份数据库不同的是，脱机备份不需要配置归档。一个完整的创建脱机数据库备份的示例如下：

1) 启动 DMRMAN 命令行工具。

2) 保证数据库处于脱机状态。

3) DMRMAN 中输入以下命令：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' FULL BACKUPSET
'/home/dm_bak/db_full_bak_01';
```

命令中的 FULL 参数表示执行的备份为完全备份，也可以不指定该参数，DMRMAN 默认执行的备份类型为完全备份。

创建增量备份

增量备份指基于指定的库的某个备份（完全备份或者增量备份），备份自该备份以来所有发生修改了的数据页。脱机增量备份要求两次备份之间数据库必须有操作，否则备份会报错。关于如何制定备份策略请参考 3.2.2.1.3 小节。

增量备份示例如下：

1) 启动 DMRMAN 命令行工具。

2) 保证数据库处于脱机状态。

3) DMRMAN 中输入以下命令：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' INCREMENT WITH BACKUPDIR
'/home/dm_bak ' BACKUPSET '/home/dm_bak/db_increment_bak_02';
```

命令中的 INCREMENT 参数表示执行的备份为增量备份，增量备份该参数不可省略。如果增量备份的基备份不在默认备份目录，必须指定 WITH BACKUPDIR 参数用于搜索基备份集，或者使用 CONFIGURE...BACKUPDIR 命令配置默认的基备份集搜索目录（参考 3.3.2.3 小节）。

3.3.3.2 备份归档

3.3.3.2.1 概述

在 DMRMAN 工具中使用 BACKUP 命令你可以备份库的归档。使用 DMRMAN 备份数据库需要设置归档，否则会报错。关闭服务器在 DMRMAN 中输入以下命令即可备份数据库：

```
RMAN>BACKUP ARCHIVE LOG DATABASE '/opt/dmdbms/data/DAMENG/dm.ini';
```

命令执行完后会在默认的备份路径下生成备份集目录，默认的备份路径为 dm.ini 同级目录的 bak 目录。这是最简单的脱机归档备份语句，如果要设置其他备份选项请参考下文的语法及使用说明。

语法如下：

```
BACKUP <ARCHIVE LOG | ARCHIVELOG>
[ALL | [FROM LSN <lsn 值>]|[UNTIL LSN <lsn 值>] | [LSN BETWEEN < lsn 值> AND < lsn 值>] | [FROM TIME '时间串'] | [UNTIL TIME '时间串'] | [TIME BETWEEN '时间串' AND '时间串']] [<notBackedUpSpec>][DELETE INPUT]
DATABASE '<INI 文件路径>'
[TO <备份名>] [BACKUPSET '<备份集目录>'] [DEVICE TYPE <介质类型>[PARMS '<介质参数>']
[BACKUPINFO '<备份描述>'] [MAXPIECESIZE <备份片限制大小>]
[IDENTIFIED BY <加密密码>[WITH ENCRYPTION<TYPE>][ENCRYPT WITH <加密算法>]]
[COMPRESSED [LEVEL <压缩级别>]] [PARALLEL [<并行数>]];
<notBackedUpSpec>::=NOT BACKED UP
                | NOT BACKED UP num TIMES
                | NOT BACKED UP SINCE TIME 'datetime_string'
```

ALL：备份所有的归档；

FROM LSN，UNTIL LSN：备份的起始和截止 lsn。请参考 3.2.2.4 归档备份

FROM TIME：指定备份的开始时间点。例如，'2015-12-10'。

UNTIL TIME：指定备份的截止时间点。

BETWEEN ...AND ...：指定备份的的区间，仅仅被备份区间内的归档文件。

<notBackedUpSpec>：搜索过滤。搜索过滤仅限于根据备份指定条件能找到的所有归档备份集。1） num TIMES，指若归档文件已经备份了 num 次，则不再备份；否则备份。如 num=3，则认为已经备份了 3 次的归档文件就不再备份。若 num =0，则认为所有都不需要备份。2）SINCE TIME 'datetime_String'，指定时间开始没有备份的归档文件进行备份。3）若以上两种均未指定，则备份所有未备份过的归档日志文件。

DELETE INPUT:用于指定备份完，是否删除归档操作。

DATABASE：必选参数。指定备份目标库的 INI 文件路径。

TO：指定生成备份名称。若未指定，系统随机生成，默认备份名格式为：ARCH_备份

时间。

BACKUPSET：指定当前备份集生成目录。若指定为相对路径，则在默认备份路径中生成备份集。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

PARMS：只对介质类型为 TAPE 时有效。

BACKUPINFO：备份的描述信息。最大不超过 256 个字节。

MAXPIECESIZE：最大备份片文件大小上限，以 M 为单位，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

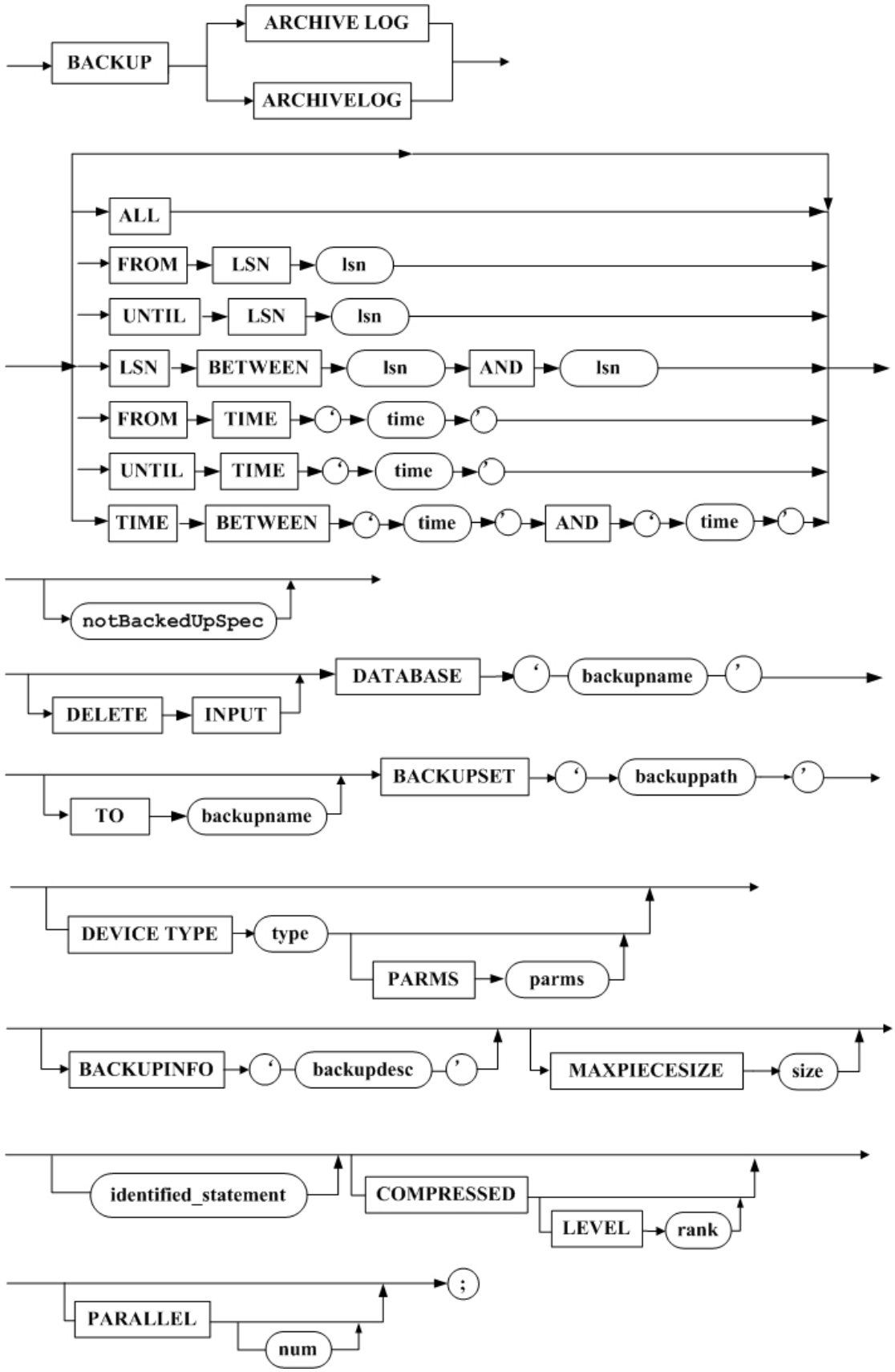
IDENTIFIED BY：指定备份时的加密密码。密码应用双引号括起来，这样避免一些特殊字符通不过语法检测。

WITH ENCRYPTION：指定加密类型，0 表示不加密，不对备份文件进行加密处理；1 表示简单加密，对备份文件设置口令，但文件内容仍以明文存；2 表示完全数据加密，对备份文件进行完全的加密，备份文件以密文方式存储。

ENCRYPT WITH：加密算法。缺省情况下，算法为 AES256_CFB。具体可以使用的加密算法参考联机数据库备份章节的参数说明。

COMPRESSED：取值范围 0~9。0 表示不压缩，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。若未指定，但指定 COMPRESSED，则默认 1；否则，默认 0。

图例



identified_statement：请参考 3.2.2.1.1 节数据库备份图例

notBackedUpSpec：请参考 3.2.2.4.1 节归档备份图例

3.3.3.2.2 备份归档

本节主要描述使用 DMRMAN 如何执行基本的脱机归档备份及实施一些备份策略，包括：

- 设置备份选项
- 创建归档备份
- 创建设置条件的归档备份

设置备份选项

备份命令如果仅指定了必选参数如“BACKUP ARCHIVE LOG DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'”，那么 DMRMAN 会根据配置的环境及内置的参数默认值自动指定备份介质类型、备份路径、备份片大小等参数。用户备份时也可以指定这些参数来覆盖默认值，常见的备份选项有设置备份集路径、指定备份名、限制备份片大小、添加描述信息、并行备份等。DMRMAN 脱机备份归档的语法与使用 DIsql 联机备份类似，关于备份选项的详细介绍及使用参见3.2.2.3.2 设置备份选项。

创建归档备份

执行归档备份要求数据库处于脱机状态。与联机备份数据库一样，脱机归档备份需要配置归档。一个完整的创建脱机数据库备份的示例如下：

1）启动 DMRMAN 命令行工具。

2）保证数据库处于脱机状态。

3）DMRMAN 中输入以下命令：

```
RMAN>BACKUP ARCHIVE LOG ALL DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'BACKUPSET
'/home/dm_bak/arch_all_bak_01';
```

命令中的 ALL 参数表示执行的备份为备份所有的归档，也可以不指定该参数，DMRMAN 默认执行的备份类型为 ALL 类型归档备份。

创建设置条件的归档备份

设置条件的归档备份指通过设置 LSN 或者时间点，控制归档需要备份归档的范围。脱机设置条件归档备份。

增量备份示例如下：

1） 启动 DMRMAN 命令行工具。

2） 保证数据库处于脱机状态。

3） DMRMAN 中输入以下命令：

```
RMAN>BACKUP ARCHIVE LOG LSN BETWEEN 50000 AND 120000 DATABASE
'/opt/dmdbms/data/DAMENG/dm.ini'BACKUPSET '/home/dm_bak/db_increment_bak_02';
```

命令中的 LSN BETEEWN start AND end 参数表示执行的备份为执行备份的区间，或者设置 FROM LSN(TIME)以及 UNTIL LSN(TIME)。

3.3.3.3 数据备份高级主题

本节介绍使用 DMRMAN 工具执行高级的数据库脱机备份过程，主要内容包括：

- 加密备份
- 设置跟踪日志文件

加密备份

DMRMAN 同 DIsql 工具一样可使用加密的方式备份数据库，没有权限的用户无法访问加密的备份集。

DMRMAN 备份命令中通过指定 IDENTIFIED BY...WITH ENCRYPTION...ENCRYPT WITH...命令执行加密备份。其中，加密备份相关参数的含义与联机备份中的含义相同（参考 3.2.3 小节）。

加密备份过程中参数 IDENTIFIED BY 必须指定，参数 WITH ENCRYPTION 和参数 ENCRYPT WITH 可不指定。加密备份时不指定 WITH ENCRYPTION 参数，该参数默认为 1，不指定 ENCRYPT WITH 参数，该参数默认值为 AES256_CFB。

例如，以下两种加密备份命令都是合法的：

```
RMAN> BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET
'/home/dm_bak/db_bak_for_encrypt_01' IDENTIFIED BY "cdb546";
RMAN> BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET
'/home/dm_bak/db_bak_for_encrypt_02' IDENTIFIED BY "cdb546" ENCRYPT WITH RC4;
```

若指定了加密密码，但加密类型 WITH ENCRYPTION 参数指定为 0，则为非加密备份，如下所示：

```
RMAN> BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET
'/home/dm_bak/db_bak_for_encrypt_03' IDENTIFIED BY "cdb546" WITH ENCRYPTION 0;
```

下面以数据库完全备份为例，创建加密密码为“cdb546”，加密算法为“rc4”的复杂加密类型的数据库加密备份，完整步骤如下：

1) 保证数据库处于关闭状态。

2) 备份数据库。启动 DMRMAN 工具并输入以下命令。

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET
'/home/dm_bak/db_bak_for_encrypt_04' IDENTIFIED BY "cdb546" WITH ENCRYPTION 2
ENCRYPT WITH RC4;
```

对于增量备份加密，如果基备份存在加密，则使用的加密算法和加密密码必须与基备份中一致；如果基备份未进行加密处理，则对增量备份使用的加密密码和算法没有特殊要求。

设置跟踪日志文件

DMRMAN 备份时可选择生成跟踪日志文件，跟踪日志记录了 SBT 接口的调用过程，用户通过查看日志可跟踪备份还原过程。

与生成跟踪日志文件相关的参数有两个，TRACE FILE 和 TRACE LEVEL。TRACE FILE 用于指定生成的跟踪日志文件路径。与 DIsql 工具不同的是，DMRMAN 不可在备份时指定参数生成跟踪文件，只能使用 CONFIGURE 命令进行事先配置。

使用 CONFIGURE DEFAULT...TRACE FILE...TRACE LEVEL 命令启用 TRACE 功能并设 TRACE 文件路径（参考 3.3.2.3 小节），以下命令生成 TRACE 文件到 /home/dm_trace 目录：

```
RMAN>CONFIGURE DEFAULT TRACE FILE '/home/dm_trace/trace.log ' TRACE LEVEL 2;
```

指定参数 TRACE FILE 但 TRACE LEVEL 值设置为 1 即不启用 TRACE 功能，会生成 TRACE 文件，但不会写入 TRACE 信息。如下所示：

```
RMAN> CONFIGURE DEFAULT TRACE FILE'/home/dm_log/db_trace.log' TRACE LEVEL 1;
```

TRACE LEVEL 值设置为 2 即启用 TRACE 功能，但若 TRACE FILE 没有指定，系统默认在执行码路径的 log 目录下生成 DM_SBTTRACE_年月.log 文件。如下所示：

```
RMAN> CONFIGURE DEFAULT TRACE LEVEL 2;
```

若 TRACE FILE 使用相对路径，日志文件则生成在执行码同级目录下。

为数据库脱机备份设置跟踪日志文件的操作步骤如下：

1) 保证数据库处于关闭状态。

2) 使用 CONFIGURE 命令配置生成跟踪日志文件。

```
RMAN>CONFIGURE DEFAULT TRACE FILE '/home/dm_trace/trace.log ' TRACE LEVEL 2;
```

3) 备份数据库。启动 DMRMAN 工具输入以下命令：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET
```

```
'/home/dm_bak/db_bak_for_trace_01' ;
```

查看/home/dm_trace/trace.log 文件即可跟踪本次备份的 SBT 接口调用过程。

如果指定的 TRACE 文件已存在，服务器不会覆盖已存在的文件而是在已有文件基础上继续记录日志。

3.3.4 管理备份

本章节主要介绍如何使用 DMRMAN 工具管理数据库备份、表空间备份及表备份。本节内容主要包括：

- 概述
- 备份集查看
- 备份集校验
- 备份集删除
- 备份集导入
- 备份集映射文件导出

3.3.4.1 概述

管理备份一个重要的目的是删除不再需要的备份。DMRMAN 工具提供 SHOW、CHECK、DELETE、LOAD 等命令分别用来查看、校验和删除备份集。下文将对这些命令进行详细介绍。

3.3.4.2 备份集查看

3.3.4.2.1 概述

DMRMAN 中使用 SHOW 命令可以查看备份集的信息，包括：

- ✓ 备份集的数据库信息
- ✓ 备份集的元信息
- ✓ 备份集中文件信息（如备份数据文件 DBF 和备份片文件）
- ✓ 备份集中表信息（仅对表备份集有效）

若指定具体备份集目录，则会生成相应的备份集链表信息。

语法如下：

```
SHOW BACKUPSET '<备份集目录>' [<device_type_stmt>][RECURSIVE]
[<database_bakdir_lst_stmt>] [<info_type_stmt>] [<to_file_stmt>];
|SHOW BACKUPSETS [<device_type_stmt>] <database_bakdir_lst_stmt>
[<info_type_stmt>] [<use_db_magic_stmt>] [<to_file_stmt>];
<database_bakdir_lst_stmt>::= DATABASE '<INI_PATH>' |
WITH BACKUPDIR '<备份集搜索目录>'{,'<备份集搜索目录>'} |
DATABASE '<INI_PATH>' WITH BACKUPDIR '<备份集搜索目录>'{, '<备份集搜索目录>'}
<device_type_stmt>::= DEVICE TYPE DISK|TAPE [PARMS '<介质参数>']
<info_type_stmt>::= INFO DB[,META][,FILE] [,TABLE]
<use_db_magic_stmt>::= USE DB_MAGIC <db_magic>
<to_file_stmt>::= TO '<输出文件路径> [FORMAT TXT | FORMAT XML]
```

BACKUPSET：指定显示目标备份集信息，若指定，则显示以该备份集为最新备份集的备份集链表。若为完全备份，则仅显示该备份集自身信息。

DATABASE：指定数据库 dm.ini 文件路径，若指定，则该数据库的默认备份目录作为备份集搜索目录之一。

WITH BACKUPDIR：备份集搜索目录，最大长度为 256 个字节。若查看完全备份的备份集信息且指定的备份集路径为相对路径可通过设置此参数搜索备份集;若查看的为增量备份集信息，设置该参数除上述功能外还用于搜索基备份集。

<device_type_stmt>: 指定介质类型和介质参数，介质类型支持 DISK 和 TAPE，默认 DISK。

RECURSIVE:指定是否递归显示备份集信息。若不指定，则仅显示目标备份集信息；若指定且目标备份集为增量备份，则递归显示其基备份集。

<info_type_stmt>：指定显示备份集信息内容，可以组合指定，若未指定，则显示全部。具体说明如下：

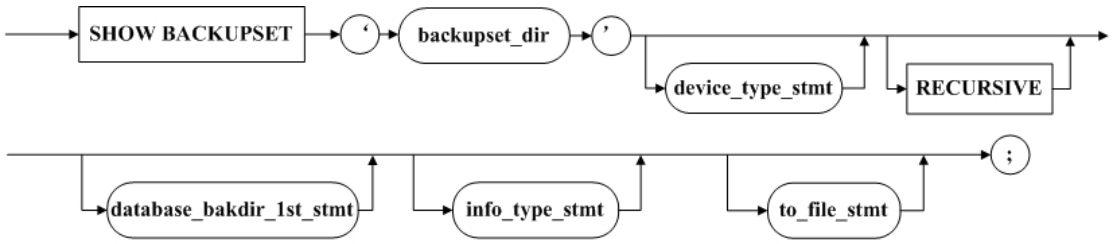
- ✓ DB 表示仅显示备份集的数据库信息；
- ✓ META 表示仅显示备份集的元信息；
- ✓ FILE 表示仅显示备份集中文件信息，如备份数据文件 DBF 和备份片文件；

✓ TABLE 表示显示备份集中表信息，仅对表备份集有效。

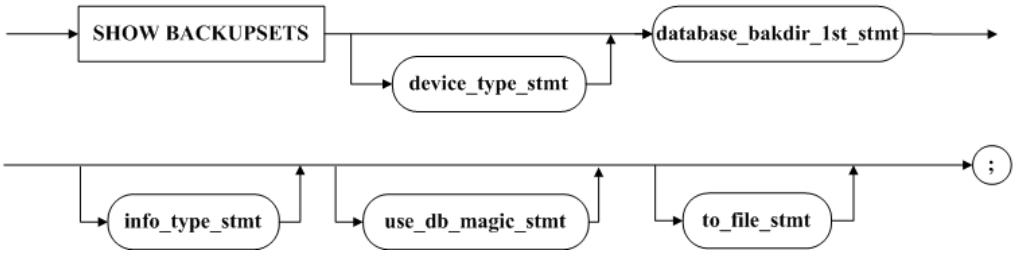
<use_db_magic_stmt>:SHOW BACKUPSETS 可以指定仅显示指定 DB_MAGIC 即指定数据库的备份集信息。

<to_file_stmt>：指定备份集信息输出的目标文件路径，若不指定，仅控制台打印。文件格式有两种类型，TXT 和 XML 格式，默认是 TXT 格式。

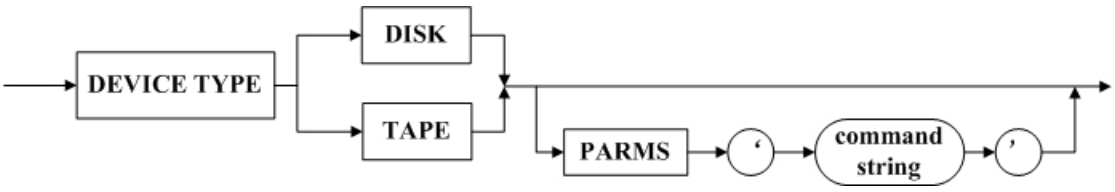
图例：



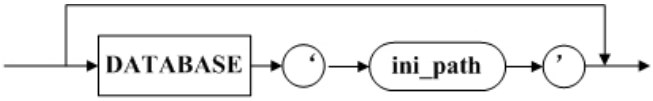
或

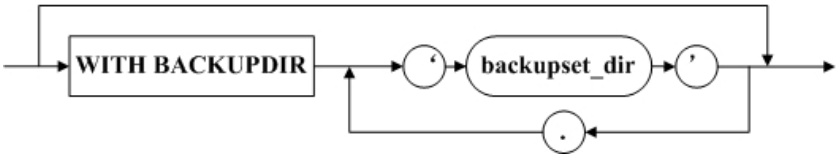


<device_type_stmt>:

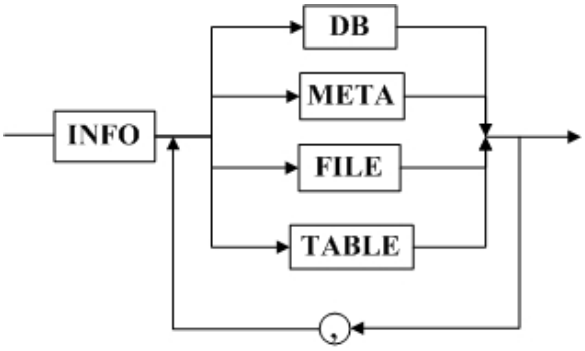


<database_bakdir_lst_stmt>:

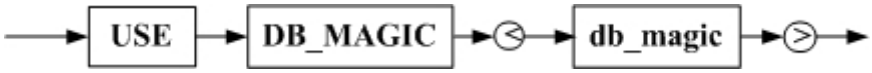




<info_type_stmt>:



<use_db_magic_stmt>:



3.3.4.2.2 备份信息查看

SHOW 命令支持查看单个备份集信息也支持批量查看多个备份集的信息。本节主要举例说明如何执行简单的常用的查看备份集信息操作，包括：

- 查看指定的备份集信息
- 批量显示备份集信息
- 查看指定数据库所有备份集的信息
- 指定显示部分备份集信息
- 以 xml 格式输出备份信息到文件

查看指定的备份集信息

SHOW BACKUPSET...命令用于显示特定的备份集信息，每次只能显示一个备份集。当仅需要查看某个特定备份集信息时可以使用此命令。执行步骤如下：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'

RMAN>SHOW BACKUPSET '/home/dm_bak/db_bak_for_show_01';

<backup set DB_FULL_DAMENG_20151120_144941_000027 info start ..........>


<DB INFO>

db magic:            1447060265

permanent magic:      1447060266

rac node:            0

page check:          0
```

```
rlog encrypt:        0
external cipher[id/name]:  0/
external hash[id/name]:  0/
length in char:      0
use new hash:        1
page size:           8 KB
extent size:         16
case sensitive:      1
log page size:       512 B
unicode_flag/charset:  0
data version:        0x70009
sys version:         V7.1.5.22-Build(2015.11.17-62910trunc)
enable policy:       0
archive flag:        1


<META INFO>
backupset sig:       BA
backup tool version:   16387
backupset db name:    DAMENG
backupset name:      DB_FULL_DAMENG_20151120_144941_000027
backupset description:
backupset MAGIC :     1448002180
parent MAGIC:        -1
META file size :      45568
compressed level:     0
encrypt type:        0
parallel num:        1
backup range:        database
mpp_timestamp:       1448002173
ddl_clone:           FALSE
```

```
mpp_flag:            FALSE
backup level:         offline
backup type:          full
use_pwr:             FALSE
without log:          FALSE
START_LSN:           828917
START_RPAG_OFF:       8
END_LSN:             828916
base START_LSN:       0
base END_LSN:         0
base name:
base backup set:
backup time:          2015-11-20 14:49:41

<FILE INFO>
backup set directory: /home/dm_bak/db_bak_for_show_01
backup set piece name: db_bak_for_show_01
backup data file num:  8
backup piece num:     1

<backup_piece_list>
$file_seq |$size(KB) |$pos_desc
0        |10040    |db_bak_for_show_01.bak

<data_file_list>
$file_seq |$group_id |$file_id  |$file_path                    |$mirror_path
|$file_len          |$start_bkp_seq     |$start_bkp_off     |$end_bkp_seq
|$end_bkp_off
1        |0        |0        |/opt/dmdbms/data/DAMENG/SYSTEM.DBF|
|25165824           |0                |4096             |0
```

```
|4973056
2       |0       |1       |/opt/dmdbms/data/DAMENG/SYSTEM_01.DBF|
|104857600         |0               |4973056          |0
|4985856
3       |1       |0       |/opt/dmdbms/data/DAMENG/ROLL.DBF |
|268435456         |0               |4985856          |0
|5285376
4       |1       |1       |/opt/dmdbms/data/DAMENG/ROLL_01.DBF|
|104857600         |0               |5285376          |0
|5298176
5       |1       |2
|/opt/dmdbms/data/DAMENG/bug52821109_37_28_07_00.dbf|
|134217728         |0               |5298176          |0
|5310976
6       |4       |0       |/opt/dmdbms/data/DAMENG/MAIN.DBF |
|134217728         |0               |5310976          |0
|9862144
7       |5       |0       |/opt/dmdbms/data/DAMENG/SYSAWR.DBF|
|134217728         |0               |9862144          |0
|10243584
8       |30      |0       |/opt/dmdbms/data/DAMENG/TS01_OFFLINE_01.DBF|
|134217728         |0               |10243584         |0
|10280960


<arch_file_list>
$file_seq |$file_path                   |$file_len         |$begin_lsn
|$begin_seqno       |$begin_rpag_off    |$end_lsn          |$end_seqno
|$create_time       |$close_time        |$start_bkp_seqno
|$start_bkp_offset   |$end_bkp_seq       |$end_bkp_off
```

```
<backup set DB_FULL_DAMENG_20151120_144941_000027 info end .........>
show backup sets successfully.
```

显示的备份集信息分为三类，依次是元数据信息（META INFO）、文件信息（FILE INFO）和数据库信息（DB INFO）。用户可根据自身需要只显示部分备份集信息，本节下文中会有介绍。

批量显示备份集信息

SHOW BACKUPSETS...命令用于批量显示指定搜索目录下的备份集信息。如需要查看的多个备份集不在同一个目录下，可通过多次指定 WITH BACKUPDIR 参数同时查看所有的备份集。操作如下：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'/home/dm_bak1/db_bak_for_show_01';
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'/home/dm_bak2/db_bak_for_show_01';
RMAN>SHOW BACKUPSETS WITH BACKUPDIR '/home/dm_bak1','/home/dm_bak2';
```

查看指定数据库所有备份集的信息

如果指定的备份搜索目录下包含不同数据库的备份集，而我们只想查看某个特定数据库的所有备份集信息，此时可以使用 SHOW BACKUPSETS...USE DB_MAGIC 命令实现。具体操作如下：

1) 备份不同数据库至同一目录下。

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'/home/dm_bak/db_bak_for_show_db_magic_01';
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG2/dm.ini' BACKUPSET'/home/dm_bak/
db_bak_for_show_db_magic_02';
```

2) 查看指定数据库备份集的信息，获取 DB_MAGIC 信息。

```
RMAN>SHOW BACKUPSET'/home/dm_bak/db_bak_for_show_db_magic_01';
```

通过查看备份集信息可知，数据库/opt/dmdbms/data/DAMENG 的 DB_MAGIC 值为 1447060265。

3) 查看指定目录下数据库/opt/dmdbms/data/DAMENG 的所有备份集信息。

```
RMAN>SHOW BACKUPSETS WITH BACKUPDIR '/home/dm_bak' USE DB_MAGIC 1447060265;
```

指定查看备份集的元数据信息

SHOW BACKUPSET...INFO META命令用于查看备份集的元数据信息。操作步骤如下：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'

BACKUPSET'/home/dm_bak1/db_bak_for_metainfo_01';

RMAN>SHOW BACKUPSET'/home/dm_bak1/db_bak_for_metainfo_01' INFO META;

<backup set DB_FULL_DAMENG_20151110_160747_000366 info start .........>


<META INFO>

backupset sig:        BA

backup tool version:   16387

backupset db name:    DAMENG

backupset name:       DB_FULL_DAMENG_20151110_160747_000366

backupset description:

backupset MAGIC :     1447142867

parent MAGIC:         -1

META file size :      25088

compressed level:     0

encrypt type:         0

parallel num:         1

backup range:         database

mpp_timestamp:        1447142859

backup level:         offline

backup type:          full

use_pwr:              FALSE

without log:          FALSE

START_LSN:            25328

START_RPAG_OFF:       8

END_LSN:              25327

base START_LSN:       0

base END_LSN:         0

base name:
```

```
base backup set:
backup time:         2015-11-10 16:07:47
<backup set DB_FULL_DAMENG_20151110_160747_000366 info end ..........>
```

这里获取的是单个备份集的元数据信息，也可以使用 SHOW BACKUPSETS...INFO META 命令批量获取元数据信息。同样地，我们可以使用类似的命令指定获取备份集的数据库信息、文件信息和表信息，或这些信息的任意组合。

以 xml 格式输出备份信息到文件

DMRMAN 可以将显示的备份集信息输出到文件，目前支持的格式包括 TXT 和 XML，默认为 TXT 文件格式。使用 SHOW BACKUPSETS...TO 'file_path' FORMAT XML 命令可将备份信息以 XML 格式显示并输出到文件，如下所示：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'/home/dm_bak1/db_bak_for_xml_01';
RMAN>SHOW BACKUPSET'/home/dm_bak1/db_bak_for_xml_01' TO
'/home/dm_info/bkp_info.txt' FORMAT XML;
```

3.3.4.3 备份集校验

DMRMAN 中使用 CHECK 命令对备份集进行校验，校验备份集是否存在及合法。

语法如下：

```
CHECK BACKUPSET '<备份集目录>'
[DEVICE TYPE<介质类型> [PARMS '<介质参数>']][DATABASE '<INI_PATH>']
```

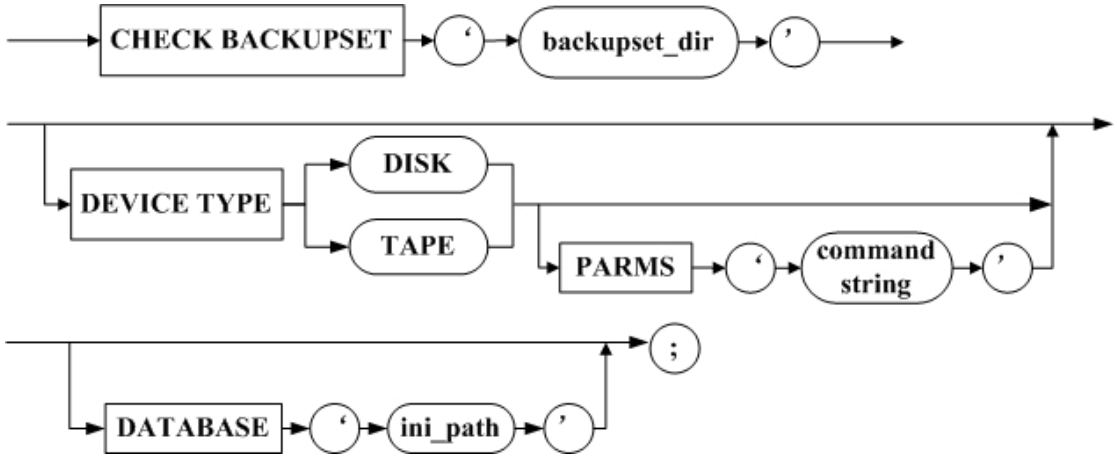
BACKUPSET：指定目标校验备份集目录。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示备份集存储为磁盘，TAPE 表示存储介质为磁带。

PARMS：介质类型为 TAPE 时，第三方介质管理实现所需的参数字符串。

INI_PATH：数据库 dm.ini 文件路径，若指定，则该数据库的默认备份目录作为备份集搜索目录之一。

图例：



校验特定的备份集

CHECK BACKUPSET...命令用于校验特定备份集，每次只能检验一个备份集。

```
SQL>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'/home/dm_bak/db_bak_for_check_01';
RMAN>CHECK BACKUPSET '/home/dm_bak/db_bak_for_check_01';
```

若备份集在默认备份路径下，可指定相对路径校验备份集，如下所示：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'db_bak_for_check_02';
RMAN>CHECK BACKUPSET 'db_bak_for_check_02' DATABASE
'/opt/dmdbms/data/DAMENG/dm.ini';
```

3.3.4.4 备份集删除

备份集删除是备份管理的主要功能，本节主要对如何删除备份进行描述。内容包括：

- 概述
- 删除备份集

3.3.4.4.1 概述

DMRMAN 中使用 REMOVE 命令删除备份集，可删除指定备份集，也可批量删除备份集。单个备份集删除时并行备份中的子备份集不允许单独删除；在指定备份集搜集目录中，发现存在引用目标备份集作为基备份的需要执行级联删除，默认报错。批量删除备份集时，跳过收集到的单独的子备份集。

语法如下：

```
REMOVE BACKUPSET '<备份集目录>'
[DEVICE TYPE< 介 质 类 型 > [PARMS '< 介 质 参 数 >']][<
database_bakdir_lst_stmt>][CASCADE];
REMOVE [DATABASE | TABLESPACE[<ts_name>] | TABLE "<schema_name>"."<tab_name>"
| ARCHIVELOG|ARCHIVE LOG] BACKUPSETS
[<device_type_stmt>]<database_bakdir_lst_stmt>
{[UNTIL TIME '<截止时间串>'] | [ BEFORE  n]}
<device_type_stmt>::= DEVICE TYPE<介质类型> [PARMS '<介质参数>']
<database_bakdir_lst_stmt>::= DATABASE '<INI_PATH>' |
WITH BACKUPDIR '<备份集搜索目录>' {, '<备份集搜索目录>' } |
DATABASE '<INI_PATH>' WITH BACKUPDIR '<备份集搜索目录>' {, '<备份集搜索目录>' }
```

BACKUPSET：指定待删除的备份集目录。

DATABASE：指定数据库 dm.ini 文件路径，若指定，则该数据库的默认备份目录作为备份集搜索目录之一。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认 DISK。DISK 表示备份集存储介质磁盘，TAPE 表示存储介质为磁带。若未指定，则 DISK 和 TAPE 介质上满足条件备份集均会删除。

PARMS：介质类型为 TAPE 时，第三方介质管理实现所需的参数字符串。

CASCADE：当目标备份集已经被其他备份集引用为基备份集，默认不允许删除，若指定 CASCADE，则递归删除所有引用的增量备份。

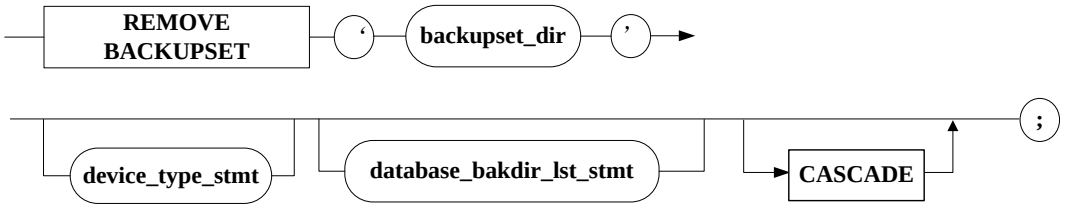
DATABASE|TABLESPACE|TABLE|ARCHIVELOG|ARCHIVE LOG：指定删除备份集的类型，分别为库级、表空间级、表级备份，以及归档级别，其中 ARCHVELOG 和 ARCHIVE LOG 等价。若不指定，全部删除。指定 TABLESPACE 时，若指定目标表空间名，则仅会删除满足条件的指定表空间名称的表空间备份集，否则，删除所有满足条件的表空间级备份集；指定 TABLE 时，若指定目标表名，则仅会删除满足条件的指定表名的表备份集；否则，删除所有满足条件的表备份集。

UNTIL TIME：删除备份集生成的最大时间，即删除指定时间之前的备份集，若未指定，则删除所有备份集。

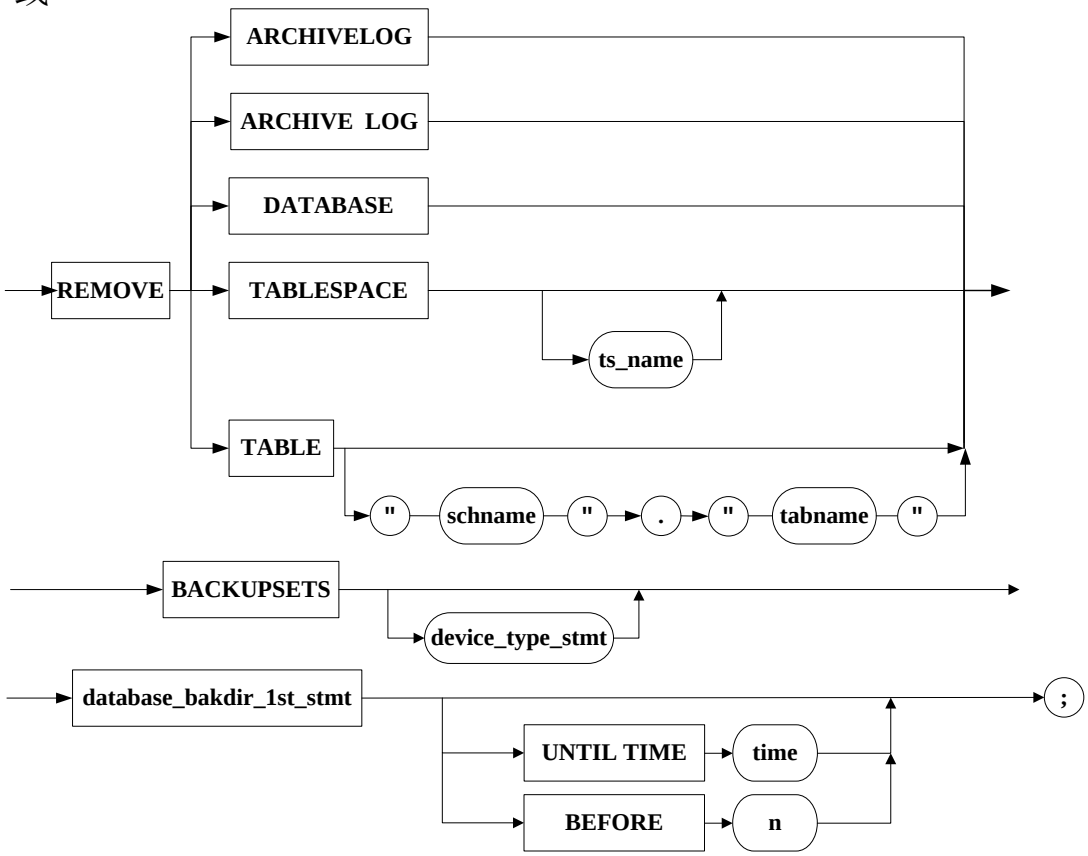
BEFORE n：删除距离当前时间前 n 天产生的备份集；n 取值范围 0~365，单位：天。

WITH BACKUPDIR：备份集搜索目录，用于搜索指定目录下的所有备份集。

图例：

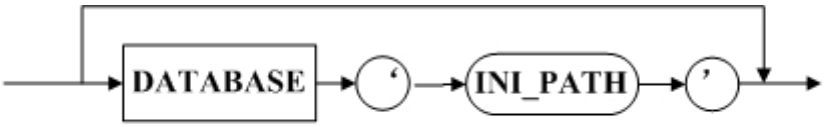


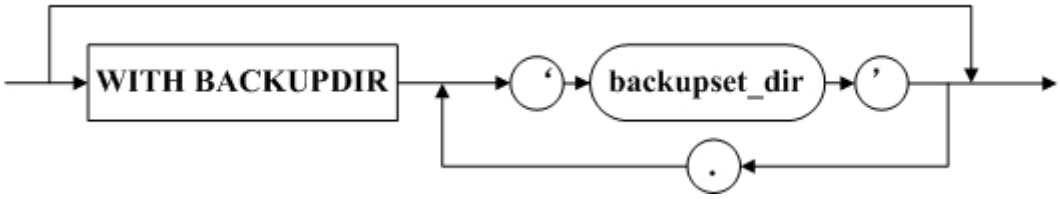
或



<device_type_stmt>:请参考 3.3.4.2.1 节

<database_bakdir_lst_stmt>:





3.3.4.4.2 删除备份集

本节举例说明如何使用 DMRMAN 的 REMOVE 命令删除备份集，主要内容包括：

- 删除特定的备份集
- 批量删除所有备份集
- 批量删除指定时间之前的备份集

删除特定的备份集

使用 REMOVE BACKUPSET...命令可删除特定备份集，每次只能删除一个备份集。若删除备份集已经被引用为其他备份集的基备份且未指定 CASCADE，则报错。

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'/home/dm_bak/db_bak_for_remove_01';
RMAN>REMOVE BACKUPSET '/home/dm_bak/db_bak_for_remove_01';
```

如果备份集在数据库默认备份目录下还可使用以下方式删除备份集：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET'db_bak_for_remove_01';
RMAN>REMOVE BACKUPSET 'db_bak_for_remove_01' DATABASE
'/opt/dmdbms/data/DAMENG/dm.ini';
```

如果备份集为其他备份集的基备份且备份集都在数据库默认备份目录下还可使用以下方式删除备份集：

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET'db_bak_for_remove_01';
--增量备份之前，启动数据库，以便成功生成增量备份
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' INCREMENT
BACKUPSET'db_bak_for_remove_01_incr';
```

```
RMAN>REMOVE BACKUPSET 'db_bak_for_remove_01_incr' DATABASE
'/opt/dmdbms/data/DAMENG/dm.ini' CASCADE;
```

批量删除所有备份集

使用 REMOVE BACKUPSETS...命令可批量删除备份集。批量删除可选择删除的备份类型，数据库备份、表空间备份、表备份，以及归档备份，不指定则全部删除。下面示例为删除/home/dm_bak 目录下的所有备份集，可以是联机生成的备份集，也可以脱机 DMRMAN 工具生成的备份集。

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET
'/home/dm_bak/db_bak_for_remove_02';
RMAN>REMOVE BACKUPSETS WITH BACKUPDIR '/home/dm_bak';
```

批量删除指定时间之前的备份集

REMOVE BACKUPSETS...UNTIL TIME 命令用来批量删除指定时间的备份集。通常情况下，用户并不想删除指定目录下所有的备份集，这时可以选择只删除指定时间之前的备份。如何确定删除备份的时间点，需要结合用户的备份计划合理指定。若用户每周做一次完全备份，每天进行增量备份，那么删除的时间可指定为 7 天前的某个特定时间。假设今天的日期为 2015-11-8，要删除 7 天前/home/dm_bak 目录下的所有备份在 DMRMAN 中输入以下命令：

```
RMAN>REMOVE BACKUPSETS WITH BACKUPDIR '/home/dm_bak' UNTIL TIME'2015-11-1
00:00:00';
```

3.3.4.5 备份集导入

备份集导入是备份管理的主要功能，本节主要对如何导入备份集进行描述。内容包括：

- 概述
- 导入备份集

3.3.4.5.1 概述

DMRMAN 中使用 LOAD 命令导入备份集。

语法如下：

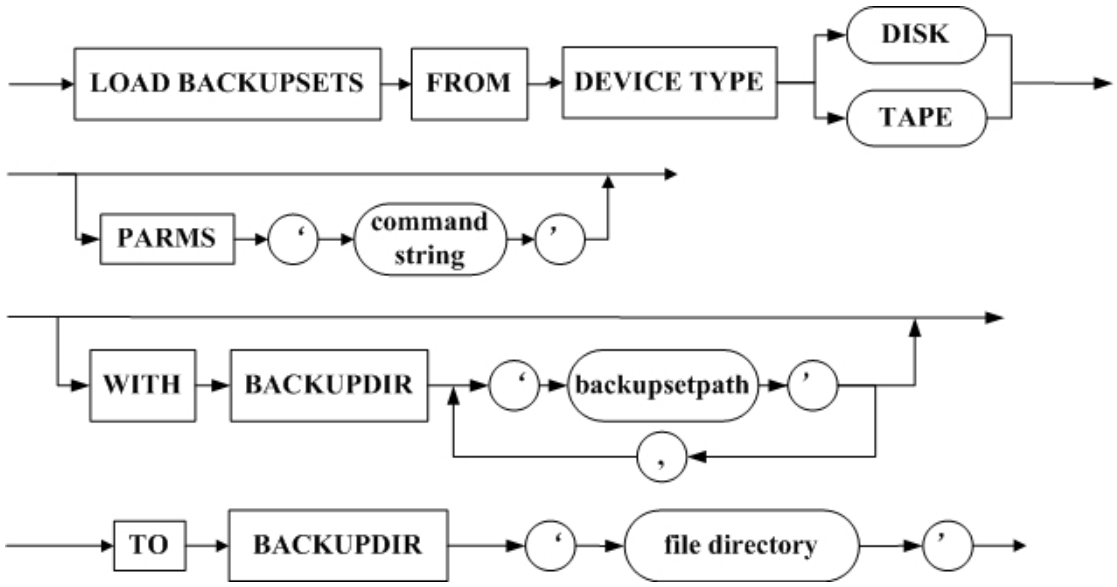
```
LOAD BACKUPSETS FROM DEVICE TYPE<介质类型> [PARMS '<介质参数>'] [WITH BACKUPDIR '<备份集搜索目录>'{,'<备份集搜索目录>'}]TO BACKUPDIR '<备份集存放目录>';
```

介质类型：指存储备份集的设备类型，暂支持 DISK 和 TAPE。

介质参数：介质类型为 TAPE 时，第三方介质管理实现所需的参数字符串。

备份集存放目录：指从 TAPE 上导出的备份集 meta 文件存放到本地磁盘的目标目录，要求为空或不存在。

图例：



3.3.4.5.2 导入备份集

本节举例说明如何使用 DMRMAN 的 LOAD 命令导入备份集，主要内容包括：

- 导出磁盘上备份集
- 导出磁带上所有的备份集

导出磁盘上备份集

导出磁盘上备份集的 meta 文件。

```
RMAN>LOAD BACKUPSETS FROM DEVICE TYPE DISK TO BACKUPDIR '/mnt/dm7src/bak_dir';
```

导出磁带上所有的备份集

导出磁带 /dev/nst0 上所有备份集的 meta 文件到目录 /mnt/hgfs/dm7src/bak_dir。

```
RMAN>LOAD BACKUPSETS FROM DEVICE TYPE TAPE TO BACKUPDIR
'/mnt/hgfs/dm7src/bak_dir';
```

```
LOAD BACKUPSETS FROM DEVICE TYPE TAPE TO BACKUPDIR '/mnt/hgfs/dm7src/bak_dir';

load backupsets failed.error code：-10000

[-10000]：[错误码：-20022]磁带打开失败
```

退出 dmrman，设置环境变量 TAPE，值为/dev/nst0。

```
[root@192 debug]# export TAPE=/dev/nst0

[root@192 debug]#echo $TAPE

/dev/nst0
```

启动 dmrman，再次执行：

```
[root@192 debug]# ./dmrman

dmrman V7.1.3.141-Build(2014.09.09-48301trunc)

RMAN>LOAD BACKUPSETS FROM DEVICE TYPE TAPE TO BACKUPDIR

'/mnt/hgfs/dm7src/bak_dir';

LOAD BACKUPSETS FROM DEVICE TYPE TAPE TO BACKUPDIR '/mnt/hgfs/dm7src/bak_dir';

load meta file [SBT_TEST_T-2014090919262900000-4966] to path

[/mnt/hgfs/dm7src/bak_dir/0/0.meta]...

load meta file [SBT_TEST_T-2014090919262900000-4966] to path

[/mnt/hgfs/dm7src/bak_dir/0/0.meta]...success

load meta file [SBT_TEST_T2-2014090919274600000-9983] to path

[/mnt/hgfs/dm7src/bak_dir/1/1.meta]...

load meta file [SBT_TEST_T2-2014090919274600000-9983] to path

[/mnt/hgfs/dm7src/bak_dir/1/1.meta]...success

load backupsets successfully.
```

退出 dmrman，查看本地磁盘目录/mnt/hgfs/dm7src/bak_dir：

```
[root@192 debug]# ls -1 /mnt/hgfs/dm7src/bak_dir

total 0

drwxrwxrwx 1 root root 0 Sep 11 00:23 0

drwxrwxrwx 1 root root 0 Sep 11 00:23 1

[root@192 debug]# ls -1 /mnt/hgfs/dm7src/bak_dir/0

total 12

-rwxrwxrwx 1 root root 24576 Sep 11 00:23 0.meta
```

```
[root@192 debug]# ls -1 /mnt/hgfs/dm7src/bak_dir/1
total 12
-rwxrwxrwx 1 root root 24576 Sep 11 00:23 1.meta
```

3.3.4.6 备份集映射文件导出

备份集映射文件，又称为 mapped file。备份集映射文件导出，是备份管理的主要功能，是将备份集中各数据文件的原始路径或者调整后的路径生成到一个本地文件中，可通过关键字 MAPPED FILE 应用于表空间和库的还原操作中。本节主要对如何将备份集中数据文件路径导出到本地进行描述。内容包括：

- 概述
- 备份集映射文件导出

3.3.4.6.1 概述

DMRMAN 中使用 DUMP 命令导入备份集。

语法如下：

```
DUMP BACKUPSET '<备份集目录>'
[DEVICE TYPE<介质类型> [PARMS '<介质参数>']]
[DATABASE '<INI_PATH>'|TO'<SYSTEM_DIR>']
MAPPED FILE '<映射文件路径>';
```

备份集目录：待导出映射文件的目标备份集，仅支持库级和表空间级备份。

介质类型：指存储备份集的设备类型，暂支持 DISK 和 TAPE。

介质参数：介质类型为 TAPE 时，第三方介质管理实现所需的参数字符串。

SYSTEM_DIR：数据库目录中 SYSTEM.DBF 数据文件所在目录，作为数据库系统目录处理。

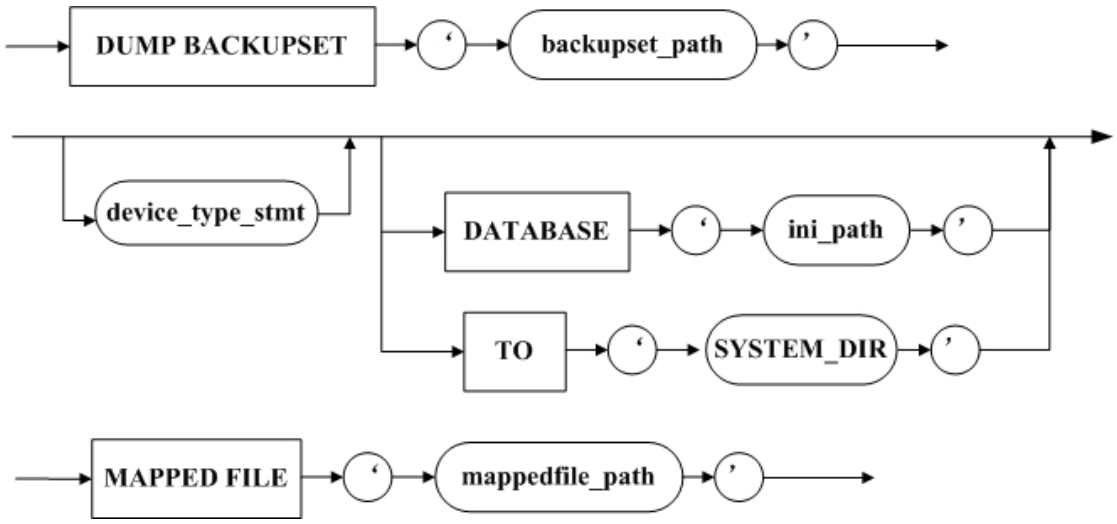
INI_PATH：备份集还原到目标库的 INI 路径。若指定，则根据 INI 对应库的系统目录调整数据文件路径；若不指定，则保持备份集中数据文件的原始路径。



说明：数据文件路径指定 INI_PATH 或者 SYSTEM_DIR 调整策略：若指定 INI_PATH，则取出配置中 SYSTEM_PATH 作为数据库系统目录，若指定 SYSTEM_DIR，则直接作为数据库系统目录。对于库备份集，与还原过程（2.1.2.1 节）中数据库文件路径构造策略描述一致。对于表空间级备份集仅支持原库还原，若指定 INI_PATH 或者 SYSTEM_DIR，则认为需要构造，构造策略与库备份集中数据文件路径构造策略一致。

映射文件路径：输出到本地的目标映射文件路径。若不存在，则创建；若存在，则覆盖。文件生成之后，允许手动调整数据文件路径到其他路径，真正使用映射文件时，为再次校验。

图例：



3.3.4.6.2 导出备份集映射文件

本节举例说明如何使用 DMRMAN 的 DUMP 命令导出备份集数据文件路径到映射文件，主要内容包括：

- 导出原始路径
- 导出指定 INI_PATH 调整后的路径

导出原始路径

导出备份集中数据文件的原始路径。

```
RMAN>DUMP BACKUPSET'/mnt/dm7src/db_bak'DEVICE TYPE DISK MAPPED FILE '/mnt/dm7src/db_bak_mapped.txt';
```

导出指定 INI_PATH 调整后的路径

指定 INI_PATH，导出调整后的数据文件路径到映射文件。

```
RMAN>DUMP      BACKUPSET'/mnt/dm7src/db_bak'DEVICE      TYPE      DISKDATABASE
'/opt/dmdbms/data/DAMENG/dm.ini'MAPPED FILE '/mnt/dm7src/db_bak_mapped.txt';
```

3.3.5 数据库还原和恢复

本章节介绍在数据文件损坏后如何使用 DMRMAN 还原和恢复数据库，主要内容包括：

- 数据库还原
- 数据库恢复

3.3.5.1 执行数据库还原和恢复

3.3.5.1.1 数据库还原

使用 RESTORE 命令完成脱机还原操作，在还原语句中指定库级备份集，可以是脱机库级备份集，或是联机库级备份集。数据库的还原包括数据库配置文件还原和数据文件还原，目前可能需要还原的数据库配置文件包括 dm.ini、dm.ctl、服务器秘钥文件(dm_service.private 或者 dm_external.config，若备份库指定 usbkey 加密，则无秘钥文件)、联机日志文件。

语法如下：

```
RESTORE DATABASE <restore_type> FROM BACKUPSET '<备份集目录>'
[DEVICE TYPE DISK|TAPE[PARMS '<介质参数>']]
[IDENTIFIED BY <密码> [ENCRYPT WITH <加密算法>]]
[WITH BACKUPDIR '<基备份集搜索目录>'{,'<基备份集搜索目录>'}]
[MAPPED FILE '<映射文件>'][TASK THREAD <任务线程数>] [NOT PARALLEL];
<restore_type>::=<type1>|<type2>
<type1>::='<INI 文件路径>'[REUSE DMINI]
<type2>::= TO '<system_dbf 所在路径>' [OVERWRITE]
```

DATABASE： 指定还原库目标的 dm.ini 文件路径。

BACKUPSET：指定用于还原目标数据库的备份集目录。若指定为相对路径，会在默认备份目录下搜索备份集。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认为 DISK。DISK 表示备份集存储介质为磁盘，TAPE 表示存储介质为磁带。

PARMS：介质参数，供第三方存储介质管理使用。

IDENTIFIED BY：指定备份时使用的加密密码，供还原过程解密使用。

ENCRYPT WITH：指定备份时使用的加密算法，供还原过程解密使用，若未指定，则使用默认算法。

WITH BACKUPDIR：指定备份集搜索目录。

MAPPED FILE：指定存放还原目标路径的文件，参见 3.3.5.2.1 数据库还原 。当<备份集目录>和<映射文件>指定的路径不一致时，以<映射文件>指定的路径为主

TASK THREAD：指定还原过程中用于处理解压缩和解密任务的线程个数，取值范围 1~64。若未指定，则默认为 4。

NOT PARALLEL：指定并行备份集使用非并行方式还原。对于非并行备份集，不论是否指定该关键字，均采用非并行还原。

<restore_type>：

- <type1>

1> 除 dm.ini 文件外，其他文件均可不存在；但 dm.ini 参数配置必须正确；

2>数据库配置文件中除已经存在的 dm.ini 外，其他文件均采用删除重建的处理，避免存在非法的文件；

4>若指定 REUSE DMINI，则会将备份集中备份的 dm.ini 中除路径相关的 INI 参数外，均拷贝到当前 dm.ini 上。

- <type2>说明：

1> 所有文件均可不在，system_dir 需为有效路径，若不存在，restore 过程中会自动创建；

2>所有前面提到到数据库配置文件均会在指定的 system_dbf_所在路径还原，但非单机环境中相关其他文件均不修改或者重建，如 mpp 中 dmmpp.ini、dmmal.ini 等；

3>若未指定 overwrite，若 system_dbf_所在路径中存在待还原的库配置文件，则报错；若指定，则将已经存在的文件删除重建；

4>由于 RAC 环境中 dm.ini 可能存在多个，且可能不在一个主库上，或者即使在一个主库上也可能不在 system_dbf_所在路径中，故暂时不支持 RAC 环境的指定目录还原。

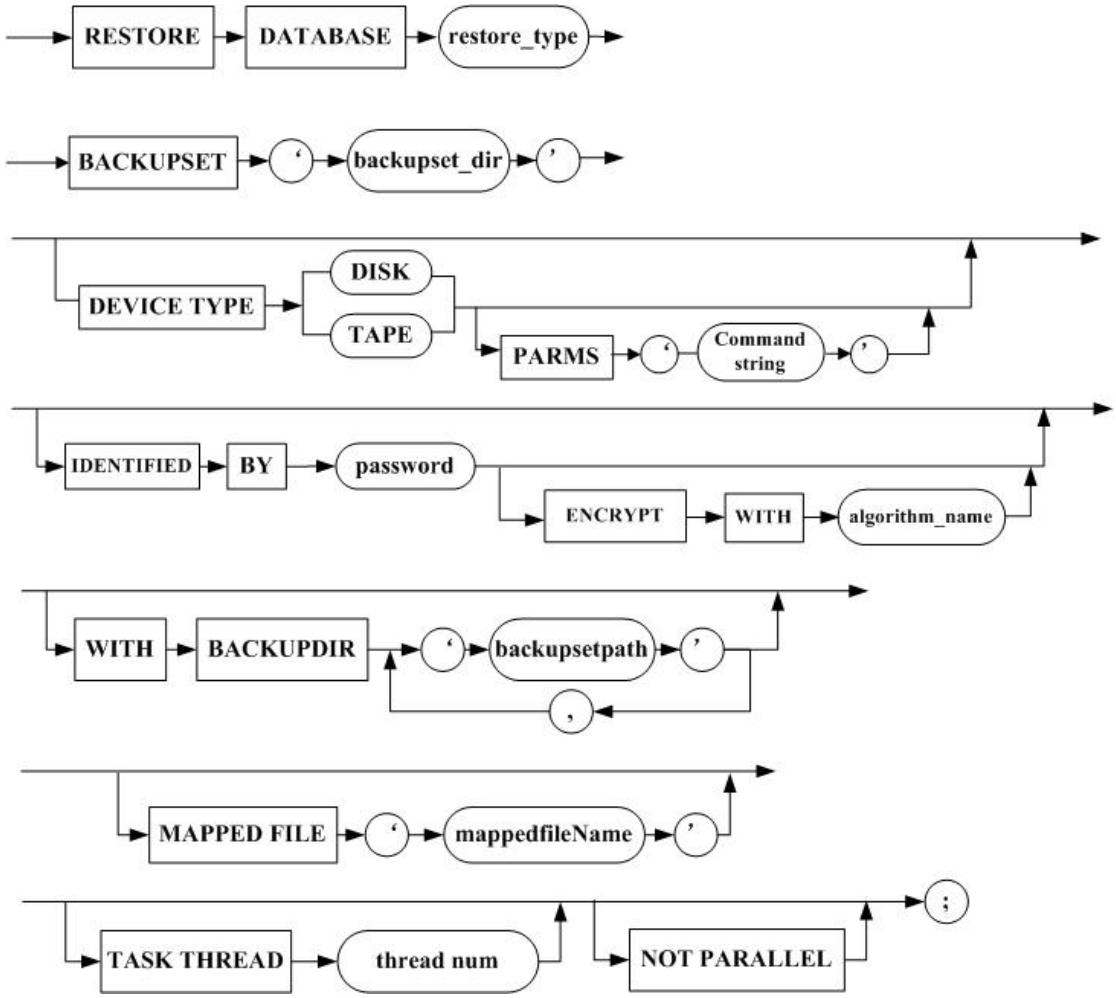
- 联机日志

上述<type1>和<type2>中的还原后的联机日志文件至少会有两个，因为源库中的日志文件可能大于等于两个，如果小于两个（被误删的情况）的则补齐为两个。已经存在的联机日志配置，使用原路径；缺少的使用缺省命名重建。联机日志文件的命名规则：

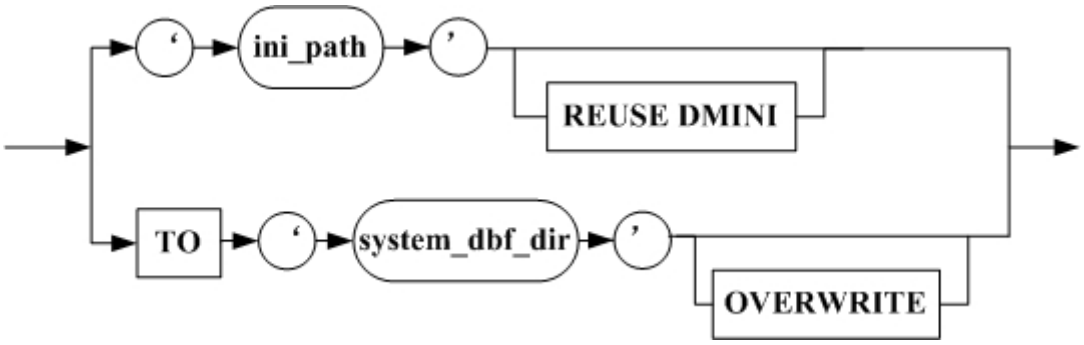
1> 单机：db_name+(file_id+1).log，其中 fil_id + 1，占 2 个位置，如：db_name=DAMENG，则相应的名称为 DAMENG01.log，DAMENG02.log；

2> RAC：rac+(ep_no + 1)_+(file_id+1).log，其中 ep_no + 1 和 file_id+1 均占 2 个位置，如 0 号站点日志名称为 rac01_01.log，rac01_02.log。

图例：



restore_type：



使用说明：

通过 RESTORE 命令还原后的数据库不可用，需进一步执行 RECOVER 命令，将数据库恢复到备份结束时的状态。

数据库备份集分为联机和脱机两种类型。通常情况下，用户会在联机的情况下备份数据库，因此下面以联机数据库备份为例说明使用 DMRMAN 如何执行数据库还原操作。

1） 联机备份数据库，保证数据库运行在归档模式及 OPEN 状态。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_full_bak_for_restore';
```

2） 准备目标库。还原目标库可以是已经存在的数据库，也可使用 dminit 工具初始化一个新库。如下所示：

```
./dminit path=/opt/dmdbms/data db_name=DAMENG_FOR_RESTORE
```

注意： **如果还原目标库与故障库是同一个，建议先执行故障库的归档修复操作。具体详见 3.3.5.4 归档修复 。**

3） 校验备份，校验待还原备份集的合法性。校验备份有两种方式，联机和脱机，此处使用脱机校验。

```
RMAN>CHECK BACKUPSET '/home/dm_bak/db_full_bak_for_restore';
```

4） 还原数据库。启动 DMRMAN，输入以下命令：

```
RMAN>RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_restore';
```

3.3.5.1.2 数据库恢复

使用 RECOVER 命令完成数据库恢复工作，可以是基于备份集的恢复工作，也可以是使用本地归档日志的恢复工作。

语法如下：

```
RECOVER DATABASE '<INI 文件路径>' [<with_archdir_lst_stmt>] [TYPE <归档执行类型>]
[USE DB_MAGIC <db_magic>] [UNTIL TIME '<时间串>'] [UNTIL LSN <LSN>]; |
RECOVER DATABASE '<INI 文件路径>' FROM BACKUPSET '<备份集目录>'[DEVICE TYPE
DISK|TAPE[PARMS '<介质参数>']] [IDENTIFIED BY <密码> [ENCRYPT WITH <加密算法>]]
[TYPE <归档执行类型>]; |
RECOVER DATABASE '<INI 文件路径>' UPDATE DB_MAGIC;
<with_archdir_lst_stmt> ::=
WITH ARCHIVEDIR '<归档日志目录>'{,'<归档日志目录>'}
```

DATABASE： 指定还原库目标的 dm.ini 文件路径。

TYPE：归档执行类型。指定是否要优化重做本地归档过程，0 表示不优化，1 表示优化，默认不优化。当需要重做的归档日志文件很大时应设置该参数为 1，可加快重做归档的速度，否则可任意设置。

USE DB_MAGIC：指定本地归档日志对应数据库的 DB_MAGIC，若不指定，则默认使用目标恢复数据库的 DB_MAGIC。

UNTIL TIME：恢复数据库到指定的时间点。

UNTIL LSN：恢复数据库到指定的 LSN。

BACKUPSET：指定用于还原目标数据库的备份集目录。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认为 DISK。DISK 表示存储备份集到磁盘，TAPE 表示存储到磁带。

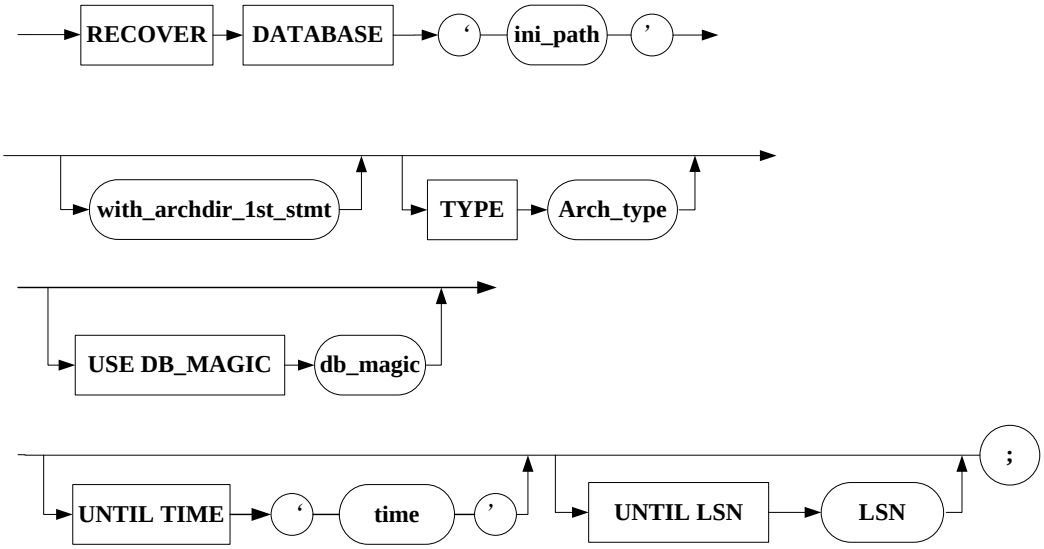
PARMS：介质参数，供第三方存储介质管理使用。

IDENTIFIED BY：指定备份时使用的加密密码，供还原过程解密使用。

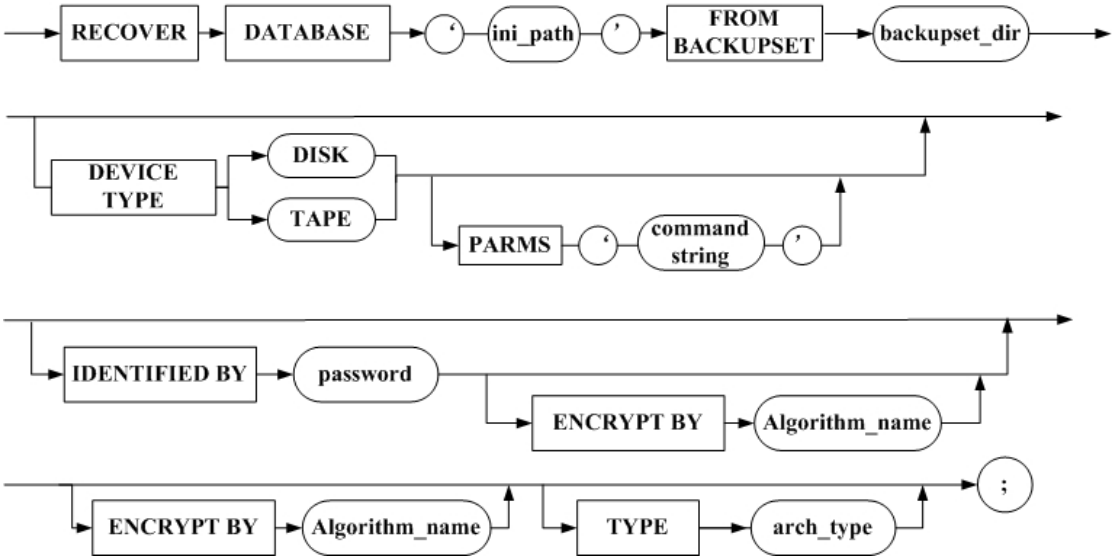
ENCRYPT WITH：指定备份时使用的加密算法，供还原过程解密使用，若未指定，则使用默认算法。

WITH ARCHIVEDIR：本地归档日志搜索目录，若未指定，则仅使用目标库配置的第一个本地归档目录。

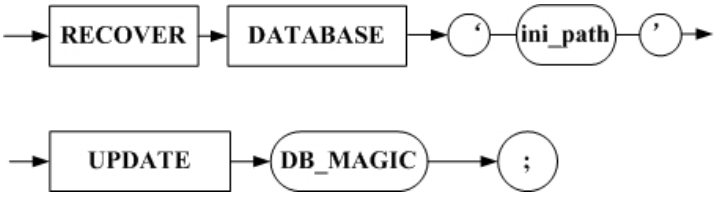
图例：



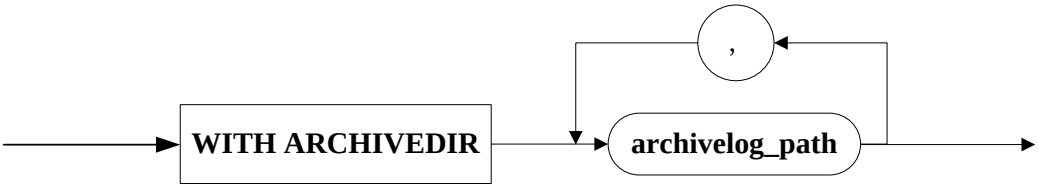
或



或



<with_archdir_lst_stmt>



数据库恢复有三种方式：更新 DB_MAGIC 恢复、从备份集恢复和从归档恢复，下面逐

一对这三种恢复方式进行说明。

更新 DB_MAGIC 恢复

当备份集为脱机备份即备份过程中无日志生成，那么此时还原后的数据库中数据与备份时数据状态一致。在不需要重做归档日志恢复数据的情况下，可以直接更新 DB_MAGIC（见 1.2.9.1 概念）完成数据库恢复。完整的示例如下：

1) 启动 RMAN 备份数据库，保证服务器处于脱机状态。

```
RMAN>BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini' BACKUPSET
'/home/dm_bak/db_full_bak_for_recover_dbmagic';
```

2) 准备目标库，可以使用备份库，也可以重新生成库。重新生成库操作如下：

```
./dminit path=/opt/dmdbms/data db_name=DAMENG_FOR_RESTORE auto_overwrite=1
```

3) 校验备份。

```
RMAN>CHECK BACKUPSET '/home/dm_bak/db_full_bak_for_recover_dbmagic';
```

4) 还原数据库。

```
RMAN>RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_recover_dbmagic ';
```

5) 恢复数据库。

```
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' UPDATE
DB_MAGIC;
```

从备份集恢复

如果备份集在备份过程中生成了日志，且这些日志在备份集中有完整备份（如联机数据库备份），在执行数据库还原后，可以重做备份集中备份的日志，将数据库恢复到备份时的状态，即从备份集恢复。完整的示例如下：

1) 启动 DIsql 联机备份数据库。

```
SQL>BACKUP DATABASE BACKUPSET
'/home/dm_bak/db_full_bak_for_recover_backupset';
```

2) 准备目标库，可以使用备份库，也可以重新生成库。重新生成库操作如下：

```
./dminit path=/opt/dmdbms/data db_name=DAMENG_FOR_RESTORE auto_overwrite=1
```

3) 启动 RMAN，校验备份。

```
RMAN>CHECK BACKUPSET '/home/dm_bak/db_full_bak_for_recover_backupset';
```

4) 还原数据库。

```
RMAN>RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_recover_backupset';
```

5) 恢复数据库。

```
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_recover_backupset';
```

从归档恢复

从归档恢复是利用重做本地归档日志来恢复数据的过程。从归档恢复允许恢复到指定的时间点及指定的 LSN 值。若同时指定了时间点和 LSN，则以较早的为结束点。



警告：由于从本地归档恢复允许使用 USE DB_MAGIC 指定待收集归档的 DB_MAGIC，那么就会存在跨库恢复情况，所以，归档日志的正确使用完全由用户保证。因此，为了保证用户能正确使用从归档恢复，除了下文这两种情况，其他情况（可能导致数据被破坏）不建议用户使用从本地归档恢复。

可以使用从归档恢复的情况：

1、执行过从备份集还原，以及执行过从备份集还原恢复的库，本地归档日志属于备份集备份时的库，本地归档日志的 DB_MAGIC 与备份集中记录 DB_MAGIC 值；

2、未执行过还原的库，但恢复目标库与本地归档日志来源于同一个库，且目标库与原来的库分离后没有再单独执行过操作或者故障重启过。比如一个数据库 A 因故障关闭后，拷贝一份作为数据库 B，此时数据库 A 和 B 完全相同。此后，数据库 A 故障重启，并正常执行其他操作，生成了新的本地归档日志。如果想利用 A 的本地归档日志去将 B 恢复到 A 的状态，那么可以利用 DMRMAN 工具执行从本地归档恢复。若中间 B 启动过或者执行过其他操作，则均不能再使用 A 的归档日志进行恢复操作。

利用归档恢复数据库至最新状态的完整示例如下：

1) 启动 DIsql 联机备份数据库，以及备份从检查点开始的本地归档日志。

```
SQL>BACKUP ARCHIVELOG FROM LSN 421401 BACKUPSET
'/home/dm_bak/arch_bak_lsn_421401';
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_full_bak_for_recover_arch';
```



查看动态视图 V$RLOG 中的 CKPT_LSN 列，可以得出当前检查点 LSN。如：

```
SQL> select CKPT_LSN from V$RLOG;

行号        CKPT_LSN
---------- --------------------
1          421401
```

2) 准备目标库，可以使用备份库，也可以重新生成库。

如果使用原备份库，且备份库故障，需要先执行目标库归档修复：

```
RMAN>REPAIR ARCHIVELOG DATABASE 'opt/dmdbms/data/dm.ini';
```

如果使用新生成的库，生成之后需要先重启一下服务器，才可以被还原。重新生成的库不需要执行归档修复（repair）操作。重新生成库操作如下：

```
./dminit path=/opt/dmdbms/data db_name=DAMENG_FOR_RESTORE auto_overwrite=1
```

3) 启动 RMAN，校验备份。

```
RMAN>CHECK BACKUPSET '/home/dm_bak/db_full_bak_for_recover_arch';
```

4) 还原数据库。

```
RMAN>RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_recover_arch';
```

5) 查看备份集的数据库信息，获取源库的 DB_MAGIC。



若还原后，立即执行执行恢复，可以不用获取源库 DB_MAGIC 。因为 DMRMAN 执行库级备份集还原后，会将备份集中的 DB_MAGIC 刷入还原后的库中。

```
RMAN>SHOW BACKUPSET '/home/dm_bak/db_full_bak_for_recover_arch' INFO DB;
<backup set DB_FULL_DAMENG_20151113_172548_000201 info start ..........>


<DB INFO>
db magic:            1447060265
permanent magic:      1447060266
rac node:            0
page check:           0
rlog encrypt:         0
external cipher[id/name]:  0/
```

```
external hash[id/name]:  0/

length in char:       0

use new hash:         1

page size:           8 KB

extent size:         16

case sensitive:       1

log page size:        512 B

unicode_flag/charset:  0

data version:         0x70009

sys version:          V7.1.5.20-Build(2015.11.06-62552trunc)

enable policy:        0

archive flag:         1

<backup set DB_FULL_DAMENG_20151113_172548_000201 info end ..........>
```

6) 利用归档恢复数据库。由步骤 5 显示的备份集信息可知，源库的 DB_MAGIC 值为 1447060265。

```
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' WITH

ARCHIVEDIR'/home/dm_arch/arch' USE DB_MAGIC 1447060265;
```



若还原后，立即执行执行恢复，可以不用指定 DB_MAGIC 。

7) 若执行后归档恢复过程中，出现归档不足的错误，则利用归档校验工具 dmrachk，查看归档目录中归档连续性情况，然后再利用备份的本地归档日志，还原到归档目录后，再次执行恢复操作。

```
./dmrachk arch_path=/home/dm_arch/arch

......

ARCH_PATH 的归档文件链表是不连续的。

breakpoint 0:

the left file /home/dm_arch/arch/ARCHIVE_LOCAL1_20160112161534890.log, clsn:

421311
```

```
the right file /home/dm_arch/arch/ARCHIVE_LOCAL1_20160113140135670.log,
arch_lsn: 423208
breakpoint 1:
the left file /home/dm_arch/arch/ARCHIVE_LOCAL1_20160114085528401.log, clsn:
424273
the right file /home/dm_arch/arch/ARCHIVE_LOCAL1_20160114092356075.log,
arch_lsn: 425285
the rachk tool running cost 949.011 ms
```

DMRMAN 查看归档备份的范围：

```
RMAN>show backupset '/home/dm_bak/arch_bak_lsn_421401' INFO META;
show backupset ' /home/dm_bak/arch_bak_lsn_421401 ' INFO META;

<backup set ARCH_20160115_153738_000536 info start ..........>

<META INFO>
backupset sig:        BA
backup tool version:   16387
backupset db name:    DAMENG
backupset name:       ARCH_20160115_153738_000536
backupset description:
backupset MAGIC :     1452843457
parent MAGIC:         -1
META file size :      20992
compressed level:     0
encrypt type:         0
parallel num:         1
backup range:         archivelog
arch start lsn:       421300
arch end lsn:       425550
backup level:         online
```

```
backup type:          archive
use_pwr:             FALSE
without log:          TRUE
START_LSN:           421401
START_RPAG_OFF:       8
END_LSN:             425551
base START_LSN:       0
base END_LSN:         0
base name:
base backup set:
backup time:          2016-01-15 15:37:39


<backup set ARCH_20160115_153738_000536 info end ..........>
show backup sets successfully.
```

可见本地归档缺失部分(421311,423208]、(424273,425285]在归档备份范围内[421300,425550]中，执行归档还原后，再次执行利用本地归档恢复操作。

```
RMAN>RESTORE ARCHIVELOG FROM BACKUPSET '/home/dm_bak/arch_bak_lsn_421401' ALL
TO ARCHIVEDIR '/home/dm_arch/arch';
```

3.3.5.2 执行数据库还原和恢复：高级场景

3.3.5.2.1 数据库还原

本节主要介绍数据库还原的高级过程，指定映射文件还原。

指定映射文件还原

还原后的数据文件默认地生成到还原目标库的路径下，如果用户想生成数据文件到特定的路径，就需要指定映射文件参数来实现。

映射文件（mappedfile）用于指定存放还原目标路径，即备份集里面的数据文件的路径。可以手动修改自动生成的映射文件。当参数 BACKUPSET 和 MAPPED FILE 指定的路径不一致时，以 MAPPED FILE 中指定的路径为主。

映射文件可用于库级脱机还原和表空间还原。

使用 DUMP 命令可以将指定备份集还原目标信息生成到目标映射文件中，该文件可被重新编辑后，用于数据库的还原过程。

语法如下：

```
DUMP BACKUPSET '<备份集目录>' [DEVICE TYPE DISK|TAPE [PARMS '介质参数']]
[DATABASE '<INI 文件路径>'] MAPPED FILE '<映射文件>';
```

BACKUPSET：待生成映射文件的备份集目录。

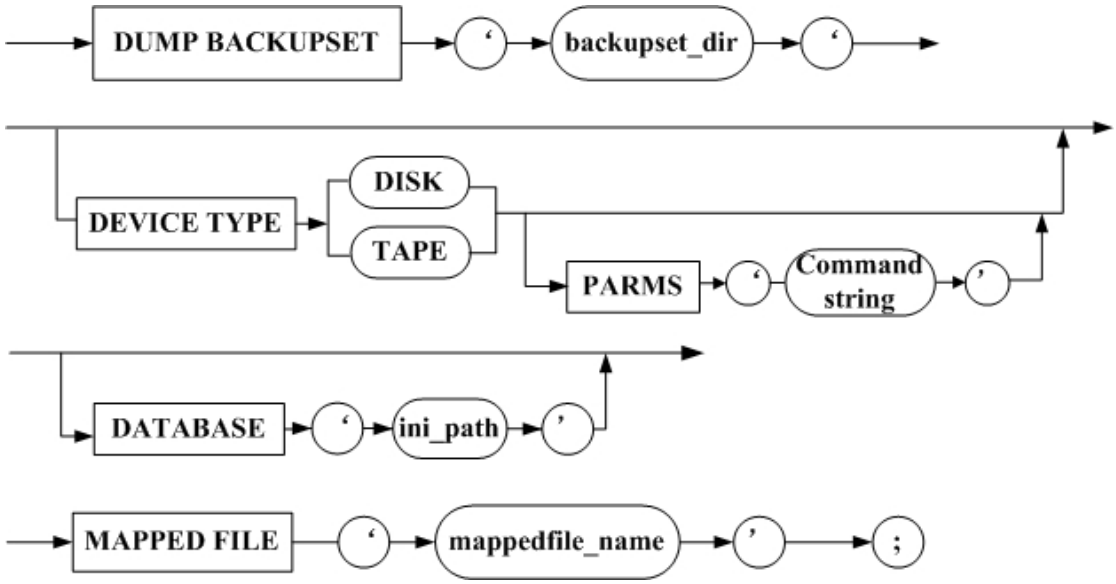
DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认为 DISK。

PARMS：介质参数，供第三方存储介质管理使用。

DATABASE： 指定目标还原库的 dm.ini 文件路径。

MAPPED FILE：生成映射文件路径。若指定 DATABASE 参数，则生成内容调整为与指定数据库相适应的数据文件目标还原路径；否则，仅将备份集中备份时记录的路径输出。

图例：



下面以脱机备份还原为例说明使用映射文件还原的具体步骤。

1) 启动 RMAN，脱机备份数据库。

```
RMAN> BACKUP DATABASE '/opt/dmdbms/data/DAMENG/dm.ini'
BACKUPSET'/home/dm_bak/db_bak_for_map_01';
```

2) 生成映射文件 map_file_01.txt 至/home/dm_mapfile 目录。此处指定生成映射文件中的数据文件路径与数据库/opt/dmdbms/data/DAMENG_FOR_RESTORE 中的数据文件一致。若不指定，与备份集中的源数据库的数据文件路径一致。

```
RMAN> DUMP BACKUPSET'/home/dm_bak/db_bak_for_map_01' DATABASE
'/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' MAPPED
FILE'/home/dm_mapfile/map_file_01.txt';
```

生成的映射文件内容如下：

```
/***********************************************************/
/***  Delete the unnecessary modified groups              **/
/***  Modify the data_path or mirror_path only in one group  **/
/***********************************************************/
/**=================================================================**/
/*[DAMENG_SYSTEM_FIL_0]*/
fil_id       = 0
ts_id        = 0
ts_name      = SYSTEM
data_path     = /opt/dmdbms/data/DAMENG_FOR_RESTORE/SYSTEM.DBF
mirror_path   =

/**=================================================================**/
/*[DAMENG_ROLL_FIL_0]*/
fil_id       = 0
ts_id        = 1
ts_name      = ROLL
data_path     = /opt/dmdbms/data/DAMENG_FOR_RESTORE/ROLL.DBF
mirror_path   =

/**=================================================================**/
/*[DAMENG_MAIN_FIL_0]*/
fil_id       = 0
ts_id        = 4
ts_name      = MAIN
data_path     = /opt/dmdbms/data/DAMENG_FOR_RESTORE/MAIN.DBF
```

```
mirror_path    =


/***************************** END ****************************/
```

如果需要恢复数据文件或镜像文件到指定路径，可手动编辑映射文件中表空间对应的 data_path 属性。例如，要还原 MAIN 表空间中的数据文件 MAIN.DBF 到/home/dm_dbf 路径下，修改组 DAMENG_MAIN_FIL_0 的内容如下：

```
/*[DAMENG_MAIN_FIL_0]*/
fil_id        = 0
ts_id         = 4
ts_name       = MAIN
data_path     = /home/dm_dbf/MAIN.DBF
```

3) 指定映射文件还原。还原前可选择对备份文件进行校验。

```
RMAN> RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_bak_for_map_01' MAPPED
FILE'/home/dm_mapfile/map_file_01.txt';
```

3.3.5.2.2 数据库恢复

前面章节我们已经对数据库恢复的三种方式进行了简单介绍，更新 DB_MAGIC、从备份集恢复和从归档恢复。本节将介绍数据库恢复中比较典型且复杂的场景，主要内容包括：

- 恢复数据库到指定时间点/LSN
- RAC 环境下的数据库恢复
- 多次故障恢复后使用不同数据库归档恢复

恢复数据库到指定时间点/LSN

恢复数据库到指定时间点/LSN 是从归档恢复的一种方式，也称为不完全恢复。从归档恢复允许恢复到指定的时间点及指定的 LSN 值。若同时指定了时间点和 LSN，则以较早的为结束点。用户可以通过指定一个时间点/LSN，使数据库恢复到这个指定的时间点/LSN。

例如，用户在下午 5 点做了一个误操作，删除了某些重要数据；我们可以指定恢复时间点到下午 4:59 分，恢复被误删除的数据。

下面以联机数据库备份为例说明如何恢复数据库到指定的时间点/LSN。

1) 准备数据。

```
SQL> CREATE TABLE TAB_FOR_RECOVER_01(C1 INT);
SQL>INSERT INTO TAB_FOR_RECOVER_01 VALUES(1);
SQL> COMMIT;
```

2) 备份数据库。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_full_bak_for_time_lsn';
```

3) 正确操作数据库，产生一些归档。

```
SQL> CREATE TABLE TAB_FOR_RECOVER_02(C1 INT);
SQL>INSERT INTO TAB_FOR_RECOVER_02 VALUES(1);
SQL> COMMIT;
```

使用 SELECT SYSDATE 命令查询此时的时间为：2015-11-16 10:56:40.624931；

使用 SELECT FILE_LSN FROM V$RLOG 命令查询此时的 LSN 为：50857

4) 误操作数据库。此步骤误删除了表 TAB_FOR_RECOVER_01 中数据。

```
SQL> DELETE FROM TAB_FOR_RECOVER_01;
SQL>COMMIT;
```

操作步骤同步骤 3，此时的时间和 LSN 分别为：2015-11-16 10:57:20.977265、50861。

5) 还原数据库。步骤 4 为误操作，因此我们需要将数据库恢复到步骤 3 的状态。首先需要还原数据库：

```
RMAN> RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_time_lsn';
```

6) 恢复数据库到指定时间点/LSN。还原后数据库的数据与备份时一致，如果要恢复数据库至步骤 3 的状态可以指定 UNTIL TIME 或 UNTIL LSN 参数重做部分归档。

使用 RECOVER DATABASE...UNTIL TIME 命令恢复到指定的时间：

```
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' WITH
ARCHIVEDIR'/home/dm_arch/arch' USE DB_MAGIC 1447060265 UNTIL TIME'2015-11-16
10:56:40.624931';
```

或者使用 RECOVER DATABASE...UNTIL TIME 命令恢复到指定的 LSN：

```
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' WITH
ARCHIVEDIR'/home/dm_arch/arch' USE DB_MAGIC 1447060265 UNTIL LSN 50857;
```

RAC 环境下的数据库恢复

DM7 支持 RAC 环境下的备份还原。RAC 环境的备份还原与普通的单机不同的是，它包含多个站点（目前仅支持两个站点），有关 RAC 的详细介绍及环境搭建参见《DM7-RAC 配置及操作手册》。下面以从归档恢复为例说明 RAC 环境下的备份恢复：

1) 搭建 RAC 环境。

2) 启动 DIsql，联机备份数据库。备份其中任意一个站点即可备份整个 RAC 环境。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_full_bak_for_rac';
```

3) 还原数据库。还原数据库之前可选择对备份文件进行校验。需要注意的是，待还原的目标库必须也是 RAC 环境，否则不能执行还原操作。

```
RMAN> RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_rac';
```

4) 恢复数据库。假设站点 1 和站点 2 的归档分别在/home/dm_arch1、/home/dm_arc2 目录下。

```
RMAN> RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' WITH
ARCHIVEDIR'/home/dm_arch1','/home/dm_arch2' USE DB_MAGIC 1447060265;
```

多次故障恢复后使用不同数据库归档恢复

在实际应用中可能会遇到以下还原场景：

1) 创建一个数据库 D1。

2) 操作数据库并执行数据库备份 B1。

3) 继续操作数据库的过程中数据库故障，此时生成的归档为 A1。

4) 利用备份 B1 和归档 A1 将数据库 D1 恢复到目标库 D2，此时数据库 D2 为 D1 故障前的状态。

5) 启动数据库 D2，操作数据库过程中数据库第二次故障，此时生成的归档为 A2。

归档 A1、A2 属于不同的数据库，使用备份 B1 和归档 A1、A2 是否可以恢复数据库到第二次故障发生前的状态呢?下面将举例详细说明如何使用不同数据库的归档恢复数据库到最新状态，具体步骤如下：

1) 创建源库 D1，即待备份的数据库，然后启动数据库并配置归档，归档目录为/home/dm_arch/arch。

```
./dminit path=/opt/dmdbms/data
./dmserver /opt/dmdbms/data/DAMENG/dm.ini
```

```
--连接数据库配置归档
SQL>ALTER DATABASE MOUNT;
SQL>ALTER DATABASE ADD ARCHIVELOG 'DEST = /home/dm_arch/arch, TYPE = local,
FILE_SIZE = 1024, SPACE_LIMIT = 2048';
SQL> ALTER DATABASE ARCHIVELOG;
SQL>ALTER DATABASE OPEN;
```

2) 操作数据库的同时备份数据库，备份集为 B1。此处以向表中循环插入数据为例操作数据库。

```
SQL>DROP TABLE TAB_FOR_RECOVER;
SQL>CREATE TABLE TAB_FOR_RECOVER(C1 INT);
SQL>BEGIN
    FOR I IN 1..100000 LOOP
        INSERT INTO TAB_FOR_RECOVER VALUES(I);
        COMMIT;
    END LOOP;
END;
/
--插入数据的同时，另外一个会话备份数据库
SQL>BACKUP DATABASE FULL TO B1 BACKUPSET '/home/dm_bak/B1' DEVICE TYPE DISK
BACKUPINFO 'DAMENG FULL BACKUP ONLINE' MAXPIECESIZE  2048;
```

3) 继续向数据库插入数据一段时间后杀掉服务器，模拟服务器故障。从备份后到服务器故障这段时间的生成的归档称为 A1。

4) 创建目标库 D2，即待还原的库，并配置归档（参考 3.1.2 小节），归档目录与数据库 D1 相同。

```
./dminit path=/opt/dmdbms/data db_name=DAMENG_FOR_RES
```

5) 启动 DMRMAN 工具,利用备份集 B1 和归档 A1 恢复数据库 D2 到 D1 故障前的状态。

```
RMAN>RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RES/dm.ini' FROM BACKUPSET
'/home/dm_bak/B1';
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RES/dm.ini' WITH ARCHIVEDIR
'/home/dm_arch/arch';
```

启动目标库 D2，向数据库中继续插入数据。

```
./dmserver /opt/dmdbms/data/DAMENG/dm.ini
SQL>BEGIN
    FOR I IN 1..100000 LOOP
        INSERT INTO TAB_FOR_RECOVER VALUES(I);
        COMMIT;
    END LOOP;
END;
/
```

6) 插入数据一段时间后，杀掉服务器，模拟服务器第二次故障。数据库 D2 启动到第二次故障之间产生的归档为 A2。

7) 恢复目标库 D2 到第一次故障前的状态。恢复一次，目标库的 DB_MAGIC 都会改变，因此归档 A1、A2 的 DB_MAGIC 不同，属于不同的数据库。第一次恢复只能先重做归档 A1，即恢复到第一次故障前的状态。

```
RMAN>RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RES/dm.ini' FROM BACKUPSET
'/home/dm_bak/B1';
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RES/dm.ini' WITH ARCHIVEDIR
'/home/dm_arch/arch';
```

8) 查看归档 A2 的 DB_MAGIC。

```
./dmrachk arch_fil=/home/dm_arch/arch/ARCHIVE_LOCAL1_20151208085947282.log
rachk V7.1.5.27-Build(2015.12.03-63445trunc)
/*****************************************************************/
/home/dm_arch/arch/ARCHIVE_LOCAL1_20151208085947282.log.
arch_status    : INACTIVE
arch n_rpags   : 7369
arch_db_magic  : 1449535319
arch_pemnt_magic : 1449476774
arch_next_seq  : 7368
arch_lsn       : 39891
arch_seq       : 0
```

```
clsn         : 76829

rarch file len : 3777024

rarch file free: 3777024

rarch create time  : 2015-12- 8  8:59:47

rarch close time  : 2015-12- 8  8:59:47

/*************************************************************/
```

9) 修改目标库 D2 的 DB_MAGIC 与归档 A2 一致。若要利用归档恢复，必须保证数据库的 DB_MAGIC 与归档的 DB_MAGIC 一致。由步骤 8 的查询可知，归档 A2 的 DB_MAGIC 为 1449535319。修改数据库的 DB_MAGIC 需要修改 SYSTEM.DBF 和 DAMENG_FOR_RES01.log 两个文件，步骤如下：

```
--修改文件 SYSTEM.DBF

./dmmdf /opt/dmdbms/data/DAMENG_FOR_RES/SYSTEM.DBF 1

*********************************************************

1 db_magic=1449536573

2 next_trxid=3384

3 pemnt_magic=1449476774

*********************************************************

Please input which parameter you want to change(1-3), q to quit: 1

Please input new value: 1449535319

*********************************************************

1 db_magic=1449535319

2 next_trxid=3384

3 pemnt_magic=1449476774

*********************************************************

Do you want to save the change to file (y/n): y

Save to file success!

--修改文件 DAMENG_FOR_RES01.log

./dmmdf /opt/dmdbms/data/DAMENG_FOR_RES/DAMENG_FOR_RES01.log 2

*********************************************************

1 sig = DMRLOG
```

```
2 ver = 7001

3 chksum = 0

4 dbversion = 0x70009

5 sta = 0

6 n_magic = 35

7 db_magic = 1449536573

8 clsn_fil = 0

10 next_seq = 7369

11 arch_seq = 0

12 len = 268435456

13 free = 4096

14 clsn = 39890

15 clsn_off = 4096

16 arch_lsn = 0

17 pemnt_magic = 1449534719

*********************************************************

You can only reset sta(5) or db_magic (7) or clsn (14) or pemnt_magic(17).

Please input the num which one you want to change, q to quit: 7

Input the new value: 1449535319

*********************************************************

1 sig = DMRLOG

2 ver = 7001

3 chksum = 0

4 dbversion = 0x70009

5 sta = 0

6 n_magic = 35

7 db_magic = 1449535319

8 clsn_fil = 0

10 next_seq = 7369

11 arch_seq = 0
```

```
12 len = 268435456

13 free = 4096

14 clsn = 39890

15 clsn_off = 4096

16 arch_lsn = 0

17 pemnt_magic = 1449534719

************************************************************

Do you want to quit and save the change to file (y/n): y

Save to file success!
```

10)利用归档 A2 恢复数据库至第二次故障前的状态。

```
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RES/dm.ini' WITH ARCHIVEDIR

'/home/dm_arch/arch';

file dm.key not found, use default license!

start redo arch file: /home/dm_arch/arch/ARCHIVE_LOCAL1_20151208085947282.log,

rpages: 7377, start lsn: 39891, end lsn: 76829


total redo pages:7369


end redo arch file: /home/dm_arch/arch/ARCHIVE_LOCAL1_20151208085947282.log

used time: 0s


recover successfully!

recover time used: 7578.080(ms)
```

3.3.6 归档还原

使用 RESTORE 命令完成脱机还原归档操作，在还原语句中指定归档备份集。备份集可以是脱机归档备份集，也可以是联机归档备份集。

语法如下：

```
RESTORE <ARCHIVE LOG | ARCHIVELOG> FROM BACKUPSET '<备份集目录>'
```

```
[DEVICE TYPE DISK|TAPE[PARMS '<介质参数>']]
[IDENTIFIED BY <密码> [ENCRYPT WITH <加密算法>]]
[TASK THREAD <任务线程数>] [NOT PARALLEL]
[ALL | [FROM LSN <lsn 值>] | [UNTIL LSN <lsn 值>] | [LSN BETWEEN < lsn 值> AND <
lsn 值>] | [FROM TIME '时间串'] | [UNTIL TIME '时间串'] | [TIME BETWEEN '时间串' AND
'时间串'] ] TO < 还原目录>
[OVERWRITE level];
<还原目录>:: = ARCHIVEDIR<归档目录>  |  DATABASE 'INI 文件路径'
```

BACKUPSET：指定用于还原目标数据库的备份集目录。若指定为相对路径，会在默认备份目录下搜索备份集。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认为 DISK。DISK 表示备份集存储介质为磁盘，TAPE 表示存储介质为磁带。

PARMS：介质参数，供第三方存储介质管理使用。

IDENTIFIED BY：指定备份时使用的加密密码，供还原过程解密使用。

ENCRYPT WITH：指定备份时使用的加密算法，供还原过程解密使用，若未指定，则使用默认算法。

TASK THREAD：指定还原过程中用于处理解压缩和解密任务的线程个数，取值范围 1~64。若未指定，则默认为 4。

NOT PARALLEL：指定并行备份集使用非并行方式还原。对于非并行备份集，不论是否指定该关键字，均采用非并行还原。

ALL：备份所有的归档。

FROM LSN ， FROM TIME：指定备份的开始 LSN 或者开始的时间点。

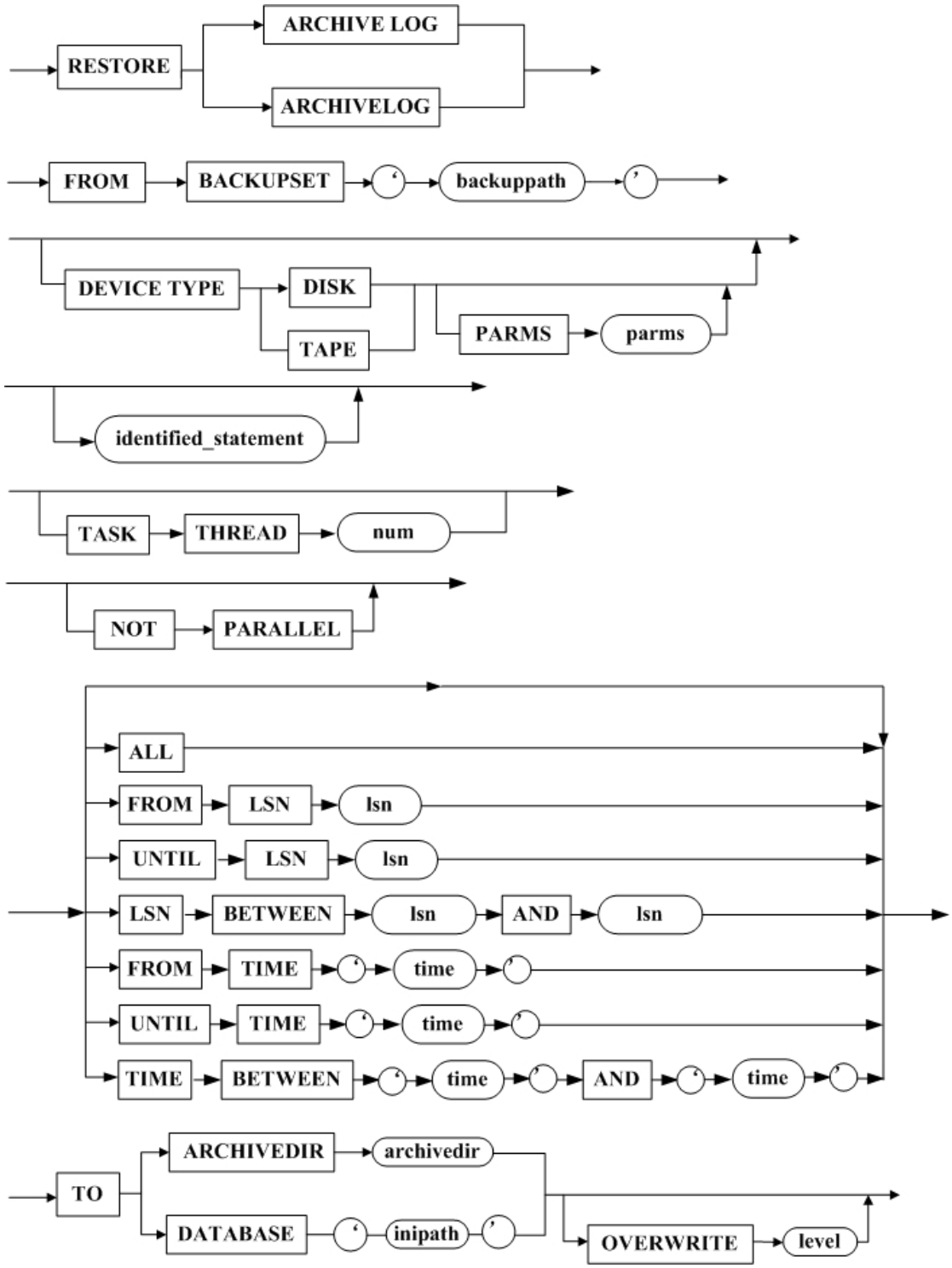
UNTIL LSN， UNTIL TIME：指定备份的截止 LSN 或者截止的时间点。

BETWEEN ... AND ...：指定备份的的区间，仅仅被备份区间内的归档文件。

<还原目录>：用户可以指定还原到指定的目录 ARCHIVEDIR，也可以指定还原指定的库的目录 DATABASE。

OVERWRITE：还原归档时，如果遇到归档已经存在的处理，1：跳过已存在的归档日志，继续其他日志的还原。跳过的信息会生成一条日志记录在安装目录/log/dm_BAKRES_年月.log 里；2：直接报错返回；3：强制覆盖存在的归档日志。缺省为 1。

图例：



identified_statement：请参考3.2.5.1.1节表空间还原图例

使用说明：

1） 联机备份归档，保证数据库运行在归档模式及 OPEN 状态。

```
SQL>BACKUP ARCHIVE LOG ALL BACKUPSET '/home/dm_bak/arch_all_for_restore';
```

2） 校验备份，校验待还原备份集的合法性。校验备份有两种方式，联机和脱机，此

处使用脱机校验。

```
RMAN>CHECK BACKUPSET '/home/dm_bak/arch_all_for_restore';
```

校验成功：

```
RMAN> CHECK BACKUPSET '/home/dm_bak/arch_all_for_restore';

CHECK BACKUPSET '/home/dm_bak/arch_all_for_restore';

CMD END.CODE:[0]

check backup set successfully.

time used: 206.026(ms)
```

3) 还原归档。启动 DMRMAN，设置 OVERWRITE 为 2，如果归档文件已存在，会报错。

```
RMAN>RESTORE ARCHIVE LOG FROM BACKUPSET '/home/dm_bak/arch_all_for_restore' TO
DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' OVERWRITE 2;
```

或者

```
RMAN>RESTORE ARCHIVE LOG FROM BACKUPSET '/home/dm_bak/arch_all_for_restore' TO
ARCHIVEDIR'/opt/dmdbms/data/DAMENG_FOR_RESTORE/arch_dest' OVERWRITE 2;
```

3.3.7 归档修复

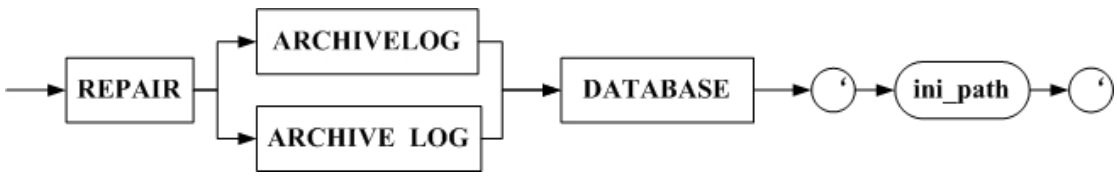
使用 REPAIR 命令完成指定数据库的归档修复，归档修复会将目标库 dmarch.ini 中配置的所有本地归档日志目录执行修复。若目标库没有配置本地归档，则不执行修复。执行修复时，目标库一定不能处于运行状态。一般建议在数据库故障后，应立即执行归档修复，否则后续还原恢复导致联机日志中未刷入本地归档的 REDO 日志丢失，届时再利用本地归档恢复将无法恢复到故障前的最新状态。

语法如下：

```
REPAIR <ARCHIVE LOG | ARCHIVELOG> DATABASE <'INI 文件路径'>;
```

INI 文件路径：指定待修复归档的数据库对应的 dm.ini 路径。

图例：



使用说明：

1） 单机环境时，确保目标库已经停止工作后，执行归档修复。

```
RMAN>REPAIR ARCHIVELOG DATABASE '/opt/dmdbms/data/dm.ini';
```

2） RAC 环境时，需要每个站点停止工作后，且每个站点独立执行修复操作。对于两站点 RAC01、RAC02 执行修复如下。

```
RMAN> REPAIR ARCHIVELOG DATABASE '/opt/dmdbms/rac/dm01.ini';

RMAN> REPAIR ARCHIVELOG DATABASE '/opt/dmdbms/rac/dm02.ini';
```

3.3.8 表空间还原

本节主要介绍表空间的脱机还原。表空间的脱机还原除了支持联机还原的所有功能外，也放开了 SYSTEM 和 ROLL 表空间还原操作。与联机表空间还原不同的是，脱机表空间还原不需要事先置目标表空间为 OFFLINE 状态。

语法如下：

```
RESTORE DATABASE '<INI 文件路径>' TABLESPACE <表空间名>
[DATAFILE < <文件编号> {,<文件编号>} | '<文件路径>' {,'<文件路径>'} >]
FROM BACKUPSET '<备份集目录>'
[DEVICE TYPE DISK|TAPE[PARMS '<介质参数>']]
[IDENTIFIED BY <密码> [ENCRYPT WITH <加密算法>]]
[WITH BACKUPDIR '<基备份集搜索目录>'{,'<基备份集搜索目录>'}]
[<with_archdir_lst_stmt>]
[MAPPED FILE '<映射文件>'][TASK THREAD <任务线程数>] [NOT PARALLEL]
[UNTIL TIME '<时间串>'] [UNTIL LSN <LSN>];
<with_archdir_lst_stmt> ::=
WITH ARCHIVEDIR '<归档日志目录>'{,'<归档日志目录>'}
```

DATABASE：指定还原库目标的 dm.ini 文件路径。

TABLESPACE：指定还原的表空间，只能还原用户表空间。

DATAFILE：还原指定的数据文件。可以指定数据文件编号或数据文件路径。文件编号，对应动态视图 V$DATAFILE 中 ID 列的值；文件路径，对应动态视图 V$DATAFILE 中 PATH 或者 MIRROR_PATH 列的值，也可以仅指定数据文件名称（相对路径），与表空间中数据文件匹配时，会使用 SYSTEM 目录补齐。

BACKUPSET：指定用于还原目标数据库的备份集目录。若指定为相对路径，会在默认备份目录下搜索备份集。

DEVICE TYPE：指存储备份集的介质类型，支持 DISK 和 TAPE，默认为 DISK。DISK 表示备份集存储介质为磁盘，TAPE 表示存储介质为磁带。

PARMS：介质参数，供第三方存储介质管理使用。

IDENTIFIED BY：指定备份时使用的加密密码，供还原过程解密使用。

ENCRYPT WITH：指定备份时使用的加密算法，供还原过程解密使用，若未指定，则使用默认算法。

WITH BACKUPDIR：指定备份集搜索目录。

MAPPED FILE：指定存放还原目标路径的文件，参见 3.3.5.2.1 数据库还原 。当<备份集目录>和<映射文件>指定的路径不一致时，以<映射文件>指定的路径为主

TASK THREAD：指定还原过程中用于处理解压缩和解密任务的线程个数，取值范围 1~64。若未指定，则默认为 4。

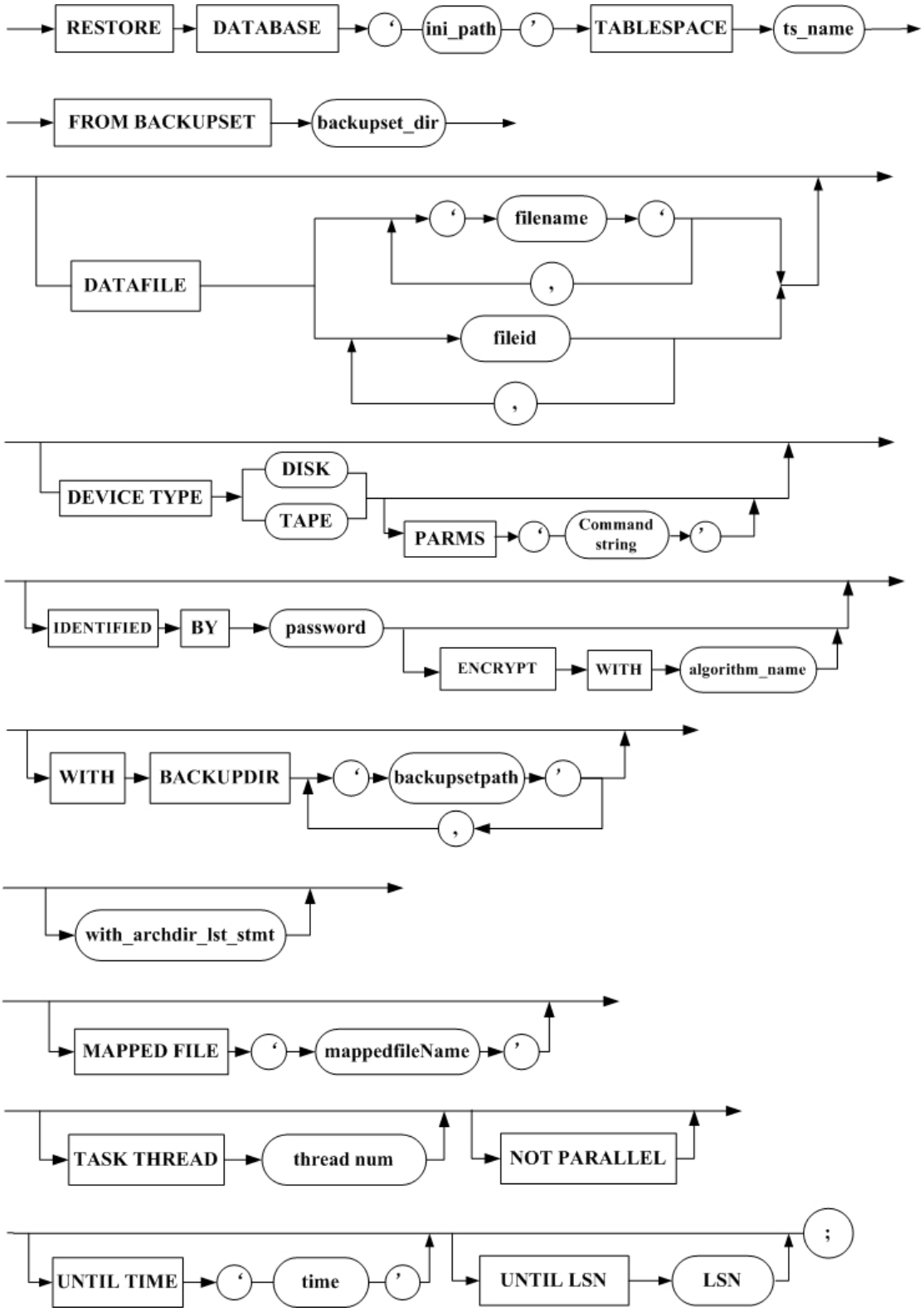
NOT PARALLEL：指定并行备份集使用非并行方式还原。对于非并行备份集，不论是否指定该关键字，均采用非并行还原。

WITH ARCHIVEDIR：本地归档日志搜索目录，若未指定，则仅使用目标库配置的第一个本地归档。

UNTIL TIME：恢复数据库到指定的时间点。

UNTIL LSN：恢复数据库到指定的 LSN。

图例：



<with_archdir_lst_stmt> 请参考 3.3.5.1.2 数据库恢复

使用说明：

1. 表空间还原不能是 TEMP 和 RLOG 表空间，指定文件还原也不能为 TEMP 表空间或者 RLOG 表空间中文件。

2. 表空间还原要求还原目标库与备份库为同一个库，且若还原目标库中 SYSTEM 表空间没有故障且还原目标表空间不是 SYSTEM 表空间的话，则要求目标库校验通过，比如加密校验和日志校验等；否则将使用备份集中信息与当前的进行校验。

3. 还原目标库不能为已经执行过 restore，但是未指定过 recover 的库。

4. SYSTEM 表空间和 ROLL 表空间不允许指定 UNTIL LSN 或者 UNTIL TIME 还原；其他用户表空间可以。若未指定 UNTIL TIME 或者 UNTIL LSN，则均还原到最新状态。使用 UNTIL TIME（或者 UNTIL LSN）的同时不能指定 DATAFILE 还原，否则报语法错误。

5. 处于 RES_OFFLINE 状态的表空间不允许指定表空间中数据文件还原。

6. 整个还原过程中不会修改数据库本身状态或者调整 CKPT_LSN。

7. 不管是 RAC 环境，还是单机环境，若异常退出，需手动指定各站点归档修复后，使用各站点完整的归档日志执行还原恢复；否则，将可能无法恢复到最新。

8. 异常退出库指定 UNTIL TIME 或者 UNTIL LSN 还原失效，因为库故障重启时会执行故障修复，UNTIL 失效，表空间仍会恢复到最新状态。

如 RAC 环境时，需要每个站点停止工作后，且每个站点独立执行备份与还原操作。对于两站点 RAC01、RAC02，表空间的还原操作如下：

```
RMAN>backup database '/home/DMDBMS/rac/dm01.ini' backupset '/home/ DMDBMS
/bak/rman_dm01_01';
RMAN>restore database'/home/DMDBMS/rac/dm01.ini' tablespace main from backupset
'/home/DMDBMS/bak/rman_dm01_01' with archivedir'/home/DMDBMS/rac/arch_dest1';
```

3.3.9 查看操作日志

本节主要介绍整个操作过程中可能涉及的日志或跟踪文件。默认情况下，这些文件都位于与 DM 数据库的 log 目录中。文件名中的“XXX”为生成文件的年月，如 2015 年 5 月生成日志文件其文件名中的“XXX”部分就是“201505”，每个月生成一个日志文件，在该月中日志内容都是不断增加的。操作日志类型包括：

- 全局日志文件 dm_DMSERVER_xxx.log
- 备份还原日志文件 dm_BAKRES__xxx.log
- SBT 接口跟踪文件 dm_SBTTRACE_xxx.log

全局日志文件 dm_DMSERVER_xxx.log

全局日志文件不可配置。dm_DMSERVER_xxx.log 是 DM7 数据库服务器的运行日志，服务器运行过程中的日志信息都包含在这个日志中，备份还原过程中备份还原操作的阶段性信息会记录在这个日志文件中，包括日志生成时间、日志级别、日志生成进程名称和进程 ID、日志生成线程和内容等。

日志格式：时间+日志类型（INFO/WARN/ERROR/FATAL）+进程(database)+进程 ID(Pxxxx)+线程(dm_sql_thd/main_thread 等)+日志内容。

如执行备份时可能会生成如下日志信息：

```
    2016-05-05 11:09:12 INFO database P0000007584 dm_sql_thd  backup database
backupset 'db_bak_new';
    2016-05-05 11:09:12 INFO database P0000007584 dm_sql_thd  CMD START......
    2016-05-05 11:09:25 INFO database P0000007584 dm_sql_thd  CMD END.CODE:[0]
    2016-05-05 11:09:47 INFO database P0000007584 dm_sql_thd  backup archivelog
backupset 'arch_bak_new';
    2016-05-05 11:09:47 INFO database P0000007584 dm_sql_thd  CMD START......
    2016-05-05 11:09:47 INFO database P0000007584 dm_rsyswrk_thd  rfil_close_low
set arch rfil[D:\del1\arch_local\ARCHIVE_LOCAL1_20160505110348437.log]'s sta to
inactive, next_seq = 1421522
    2016-05-05 11:09:47 INFO database P0000007584 dm_rsyswrk_thd  rfil_close_low
set arch rfil[D:\del1\arch_local\ARCHIVE_LOCAL1_20160505110947964.log]'s sta to
inactive, next_seq = 0
    2016-05-05 11:09:47 INFO database P0000007584 dm_rsyswrk_thd
rafil_file_create [D:\del1\arch_local\ARCHIVE_LOCAL1_20160505110947964.log]
success
    2016-05-05 11:10:19 INFO database P0000007584 dm_sql_thd  CMD END.CODE:[0]
    2016-05-05 11:11:23 INFO database P0000007584 dm_sql_thd  backup database
increment backupset 'db_bak_incr2_new';
    2016-05-05 11:11:23 INFO database P0000007584 dm_sql_thd  CMD START......
    2016-05-05 11:11:25 INFO database P0000007584 dm_chkpnt_thd  ckpt info adjust:
ckpt_lsn(288035), ckpt_fil(0), ckpt_off(193608704), cur_lsn(288201)
```

```
    2016-05-05 11:11:25 INFO database P0000007584 dm_chkpnt_thd  ckpt info adjust:
ckpt_lsn(288193), ckpt_fil(0), ckpt_off(193943040), cur_lsn(288201)
    2016-05-05 11:11:25 INFO database P0000007584 dm_chkpnt_thd  ckpt info adjust:
ckpt_lsn(288201), ckpt_fil(0), ckpt_off(194283520), cur_lsn(288201)
    2016-05-05 11:11:29 INFO database P0000007584 dm_sql_thd  CMD END.CODE:[0]
```

备份还原日志文件 dm_BAKRES_xxx.log

备份还原日志文件不可配置。dm_BAKRS_xxx.log 是 DM7 备份还原在执行备份与还原时生成的日志，包括主进程（联机时为 dmserver 工作线程，脱机时为 dmrman）发送，以及子进程处理过程信息。每条信息，包括生成时间、信息级别、生成日志的进程名称和进程 ID，对于子进程还会生成对应主进程的 ID。另外，为了保证日志内容的完整性，还会记录由全局日志生成接口生成的日志记录，这部分日志格式参照全局日志文件说明。

备份还原日志格式为：

时间+日志类型（INFO/WARN/ERROR/FATAL/CMD/CMD_PARSE）+进程（dmrman/database/dmbakres)+进程 ID(P 开头)+主进程 ID(PP 开头,非 dmbakres 显示 4294967295，非法值，否则为相应 dmrman 或者 database 的进程号)+日志内容。

如下所示为一段 dm_BAKRES_xxx.log 日志信息示例：

```
2016-05-19 17:20:02 CMD_PARSE dmrman P0000006556 PPID4294967295  recover
database 'd:\del1_res\dm.ini' from backupset
'd:\del1\dameng\bak\db_bak_rename';
2016-05-19 17:20:02 INFO dmrman P0000006556 T0000000000000006468  fil_sys_init
2016-05-19 17:20:02 INFO dmrman P0000006556 T0000000000000006468  fil_ts_create
ts_id=0
2016-05-19 17:20:02 INFO dmrman P0000006556 T0000000000000006468  fil_ts_create
ts_id=1
2016-05-19 17:20:02 INFO dmrman P0000006556 T0000000000000006468  fil_ts_create
ts_id=2
2016-05-19 17:20:02 INFO dmrman P0000006556 T0000000000000006468  fil_ts_create
ts_id=4
2016-05-19 17:20:02 INFO dmrman P0000006556 T0000000000000006468  fil_ts_create
ts_id=5
```

```
2016-05-19 17:20:02 INFO dmrman P0000006556  PPID4294967295  checking if the
database under system path [d:\del1_res] is running...[4].
2016-05-19 17:20:03 INFO dmrman P0000006556  PPID4294967295  checking if the
database under system path [d:\del1_res] is running...[3].
2016-05-19 17:20:04 INFO dmrman P0000006556  PPID4294967295  checking if the
database under system path [d:\del1_res] is running...[2].
2016-05-19 17:20:05 INFO dmrman P0000006556  PPID4294967295  checking if the
database under system path [d:\del1_res] is running...[1].
2016-05-19 17:20:06 INFO dmrman P0000006556  PPID4294967295  checking if the
database under system path [d:\del1_res] is running...[0].
2016-05-19 17:20:08 INFO dmrman P0000006556  PPID4294967295  checking if the
database under system path [d:\del1_res] is running, write dmrman info.
2016-05-19 17:20:08 INFO dmrman P0000006556 T0000000000000006468
rfil[d:\del1_res\DAMENG01.log] sta: 1, arch_lsn = 0, arch_seq = 0, clsn = 15551333,
next_seq = 0, free = 4096
2016-05-19 17:20:08 INFO dmrman P0000006556 T0000000000000006468  main
rfil[d:\del1_res\DAMENG01.log]'s sta is inactive


2016-05-19 17:20:08 CMD dmrman P0000006556 PPID4294967295  recover database
'd:\del1_res\dm.ini' from backupset 'd:\del1\dameng\bak\db_bak_rename';
2016-05-19 17:20:08 INFO dmrman P0000006556  PPID4294967295  CMD START....
2016-05-19 17:20:08 INFO dmrman P0000006556  PPID4294967295  create
channel[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] and subprocess,
code: [0].
2016-05-19 17:20:08 INFO dmrman P0000006556  PPID4294967295  send CMD [BMML_INIT]
by channel [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129].
2016-05-19 17:20:08 INFO dmrman P0000006556  PPID4294967295  RESTORE RLOG
CHECK......
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID4294967295  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
```

```
CMD [BMML_INIT].
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID4294967295  file dm.key not
found, use default license!
2016-05-19 17:20:08 ERROR dmbakres P0000004596 T0000000000000003684
os_file_delete file error! name: d:\del1_res\rep_debug.log, code: 2, desc: No
such file or directory
2016-05-19 17:20:08 INFO dmbakres P0000004596 T0000000000000003684  fil_sys_init
2016-05-19 17:20:08 INFO dmbakres P0000004596 T0000000000000003684
fil_ts_create ts_id=0
2016-05-19 17:20:08 INFO dmbakres P0000004596 T0000000000000003684
fil_ts_create ts_id=1
2016-05-19 17:20:08 INFO dmbakres P0000004596 T0000000000000003684
fil_ts_create ts_id=2
2016-05-19 17:20:08 INFO dmbakres P0000004596 T0000000000000003684
fil_ts_create ts_id=4
2016-05-19 17:20:08 INFO dmbakres P0000004596 T0000000000000003684
fil_ts_create ts_id=5
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID0000006556  CMD [BMML_INIT]
is ready
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0]
2016-05-19 17:20:08 INFO dmrman P0000006556  PPID4294967295  send CMD
[BMML_CFG_FOR_NON_BACKUP] by channel
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129].
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [BMML_CFG_NON_BACKUP].
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID0000006556  CMD
[BMML_CFG_NON_BACKUP] is ready
```

```
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0]
2016-05-19 17:20:08 INFO dmrman P0000006556  PPID4294967295  send CMD
[RESTORE_START] by channel
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129].
2016-05-19 17:20:08 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [RESTORE_START].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  CMD
[RESTORE_START] is ready
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0],
DATA_LEN[90719]
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  BPSP notify
subprocess complete, pipe send successfully
_3_p6556_-3689348817886803129] success, CMD [COMPLETE], PIPE_RESUSE [TRUE].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [COMPLETE] AND PIPE_RESUSE.
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0]
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  RESTORE RLOG ,gen
tmp file......
2016-05-19 17:20:09 INFO dmrman P0000006556 T0000000000000006468  rfil_close_low
set arch
rfil[d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log]'s
sta to inactive, next_seq = 0
```

```
2016-05-19 17:20:09 INFO dmrman P0000006556 T0000000000000006468
rafil_file_create
[d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log]
success
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  RESTORE RLOG FROM
BACKUPSET [d:\del1\dameng\bak\db_bak_rename] START......
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  send CMD
[RESTORE_SET_DBFS] by channel
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [RESTORE_SET_DBFS].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  CMD
[RESTORE_SET_DBFS] is ready
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0]
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  send CMD
[RESTORE_INIT] by channel
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], bakset_dir:
[d:\del1\dameng\bak\db_bak_rename].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [RESTORE_INIT].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  CMD [RESTORE_INIT]
is ready
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0]
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  send CMD
```

```
[RESTORE_DBF] by channel
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], bakset_dir:
[d:\del1\dameng\bak\db_bak_rename].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [RESTORE_RLOG].
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  CMD [RESTORE_RLOG]
restore BACKUPSET[d:\del1\dameng\bak\db_bak_rename]......
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  start restore
backup set d:\del1\dameng\bak\db_bak_rename ...
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  start restore temp
rlog file:
d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log, rpages:
226
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  end restore file:
d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log
2016-05-19 17:20:09 INFO dmbakres P0000004596 T0000000000000004156
rfil_close_low set arch
rfil[d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log]'s
sta to inactive, next_seq = 0
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  end restore rlog
from backup set d:\del1\dameng\bak\db_bak_rename, code:[0]
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  CMD [RESTORE_RLOG]
is ready
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0]
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [COMPLETE], PIPE_RESUSE [TRUE].
```

```
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  BPSP notify
subprocess complete, pipe send successfully
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [COMPLETE] AND PIPE_RESUSE.
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres send
EXECUTE OVER status by named pipe
[\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129], CODE [0]
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres receive
from named pipe [\\.\pipe\DM_PIPE_NUMBER_3_p6556_-3689348817886803129] success,
CMD [COMPLETE], PIPE_RESUSE [FALSE].
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  BPSP notify
subprocess complete, pipe send successfully
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  code = 0, bakres
CMD [COMPLETE] begin...
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres CMD
[COMPLETE] tsk node get...
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556
brtsk_tsk_complete begin...
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556
brtsk_tsk_complete end
2016-05-19 17:20:09 INFO dmbakres P0000004596  PPID0000006556  bakres CMD
[COMPLETE] end.
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  CMD END.CODE:[0]
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  redo tmp archive log
of backup set d:\del1\dameng\bak\db_bak_rename\db_bak_rename.meta ...
2016-05-19 17:20:09 INFO dmrman P0000006556 T0000000000000006468  start redo arch
file: d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log,
rpages: 233, start lsn: 15551334, end lsn: 15552769
2016-05-19 17:20:09 INFO dmrman P0000006556 T0000000000000006468  try to apply
```

```
archive log from LSN: 15551334 to LSN: 15552769.
2016-05-19 17:20:09 INFO dmrman P0000006556 T0000000000000006468  apply archive
log d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log, redo
1568 ptxes.
2016-05-19 17:20:09 INFO dmrman P0000006556 T0000000000000006468  end redo arch
file: d:\del1\dameng\bak\db_bak_rename\RES2_TMP_ARCH_20160519172009017.log
2016-05-19 17:20:09 INFO dmrman P0000006556 T0000000000000006468  used time: 0s
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  RECOVER LSN
(15551334, 15552769).
2016-05-19 17:20:09 INFO dmrman P0000006556  PPID4294967295  end redo archive log
of backup set d:\del1\dameng\bak\db_bak_rename\db_bak_rename.meta ...
2016-05-19 17:20:11 INFO dmrman P0000006556 T0000000000000006468
fil_sys_destroy
2016-05-19 17:20:11 INFO dmrman P0000006556 T0000000000000006468  fil_ts_drop
ts_id=0
2016-05-19 17:20:12 INFO dmrman P0000006556 T0000000000000006468  fil_ts_drop
ts_id=1
2016-05-19 17:20:12 INFO dmrman P0000006556 T0000000000000006468  fil_ts_drop
ts_id=2
2016-05-19 17:20:12 INFO dmrman P0000006556 T0000000000000006468  fil_ts_drop
ts_id=4
2016-05-19 17:20:12 INFO dmrman P0000006556 T0000000000000006468  fil_ts_drop
ts_id=5
2016-05-19 17:20:12 INFO dmrman P0000006556  PPID4294967295  recover
successfully!
2016-05-19 17:20:12 INFO dmrman P0000006556  PPID4294967295  time used:
9795.051(ms)


2016-05-19 17:20:17 CMD_PARSE dmrman P0000006556 PPID4294967295  exit
2016-05-19 17:20:17 INFO dmrman P0000006556  PPID4294967295  time used: 0.001(ms)
```

SBT 接口跟踪文件 dm_SBTTRACE_xxx.log

SBT 接口跟踪文件可配置，相关 TRACE 文件详见 3.2.3 数据备份高级主题 。SBT 接口在打开 TRACE 跟踪（TRACE LEVEL > 1）的情况下生成执行日志，默认文件名为 dm_SBTTRACE_xxx.log，用户可根据需要自行设置文件名。

如下所示为一段 dm_SBTTRACE_xxx.log 日志信息示例：

```
2015-05-05 17:02:28 start restore backup set
d:\dbtest\vdata\DAMENG\bak\ts2_bak ...
2015-05-05 17:02:28 start restore file: d:\dbtest\vdata\DAMENG\ts2.dbf(TS2),
pages: 4
2015-05-05 17:02:28 end restore file: d:\dbtest\vdata\DAMENG\ts2.dbf(TS2)
2015-05-05 17:02:28 start restore file: d:\dbtest\vdata\DAMENG\ts2_1.dbf(TS2),
pages: 1
2015-05-05 17:02:28 end restore file: d:\dbtest\vdata\DAMENG\ts2_1.dbf(TS2)
2015-05-05 17:02:28 end restore backup set d:\dbtest\vdata\DAMENG\bak\ts2_bak,
code:[0]
```

3.4 使用图形化客户端工具进行备份还原

DM7 数据库提供几种类型的图形化客户端管理工具。在 WINDOWS 的 DM7 数据库系统，可以通过选择“开始”-“程序”-“达梦数据库”-“客户端”，然后选择具体的某个客户端工具。比如，通过 MANAGER 管理工具可以与数据库进行交互，操作数据库对象和从数据库获取信息；通过控制台工具 CONSOLE 可以完成数据库的配置与监控等。下文分别介绍使用 MANAGER 管理工具联机执行备份还原和使用 CONSOLE 控制台工具脱机执行备份还原。

3.4.1 使用 MANAGER 工具进行联机备份还原

下面将介绍如何使用 DM7 的 MANAGER 管理工具来执行联机的备份与还原的操作。MANAGER 管理工具主界面如图 3.1 所示：

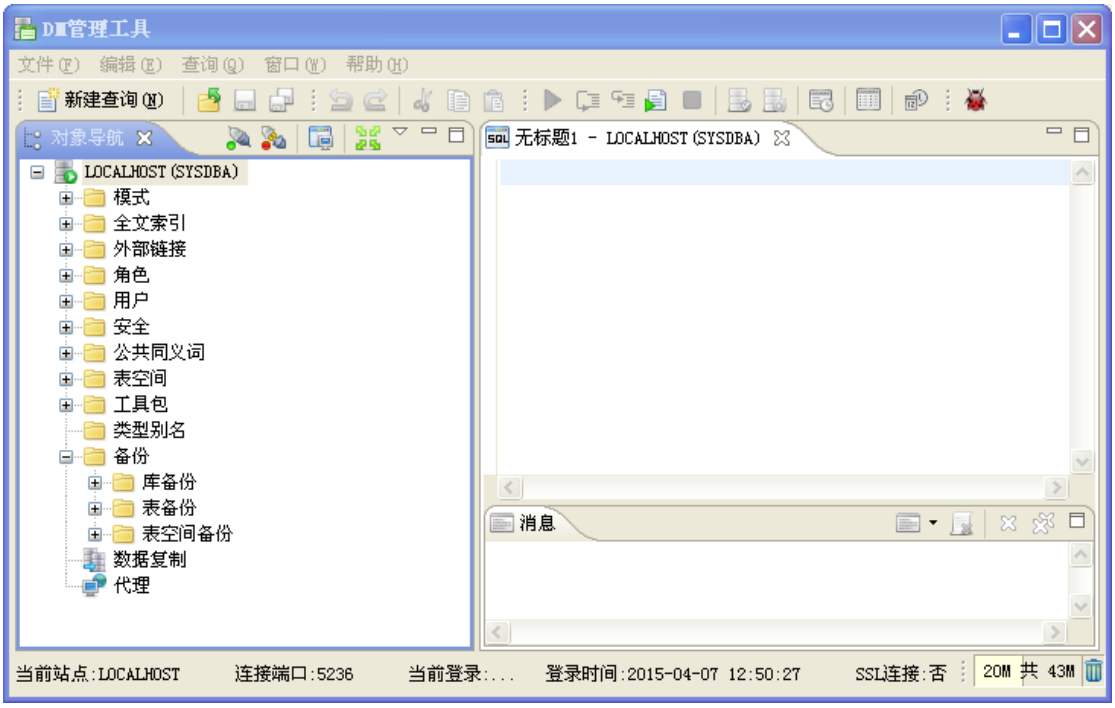


图 3.1 MANAGER 工具主页面

MANAGER 工具的对象导航树中的备份节点提供了联机备份还原相关的各项操作。导航树备份节点下面包含三个子节点：库备份、表备份和表空间备份。各备份节点的右键菜单中提供了所有可操作的选项，包括新建备份、指定工作目录、备份恢复、备份校验、属性等。备份文件夹节点右键菜单如下：



图 3.2 备份节点操作选项

备份节点右键菜单如下：



图 3.3 备份集节点操作选项

3.4.1.1 数据备份

数据备份包括库备份、表备份和表空间备份，分别对应备份文件夹节点中库备份、表备份和表空间备份的子节点。点开各子节点，显示当前已经存在的相应备份集名称。三者的备份流程基本一致，只是具体可配置的参数有些不同，下面以库备份为例介绍。

库备份节点右键菜单->新建备份，可以打开新建库备份对话框，首页显示的是常规选项页面，如图 3.4 所示：

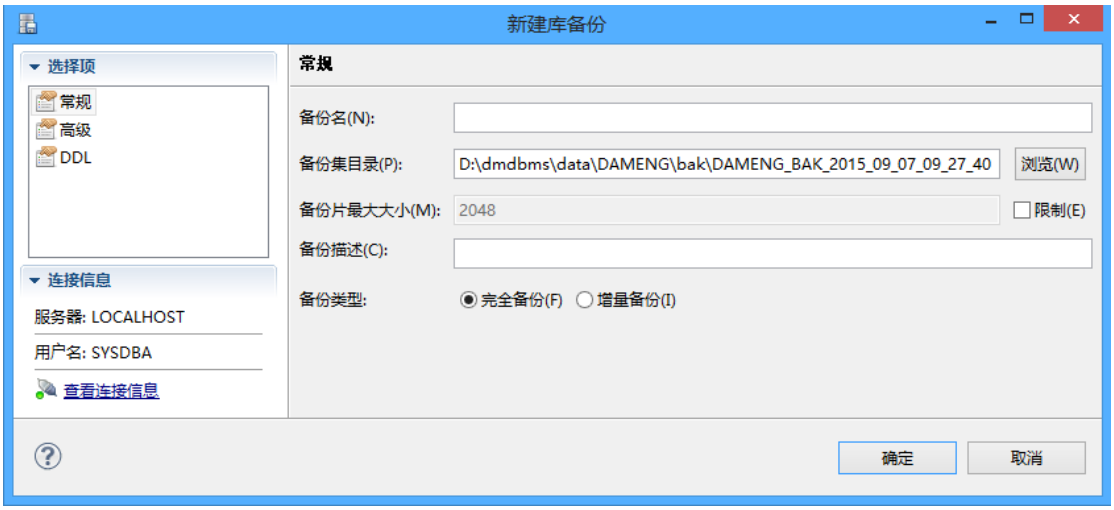


图 3.4 数据库备份常规选项页面

常规页面可配置库备份的常规参数信息，具体如下：

备份名：备份的名称。

备份集目录：备份集的目录，默认为系统默认备份集目录，即 DM 数据库安装目录下数据文件目录加备份集名称。

备份片最大大小：备份片的最大大小，如果备份文件超过该大小，则系统自动将备份文件分为多个文件。以兆为单位。不限制时，系统默认备份文件的最大大小为 2048M 。限制时，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

备份描述：对备份文件的描述，用户可以根据需要填写。

备份类型：

- ✓ 完全备份：对数据库的完全备份，不依赖于其他备份，可以独立地进行数据库恢复。
- ✓ 增量备份：在完全备份或增量备份的基础上进行的备份。进行增量备份时，允许用户自己指定基备份来进行备份，默认使用最近一次备份作为基备份，在该数据库不存在任何备份时，备份类型不能为增量备份。

库备份和表空间备份都可指定备份类型为增量备份，如果选择增量备份，则还有如图 3.5 所示的增量备份相关选项：

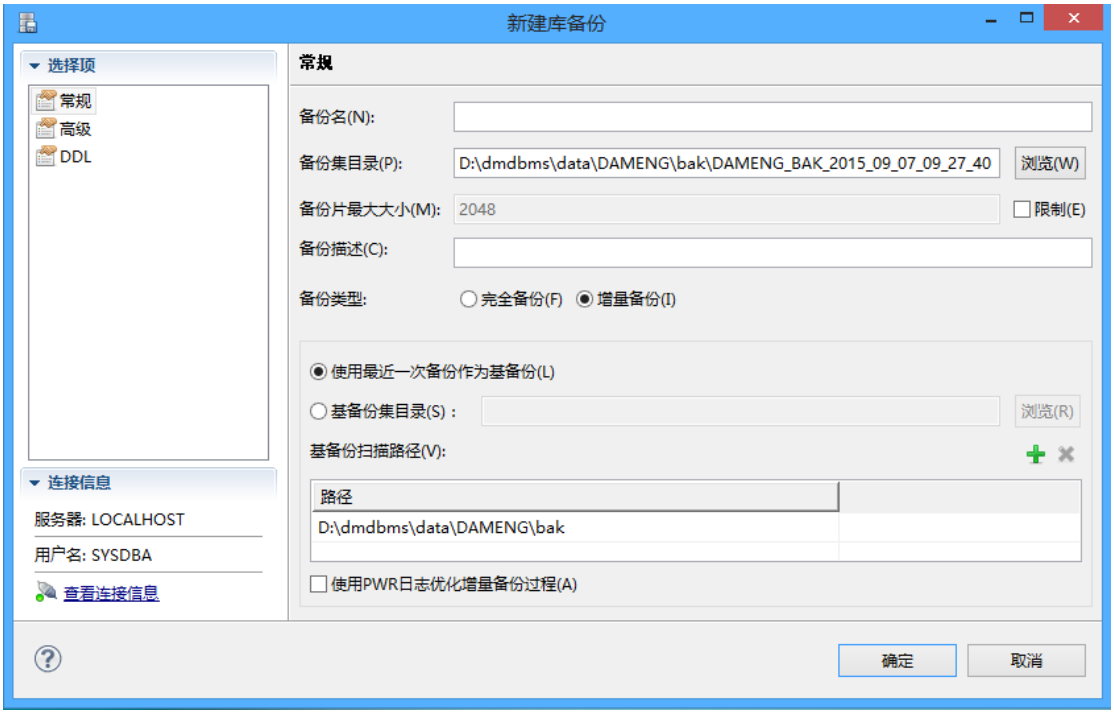


图 3.5 数据库备份增量备份页面

可以为增量备份配置以下参数：

指定基备份：可以指定基备份集目录，默认系统会自己在指定的基备份扫描路径下查找最近一次备份作为基备份。

基备份扫描路径：指定基备份的扫描路径，可以同时指定多个，增量备份时必须保证依赖的所有基备份都可以在指定的基备份扫描路径中找到。

PWR 日志优化：是否使用 PWR 日志优化增量备份。

高级选项页面中可以配置一些其他的高级选项，如图 3.6 所示：

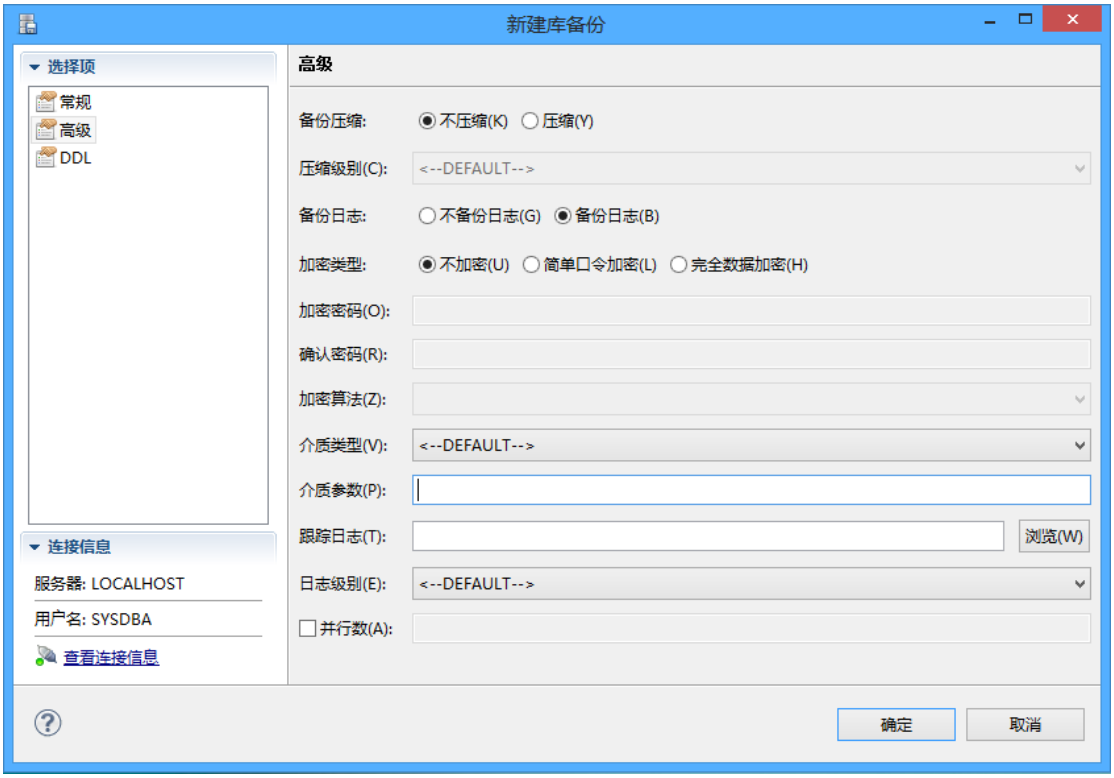


图 3.6 数据库备份高级选项页面

可为备份配置以下高级参数：

备份压缩：可以选择压缩或不压缩，通过压缩可以减少备份文件所需要的磁盘空间。默认为不压缩。

压缩级别：压缩的级别。可以设置压缩级别 0~9，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。

备份日志 ：是否需要备份归档日志中的内容。默认备份日志。

加密类型：加密的类型，默认为不加密。

- ✓ 不加密：不对备份文件进行加密处理。
- ✓ 简单口令加密：对备份文件设置口令，但文件内容仍以明文存储。
- ✓ 完全数据加密：对备份文件进行完全的加密，备份文件以密文方式存储。

加密密码：当选择加密时，需要输入加密密码。

确认密码：当选择加密时，需要确认输入的加密密码。

加密算法：加密时使用的算法。在选择简单口令加密和完全数据加密的情况下适用。支持的加密算法参考数据库备份章节的参数说明。

介质类型：指定存储备份集的设备类型，默认选择磁盘，目前暂支持磁盘和磁带。

介质参数：使用第三方介质类型时使用的参数。

跟踪日志：备份过程的跟踪日志路径。系统有默认路径，当日志级别为记录跟踪日志时启用。

日志级别：是否记录跟踪日志。默认不记录。

并行数：指定是否执行并行备份，以及执行并行备份的并行数。并行数没有上限，但实际可使用的的最大并行数就是目标备份数据文件的个数。若勾选但没指定并行数，则默认并行数为 4。若未勾选，则认为非并行备份。

DDL 页面可以查看新建库备份的 SQL 语句，右下角的保存按钮也可以保存 SQL 脚本到文件中，如图 3.7 所示：

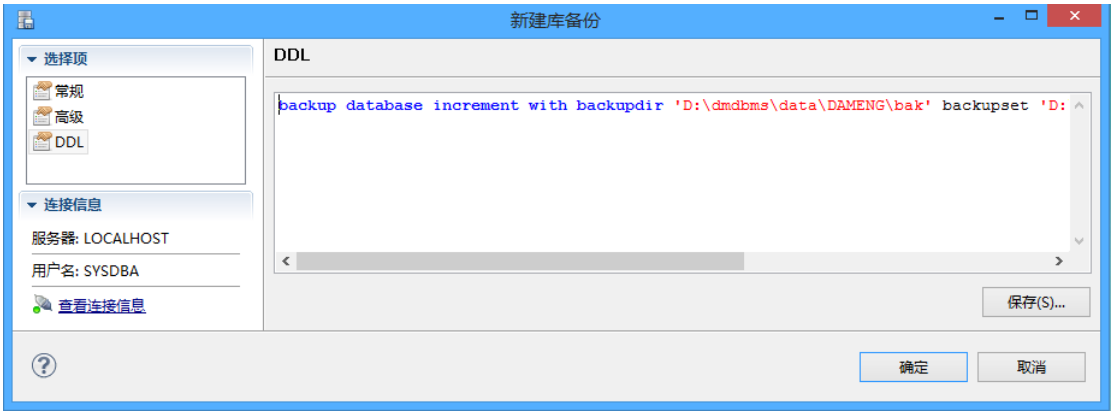


图 3.7 数据库备份查看 DDL 页面

3.4.1.2 备份管理

备份管理包括备份集信息、备份校验、备份删除和指定工作目录，其中备份删除是备份管理的主要功能。下面将分别对这些功能的使用进行介绍。

备份集查看

点击某个备份节点右键菜单->属性，可以打开备份属性查看对话框，属性列表中可以查看该备份集的信息，包括备份集的元数据信息集数据库信息，分别如图 3.8、图 3.9 所示：

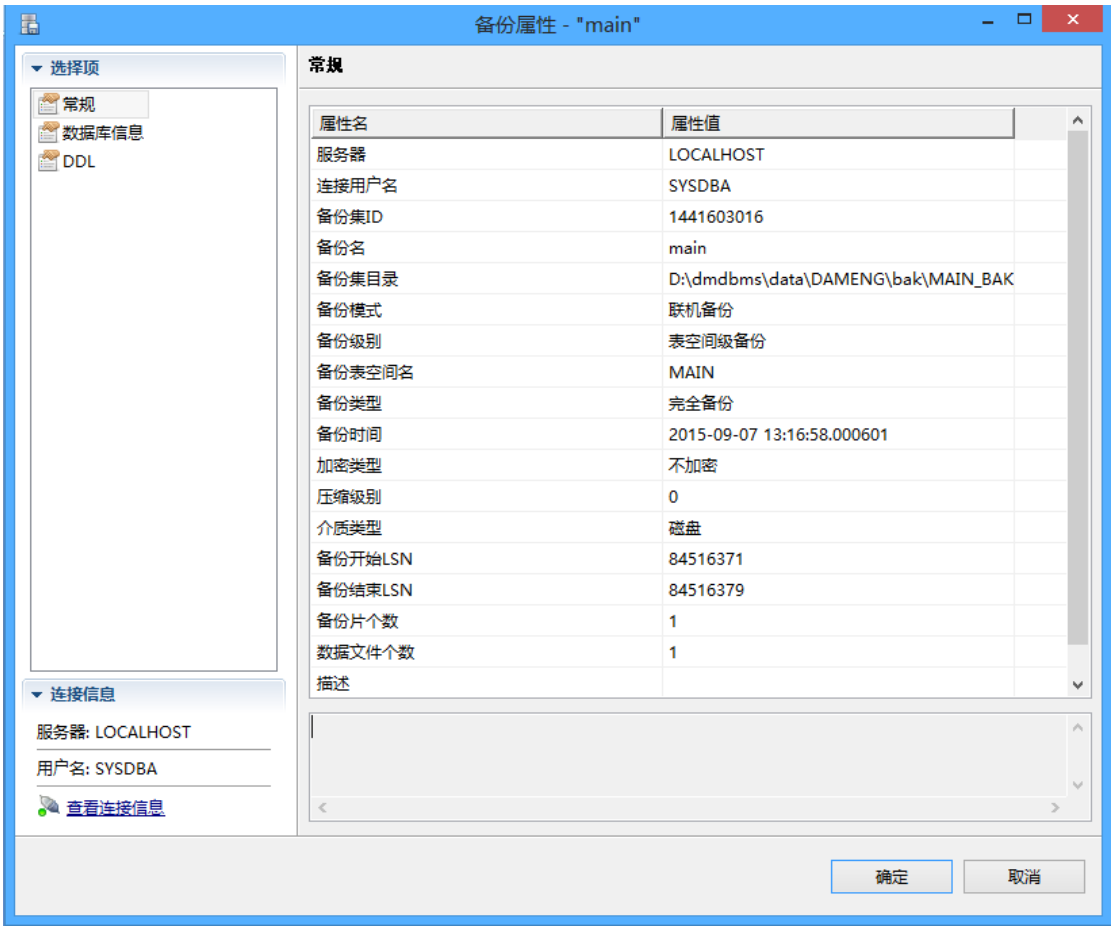


图 3.8 备份集常规信息显示页面

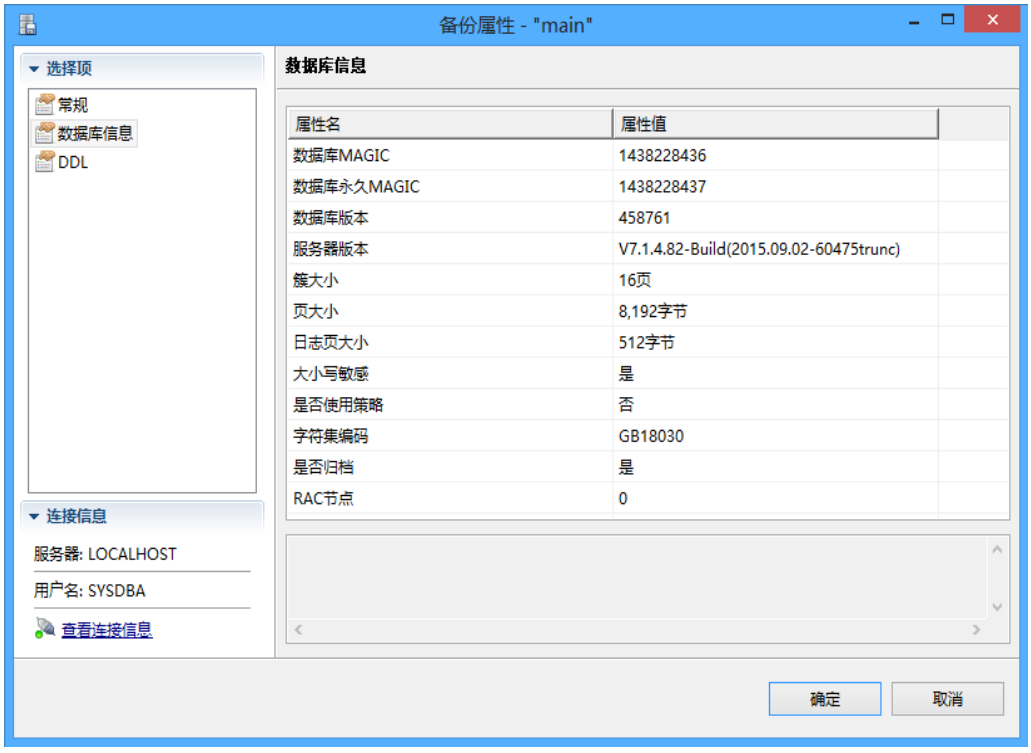


图 3.9 备份集数据库信息显示页面

备份校验

点击某备份节点右键菜单->备份校验，即可校验备份集的合法性。

备份删除

点击某备份节点右键菜单->删除，可以删除该备份集。也可以同时选中多个备份节点进行批量删除。

指定工作目录

点击备份文件夹节点右键菜单->指定工作目录，可以指定备份的工作目录，允许同时指定多个。如果设置了备份工作目录，备份文件夹下会显示所有工作目录下的所有备份。指定工作目录的对话框如下：

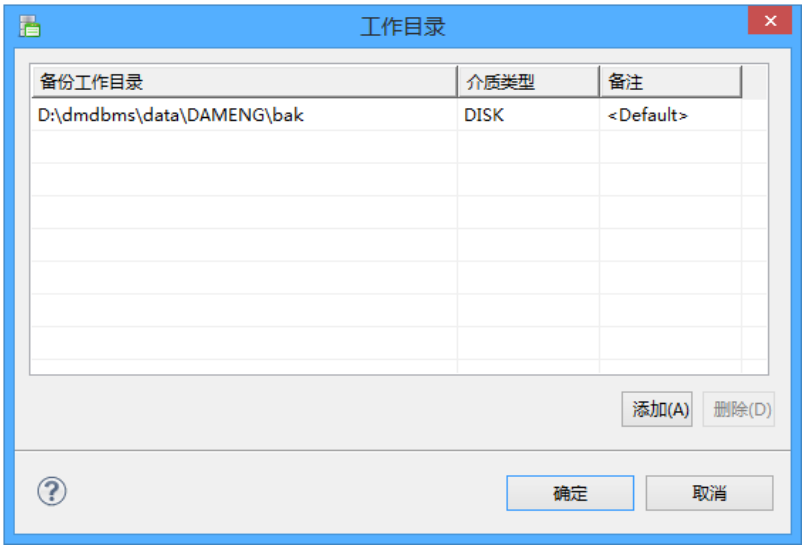


图 3.10 指定工作目录页面

点击添加按钮可以添加一个工作目录，选中一个工作目录点击删除按钮可以删除一个工作目录。系统有一个默认工作目录，该目录会一直保留不被删除。

3.4.1.3 数据还原

库备份不支持联机还原，表空间备份和表备份支持联机还原。

表空间还原时，首先应将表空间脱机，使待还原表空间处于 OFFLINE 状态；然后执行还原；还原成功后再将表空间联机，使其恢复 ONLINE 状态。还原前后 MANAGER 工具会默认地脱机和联机表空间，不需要用户执行这些操作。还原过程中表空间不能对外提供工作。如果还原失败，表空间会处于异常警告状态，需要管理员干预恢复到正常状态才能再次对外提供工作。

表还原可直接进行，且表还原过程中表空间中其他的表还可以正常操作。

除此之外，表空间和表的还原操作流程基本类似，下面以表空间还原为例进行说明。

表空间备份节点右键菜单->备份恢复，可以打开表空间备份恢复对话框的常规页面如图 3.11 所示：

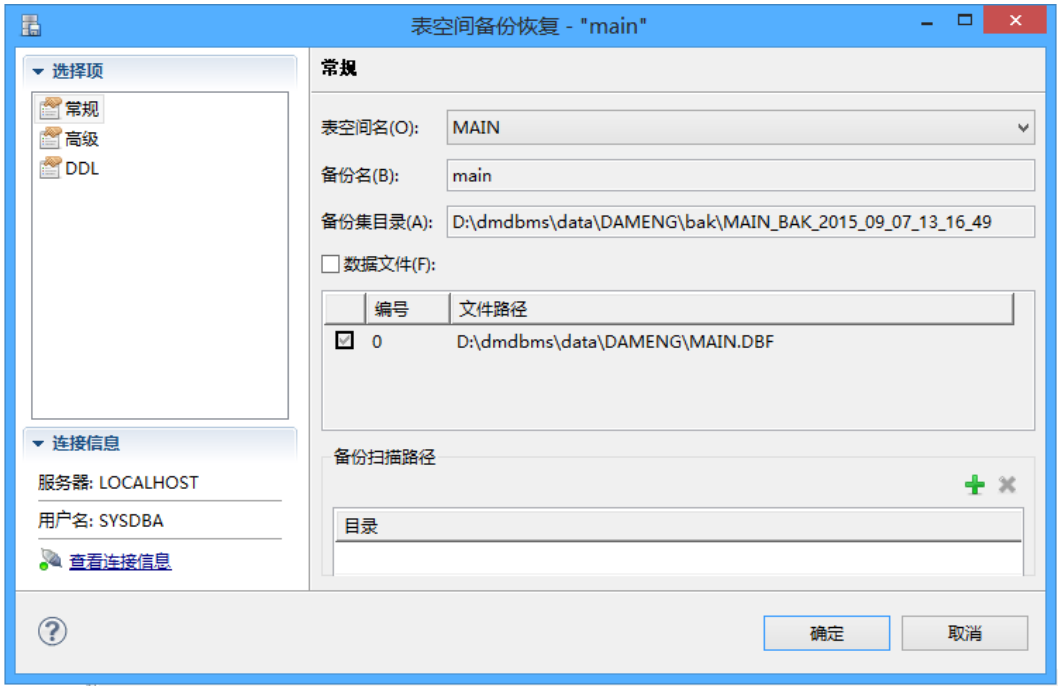


图 3.11 表空间恢复常规选项页面

可为表空间还原配置以下常规参数：

表空间名：表空间的名称。

备份名：备份的名称，无需填写，系统自动获取。

备份集目录：备份集所在的目录，无需填写，系统自动获取。

数据文件：表空间中指定还原数据文件路径。若不指定，默认全部还原到原来的路径。

备份扫描路径：如果还原的备份为增量备份，则必须指定备份扫描路径，否则可能搜索不到基备份导致无法还原。

高级选项页面如下：

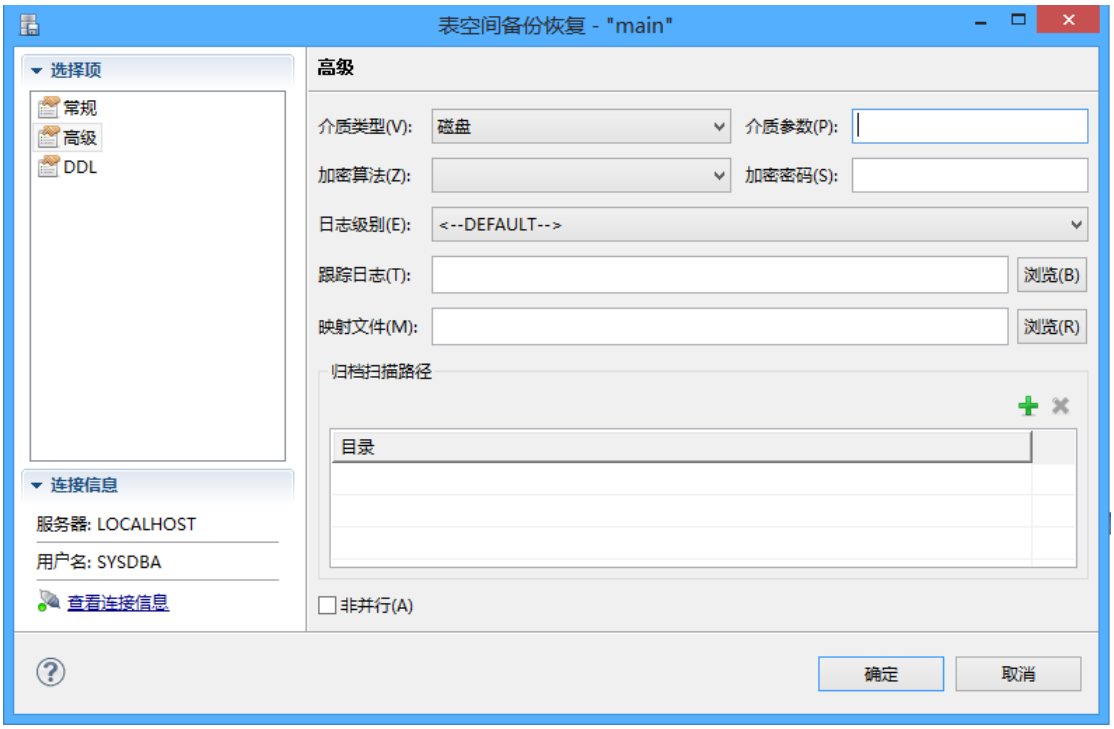


图 3.12 表空间恢复常规选项页面

可配置以下参数：

介质类型：指存储备份集的介质类型，暂支持 DISK 和 TAPE，默认 DISK。

介质参数：使用第三方介质类型时使用的参数。只对介质类型为 TAPE 时有效。

加密密码/加密算法：如果备份时使用了加密，则还原时必须输入加密密码和加密算法。

日志级别：是否记录跟踪日志。有效值 1、2，默认为 1 表示不启用 trace，为 2 表示启用 trace。

跟踪日志：指定生成的 trace 文件名。

映射文件：指定存放还原目标路径的文件，参见 DUMP 命令中的<映射文件>。

归档扫描路径：还原时，搜集归档文件的目录。

非并行：若勾选，则认为并行备份不使用并行还原，默认并行备份执行并行还原。对于非并行备份集还原，则忽略。

DDL 页面同新建备份，可以查看表空间备份集还原的 SQL 语句，右下角的保存按钮也可以保存 SQL 脚本到文件中。

3.4.2 使用 CONSOLE 工具进行脱机备份还原

下面将介绍如何使用 DM 控制台工具 CONSOLE 来执行库级脱机备份与还原的操作。控

制台主界面如图 3.13 所示。

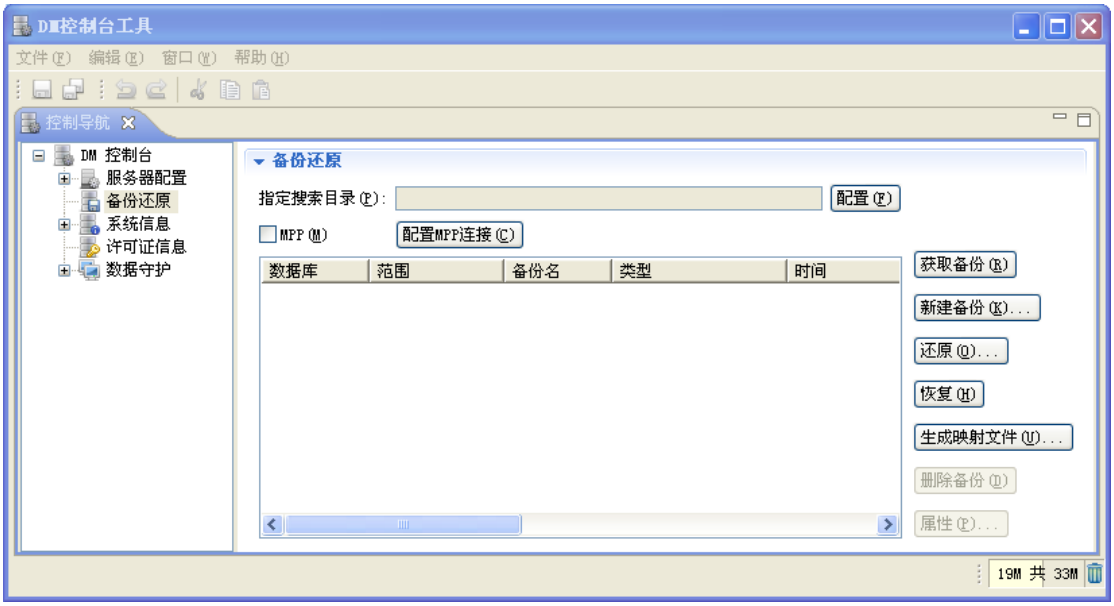


图 3.13 CONSOLE 工具备份还原主页面

3.4.2.1 数据备份

CONSOLE 工具的控制导航树中提供备份还原节点，单击备份还原节点可以打开备份还原管理界面。在备份还原管理界面中，点击新建备份按钮。打开新建备份对话框，如图 3.14 所示。

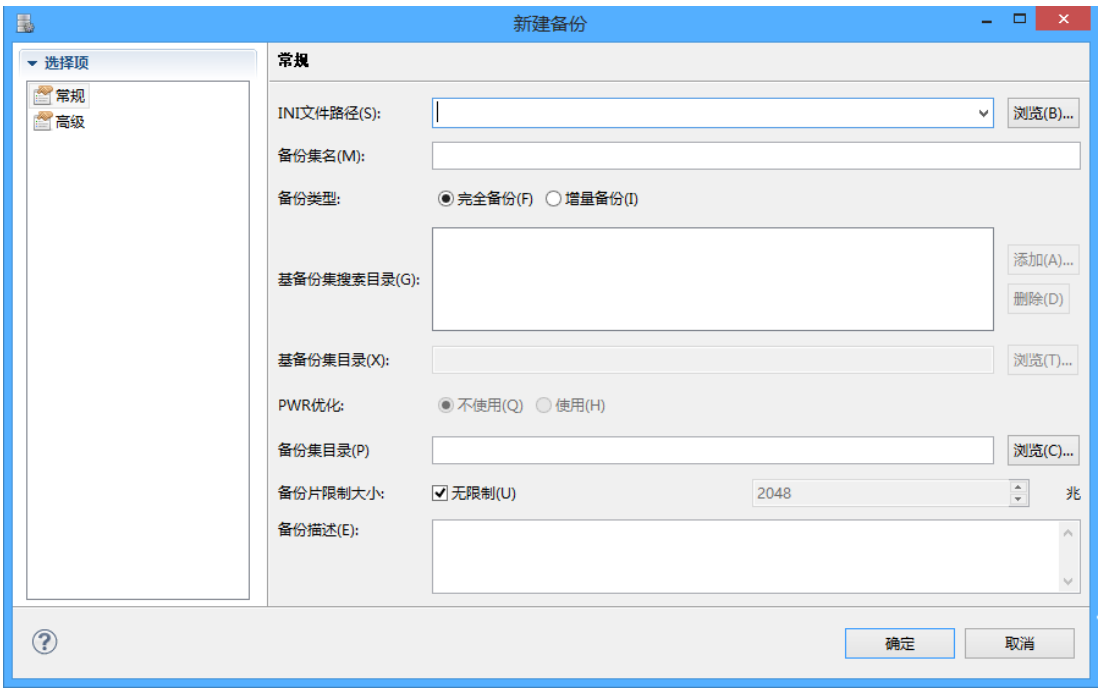


图 3.14 数据库备份常规选项页面

需要为脱机备份配置以下参数：

INI 文件路径：必填，待备份目标数据库 dm.ini 文件路径。

备份集名：选填，指定生成备份集名称，若未指定，则使用约定方法随机生成。

备份类型：默认选完全备份，在已经存在完全备份的情况下，可以选择增量备份。

基备份搜索目录：选填，并且仅适用于增量备份。

基备份集目录：选填，并且仅适用于增量备份。

PWR 优化：仅适用于增量备份，默认不使用。

备份集目录 ：选填，指定当前备份集生成目录，若未指定，则在默认备份目录中生成备份集目录。

备份片限制大小：默认无限制，取消选中无限制，则可以设置备份片的限制大小，以兆为单位，最小 128M，32 位系统最大 2G，64 位系统最大 128G。

备份描述：选填，根据实际情况对备份集进行描述。

根据实际需要配置完上述参数后，点击确定按钮就开始执行脱机备份操作了。备份操作可能需要花费一些时间，备份结束后，如果备份成功则弹出备份成功的对话框；如果备份失败，则弹出对话框提示失败原因。

高级选项页面如下：

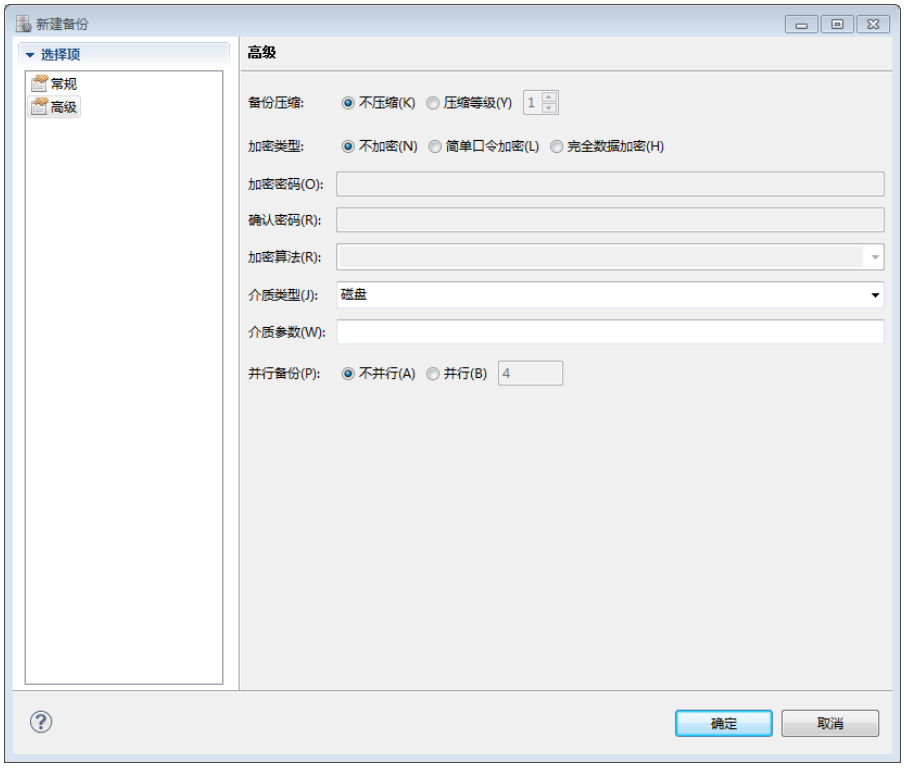


图 3.15 数据库备份高级选项页面

可为脱机备份配置以下参数：

备份压缩：默认不压缩。选择压缩等级，则可以设置压缩级别 0~9，1 表示 1 级压缩，9 表示 9 级压缩。压缩级别越高，压缩越慢，但压缩比越高。

加密类型：默认不加密。

加密密码：在选择简单口令加密和完全数据加密的情况下，需输入加密密码。

加密算法：在选择简单口令加密和完全数据加密的情况下适用。

介质类型：指定存储备份集的设备类型，默认选择磁盘，目前暂支持磁盘和磁带。

介质参数：使用第三方介质类型时使用的参数。

并行备份：指定是否执行并行备份，以及执行并行备份的并行数。并行数没有上限，但实际可使用的的最大并行数就是目标备份数据文件的个数。若勾选但没指定并行数，则默认并行数为 4。若未勾选，则认为非并行备份。

3.4.2.2 备份管理

查看备份集

在备份还原管理页面中，指定搜索目录，点击获取备份按钮，即可获取备份集列表。操作步骤如图 3.16、图 3.17、图 3.18 所示。

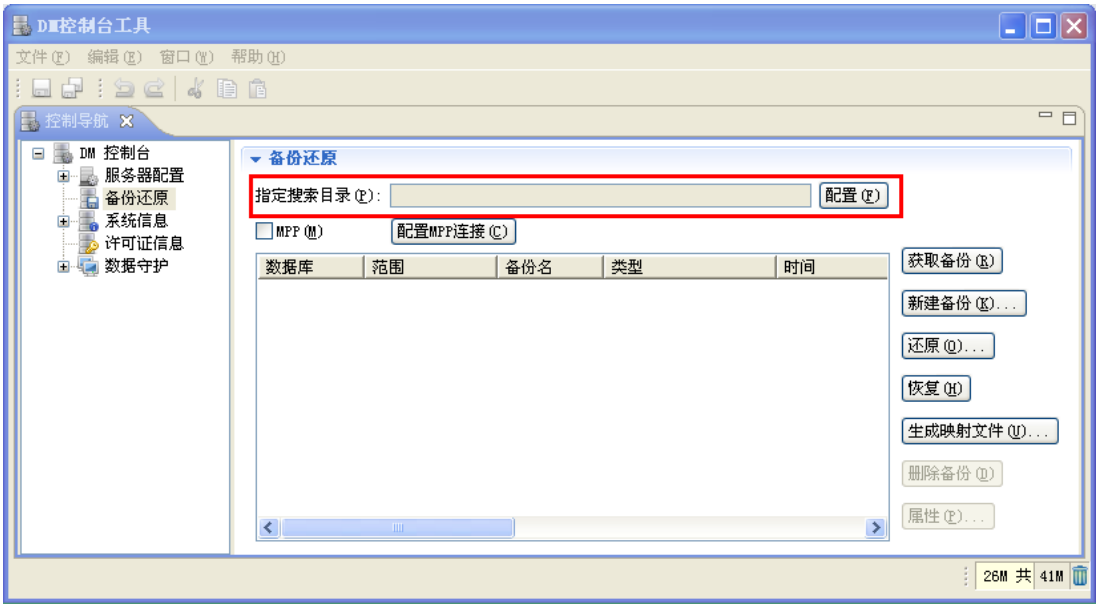


图 3.16 指定搜索目录

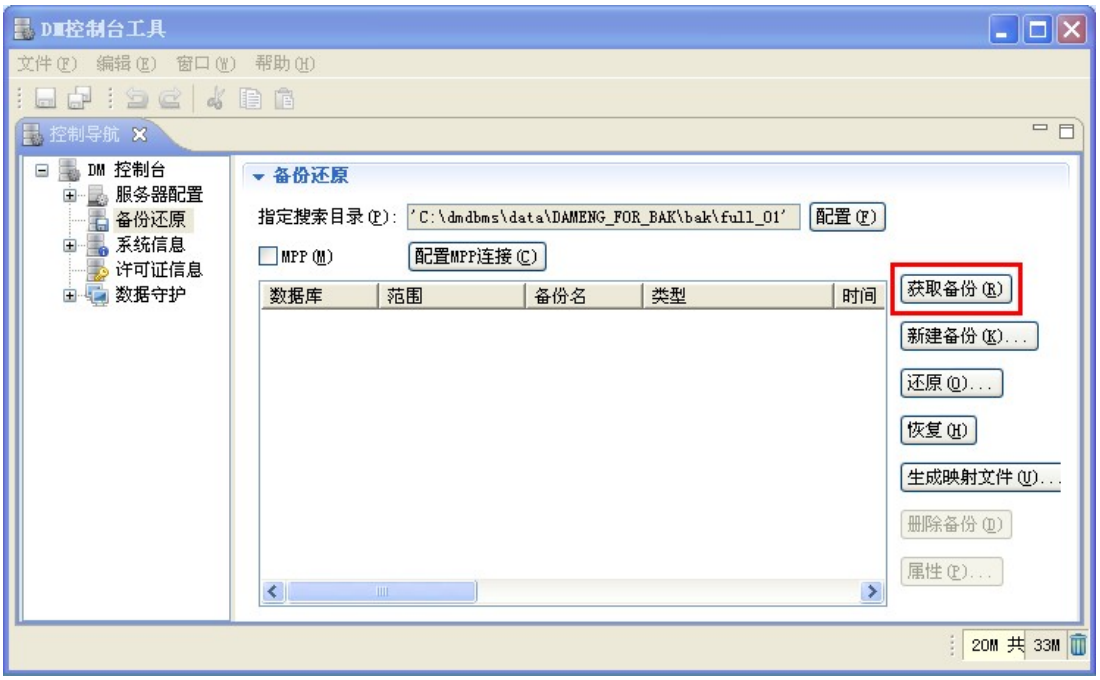


图 3.17 获取备份

可以获取到指定搜索目录下的所有备份集，显示在备份列表中。选择要查看的备份集，点击属性按钮。

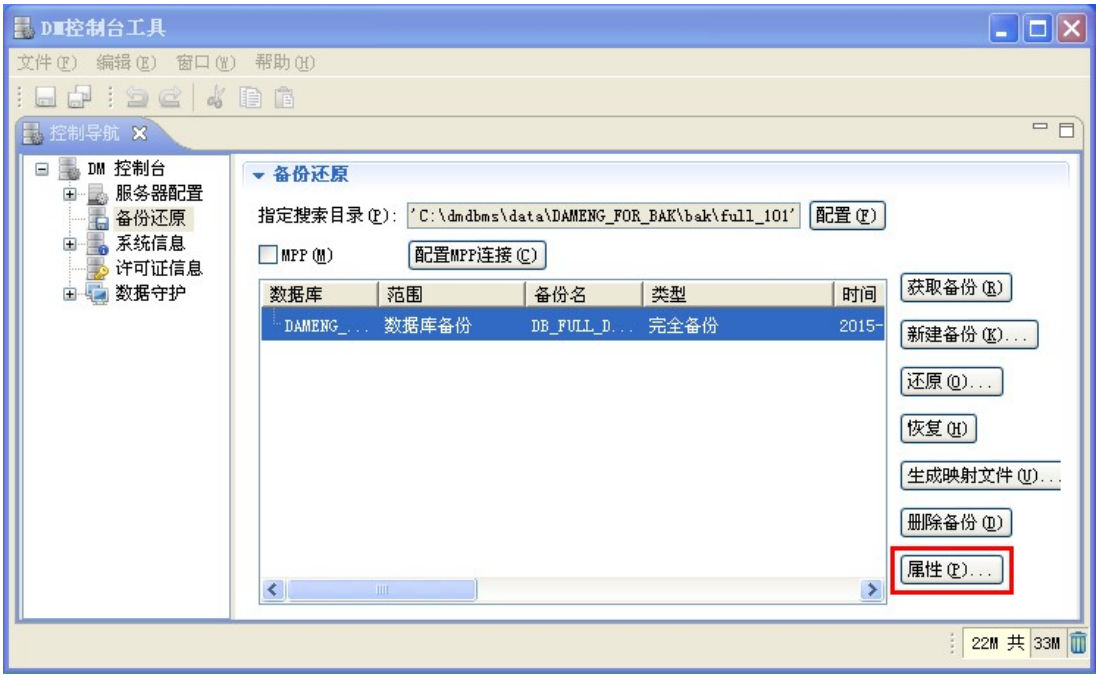


图 3.18 备份集列表显示页面

打开备份属性对话框，可查看备份属性。备份属性分为常规、元信息、文件信息、数据库信息几类。常规属性主要显示备份集相关的属性，如图 3.19 所示。

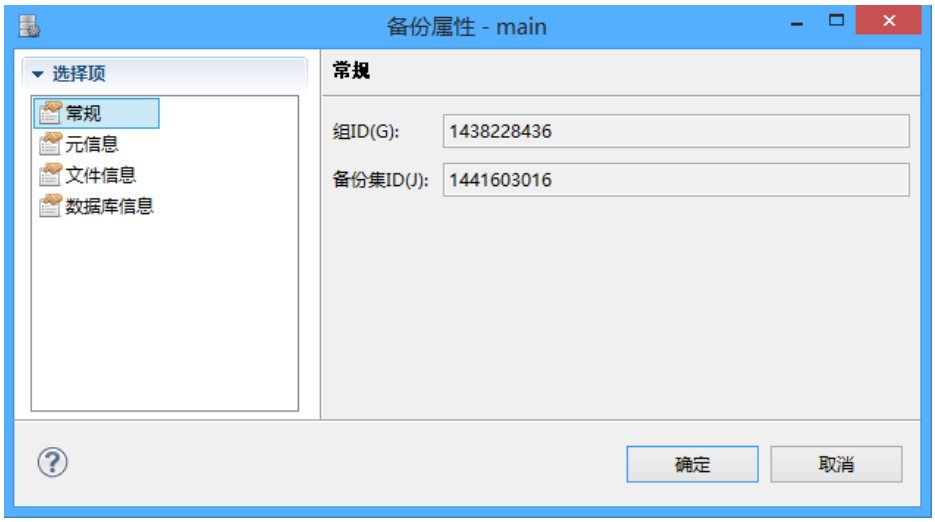


图 3.19 备份集常规信息显示页面

元信息主要显示备份集本身相关的信息如备份是否联机备份、备份的范围、备份的加密信息，以及备份的压缩信息等，如下图所示。

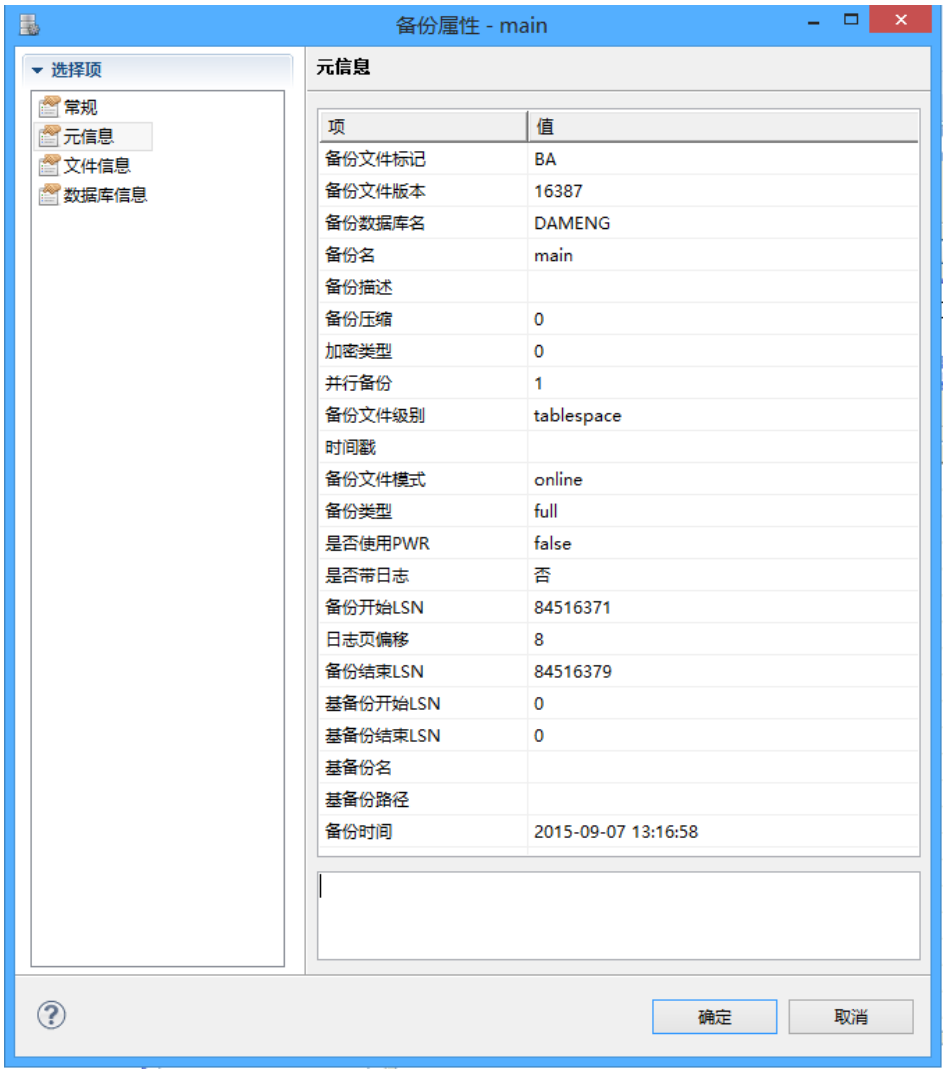


图 3.20 备份集元数据信息显示页面

文件信息显示备份集的所有数据文件和备份片信息，如下图所示。

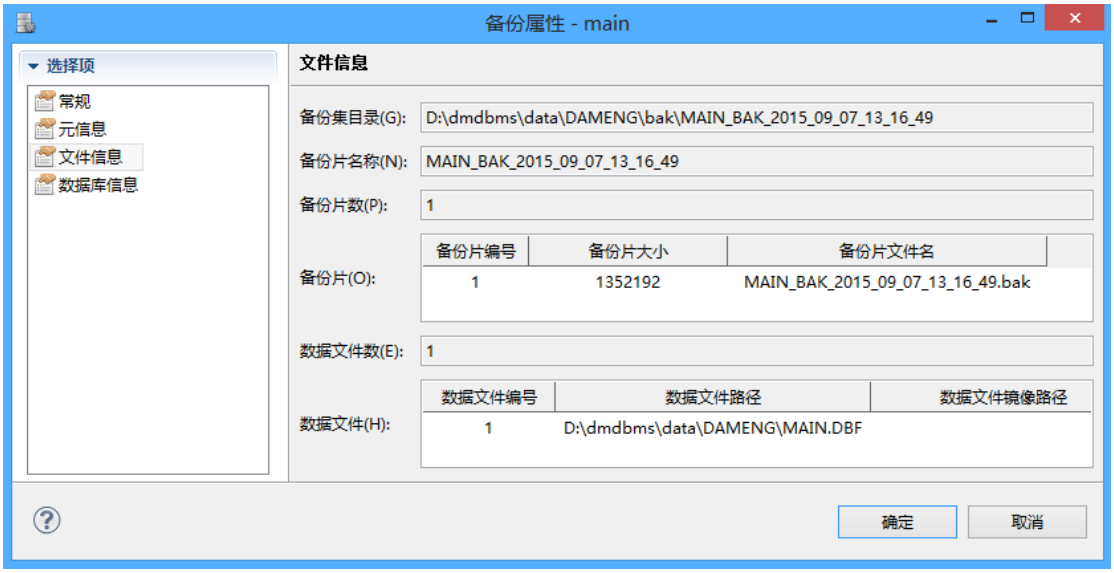


图 3.21 备份集元数据文件信息显示页面

数据库信息显示备份库的固有信息，如下图所示。

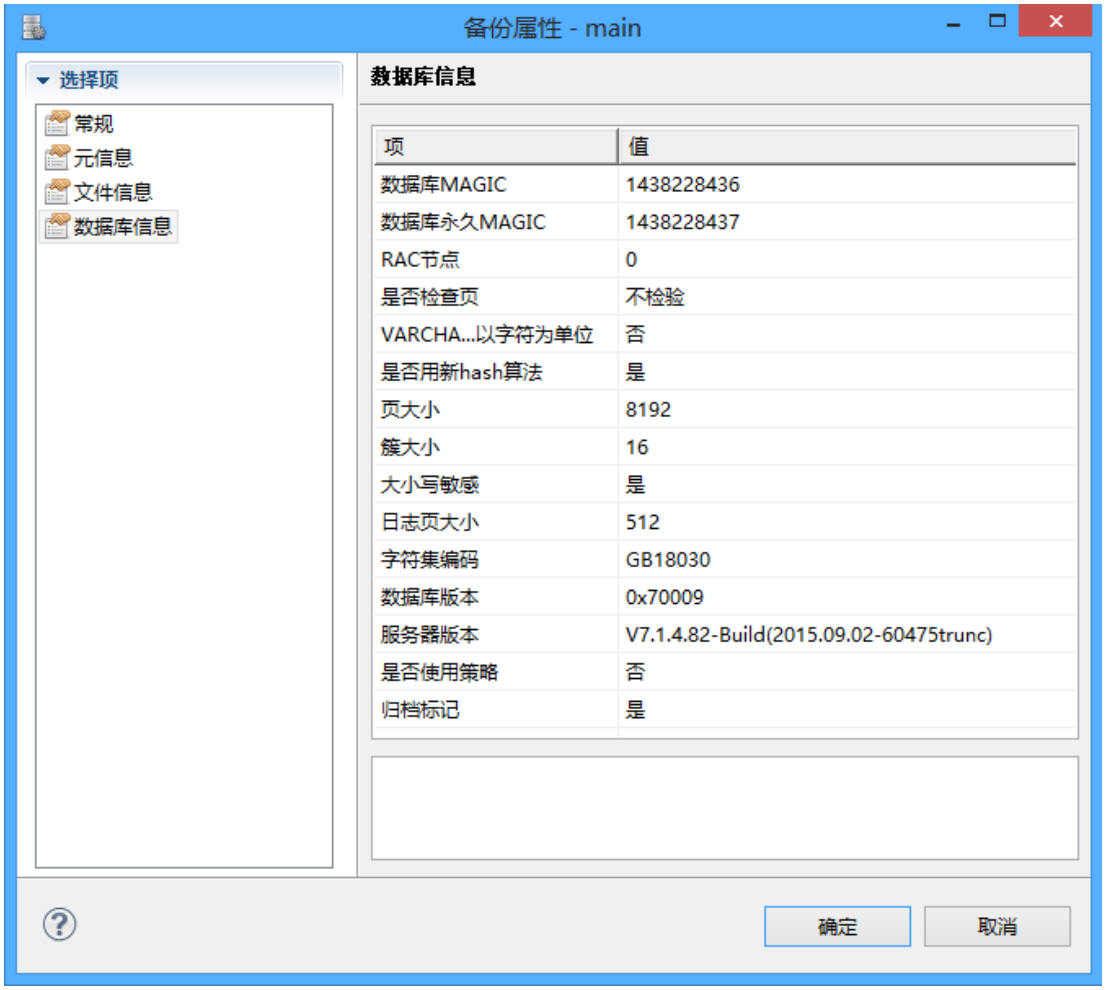


图 3.22 备份集数据库信息显示页面

3.4.2.3 数据还原

数据库还原

在备份还原管理主界面中，点击还原按钮，打开备份还原对话框，如图 3.23 所示。

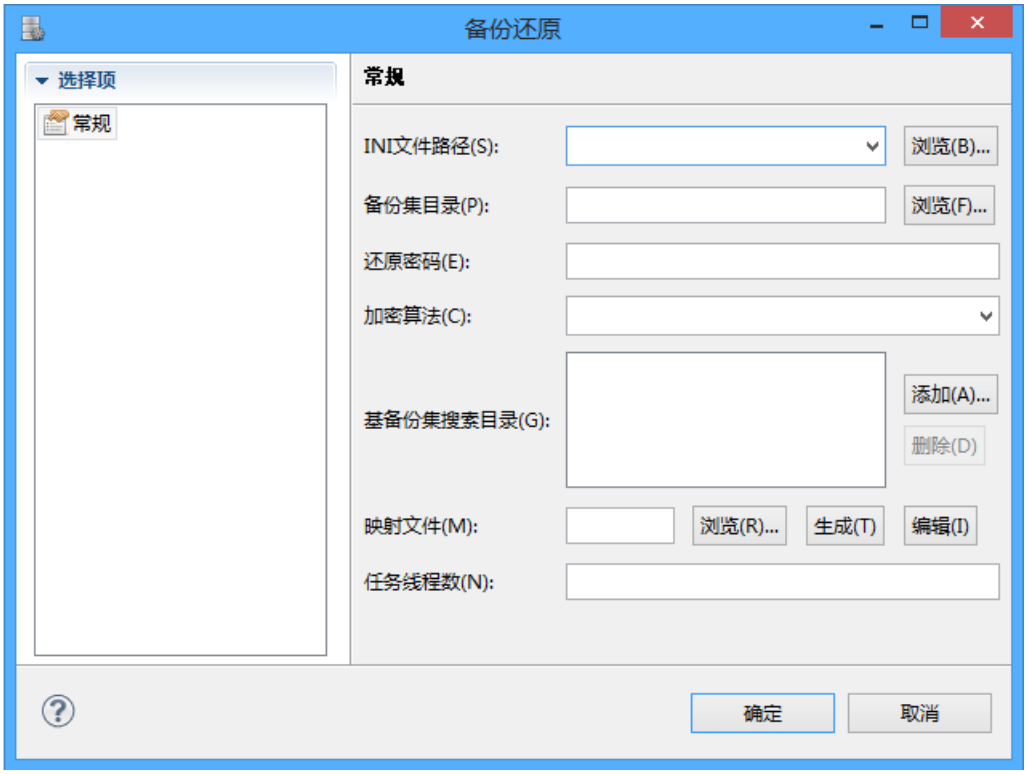


图 3.23 备份集还原常规页面

需要为脱机还原配置以下参数：

INI 文件路径：必填，待还原目标数据库 dm.ini 文件路径。

备份集目录：必填，指定待还原备份集目录。

还原密码：待还原备份集，如果备份时使用了加密密码，则还原时必须指定与备份时一致的密码，供还原过程解密使用。

加密算法：待还原备份集，如果备份时使用了加密算法，在还原时必须指定与备份时一致的加密算法供还原过程解密使用。

基备份集搜索目录：待还原备份集，如果是增量备份集，则可以在此处指定基备份集搜索目录。

映射文件：选填，指定存放还原目标路径的文件。可以通过点击生成按钮，按给定的备份集目录、INI 文件路径以及映射文件参数生成映射文件；也可以通过编辑按钮，编辑映射文件。

任务线程数：选填，指定还原过程中用于处理解压缩和解密任务的线程个数，若未指定，则默认为 4，上限为 64。任务线程数越多使用的系统资源越多，实际应用中应根据系统资源及数据文件个数来设置该参数。

根据实际需要配置完上述参数后，点击确定按钮开始执行脱机还原操作。如果还原成功，则弹出还原成功的对话框；如果还原失败，则弹出对话框提示失败原因。

生成映射文件

数据库还原时可选择映射文件进行还原，映射文件用来指定还原后数据库的数据文件路径（具体参见 3.3.5.2.1 数据库还原）。CONSOLE 工具中生成映射文件有两种方式，一种是在备份还原页面的映射文件选项中点击生成按钮，另外一种是在备份还原主页面中，点击生成映射文件按钮。这里介绍后面一种映射文件生成方式。

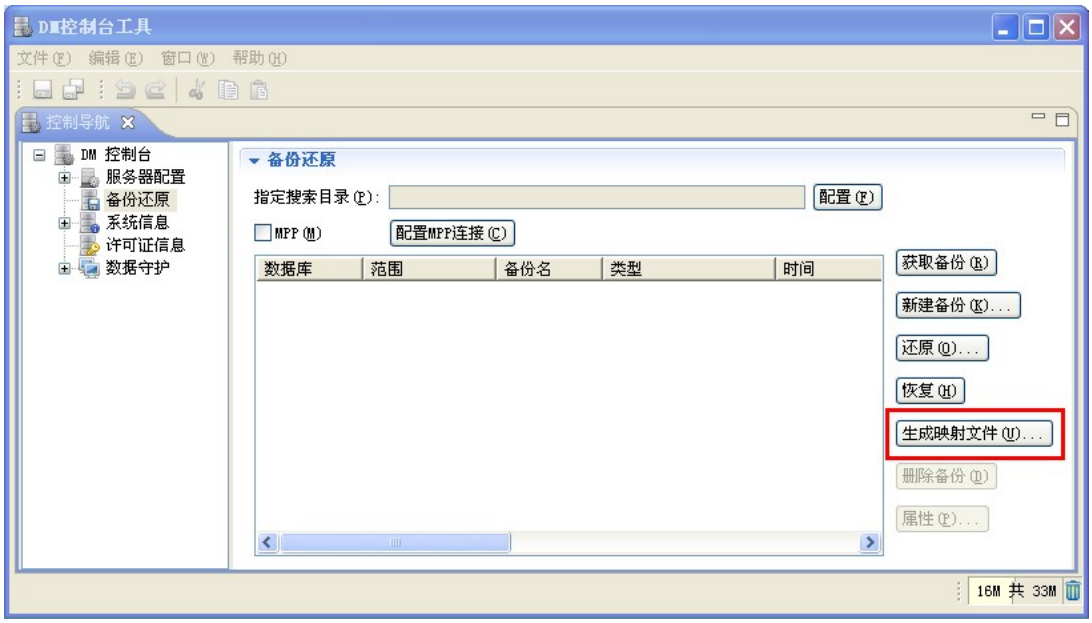


图 3.24 生成映射文件按钮

打开生成映射文件对话框，显示如图 3.25 所示。



图 3.25 生成映射文件对话框

需要配置以下参数：

备份集目录：必填，待生成映射文件的备份集目录。

INI 文件路径：选填，目标还原库的 dm.ini 文件路径。

映射文件：必填，生成映射文件路径。

根据实际需要配置好上述参数后，点击确定按钮，执行生成映射文件操作。如果生成映射文件成功，则弹出生成映射文件成功的对话框；如果生成映射文件失败，则弹出对话框提示失败原因。

3.4.2.4 数据恢复

在备份还原管理界面中，点击恢复按钮，打开备份恢复对话框。有如下三种类型的恢复，默认选中普通恢复。

- 更新 DB_MAGIC
- 从备份集恢复
- 从归档恢复

以下分别进行介绍。

更新 DB_MAGIC

对于联机备份的备份集，对目标库执行完还原操作后，如果仅希望把目标库恢复到备份的时间点，可以借助更新 DB_MAGIC 把目标库恢复到备份结束的时间点。这种情况的恢复要求联机备份集是使用 WITHOUT LOG 类型的备份，且要注意只能恢复到备份结束的时间点。

对于脱机备份的备份集，或者联机备份的备份集中不需要重做联机日志的情况，在执行完还原操作后允许执行更新 DB_MAGIC 操作，更新 DB_MAGIC 恢复仅仅把目标库恢复到备份结束的时间点。

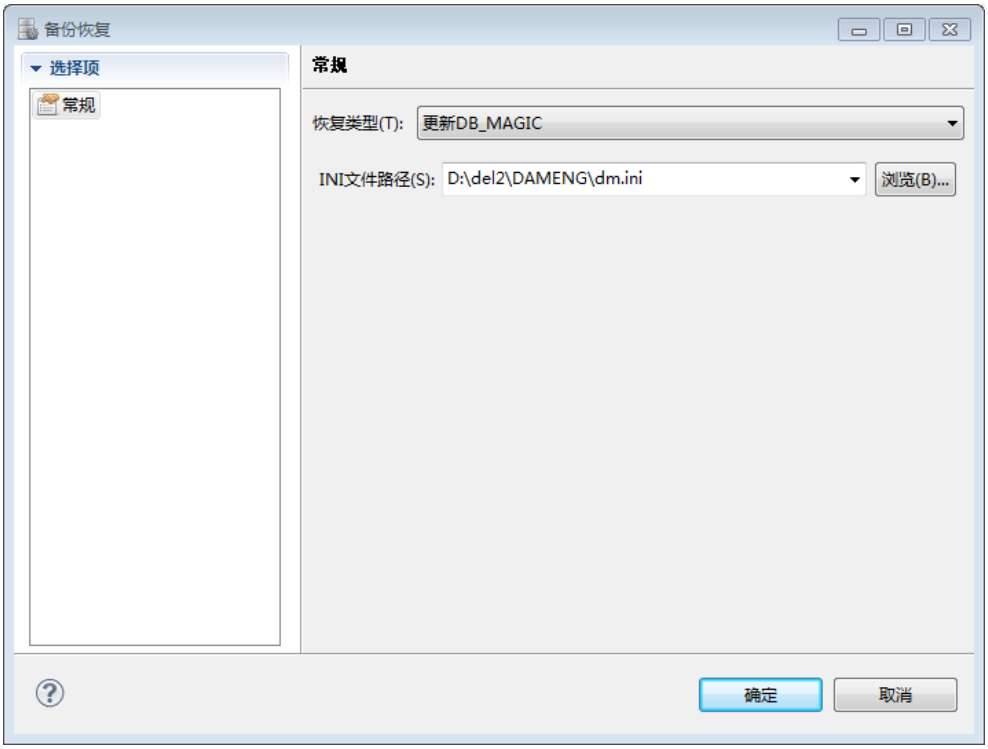


图 3.26 更新 DB_MAGIC 恢复页面

如果选择更新 DB_MAGIC 恢复，则需要配置以下参数：

INI 文件路径：必填，待恢复目标数据库 dm.ini 文件路径。

从备份集恢复

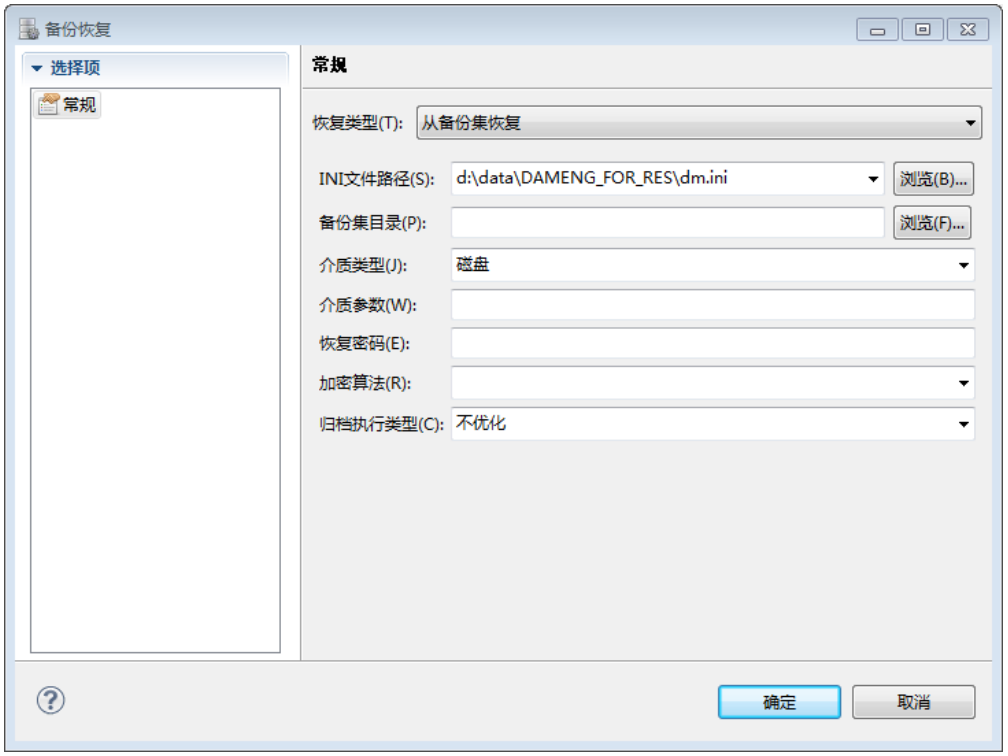


图 3.27 从备份集恢复页面

如果选择从备份集恢复，则需要配置以下参数：

INI 文件路径：必填，待恢复目标数据库 dm.ini 文件路径。

备份集目录：必填，指定待恢复备份集目录。

介质类型：指存储备份集的介质类型，暂支持 DISK 和 TAPE，默认 DISK。

介质参数：介质类型为 TAPE 时，第三方介质管理实现所需的参数字符串。

恢复密码：待恢复备份集，如果备份时使用了加密密码，则恢复时必须指定与备份时一致的密码，供恢复过程解密使用。

加密算法：待恢复备份集，如果恢复时使用了加密算法，在恢复时必须指定与备份时一致的加密算法供恢复过程解密使用。

归档执行类型：指定是否要优化重做本地归档过程，默认不优化。

从归档恢复

从归档恢复，主要有两种类型：

1）没有经过还原操作，直接在目标库上执行恢复操作。这种恢复主要针对一些已经滞后的库，用最新的归档把目标库恢复到某个时间点，或者恢复到最新的状态。

2）经过了还原操作，这时需要借助归档把目标恢复到一个指定的时间点。可能还原的备份集是 WITHOUT LOG 的备份集，没有办法执行从备份集恢复；或者希望恢复到更新的状态，而从备份集恢复又不能够满足要求，所以需要借助归档恢复到某个时间点。

在未指定恢复时间或者恢复 LSN 号时，指定归档恢复默认恢复到最新状态，有多少归档就会重做多少。

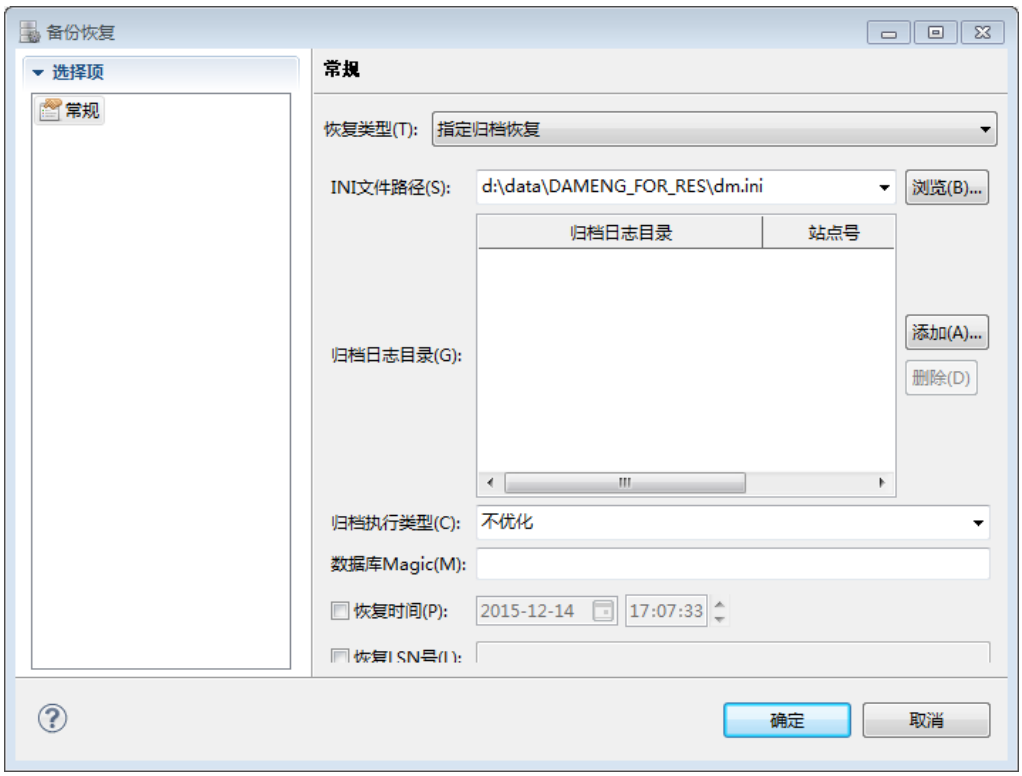


图 3.28 从归档恢复页面

如果选择指定归档恢复，则需要配置以下参数：

INI 文件路径：必填，待恢复目标数据库 dm.ini 文件路径。

归档日志目录：必填，通过添加删除按钮配置归档日志目录，在多站点归档日志恢复环境中，需要配置对应各站点的站点编号。

归档执行类型：指定是否要优化重做本地归档过程，默认不优化。

数据库 Magic：指定本地归档日志对应数据库的 Magic，若不指定，则默认使用目标恢复数据库的 Magic。

恢复时间：指定重做本地归档结束时间。

恢复 LSN 号：指定重做本地归档的结束 LSN 值。

根据实际需要选择对应的恢复类型，配置对应的参数后，点击确定按钮即开始执行对应恢复操作。如果恢复成功，则弹出恢复成功的对话框；如果恢复失败，则弹出对话框提示失败原因。